

Deponentlanadigan materiallarning titul varag'i

EHM uchun dastur (Ma'lumotlar bazasi) nomi:

“«AI Scan» — sun'iy intellekt va chuqur o'rganish algoritmlariga asoslangan, sut bezi o'smalarini erta aniqlash hamda BI-RADS toifalashtirish maqsadida mammografiya DICOM tasvirlarini avtomatlashtirilgan tahlil qilishga mo'ljallangan dasturiy kompleks.”

Huquq ega(lar)si :

1. TURAQULOV SHOXRUX XUDAYAROVICH

Muallif(lar):

1. TURAQULOV SHOXRUX XUDAYAROVICH

EHM uchun dasturni identifikatsiya qiluvchi materiallar dastlabki matni (Dastur kodi)

Dasturiy mahsulot annotatsiyasi (qisqacha tavsifi):

Dasturiy kompleks sut bezi saratonini erta aniqlash maqsadida mammografiya rentgenologik tasvirlarini (DICOM standartida) qabul qilish, bemor identifikatorlarini avtomatik anonimlashtirish (de-identifikatsiya), sun'iy intellekt yordamida shubhali o'choqlarni avtomatik aniqlash (BCA-YOLO, TILLNet-Det chuqur neyron tarmoq arxitekturalari), radiomika belgilari asosida BI-RADS toifalashtirish (XS-Classifer), shifokorlar tomonidan annotatsiyalash, klinik PACS tizimlari bilan integratsiya va natijalarni DICOM SEG / DICOM SR formatlarida eksport qilish imkoniyatlarini birlashtirgan ko'p foydalanuvchili veb-tizimdir.

Dasturlash tili: Python 3.12 (backend), HTML/CSS/JavaScript (frontend)

Asosiy texnologiyalar: FastAPI, SQLite, Docker, Ultralytics YOLO, PyTorch, pydicom/highdicom, scikit-image, scikit-learn

Loyiha sohasi: tibbiy radiologiya, raqamli mammografiya, sun'iy intellekt

Foydalanuvchi turlari: administrator, ekspert-rentgenolog (reviewer), annotator-shifokor

Mundarija (bo'limlar)

1. 1. Konfiguratsiya va deploy fayllari
2. 2. Backend — asosiy ilova kodi (app/)
3. 3. Frontend — veb UI (app/static/)
4. 4. Model arxitekturalari (app/models_arch/)
5. 5. O'qitish va tadqiqot skriptlari (app/research/)
6. 6. Qo'shimcha utilita skriptlari

1. Konfiguratsiya va deploy fayllari

requirements.txt

Fayl yo'li: requirements.txt | Qatorlar: 22 | Hajm: 362 bayt

```
1 fastapi>=0.110
2 uvicorn[standard]>=0.27
3 pydicom>=2.4
4 numpy>=1.26
5 Pillow>=10.0
6 python-multipart>=0.0.9
7 openpyxl>=3.1
8 ultralytics>=8.0
9 PyJWT>=2.8
10 bcrypt>=4.0
11 slowapi>=0.1.9
12 highdicom>=0.22
13 pyotp>=2.9
14 qrcode>=7.0
15 pylibjpeg>=2.0
16 pylibjpeg-libjpeg>=2.0
17 pylibjpeg-openjpeg>=2.0
18 pylibjpeg-rle>=2.0
19 opencv-python-headless>=4.8
20 scipy>=1.11
21 scikit-image>=0.22
22 nibabel>=5.2
```

Dockerfile

Fayl yo'li: Dockerfile | Qatorlar: 22 | Hajm: 555 bayt

```
1 FROM python:3.12-slim
2
3 ENV PYTHONDONTWRITEBYTECODE=1 \
4     PYTHONUNBUFFERED=1 \
5     PIP_NO_CACHE_DIR=1
6
7 WORKDIR /app
8
9 RUN apt-get update && apt-get install -y --no-install-recommends \
10     libgl1 libglu1-mesa-dev \
11     && rm -rf /var/lib/apt/lists/*
12
13 COPY requirements.txt ./
14 RUN pip install --upgrade pip && pip install -r requirements.txt
15
16 COPY app ./app
17
18 RUN mkdir -p /app/app/uploads /app/app/annotations /app/app/models
19
20 EXPOSE 8000
21
22 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--proxy-headers", "--forwarded-allow-ips", "*"]
```

Dockerfile.prod

Fayl yo'li: Dockerfile.prod | Qatorlar: 28 | Hajm: 846 bayt

```
1 FROM python:3.12-slim
2
3 ENV PYTHONDONTWRITEBYTECODE=1 \
4     PYTHONUNBUFFERED=1 \
5     PIP_NO_CACHE_DIR=1 \
6     PIP_DISABLE_PIP_VERSION_CHECK=1 \
7     PIP_DEFAULT_TIMEOUT=180 \
8     PIP_RETRIES=10 \
9     # VM network runs a TLS-inspecting proxy whose CA the clean build container
10    # does not trust; mark PyPI hosts trusted so pip skips cert verification.
11    PIP_TRUSTED_HOST="pypi.org files.pythonhosted.org pypi.python.org"
12
13 WORKDIR /app
14
15 RUN apt-get update && apt-get install -y --no-install-recommends \
16     libgl1 libglu1-mesa-dev \
17     && rm -rf /var/lib/apt/lists/*
18
19 COPY requirements.txt ./
20 RUN pip install -r requirements.txt
21
22 COPY app ./app
23
24 RUN mkdir -p /app/app/uploads /app/app/annotations /app/app/models
25
26 EXPOSE 8000
27
28 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--proxy-headers", "--forwarded-allow-ips", "*"]
```

docker-compose.yml

Fayl yo'li: docker-compose.yml | Qatorlar: 46 | Hajm: 1078 bayt

```
1 services:
2   app:
3     build: .
4     container_name: mamograf-app
5     restart: unless-stopped
6     environment:
7       - LOCAL_DICOM_ROOT=/data/dicom
8       - JWT_SECRET=${JWT_SECRET:-}
9       # Avtomatik anonimlashtirish (PHI tag'lari upload paytida tozalanadi).
10      # Default: yoqilgan. O'chirish uchun: AUTO_DEIDENTIFY=0
11      - AUTO_DEIDENTIFY=${AUTO_DEIDENTIFY:-1}
12     volumes:
13       - app_uploads:/app/app/uploads
14       - app_annotations:/app/app/annotations
15       - app_models:/app/app/models
16       - app_db:/app/app
17       - ${LOCAL_DICOM_HOST_PATH:-./dicom_data}:/data/dicom:ro
18     expose:
19       - "8000"
20     networks: [internal]
21
22   caddy:
23     image: caddy:2-alpine
24     container_name: mamograf-caddy
25     restart: unless-stopped
26     ports:
```

```

27     - "80:80"
28     - "443:443"
29     - "443:443/udp"
30     volumes:
31     - ./Caddyfile:/etc/caddy/Caddyfile:ro
32     - caddy_data:/data
33     - caddy_config:/config
34     depends_on: [app]
35     networks: [internal]
36
37 volumes:
38     app_uploads:
39     app_annotations:
40     app_models:
41     app_db:
42     caddy_data:
43     caddy_config:
44
45 networks:
46     internal:

```

docker-compose.prod.yml

Fayl yo'li: *docker-compose.prod.yml* | Qatorlar: 33 | Hajm: 964 bayt

```

1 name: mamograf-prod
2
3 # Production stack for VM 10.10.0.75 (domain aiscan.airi.uz).
4 # No Caddy: port 80 is closed on this network and TLS is terminated on the
5 # upstream edge proxy, which forwards aiscan.airi.uz:443 -> 10.10.0.75:8081.
6 # (8080 is taken by another VM on this network, so we publish on 8081.)
7 # The app trusts X-Forwarded-* headers (uvicorn --proxy-headers).
8
9 services:
10     app:
11         build:
12             context: .
13             dockerfile: Dockerfile.prod
14         container_name: mamograf-app
15         restart: unless-stopped
16         environment:
17             - LOCAL_DICOM_ROOT=/data/dicom
18             - JWT_SECRET=${JWT_SECRET:-}
19             - AUTO_DEIDENTIFY=${AUTO_DEIDENTIFY:-1}
20         volumes:
21             - app_uploads:/app/app/uploads
22             - app_annotations:/app/app/annotations
23             - app_models:/app/app/models
24             - app_db:/app/app
25             - ${LOCAL_DICOM_HOST_PATH:-./dicom_data}:/data/dicom:ro
26         ports:
27             - "8081:8080"
28
29 volumes:
30     app_uploads:
31     app_annotations:
32     app_models:
33     app_db:

```

Caddyfile

Fayl yo'li: *Caddyfile* | Qatorlar: 27 | Hajm: 538 bayt

```

1  {$DOMAIN:localhost} {
2      encode zstd gzip
3
4      # WebSocket route (preserve Upgrade header)
5      @ws path /ws/*
6      reverse_proxy @ws app:8000
7
8      reverse_proxy app:8000 {
9          header_up X-Real-IP {remote_host}
10         header_up X-Forwarded-For {remote_host}
11     }
12
13     header {
14         Strict-Transport-Security "max-age=63072000; includeSubDomains; preload"
15         X-Content-Type-Options nosniff
16         Referrer-Policy no-referrer
17         X-Frame-Options DENY
18         Permissions-Policy "interest-cohort=()"
19     }
20
21     log {
22         output file /data/access.log {
23             roll_size 50mb
24             roll_keep 5
25         }
26     }
27 }

```

deploy_prod.sh

Fayl yo'li: `deploy_prod.sh` | Qatorlar: 42 | Hajm: 1369 bayt

```

1  #!/usr/bin/env bash
2  # Production bootstrap for MAMOGRAF on VM 10.10.0.75 (behind edge TLS proxy).
3  # Run from the project root: bash deploy_prod.sh
4  set -euo pipefail
5
6  cd "$(dirname "$0")"
7  COMPOSE="docker compose -f docker-compose.prod.yml"
8
9  # --- 1) .env with a generated JWT secret (only if missing) -----
10 if [ ! -f .env ]; then
11     echo "[deploy] .env topilmadi – yaratamiz (JWT_SECRET generatsiya)..."
12     cat > .env <<EOF
13 # Auto-generated by deploy_prod.sh – DO NOT commit.
14 JWT_SECRET=$(openssl rand -base64 48)
15 AUTO_DEIDENTIFY=1
16 LOCAL_DICOM_HOST_PATH=./dicom_data
17 EOF
18     chmod 600 .env
19     echo "[deploy] .env yaratildi."
20 else
21     echo "[deploy] .env mavjud – saqlanib qoldi."
22 fi
23
24 # --- 2) read-only DICOM mount point (empty is fine) -----
25 mkdir -p dicom_data
26
27 # --- 3) Build + start -----
28 echo "[deploy] Image qurilmoqda (ultralitics/torch tufayli uzoq davom etishi mumkin)..."
29 $COMPOSE build
30
31 echo "[deploy] App ishga tushmoqda..."
32 $COMPOSE up -d

```

```

33
34 echo
35 echo "[deploy] ===== HOLAT ====="
36 $COMPOSE ps
37 echo
38 echo "Keyingi qadam – birinchi admin yarating (parol so'raydi):"
39 echo "  $COMPOSE exec app python -m app.manage_users create admin --role admin"
40 echo
41 echo "Lokal tekshiruv:  curl -s http://localhost:8081/ | head"
42 echo "Edge proxy:      aiscan.airi.uz:443  -> 10.10.0.75:8081"

```

.gitignore

Fayl yo'li: .gitignore | Qatorlar: 89 | Hajm: 2012 bayt

```

1  # =====
2  # MAMOGRAF – .gitignore
3  # Qoidalar: bemor ma'lumotlarini hech qachon yuklamaslik!
4  # =====
5
6  # --- Python / virtualenv / cache ---
7  .venv/
8  venv/
9  env/
10 __pycache__/
11 *.pyc
12 *.pyo
13 *.egg-info/
14 .pytest_cache/
15 .mypy_cache/
16 .ruff_cache/
17
18 # --- IDE ---
19 .idea/
20 .vscode/
21 .claude/
22
23 # --- Maxfiy kalitlar va sertifikatlar ---
24 app/.jwt_secret
25 *.pem
26 *.key
27 .env
28 .env.*
29 !.env.example
30
31 # =====
32 # BEMOR MA'LUMOTLARI (PHI) – YUKLAMASLIK SHART
33 # =====
34 # DICOM fayllar va PACS inbox (haqiqiy bemor rasmlar)
35 app/uploads/
36 # Annotation fayllari (DICOM ID'larga bog'langan)
37 app/annotations/
38 # SQLite baza (foydalanuvchilar, sessiyalar, bemorlar)
39 app/db.sqlite3
40 app/db.sqlite3-*
41 # Bemorlar Excel
42 MamologiyaInfo_.xlsx
43 MamologiyaInfo_*.xlsx
44 # Research datasets (haqiqiy data)
45 app/research/datasets/
46 app/research/__pycache__/
47
48 # =====
49 # MODEL OG'IRLIKLARI – Git LFS yoki HuggingFace Hub orqali

```



```

50 # =====
51 app/models/*.pt
52 app/models/*.pth
53 app/models/*.onnx
54 app/models/*.engine
55 # README.md app/models/ ichida saqlanadi
56
57 # --- Installerlar / katta artifaktlar ---
58 *.msi
59 *.exe
60 *.dmg
61
62 # --- Build artefaktlar ---
63 build/
64 dist/
65 *.spec
66
67 # --- Logs ---
68 *.log
69 logs/
70
71 # --- MS Office lock fayllar (ochiq hujjat) ---
72 ~$*
73 *.tmp
74 *.bak
75
76 # =====
77 # SCREENSHOT'LAR VA QO'LLANMA PDF'LAR – ehtimol PHI bor
78 # =====
79 docs/screenshots/
80 docs/researcher_screenshots/
81 FOYDALANUVCHI_QOLLANMA.pdf
82 TADQIQOTCHI_QOLLANMA.pdf
83
84 # =====
85 # DISSERTATSIYA VA ILMIY MAQOLALAR – bu repo'ga kerak emas
86 # =====
87 dissertation/
88 paper/
89

```

2. Backend — asosiy ilova kodi (app/)

app/main.py

Fayl yo'li: app/main.py | Qatorlar: 3030 | Hajm: 103665 bayt

```
1 from __future__ import annotations
2
3 import csv
4 import io
5 import json
6 import os
7 import re
8 import uuid
9 from contextlib import asynccontextmanager
10 from datetime import datetime, timezone
11 from pathlib import Path
12 from typing import Optional
13
14 import pydicom
15 from fastapi import Depends, FastAPI, File, HTTPException, Query, Request, UploadFile, WebSocket,
WebSocketDisconnect
16 from fastapi.middleware.cors import CORSMiddleware
17 from fastapi.responses import FileResponse, StreamingResponse
18 from fastapi.staticfiles import StaticFiles
19 from pydantic import BaseModel
20 from slowapi import Limiter, _rate_limit_exceeded_handler
21 from slowapi.errors import RateLimitExceeded
22 from slowapi.util import get_remote_address
23
24 from . import annotations as annot_store
25 from . import auth as auth_mod
26 from . import db as db_mod
27 from . import deidentify as deid
28 from . import dicom_seg as dseg
29 from . import dicom_sr as dsr
30 from . import exporters as exporters
31 from . import radiomics as radiomics_mod
32 from . import inference as inf
33 from . import pacs as pacs_mod
34 from . import ws as ws_mod
35 from .dicom_utils import (
36     NoPixelDataError, extract_sr_content, load_frame_array, quick_summary,
37     read_metadata, render_frame_png,
38 )
39
40 BASE_DIR = Path(__file__).resolve().parent
41 UPLOAD_DIR = BASE_DIR / "uploads"
42 UPLOAD_DIR.mkdir(exist_ok=True)
43 ANNOT_DIR = BASE_DIR / "annotations"
44 ANNOT_DIR.mkdir(exist_ok=True)
45 STATIC_DIR = BASE_DIR / "static"
46
47 # Avtomatik anonimlashtirish: upload paytida DICOM PHI tag'lari tozalanadi.
48 # AUTO_DEIDENTIFY=0 yoki "false" o'rnatilsa – o'chiriladi (sukut: yoqilgan).
49 AUTO_DEIDENTIFY = os.environ.get("AUTO_DEIDENTIFY", "1").strip().lower() not in (
50     "0", "false", "no", "off", ""
51 )
52
53 DEFAULT_LABELS = [
54     {"name": "mass", "color": "#ff5050"},
```

```

55     {"name": "calcification", "color": "#ffb000"},
56     {"name": "asymmetry", "color": "#00c4ff"},
57     {"name": "architectural_distortion", "color": "#a070ff"},
58     {"name": "skin_thickening", "color": "#28d97f"},
59     {"name": "nipple_retraction", "color": "#ff80b4"},
60     {"name": "lymph_node", "color": "#ffe44d"},
61     {"name": "other", "color": "#9aa3b2"},
62 ]
63 BIRADS_VALUES = ["0", "1", "2", "3", "4A", "4B", "4C", "5", "6"]
64 LABELS_CONFIG = BASE_DIR / "labels.json"
65
66
67 def load_labels_config() -> tuple[list[dict], list[str]]:
68     if LABELS_CONFIG.exists():
69         try:
70             data = json.loads(LABELS_CONFIG.read_text(encoding="utf-8"))
71             labels = data.get("labels") or DEFAULT_LABELS
72             birads = data.get("birads") or BIRADS_VALUES
73             return labels, birads
74         except Exception:
75             pass
76     return DEFAULT_LABELS, BIRADS_VALUES
77
78 LOCAL_ROOT_ENV = os.environ.get("LOCAL_DICOM_ROOT", "").strip()
79 LOCAL_ROOT: Optional[Path] = Path(LOCAL_ROOT_ENV).resolve() if LOCAL_ROOT_ENV else None
80
81 @asynccontextmanager
82 async def lifespan(_app: FastAPI):
83     db_mod.init_db()
84     yield
85
86
87 def _client_key(request: Request) -> str:
88     auth = request.headers.get("Authorization", "")
89     if auth.startswith("Bearer "):
90         try:
91             payload = auth_mod.decode_token(auth.split(" ", 1)[1])
92             return f"user:{payload.get('sub','?')}"
93         except HTTPException:
94             pass
95     return f"ip:{get_remote_address(request)}"
96
97
98 limiter = Limiter(key_func=_client_key)
99
100 app = FastAPI(title="MAMOGRAF DICOM Viewer", lifespan=lifespan)
101 app.state.limiter = limiter
102 app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)
103
104 _ALLOWED_ORIGINS = [
105     o.strip() for o in os.environ.get("ALLOWED_ORIGINS", "").split(",") if o.strip()
106 ]
107 if _ALLOWED_ORIGINS:
108     app.add_middleware(
109         CORSMiddleware,
110         allow_origins=_ALLOWED_ORIGINS,
111         allow_credentials=True,
112         allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
113         allow_headers=["Authorization", "Content-Type"],
114     )
115
116 _CSP = (
117     "default-src 'self'; "
118     "img-src 'self' data: blob; "
119     "style-src 'self' 'unsafe-inline'; "
120     "script-src 'self'; "

```

```

121     "connect-src 'self' ws: wss;; "
122     "object-src 'none'; "
123     "base-uri 'self'; "
124     "frame-ancestors 'none';"
125 )
126
127
128 @app.middleware("http")
129 async def security_headers(request: Request, call_next):
130     response = await call_next(request)
131     response.headers["X-Content-Type-Options"] = "nosniff"
132     response.headers["X-Frame-Options"] = "DENY"
133     response.headers["Referrer-Policy"] = "no-referrer"
134     response.headers["Permissions-Policy"] = "interest-cohort=()"
135     response.headers["Content-Security-Policy"] = _CSP
136     return response
137
138
139 def _resolve_local(rel: str) -> Path:
140     if LOCAL_ROOT is None:
141         raise HTTPException(404, "local browsing disabled (set LOCAL_DICOM_ROOT)")
142     p = (LOCAL_ROOT / rel).resolve()
143     try:
144         p.relative_to(LOCAL_ROOT)
145     except ValueError:
146         raise HTTPException(400, "invalid path")
147     if not p.exists() or not p.is_file():
148         raise HTTPException(404, "file not found")
149     return p
150
151
152 def _looks_like_dicom(p: Path) -> bool:
153     suf = p.suffix.lower()
154     if suf in (".dcm", ".dicom"):
155         return True
156     if suf == "":
157         try:
158             with p.open("rb") as f:
159                 f.seek(128)
160                 return f.read(4) == b"DICM"
161         except OSError:
162             return False
163     return False
164
165
166 @app.get("/api/config")
167 def config():
168     return {
169         "local_browsing": LOCAL_ROOT is not None,
170         "local_root": str(LOCAL_ROOT) if LOCAL_ROOT else None,
171         "auth_required": auth_mod.has_any_user(),
172     }
173
174
175 class LoginBody(BaseModel):
176     username: str
177     password: str
178     totp_code: Optional[str] = None
179
180
181 @app.post("/api/auth/login")
182 @limiter.limit("10/minute")
183 def auth_login(request: Request, body: LoginBody):
184     user = auth_mod.get_user_by_username(body.username)
185     if not user or not user.get("is_active"):
186         raise HTTPException(401, "invalid credentials")

```

```

187     if not auth_mod.verify_password(body.password, user["password_hash"]):
188         raise HTTPException(401, "invalid credentials")
189     if user.get("totp_enrolled"):
190         if not body.totp_code:
191             raise HTTPException(
192                 status_code=401,
193                 detail={"error": "totp_required", "message": "TOTP kodi kerak"},
194             )
195         if not auth_mod.verify_totp(user.get("totp_secret"), body.totp_code):
196             raise HTTPException(401, "invalid TOTP code")
197     auth_mod.update_last_login(user["id"])
198     token, exp = auth_mod.issue_token(user)
199     pub = auth_mod.public_user(user)
200     pub["totp_setup_required"] = auth_mod.needs_totp_setup(user)
201     return {
202         "token": token,
203         "expires_at": exp,
204         "user": pub,
205     }
206
207
208 @app.get("/api/settings")
209 def get_settings(_user: dict = Depends(auth_mod.require_user)):
210     return {
211         "totp_required_roles": auth_mod.totp_required_roles(),
212     }
213
214
215 class SettingsBody(BaseModel):
216     totp_required_roles: Optional[list[str]] = None
217
218
219 @app.patch("/api/settings")
220 def update_settings(
221     body: SettingsBody,
222     user: dict = Depends(auth_mod.require_role("admin")),
223 ):
224     if body.totp_required_roles is not None:
225         valid = {r for r in body.totp_required_roles if r in auth_mod.ROLES}
226         auth_mod.set_setting(
227             "totp_required_roles", ", ".join(sorted(valid)), updated_by=user.get("sub")
228         )
229     return {"ok": True, "totp_required_roles": auth_mod.totp_required_roles()}
230
231
232 @app.post("/api/auth/totp/setup")
233 def auth_totp_setup(user: dict = Depends(auth_mod.require_user)):
234     row = auth_mod.get_user_by_username(user["sub"])
235     if not row:
236         raise HTTPException(404, "user not found")
237     if row.get("totp_enrolled"):
238         raise HTTPException(409, "TOTP allaqachon yoqilgan")
239     secret = auth_mod.generate_totp_secret()
240     auth_mod.set_totp_secret(user["sub"], secret, enrolled=False)
241     uri = auth_mod.totp_provisioning_uri(user["sub"], secret)
242     qr = auth_mod.totp_qr_data_uri(uri)
243     return {"secret": secret, "uri": uri, "qr_png_data_url": qr}
244
245
246 class TotpVerifyBody(BaseModel):
247     code: str
248
249
250 @app.post("/api/auth/totp/verify")
251 def auth_totp_verify(
252     body: TotpVerifyBody,

```

```

253     user: dict = Depends(auth_mod.require_user),
254 ):
255     row = auth_mod.get_user_by_username(user["sub"])
256     if not row or not row.get("totp_secret"):
257         raise HTTPException(400, "TOTP setup qilinmagan")
258     if not auth_mod.verify_totp(row.get("totp_secret"), body.code):
259         raise HTTPException(401, "kod noto'g'ri")
260     auth_mod.set_totp_secret(user["sub"], row.get("totp_secret"), enrolled=True)
261     return {"ok": True, "totp_enrolled": True}
262
263
264 class TotpDisableBody(BaseModel):
265     password: str
266
267
268 @app.post("/api/auth/totp/disable")
269 def auth_totp_disable(
270     body: TotpDisableBody,
271     user: dict = Depends(auth_mod.require_user),
272 ):
273     row = auth_mod.get_user_by_username(user["sub"])
274     if not row:
275         raise HTTPException(404, "user not found")
276     if not auth_mod.verify_password(body.password, row["password_hash"]):
277         raise HTTPException(401, "parol noto'g'ri")
278     auth_mod.set_totp_secret(user["sub"], None, enrolled=False)
279     return {"ok": True}
280
281
282 @app.get("/api/auth/me")
283 def auth_me(user: dict = Depends(auth_mod.require_user)):
284     row = auth_mod.get_user_by_username(user["sub"])
285     if not row:
286         raise HTTPException(401, "user not found")
287     pub = auth_mod.public_user(row)
288     pub["totp_setup_required"] = auth_mod.needs_totp_setup(row)
289     return pub
290
291
292 class PasswordChangeBody(BaseModel):
293     current_password: str
294     new_password: str
295
296
297 @app.post("/api/auth/change-password")
298 def auth_change_password(
299     body: PasswordChangeBody,
300     user: dict = Depends(auth_mod.require_user),
301 ):
302     row = auth_mod.get_user_by_username(user["sub"])
303     if not row or not auth_mod.verify_password(body.current_password, row["password_hash"]):
304         raise HTTPException(401, "current password incorrect")
305     if len(body.new_password) < 4:
306         raise HTTPException(400, "new password too short")
307     auth_mod.update_password(user["sub"], body.new_password)
308     return {"ok": True}
309
310
311 @app.get("/api/auth/users")
312 def auth_users(_admin: dict = Depends(auth_mod.require_role("admin"))):
313     return {"users": auth_mod.list_users()}
314
315
316 class CreateUserBody(BaseModel):
317     username: str
318     password: str

```

```

319     role: str
320     display_name: Optional[str] = None
321     email: Optional[str] = None
322
323
324 @app.post("/api/auth/users")
325 def admin_create_user(
326     body: CreateUserBody,
327     _admin: dict = Depends(auth_mod.require_role("admin")),
328 ):
329     if body.role not in auth_mod.ROLES:
330         raise HTTPException(400, f"role must be one of {auth_mod.ROLES}")
331     if not body.username or not body.username.strip():
332         raise HTTPException(400, "username required")
333     if len(body.password) < 4:
334         raise HTTPException(400, "password too short (min 4)")
335     if auth_mod.get_user_by_username(body.username):
336         raise HTTPException(409, "username already exists")
337     user = auth_mod.create_user(
338         username=body.username.strip(),
339         password=body.password,
340         role=body.role,
341         display_name=body.display_name,
342         email=body.email,
343     )
344     return {"ok": True, "user": auth_mod.public_user(user)}
345
346
347 class UpdateUserBody(BaseModel):
348     role: Optional[str] = None
349     display_name: Optional[str] = None
350     email: Optional[str] = None
351     is_active: Optional[bool] = None
352
353
354 @app.patch("/api/auth/users/{username}")
355 def admin_update_user(
356     username: str,
357     body: UpdateUserBody,
358     _admin: dict = Depends(auth_mod.require_role("admin")),
359 ):
360     user = auth_mod.get_user_by_username(username)
361     if not user:
362         raise HTTPException(404, "user not found")
363     fields = []
364     params: list = []
365     if body.role is not None:
366         if body.role not in auth_mod.ROLES:
367             raise HTTPException(400, "invalid role")
368         if username == admin.get("sub") and body.role != "admin":
369             raise HTTPException(400, "cannot demote yourself")
370         fields.append("role = ?"); params.append(body.role)
371     if body.display_name is not None:
372         fields.append("display_name = ?"); params.append(body.display_name)
373     if body.email is not None:
374         fields.append("email = ?"); params.append(body.email)
375     if body.is_active is not None:
376         if username == admin.get("sub") and not body.is_active:
377             raise HTTPException(400, "cannot deactivate yourself")
378         fields.append("is_active = ?"); params.append(1 if body.is_active else 0)
379     if not fields:
380         return {"ok": True, "user": auth_mod.public_user(user), "no_changes": True}
381     params.append(username)
382     with db_mod.get_conn() as c:
383         c.execute(f"UPDATE users SET {'', '.join(fields)} WHERE username = ?", params)
384         c.commit()

```

```

385     refreshed = auth_mod.get_user_by_username(username)
386     return {"ok": True, "user": auth_mod.public_user(refreshed)}
387
388
389 class AdminResetPasswordBody(BaseModel):
390     new_password: str
391
392
393 @app.post("/api/auth/users/{username}/reset-password")
394 def admin_reset_password(
395     username: str,
396     body: AdminResetPasswordBody,
397     _admin: dict = Depends(auth_mod.require_role("admin")),
398 ):
399     if not auth_mod.get_user_by_username(username):
400         raise HTTPException(404, "user not found")
401     if len(body.new_password) < 4:
402         raise HTTPException(400, "password too short (min 4)")
403     auth_mod.update_password(username, body.new_password)
404     return {"ok": True}
405
406
407 @app.delete("/api/auth/users/{username}")
408 def admin_delete_user(
409     username: str,
410     _admin: dict = Depends(auth_mod.require_role("admin")),
411 ):
412     if username == admin.get("sub"):
413         raise HTTPException(400, "cannot delete yourself")
414     if not auth_mod.get_user_by_username(username):
415         raise HTTPException(404, "user not found")
416     with db_mod.get_conn() as c:
417         c.execute("DELETE FROM users WHERE username = ?", (username,))
418         c.commit()
419     return {"ok": True}
420
421
422 _UPLOAD_CHUNK = 1024 * 1024 # 1 MB
423
424
425 @app.post("/api/upload")
426 async def upload(
427     files: list[UploadFile] = File(...),
428     _user: dict = Depends(auth_mod.require_user),
429 ):
430     saved = []
431     for f in files:
432         file_id = uuid.uuid4().hex
433         out = UPLOAD_DIR / f"{file_id}.dcm"
434         with out.open("wb") as fout:
435             while True:
436                 chunk = await f.read(_UPLOAD_CHUNK)
437                 if not chunk:
438                     break
439                 fout.write(chunk)
440             try:
441                 ds = pydicom.dcmread(out, stop_before_pixels=True, force=True)
442             except Exception as e:
443                 out.unlink(missing_ok=True)
444                 raise HTTPException(status_code=400, detail=f"{f.filename}: not a valid DICOM ({e})")
445
446         # Avtomatik anonimlashtirish: PHI tag'lari tozalanadi va fayl qayta yoziladi.
447         # Box/annotation va export PHI-siz fayl ustida bajariladi.
448         deidentified_now = False
449         if AUTO_DEIDENTIFY:
450             try:

```



```

451         deid.anonymize_in_place(out)
452         ds = pydicom.dcmread(out, stop_before_pixels=True, force=True)
453         deidentified_now = True
454     except Exception as e:
455         out.unlink(missing_ok=True)
456         raise HTTPException(
457             status_code=500,
458             detail=f"{f.filename}: anonymization failed ({e})",
459         )
460
461     rows = int(getattr(ds, "Rows", 0) or 0)
462     cols = int(getattr(ds, "Columns", 0) or 0)
463     info = {
464         "id": file_id,
465         "patient": str(getattr(ds, "PatientName", "")),
466         "patient_id": str(getattr(ds, "PatientID", "")),
467         "modality": str(getattr(ds, "Modality", "")),
468         "study_date": str(getattr(ds, "StudyDate", "")),
469         "view": str(getattr(ds, "ViewPosition", "")),
470         "laterality": str(getattr(ds, "ImageLaterality", "")),
471         "rows": rows,
472         "cols": cols,
473         "frames": int(getattr(ds, "NumberOfFrames", 1) or 1) if rows and cols else 0,
474         "has_pixels": bool(rows and cols),
475         "annotation_count": 0,
476         "original_name": f.filename,
477         "deidentified": deidentified_now,
478     }
479     saved.append(info)
480     return {"files": saved}
481
482
483 @app.get("/api/files")
484 def list_files():
485     items = []
486     for p in sorted(UPLOAD_DIR.glob("*.dcm")):
487         info = quick_summary(p)
488         if "error" in info:
489             continue
490         info["id"] = p.stem
491         info["annotation_count"] = annot_store.count(ANNOT_DIR, "upload", p.stem)
492         items.append(info)
493     return {"files": items}
494
495
496 @app.get("/api/files/{file_id}/metadata")
497 def metadata(file_id: str):
498     p = UPLOAD_DIR / f"{file_id}.dcm"
499     if not p.exists():
500         raise HTTPException(404)
501     return read_metadata(p)
502
503
504 @app.get("/api/files/{file_id}/image")
505 def image(
506     file_id: str,
507     frame: int = 0,
508     wc: Optional[float] = Query(default=None),
509     ww: Optional[float] = Query(default=None),
510     invert: bool = False,
511     max_dim: int = 2048,
512 ):
513     p = UPLOAD_DIR / f"{file_id}.dcm"
514     if not p.exists():
515         raise HTTPException(404)
516     try:

```

```

517     png_bytes = render_frame_png(p, frame=frame, wc=wc, ww=ww, invert=invert, max_dim=max_dim)
518 except NoPixelDataError as e:
519     raise HTTPException(
520         status_code=422,
521         detail={"error": "no_pixels", "modality": e.modality, "sop_class": e.sop_class},
522     )
523 except Exception as e:
524     raise HTTPException(500, f"render failed: {e}")
525 return StreamingResponse(io.BytesIO(png_bytes), media_type="image/png")
526
527
528 @app.delete("/api/files/{file_id}")
529 def delete_file(file_id: str, _user: dict = Depends(auth_mod.require_user)):
530     p = UPLOAD_DIR / f"{file_id}.dcm"
531     if p.exists():
532         p.unlink()
533     return {"ok": True}
534
535
536 @app.get("/api/local/list")
537 def local_list(subdir: str = "", limit: int = 1000):
538     if LOCAL_ROOT is None:
539         return {"enabled": False, "dirs": [], "files": []}
540     target = (LOCAL_ROOT / subdir).resolve()
541     try:
542         target.relative_to(LOCAL_ROOT)
543     except ValueError:
544         raise HTTPException(400, "invalid path")
545     if not target.exists() or not target.is_dir():
546         raise HTTPException(404, "directory not found")
547
548     dirs, files = [], []
549     try:
550         entries = sorted(target.iterdir(), key=lambda x: (not x.is_dir(), x.name.lower()))
551     except PermissionError:
552         raise HTTPException(403, "permission denied")
553
554     for p in entries[:limit]:
555         rel = p.relative_to(LOCAL_ROOT).as_posix()
556         if p.is_dir():
557             dirs.append({"name": p.name, "path": rel})
558         elif _looks_like_dicom(p):
559             try:
560                 size = p.stat().st_size
561             except OSError:
562                 size = 0
563             files.append({
564                 "name": p.name,
565                 "path": rel,
566                 "size": size,
567                 "annotation_count": annot_store.count(ANNOT_DIR, "local", rel),
568             })
569
570     parent = ""
571     if subdir:
572         parent_path = Path(subdir).parent.as_posix()
573         parent = "" if parent_path == "." else parent_path
574
575     return {
576         "enabled": True,
577         "root": str(LOCAL_ROOT),
578         "subdir": subdir,
579         "parent": parent,
580         "dirs": dirs,
581         "files": files,
582     }

```

```

583
584
585 @app.get("/api/local/metadata")
586 def local_metadata(path: str):
587     p = _resolve_local(path)
588     return read_metadata(p)
589
590
591 @app.get("/api/files/{file_id}/sr-content")
592 def upload_sr_content(file_id: str):
593     p = UPLOAD_DIR / f"{file_id}.dcm"
594     if not p.exists():
595         raise HTTPException(404, "upload not found")
596     try:
597         return extract_sr_content(p)
598     except Exception as e:
599         raise HTTPException(500, f"SR parse failed: {e}")
600
601
602 @app.get("/api/local/sr-content")
603 def local_sr_content(path: str):
604     p = _resolve_local(path)
605     try:
606         return extract_sr_content(p)
607     except Exception as e:
608         raise HTTPException(500, f"SR parse failed: {e}")
609
610
611 @app.get("/api/local/summary")
612 def local_summary(path: str):
613     p = _resolve_local(path)
614     return quick_summary(p)
615
616
617 @app.get("/api/local/image")
618 def local_image(
619     path: str,
620     frame: int = 0,
621     wc: Optional[float] = Query(default=None),
622     ww: Optional[float] = Query(default=None),
623     invert: bool = False,
624     max_dim: int = 2048,
625 ):
626     p = _resolve_local(path)
627     try:
628         png_bytes = render_frame_png(p, frame=frame, wc=wc, ww=ww, invert=invert, max_dim=max_dim)
629     except NoPixelDataError as e:
630         raise HTTPException(
631             status_code=422,
632             detail={"error": "no_pixels", "modality": e.modality, "sop_class": e.sop_class},
633         )
634     except Exception as e:
635         raise HTTPException(500, f"render failed: {e}")
636     return StreamingResponse(io.BytesIO(png_bytes), media_type="image/png")
637
638
639 @app.get("/api/labels")
640 def labels():
641     lbls, brs = load_labels_config()
642     return {"labels": lbls, "birads": brs}
643
644
645 class AnnotationsBody(BaseModel):
646     source: str
647     ref: str
648     annotations: list[dict] = []

```

```

649     rows: Optional[int] = None
650     cols: Optional[int] = None
651
652
653 def _validate_source(source: str) -> None:
654     if source not in ("upload", "local"):
655         raise HTTPException(400, "invalid source")
656
657
658 def _validate_ref(source: str, ref: str) -> None:
659     if not ref:
660         raise HTTPException(400, "missing ref")
661     if source == "upload":
662         if not ref.replace("-", "").replace("_", "").isalnum():
663             raise HTTPException(400, "invalid upload id")
664         if not (UPLOAD_DIR / f"{ref}.dcm").exists():
665             raise HTTPException(404, "upload not found")
666     else:
667         _resolve_local(ref)
668
669
670 @app.get("/api/annotations")
671 def get_annotations(source: str, ref: str):
672     _validate_source(source)
673     _validate_ref(source, ref)
674     return annot_store.load(ANNOT_DIR, source, ref)
675
676
677 _HISTORY_IGNORE_FIELDS = {"updated_at", "updated_by"}
678
679
680 def _annotation_state_eq(a: dict, b: dict) -> bool:
681     def normalize(d: dict) -> dict:
682         return {k: v for k, v in d.items() if k not in _HISTORY_IGNORE_FIELDS}
683     return normalize(a) == normalize(b)
684
685
686 def _notify(
687     recipients: list[str],
688     kind: str,
689     title: str,
690     body: str = "",
691     link_source: Optional[str] = None,
692     link_ref: Optional[str] = None,
693     link_annotation_id: Optional[str] = None,
694     actor: Optional[str] = None,
695 ) -> None:
696     if not recipients:
697         return
698     now = datetime.now(timezone.utc).isoformat()
699     with db_mod.get_conn() as c:
700         for r in recipients:
701             c.execute(
702                 "INSERT INTO notifications "
703                 "(recipient, ts, kind, title, body, link_source, link_ref, link_annotation_id, actor) "
704                 "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",
705                 (r, now, kind, title, body, link_source, link_ref, link_annotation_id, actor),
706             )
707         c.commit()
708
709
710 def _reviewer_recipients(exclude: Optional[str] = None) -> list[str]:
711     with db_mod.get_conn() as c:
712         rows = c.execute(
713             "SELECT username FROM users "

```

```

714         "WHERE role IN ('reviewer', 'admin') AND is_active = 1"
715     ).fetchall()
716     return [r[0] for r in rows if r[0] != exclude]
717
718
719     _ANNOT_CHANGE_TS = 0.0
720     _CACHE: dict[str, tuple[float, object]] = {}
721     _CACHE_TTL = 30.0
722
723
724     def _bump_annot_ts() -> None:
725         import time
726         global _ANNOT_CHANGE_TS
727         _ANNOT_CHANGE_TS = time.time()
728
729
730     def _cache_get(key: str):
731         import time
732         entry = _CACHE.get(key)
733         if not entry:
734             return None
735         ts, value = entry
736         if ts < _ANNOT_CHANGE_TS:
737             return None
738         if time.time() - ts > _CACHE_TTL:
739             return None
740         return value
741
742
743     def _cache_set(key: str, value: object) -> None:
744         import time
745         _CACHE[key] = (time.time(), value)
746
747
748     def _record_history(source: str, ref: str, rows: list[dict]) -> None:
749         if not rows:
750             return
751         with db_mod.get_conn() as c:
752             for row in rows:
753                 c.execute(
754                     "INSERT INTO annotation_history "
755                     "(source, ref, annotation_id, action, username, ts, prev_snapshot, new_snapshot) "
756                     "VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
757                     (
758                         source, ref,
759                         row["annotation_id"],
760                         row["action"],
761                         row["username"],
762                         row["ts"],
763                         json.dumps(row["prev"], ensure_ascii=False) if row.get("prev") else None,
764                         json.dumps(row["new"], ensure_ascii=False) if row.get("new") else None,
765                     ),
766                 )
767             c.commit()
768
769
770     @app.put("/api/annotations")
771     async def put_annotations(
772         body: AnnotationsBody,
773         user: dict = Depends(auth_mod.require_user),
774     ):
775         _validate_source(body.source)
776         _validate_ref(body.source, body.ref)
777         now = datetime.now(timezone.utc).isoformat()
778         username = user.get("sub") or ""
779         role = user.get("role") or ""

```

```

780
781     old_data = annot_store.load(ANNOT_DIR, body.source, body.ref)
782     old_by_id = {a.get("id"): a for a in old_data.get("annotations", []) if a.get("id")}
783     new_by_id = {a.get("id"): a for a in body.annotations if a.get("id")}
784
785     history_rows: list[dict] = []
786
787     for aid, new_a in new_by_id.items():
788         old_a = old_by_id.get(aid)
789         if old_a is None:
790             new_a.setdefault("created_by", username)
791             new_a.setdefault("created_at", now)
792             new_a["updated_by"] = username
793             new_a["updated_at"] = now
794             new_a.setdefault("status", "draft")
795             history_rows.append({
796                 "annotation_id": aid, "action": "create",
797                 "username": username, "ts": now,
798                 "prev": None, "new": new_a,
799             })
800         else:
801             new_a["created_by"] = old_a.get("created_by") or username
802             new_a["created_at"] = old_a.get("created_at") or now
803             new_a["status"] = old_a.get("status", "draft")
804             if "reviewed_by" in old_a:
805                 new_a["reviewed_by"] = old_a["reviewed_by"]
806             if "reviewed_at" in old_a:
807                 new_a["reviewed_at"] = old_a["reviewed_at"]
808             if "review_note" in old_a:
809                 new_a["review_note"] = old_a["review_note"]
810
811             if role == "annotator":
812                 if old_a.get("created_by") and old_a.get("created_by") != username:
813                     if not _annotation_state_eq(old_a, new_a):
814                         raise HTTPException(403, f"annotator cannot edit annotation owned by
{old_a.get('created_by')}")
815                 if old_a.get("status") in ("approved",):
816                     if not _annotation_state_eq(old_a, new_a):
817                         raise HTTPException(403, "approved annotation is locked; reviewer must reopen
it first")
818
819                 if not _annotation_state_eq(old_a, new_a):
820                     new_a["updated_by"] = username
821                     new_a["updated_at"] = now
822                     history_rows.append({
823                         "annotation_id": aid, "action": "update",
824                         "username": username, "ts": now,
825                         "prev": old_a, "new": new_a,
826                     })
827                 else:
828                     new_a["updated_by"] = old_a.get("updated_by") or username
829                     new_a["updated_at"] = old_a.get("updated_at") or now
830
831     for aid, old_a in old_by_id.items():
832         if aid in new_by_id:
833             continue
834         if role == "annotator":
835             if old_a.get("created_by") and old_a.get("created_by") != username:
836                 raise HTTPException(403, f"annotator cannot delete annotation owned by
{old_a.get('created_by')}")
837             if old_a.get("status") in ("approved",):
838                 raise HTTPException(403, "approved annotation cannot be deleted by annotator")
839             history_rows.append({
840                 "annotation_id": aid, "action": "delete",
841                 "username": username, "ts": now,
842                 "prev": old_a, "new": None,

```

```

843     })
844
845     annot_store.save(ANNOT_DIR, body.source, body.ref, body.model_dump())
846     _record_history(body.source, body.ref, history_rows)
847     _bump_annot_ts()
848     if history_rows:
849         try:
850             await ws_mod.rooms.broadcast(body.source, body.ref, {
851                 "type": "annotations_changed",
852                 "by": username,
853                 "changes": len(history_rows),
854             })
855         except Exception:
856             pass
857     return {
858         "ok": True,
859         "count": len(body.annotations),
860         "history_rows_added": len(history_rows),
861     }
862
863
864 VALID_STATUSES = ("draft", "submitted", "approved", "rejected")
865
866
867 class StatusChangeBody(BaseModel):
868     source: str
869     ref: str
870     annotation_id: str
871     status: str
872     note: Optional[str] = None
873
874
875 @app.post("/api/annotations/status")
876 async def patch_annotation_status(
877     body: StatusChangeBody,
878     user: dict = Depends(auth_mod.require_user),
879 ):
880     if body.status not in VALID_STATUSES:
881         raise HTTPException(400, f"invalid status (allowed: {VALID_STATUSES})")
882     _validate_source(body.source)
883     _validate_ref(body.source, body.ref)
884
885     data = annot_store.load(ANNOT_DIR, body.source, body.ref)
886     ann = next((a for a in data.get("annotations", []) if a.get("id") == body.annotation_id), None)
887     if not ann:
888         raise HTTPException(404, "annotation not found")
889
890     old_status = ann.get("status", "draft")
891     new_status = body.status
892     role = user.get("role")
893     username = user.get("sub")
894     is_owner = ann.get("created_by") == username
895
896     annotator_allowed = {("draft", "submitted"), ("submitted", "draft")}
897     if role == "annotator":
898         if not is_owner:
899             raise HTTPException(403, "annotator can only change own annotations")
900         if (old_status, new_status) not in annotator_allowed:
901             raise HTTPException(403, "annotator transitions limited to draft→submitted on own work")
902
903     now = datetime.now(timezone.utc).isoformat()
904     ann["status"] = new_status
905     ann["updated_by"] = username
906     ann["updated_at"] = now
907     if new_status in ("approved", "rejected"):
908         ann["reviewed_by"] = username

```

```

909     ann["reviewed_at"] = now
910     if body.note:
911         ann["review_note"] = body.note
912 elif new_status == "draft" and old_status in ("approved", "rejected"):
913     ann.pop("reviewed_by", None)
914     ann.pop("reviewed_at", None)
915     ann.pop("review_note", None)
916
917     annot_store.save(ANNOT_DIR, body.source, body.ref, data)
918     _record_history(body.source, body.ref, [{
919         "annotation_id": body.annotation_id, "action": "status",
920         "username": username, "ts": now,
921         "prev": {"status": old_status},
922         "new": {"status": new_status, "note": body.note},
923     ]])
924     _bump_annot_ts()
925
926     label = ann.get("label") or "annotation"
927     if new_status == "submitted":
928         recipients = _reviewer_recipients(exclude=username)
929         _notify(
930             recipients,
931             kind="submitted",
932             title=f"{username} '{label}' annotatsiyasini ko'rib chiqishga yubordi",
933             body=f"DICOM: {body.ref}",
934             link_source=body.source, link_ref=body.ref,
935             link_annotation_id=body.annotation_id,
936             actor=username,
937         )
938     elif new_status in ("approved", "rejected"):
939         owner = ann.get("created_by")
940         if owner and owner != username:
941             verb = "tasdiqladi" if new_status == "approved" else "rad etdi"
942             _notify(
943                 [owner],
944                 kind=new_status,
945                 title=f"{username} sizning '{label}' annotatsiyangizni {verb}",
946                 body=body.note or "",
947                 link_source=body.source, link_ref=body.ref,
948                 link_annotation_id=body.annotation_id,
949                 actor=username,
950             )
951
952     try:
953         await ws_mod.rooms.broadcast(body.source, body.ref, {
954             "type": "status_changed",
955             "by": username,
956             "annotation_id": body.annotation_id,
957             "status": new_status,
958         })
959     except Exception:
960         pass
961
962     return {"ok": True, "status": new_status, "previous": old_status, "snapshot": ann}
963
964
965 @app.get("/api/notifications")
966 def get_notifications(
967     unread_only: bool = False,
968     limit: int = 50,
969     user: dict = Depends(auth_mod.require_user),
970 ):
971     sql = "SELECT * FROM notifications WHERE recipient IN (?, '*')"
972     params: list = [user["sub"]]
973     if unread_only:
974         sql += " AND read_at IS NULL"

```



```

975     sql += " ORDER BY ts DESC LIMIT ?"
976     params.append(limit)
977     with db_mod.get_conn() as c:
978         rows = [dict(r) for r in c.execute(sql, params).fetchall()]
979         unread = c.execute(
980             "SELECT COUNT(*) FROM notifications "
981             "WHERE recipient IN (?, '*') AND read_at IS NULL",
982             (user["sub"],),
983         ).fetchone()[0]
984     return {"notifications": rows, "unread_count": unread}
985
986
987 class MarkReadBody(BaseModel):
988     ids: Optional[list[int]] = None
989     all: bool = False
990
991
992 @app.post("/api/notifications/mark-read")
993 def mark_notifications_read(
994     body: MarkReadBody,
995     user: dict = Depends(auth_mod.require_user),
996 ):
997     now = datetime.now(timezone.utc).isoformat()
998     with db_mod.get_conn() as c:
999         if body.all:
1000             c.execute(
1001                 "UPDATE notifications SET read_at = ? "
1002                 "WHERE recipient IN (?, '*') AND read_at IS NULL",
1003                 (now, user["sub"]),
1004             )
1005         elif body.ids:
1006             placeholders = ",".join("? " * len(body.ids))
1007             c.execute(
1008                 f"UPDATE notifications SET read_at = ? "
1009                 f"WHERE id IN ({placeholders}) AND recipient IN (?, '*')",
1010                 [now, *body.ids, user["sub"]],
1011             )
1012         c.commit()
1013     return {"ok": True}
1014
1015
1016 @app.get("/api/annotations/list")
1017 def list_annotations(
1018     status: Optional[str] = None,
1019     created_by: Optional[str] = None,
1020     own: bool = False,
1021     limit: int = 500,
1022     user: dict = Depends(auth_mod.require_user),
1023 ):
1024     if own and not created_by:
1025         created_by = user["sub"]
1026
1027     links: dict[tuple, dict] = {}
1028     with db_mod.get_conn() as c:
1029         for r in c.execute(
1030             "SELECT l.source, l.ref, p.patient_id, p.first_name, p.last_name, "
1031             "p.sex, p.birth_date "
1032             "FROM dicom_patient_links l "
1033             "JOIN patients p ON p.patient_id = l.patient_id"
1034         ).fetchall():
1035             d = dict(r)
1036             links[(d.pop("source"), d.pop("ref"))] = d
1037
1038     items: list[dict] = []
1039     for entry in annot_store.iter_all(ANNOT_DIR):
1040         src = entry.get("source")

```

```

1041         ref = entry.get("ref")
1042         if not src or not ref:
1043             continue
1044         for a in entry.get("annotations", []):
1045             ann_status = a.get("status", "draft")
1046             if status and ann_status != status:
1047                 continue
1048             if created_by and a.get("created_by") != created_by:
1049                 continue
1050             items.append({
1051                 "source": src,
1052                 "ref": ref,
1053                 "annotation_id": a.get("id"),
1054                 "type": a.get("type", "bbox"),
1055                 "label": a.get("label"),
1056                 "labels": a.get("labels") or ([a["label"]] if a.get("label") else []),
1057                 "bi_rads": a.get("bi_rads"),
1058                 "status": ann_status,
1059                 "created_by": a.get("created_by"),
1060                 "updated_by": a.get("updated_by"),
1061                 "reviewed_by": a.get("reviewed_by"),
1062                 "review_note": a.get("review_note"),
1063                 "created_at": a.get("created_at"),
1064                 "updated_at": a.get("updated_at"),
1065                 "patient": links.get((src, ref)),
1066             })
1067
1068         items.sort(key=lambda x: (x.get("updated_at") or "", x.get("created_at") or ""), reverse=True)
1069         return {"items": items[:limit], "count": min(len(items), limit), "total": len(items)}
1070
1071
1072 @app.get("/api/audit/timeline")
1073 def audit_timeline(
1074     since: Optional[str] = None,
1075     username: Optional[str] = None,
1076     action: Optional[str] = None,
1077     limit: int = 200,
1078     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
1079 ):
1080     sql = "SELECT * FROM annotation_history WHERE 1=1"
1081     params: list = []
1082     if since:
1083         sql += " AND ts >= ?"
1084         params.append(since)
1085     if username:
1086         sql += " AND username = ?"
1087         params.append(username)
1088     if action:
1089         sql += " AND action = ?"
1090         params.append(action)
1091     sql += " ORDER BY ts DESC, id DESC LIMIT ?"
1092     params.append(limit)
1093     with db_mod.get_conn() as c:
1094         rows = [dict(r) for r in c.execute(sql, params).fetchall()]
1095     for r in rows:
1096         for k in ("prev_snapshot", "new_snapshot"):
1097             v = r.get(k)
1098             if v:
1099                 try:
1100                     r[k] = json.loads(v)
1101                 except Exception:
1102                     pass
1103     return {"events": rows, "count": len(rows)}
1104
1105
1106 @app.get("/api/annotations/history")

```

```

1107 def get_annotation_history(
1108     source: str,
1109     ref: str,
1110     annotation_id: Optional[str] = None,
1111     limit: int = 200,
1112 ):
1113     _validate_source(source)
1114     _validate_ref(source, ref)
1115     sql = "SELECT * FROM annotation_history WHERE source = ? AND ref = ?"
1116     params: list = [source, ref]
1117     if annotation_id:
1118         sql += " AND annotation_id = ?"
1119         params.append(annotation_id)
1120     sql += " ORDER BY ts DESC, id DESC LIMIT ?"
1121     params.append(limit)
1122
1123     with db_mod.get_conn() as c:
1124         rows = [dict(r) for r in c.execute(sql, params).fetchall()]
1125     for r in rows:
1126         for k in ("prev_snapshot", "new_snapshot"):
1127             v = r.get(k)
1128             if v:
1129                 try:
1130                     r[k] = json.loads(v)
1131                 except Exception:
1132                     pass
1133     return {"history": rows, "count": len(rows)}
1134
1135
1136 def _polygon_area(pts: list[tuple[float, float]]) -> float:
1137     n = len(pts)
1138     if n < 3:
1139         return 0.0
1140     s = 0.0
1141     for i in range(n):
1142         x1, y1 = pts[i]
1143         x2, y2 = pts[(i + 1) % n]
1144         s += x1 * y2 - x2 * y1
1145     return s / 2.0
1146
1147
1148 def _resolve_dims(source: str, ref: str, stored_rows, stored_cols):
1149     if stored_rows and stored_cols:
1150         return int(stored_rows), int(stored_cols)
1151     try:
1152         if source == "upload":
1153             p = UPLOAD_DIR / f"{ref}.dcm"
1154         else:
1155             p = (LOCAL_ROOT / ref).resolve() if LOCAL_ROOT else None
1156         if p and p.exists():
1157             s = quick_summary(p)
1158             return int(s.get("rows") or 0), int(s.get("cols") or 0)
1159     except Exception:
1160         pass
1161     return 0, 0
1162
1163
1164 def _build_coco():
1165     labels_cfg, _ = load_labels_config()
1166     seen_labels: list[str] = [l["name"] for l in labels_cfg]
1167
1168     links: dict[tuple, dict] = {}
1169     with db_mod.get_conn() as c:
1170         for row in c.execute(
1171             "SELECT l.source, l.ref, l.confidence, l.confirmed_at, "
1172             "p.patient_id, p.first_name, p.last_name, p.sex, p.birth_date "

```

```

1173         "FROM dicom_patient_links l "
1174         "JOIN patients p ON p.patient_id = l.patient_id"
1175     ).fetchall():
1176         d = dict(row)
1177         src = d.pop("source"); ref = d.pop("ref")
1178         links[(src, ref)] = d
1179
1180     images = []
1181     annotations_out = []
1182     image_ix: dict[tuple, int] = {}
1183     label_ix: dict[str, int] = {name: i + 1 for i, name in enumerate(seen_labels)}
1184     next_image_id = 1
1185     next_ann_id = 1
1186
1187     for entry in annot_store.iter_all(ANNOT_DIR):
1188         source = entry.get("source")
1189         ref = entry.get("ref")
1190         if not source or not ref:
1191             continue
1192         rows, cols = _resolve_dims(source, ref, entry.get("rows"), entry.get("cols"))
1193         key = (source, ref)
1194         if key not in image_ix:
1195             image_ix[key] = next_image_id
1196             img_entry = {
1197                 "id": next_image_id,
1198                 "file_name": ref,
1199                 "source": source,
1200                 "width": cols or 0,
1201                 "height": rows or 0,
1202             }
1203             if key in links:
1204                 img_entry["patient"] = links[key]
1205             images.append(img_entry)
1206             next_image_id += 1
1207         img_id = image_ix[key]
1208         link_for_image = links.get(key)
1209         for a in entry.get("annotations", []):
1210             bbox = a.get("bbox") or [0, 0, 0, 0]
1211             if len(bbox) != 4:
1212                 continue
1213             labs = exporters.ann_labels(a) or ["other"]
1214             x_n, y_n, w_n, h_n = bbox
1215             if rows and cols:
1216                 x = x_n * cols
1217                 y = y_n * rows
1218                 w = w_n * cols
1219                 h = h_n * rows
1220             else:
1221                 x, y, w, h = x_n, y_n, w_n, h_n
1222
1223             ann_type = a.get("type") or "bbox"
1224             # Geometry/segmentation is computed once and shared across labels.
1225             seg_fields: dict = {}
1226             if ann_type == "polygon" and a.get("points"):
1227                 pts = a["points"]
1228                 if rows and cols:
1229                     flat_px: list[float] = []
1230                     for px, py in pts:
1231                         flat_px.append(round(px * cols, 2))
1232                         flat_px.append(round(py * rows, 2))
1233                     seg_fields["segmentation"] = [flat_px]
1234                     pts_px = [(px * cols, py * rows) for px, py in pts]
1235                     seg_fields["area"] = round(abs(_polygon_area(pts_px)), 2)
1236                 else:
1237                     flat_n: list[float] = []
1238                     for px, py in pts:

```

```

1239         flat_n.append(px)
1240         flat_n.append(py)
1241         seg_fields["segmentation_normalized"] = [flat_n]
1242         seg_fields["area"] = round(w * h, 2)
1243         seg_fields["points_count"] = len(pts)
1244     else:
1245         seg_fields["area"] = round(w * h, 2)
1246
1247     # Multi-label: emit one COCO annotation per label, sharing group_id.
1248     group_id = a.get("id") or f"g{next_ann_id}"
1249     for lab in labs:
1250         if lab not in label_ix:
1251             label_ix[lab] = len(label_ix) + 1
1252         ann_out = {
1253             "id": next_ann_id,
1254             "image_id": img_id,
1255             "category_id": label_ix[lab],
1256             "group_id": group_id,
1257             "label": lab,
1258             "labels": labs,
1259             "type": ann_type,
1260             "bbox": [round(x, 2), round(y, 2), round(w, 2), round(h, 2)],
1261             "bbox_normalized": [x_n, y_n, w_n, h_n],
1262             "iscrowd": 0,
1263             "bi_rads": a.get("bi_rads") or "",
1264             "note": a.get("note") or "",
1265             "frame": a.get("frame") or 0,
1266             "status": a.get("status") or "draft",
1267             "created_by": a.get("created_by") or "",
1268             "reviewed_by": a.get("reviewed_by") or "",
1269         }
1270         ann_out.update(seg_fields)
1271         if link_for_image and link_for_image.get("patient_id"):
1272             ann_out["patient_id"] = link_for_image["patient_id"]
1273         annotations_out.append(ann_out)
1274         next_ann_id += 1
1275
1276     categories = [{"id": v, "name": k} for k, v in sorted(label_ix.items(), key=lambda kv: kv[1])]
1277     return {
1278         "info": {
1279             "description": "MAMOGRAF DICOM Viewer annotations export",
1280             "date_created": datetime.now(timezone.utc).isoformat(),
1281             "version": "1",
1282         },
1283         "categories": categories,
1284         "images": images,
1285         "annotations": annotations_out,
1286     }
1287
1288
1289 @app.get("/api/worklist")
1290 def list_worklist(
1291     assigned_to: Optional[str] = None,
1292     status: Optional[str] = None,
1293     own: bool = False,
1294     limit: int = 500,
1295     user: dict = Depends(auth_mod.require_user),
1296 ):
1297     if own:
1298         assigned_to = user["sub"]
1299     where = []
1300     params: list = []
1301     if assigned_to:
1302         where.append("w.assigned_to = ?")
1303         params.append(assigned_to)
1304     if status:

```

```

1305         where.append("w.status = ?")
1306         params.append(status)
1307
1308     sql = (
1309         "SELECT w.*, p.first_name, p.last_name, "
1310         "(SELECT COUNT(*) FROM records r WHERE r.patient_id = w.patient_id) AS record_count "
1311         "FROM worklist w "
1312         "LEFT JOIN patients p ON p.patient_id = w.patient_id"
1313     )
1314     if where:
1315         sql += " WHERE " + " AND ".join(where)
1316     sql += " ORDER BY w.priority DESC, w.study_date DESC, w.id LIMIT ?"
1317     params.append(limit)
1318
1319     with db_mod.get_conn() as c:
1320         rows = [dict(r) for r in c.execute(sql, params).fetchall()]
1321     return {"items": rows, "count": len(rows)}
1322
1323
1324 class WorklistUpdateBody(BaseModel):
1325     assigned_to: Optional[str] = None
1326     priority: Optional[str] = None
1327     status: Optional[str] = None
1328     notes: Optional[str] = None
1329
1330
1331 @app.patch("/api/worklist/{wl_id}")
1332 def update_worklist(
1333     wl_id: int,
1334     body: WorklistUpdateBody,
1335     user: dict = Depends(auth_mod.require_user),
1336 ):
1337     role = user.get("role")
1338     fields = []
1339     params: list = []
1340     if body.assigned_to is not None:
1341         if role not in ("admin", "reviewer"):
1342             raise HTTPException(403, "only admin/reviewer can assign")
1343         fields.append("assigned_to = ?")
1344         params.append(body.assigned_to or None)
1345     if body.priority is not None:
1346         if body.priority not in ("low", "normal", "high", "urgent"):
1347             raise HTTPException(400, "invalid priority")
1348         fields.append("priority = ?")
1349         params.append(body.priority)
1350     if body.status is not None:
1351         if body.status not in ("pending", "in_progress", "done", "skipped"):
1352             raise HTTPException(400, "invalid status")
1353         fields.append("status = ?")
1354         params.append(body.status)
1355     if body.notes is not None:
1356         fields.append("notes = ?")
1357         params.append(body.notes)
1358     if not fields:
1359         raise HTTPException(400, "nothing to update")
1360     params.append(wl_id)
1361     with db_mod.get_conn() as c:
1362         cur = c.execute(f"UPDATE worklist SET {' '.join(fields)} WHERE id = ?", params)
1363         c.commit()
1364         if cur.rowcount == 0:
1365             raise HTTPException(404, "not found")
1366         row = c.execute("SELECT * FROM worklist WHERE id = ?", (wl_id,)).fetchone()
1367
1368     if body.assigned_to and role in ("admin", "reviewer"):
1369         _notify(
1370             [body.assigned_to],

```

```

1371         kind="assigned",
1372         title=f"Yangi worklist topshirig'i: {row['patient_name'] or row['patient_id']}",
1373         body=f"Sana: {row['study_date']} · Modality: {row['modality']}",
1374         actor=user.get("sub"),
1375     )
1376     return {"ok": True, "item": dict(row)}
1377
1378
1379 @app.get("/api/pacs/servers")
1380 def pacs_list_servers(_user: dict = Depends(auth_mod.require_user)):
1381     with db_mod.get_conn() as c:
1382         rows = [dict(r) for r in c.execute("SELECT * FROM pacs_servers ORDER BY name").fetchall()]
1383     return {"servers": rows}
1384
1385
1386 class PacsServerBody(BaseModel):
1387     name: str
1388     host: str
1389     port: int
1390     aet: str
1391     calling_aet: Optional[str] = "MAMOGRAF_SCU"
1392     notes: Optional[str] = None
1393
1394
1395 @app.post("/api/pacs/servers")
1396 def pacs_add_server(
1397     body: PacsServerBody,
1398     user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
1399 ):
1400     if not body.name.strip() or not body.host.strip() or not body.aet.strip():
1401         raise HTTPException(400, "name, host, aet required")
1402     if not (1 <= body.port <= 65535):
1403         raise HTTPException(400, "invalid port")
1404     now = datetime.now(timezone.utc).isoformat()
1405     with db_mod.get_conn() as c:
1406         try:
1407             c.execute(
1408                 "INSERT INTO pacs_servers (name, host, port, aet, calling_aet, notes, added_by,
1409 added_at) "
1410                 "VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
1411                 (body.name.strip(), body.host.strip(), body.port,
1412                 body.aet.strip(), (body.calling_aet or "MAMOGRAF_SCU").strip(),
1413                 body.notes, user.get("sub"), now),
1414             )
1415             c.commit()
1416             new_id = c.execute("SELECT last_insert_rowid()").fetchone()[0]
1417             row = c.execute("SELECT * FROM pacs_servers WHERE id=?", (new_id,)).fetchone()
1418         except Exception as e:
1419             raise HTTPException(409, f"insert failed: {e}")
1420     return {"ok": True, "server": dict(row)}
1421
1422
1423 @app.delete("/api/pacs/servers/{srv_id}")
1424 def pacs_delete_server(srv_id: int, _admin: dict = Depends(auth_mod.require_role("admin"))):
1425     with db_mod.get_conn() as c:
1426         cur = c.execute("DELETE FROM pacs_servers WHERE id=?", (srv_id,))
1427         c.commit()
1428         if cur.rowcount == 0:
1429             raise HTTPException(404, "not found")
1430     return {"ok": True}
1431
1432
1433 def _get_pacs_server(srv_id: int) -> dict:
1434     with db_mod.get_conn() as c:
1435         row = c.execute("SELECT * FROM pacs_servers WHERE id=?", (srv_id,)).fetchone()
1436         if not row:

```

```

1436         raise HTTPException(404, "server not found")
1437     return dict(row)
1438
1439
1440 @app.post("/api/pacs/servers/{srv_id}/echo")
1441 def pacs_server_echo(
1442     srv_id: int,
1443     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
1444 ):
1445     s = _get_pacs_server(srv_id)
1446     try:
1447         return pacs_mod.echo(s["host"], s["port"], s["aet"], s["calling_aet"])
1448     except RuntimeError as e:
1449         raise HTTPException(503, str(e))
1450     except Exception as e:
1451         raise HTTPException(502, f"echo failed: {e}")
1452
1453
1454 class PacsQueryBody(BaseModel):
1455     patient_id: Optional[str] = ""
1456     patient_name: Optional[str] = ""
1457     modality: Optional[str] = ""
1458     study_date: Optional[str] = ""
1459
1460
1461 @app.post("/api/pacs/servers/{srv_id}/query")
1462 def pacs_server_query(
1463     srv_id: int,
1464     body: PacsQueryBody,
1465     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
1466 ):
1467     s = _get_pacs_server(srv_id)
1468     try:
1469         results = pacs_mod.query(
1470             s["host"], s["port"], s["aet"],
1471             patient_id=body.patient_id or "",
1472             patient_name=body.patient_name or "",
1473             modality=body.modality or "",
1474             study_date=body.study_date or "",
1475             calling_aet=s["calling_aet"],
1476         )
1477     except RuntimeError as e:
1478         raise HTTPException(503, str(e))
1479     except Exception as e:
1480         raise HTTPException(502, f"query failed: {e}")
1481     return {"results": results, "count": len(results)}
1482
1483
1484 class PacsToWorklistBody(BaseModel):
1485     patient_id: str
1486     patient_name: Optional[str] = None
1487     sex: Optional[str] = None
1488     dob: Optional[str] = None
1489     study_date: Optional[str] = None
1490     modality: Optional[str] = None
1491
1492
1493 class PacsStoreBody(BaseModel):
1494     items: list[dict]
1495
1496
1497 @app.post("/api/pacs/servers/{srv_id}/store")
1498 def pacs_server_store(
1499     srv_id: int,
1500     body: PacsStoreBody,
1501     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),

```



```

1502 ):
1503     s = _get_pacs_server(srv_id)
1504     paths: list[Path] = []
1505     for it in body.items:
1506         src = it.get("source")
1507         ref = it.get("ref")
1508         if src not in ("upload", "local") or not ref:
1509             continue
1510         try:
1511             paths.append(_resolve_dicom_path(src, ref))
1512         except HTTPException:
1513             continue
1514     if not paths:
1515         raise HTTPException(400, "no valid items to send")
1516     try:
1517         result = pacs_mod.store(s["host"], s["port"], s["aet"], paths,
1518                                calling_aet=s["calling_aet"])
1519     except RuntimeError as e:
1520         raise HTTPException(503, str(e))
1521     except Exception as e:
1522         raise HTTPException(502, f"store failed: {e}")
1523     return {"ok": True, **result, "total": len(paths)}
1524
1525
1526 class PacsFetchBody(BaseModel):
1527     study_uid: str
1528     scp_port: Optional[int] = None
1529
1530
1531 _PACS_FETCH_DIR = UPLOAD_DIR
1532
1533
1534 @app.post("/api/pacs/servers/{srv_id}/fetch")
1535 def pacs_server_fetch(
1536     srv_id: int,
1537     body: PacsFetchBody,
1538     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
1539 ):
1540     s = _get_pacs_server(srv_id)
1541     if not body.study_uid.strip():
1542         raise HTTPException(400, "study_uid kerak")
1543     scp_port = body.scp_port or int(os.environ.get("PACS_SCP_PORT", "11112"))
1544     dest = _PACS_FETCH_DIR / "_pacs_inbox"
1545     dest.mkdir(parents=True, exist_ok=True)
1546     try:
1547         result = pacs_mod.fetch(
1548             s["host"], s["port"], s["aet"],
1549             study_uid=body.study_uid.strip(),
1550             dest=dest,
1551             calling_aet=s["calling_aet"],
1552             scp_port=scp_port,
1553         )
1554     except RuntimeError as e:
1555         raise HTTPException(503, str(e))
1556     except Exception as e:
1557         raise HTTPException(502, f"fetch failed: {e}")
1558
1559     moved: list[dict] = []
1560     for fname in result.get("received", []):
1561         src_p = dest / fname
1562         if not src_p.exists():
1563             continue
1564         new_id = uuid.uuid4().hex
1565         out_path = UPLOAD_DIR / f"{new_id}.dcm"
1566         try:
1567             src_p.rename(out_path)

```

```

1568         except OSError:
1569             try:
1570                 out_path.write_bytes(src_p.read_bytes())
1571                 src_p.unlink(missing_ok=True)
1572             except Exception:
1573                 continue
1574         try:
1575             ds = pydicom.dcmread(out_path, stop_before_pixels=True, force=True)
1576             moved.append({
1577                 "id": new_id,
1578                 "original": fname,
1579                 "patient": str(getattr(ds, "PatientName", "")),
1580                 "patient_id": str(getattr(ds, "PatientID", "")),
1581             })
1582         except Exception:
1583             moved.append({"id": new_id, "original": fname})
1584
1585     return {"ok": True, "received": len(moved), "uploads": moved}
1586
1587
1588 @app.post("/api/pacs/to-worklist")
1589 def pacs_to_worklist(
1590     body: PacsToWorklistBody,
1591     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
1592 ):
1593     now = datetime.now(timezone.utc).isoformat()
1594     with db_mod.get_conn() as c:
1595         c.execute(
1596             "INSERT OR IGNORE INTO worklist "
1597             "(patient_id, patient_name, sex, dob, study_date, modality, "
1598             "priority, status, imported_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",
1599             (body.patient_id, body.patient_name, body.sex, body.dob,
1600              body.study_date, body.modality, "normal", "pending", now),
1601         )
1602         c.commit()
1603     return {"ok": True}
1604
1605
1606 @app.delete("/api/worklist/{wl_id}")
1607 def delete_worklist(wl_id: int, _admin: dict = Depends(auth_mod.require_role("admin"))):
1608     with db_mod.get_conn() as c:
1609         cur = c.execute("DELETE FROM worklist WHERE id = ?", (wl_id,))
1610         c.commit()
1611         if cur.rowcount == 0:
1612             raise HTTPException(404, "not found")
1613     return {"ok": True}
1614
1615
1616 def _resolve_dicom_path(source: str, ref: str) -> Path:
1617     if source == "upload":
1618         p = UPLOAD_DIR / f"{ref}.dcm"
1619         if not p.exists():
1620             raise HTTPException(404, "upload not found")
1621         return p
1622     if source == "local":
1623         return _resolve_local(ref)
1624     raise HTTPException(400, "invalid source")
1625
1626
1627 @app.get("/api/deidentify/detect")
1628 def deid_detect(source: str, ref: str, _user: dict = Depends(auth_mod.require_user)):
1629     _validate_source(source)
1630     _validate_ref(source, ref)
1631     path = _resolve_dicom_path(source, ref)
1632     try:
1633         return deid.detect_phi(path)

```

```

1634     except Exception as e:
1635         raise HTTPException(500, f"detect failed: {e}")
1636
1637
1638 @app.get("/api/export/dicom-seg")
1639 def export_dicom_seg(
1640     source: str,
1641     ref: str,
1642     _user: dict = Depends(auth_mod.require_user),
1643 ):
1644     _validate_source(source)
1645     _validate_ref(source, ref)
1646     src_path = _resolve_dicom_path(source, ref)
1647     data = annot_store.load(ANNOT_DIR, source, ref)
1648     annotations = data.get("annotations", []) or []
1649     if not annotations:
1650         raise HTTPException(404, "no annotations to export")
1651     rows = data.get("rows")
1652     cols = data.get("cols")
1653     try:
1654         body = dseg.annotations_to_seg(src_path, annotations, rows=rows, cols=cols)
1655     except RuntimeError as e:
1656         raise HTTPException(503, str(e))
1657     except Exception as e:
1658         raise HTTPException(500, f"SEG build failed: {e}")
1659     base = Path(ref).stem or "annotations"
1660     fname = f"{base}_seg.dcm"
1661     return StreamingResponse(
1662         io.BytesIO(body),
1663         media_type="application/dicom",
1664         headers={"Content-Disposition": f'attachment; filename="{fname}"'},
1665     )
1666
1667
1668 @app.get("/api/export/dicom-sr")
1669 def export_dicom_sr(
1670     source: str,
1671     ref: str,
1672     user: dict = Depends(auth_mod.require_user),
1673 ):
1674     _validate_source(source)
1675     _validate_ref(source, ref)
1676     src_path = _resolve_dicom_path(source, ref)
1677     data = annot_store.load(ANNOT_DIR, source, ref)
1678     annotations = data.get("annotations", []) or []
1679     if not annotations:
1680         raise HTTPException(404, "no annotations to export")
1681     rows = data.get("rows")
1682     cols = data.get("cols")
1683     try:
1684         body = dsr.annotations_to_sr(
1685             src_path, annotations,
1686             rows=rows, cols=cols,
1687             author=user.get("sub"),
1688         )
1689     except Exception as e:
1690         raise HTTPException(500, f"SR build failed: {e}")
1691     base = Path(ref).stem or "annotations"
1692     fname = f"{base}_sr.dcm"
1693     return StreamingResponse(
1694         io.BytesIO(body),
1695         media_type="application/dicom",
1696         headers={"Content-Disposition": f'attachment; filename="{fname}"'},
1697     )
1698
1699

```

```

1700 @app.get("/api/deidentify/download")
1701 def deid_download(source: str, ref: str, _user: dict = Depends(auth_mod.require_user)):
1702     _validate_source(source)
1703     _validate_ref(source, ref)
1704     path = _resolve_dicom_path(source, ref)
1705     try:
1706         body = deid.anonymize_to_bytes(path)
1707     except Exception as e:
1708         raise HTTPException(500, f"anonymize failed: {e}")
1709     base = Path(ref).stem or "anonymous"
1710     fname = f"{base}_anon.dcm"
1711     return StreamingResponse(
1712         io.BytesIO(body),
1713         media_type="application/dicom",
1714         headers={"Content-Disposition": f'attachment; filename="{fname}"'},
1715     )
1716
1717
1718 @app.get("/api/export/mask")
1719 def export_mask(
1720     source: str,
1721     ref: str,
1722     format: str = "png",
1723     _user: dict = Depends(auth_mod.require_user),
1724 ):
1725     """Segmentation mask for one image, built from its polygons + bboxes.
1726
1727     format=png → label-indexed PNG (0 = background, 1..N = label classes)
1728     format=nifti → label-indexed .nii.gz (same indexing)
1729
1730     The class→label legend is returned in the ``X-Mask-Legend`` header (JSON).
1731     """
1732     fmt = format.lower()
1733     if fmt not in ("png", "nifti"):
1734         raise HTTPException(400, "format must be 'png' or 'nifti'")
1735     _validate_source(source)
1736     _validate_ref(source, ref)
1737     data = annot_store.load(ANNOT_DIR, source, ref)
1738     annotations = data.get("annotations", []) or []
1739     if not annotations:
1740         raise HTTPException(404, "no annotations to export")
1741     rows, cols = _resolve_dims(source, ref, data.get("rows"), data.get("cols"))
1742     if not rows or not cols:
1743         raise HTTPException(422, "image dimensions unknown; cannot rasterize mask")
1744
1745     label_names = [l["name"] for l in load_labels_config()[0]]
1746     label_index = {name: i + 1 for i, name in enumerate(label_names)}
1747     # Include any ad-hoc labels not in config so nothing is silently dropped.
1748     for a in annotations:
1749         for lab in exporters.ann_labels(a):
1750             if lab not in label_index:
1751                 label_index[lab] = len(label_index) + 1
1752
1753     try:
1754         mask = exporters.build_label_mask(annotations, rows, cols, label_index)
1755     except Exception as e:
1756         raise HTTPException(500, f"mask build failed: {e}")
1757
1758     base = Path(ref).stem or "annotations"
1759     legend = json.dumps(exporters.mask_legend(label_index), ensure_ascii=False)
1760     if fmt == "png":
1761         body = exporters.mask_to_png_bytes(mask)
1762         media, suffix = "image/png", "mask.png"
1763     else:
1764         body = exporters.mask_to_nifti_bytes(mask)
1765         media, suffix = "application/gzip", "mask.nii.gz"

```

```

1766
1767     return StreamingResponse(
1768         io.BytesIO(body),
1769         media_type=media,
1770         headers={
1771             "Content-Disposition": f'attachment; filename="{base}_{suffix}"',
1772             "X-Mask-Legend": legend,
1773         },
1774     )
1775
1776
1777 def _radiomics_for(source: str, ref: str, bins: int, only_id: Optional[str] = None):
1778     """Compute radiomics features for one or all annotations on an image.
1779
1780     Returns ``(image_meta, [ {annotation_id, label, labels, frame, features}, ... ])``.
1781     """
1782     path = _resolve_dicom_path(source, ref)
1783     data = annot_store.load(ANNOT_DIR, source, ref)
1784     annotations = data.get("annotations", []) or []
1785     if only_id:
1786         annotations = [a for a in annotations if a.get("id") == only_id]
1787     if not annotations:
1788         raise HTTPException(404, "no matching annotations")
1789     rows, cols = _resolve_dims(source, ref, data.get("rows"), data.get("cols"))
1790     if not rows or not cols:
1791         raise HTTPException(422, "image dimensions unknown")
1792
1793     # Load each needed frame once (radiomics is per-frame).
1794     frame_cache: dict[int, tuple] = {}
1795     out: list[dict] = []
1796     for a in annotations:
1797         frame = int(a.get("frame") or 0)
1798         if frame not in frame_cache:
1799             try:
1800                 frame_cache[frame] = load_frame_array(path, frame=frame)
1801             except Exception as e:
1802                 raise HTTPException(500, f"pixel load failed: {e}")
1803         image, spacing = frame_cache[frame]
1804         fr_rows, fr_cols = image.shape[:2]
1805         mask = radiomics_mod.roi_mask_from_annotation(a, fr_rows, fr_cols)
1806         if not mask.any():
1807             continue
1808         try:
1809             feats = radiomics_mod.extract(image, mask, spacing=spacing, bins=bins)
1810         except Exception as e:
1811             raise HTTPException(500, f"radiomics failed for {a.get('id')}: {e}")
1812         out.append({
1813             "annotation_id": a.get("id"),
1814             "label": a.get("label"),
1815             "labels": exporters.ann_labels(a),
1816             "bi_rads": a.get("bi_rads") or "",
1817             "frame": frame,
1818             "type": a.get("type") or "bbox",
1819             "spacing_mm": list(spacing) if spacing else None,
1820             "features": feats,
1821         })
1822     if not out:
1823         raise HTTPException(422, "ROI(s) produced no usable mask")
1824     return {"source": source, "ref": ref, "rows": rows, "cols": cols, "bins": bins}, out
1825
1826
1827 @app.get("/api/radiomics")
1828 def get_radiomics(
1829     source: str,
1830     ref: str,
1831     annotation_id: Optional[str] = None,

```

```

1832     bins: int = 32,
1833     _user: dict = Depends(auth_mod.require_user),
1834 ):
1835     """Radiomics features for one annotation (annotation_id) or all on the image."""
1836     _validate_source(source)
1837     _validate_ref(source, ref)
1838     bins = max(8, min(128, int(bins)))
1839     meta, items = _radiomics_for(source, ref, bins, only_id=annotation_id)
1840     return {**meta, "results": items}
1841
1842
1843 @app.get("/api/radiomics/csv")
1844 def get_radiomics_csv(
1845     source: str,
1846     ref: str,
1847     bins: int = 32,
1848     _user: dict = Depends(auth_mod.require_user),
1849 ):
1850     """All ROIs on an image as a CSV – one row per annotation, one column per feature."""
1851     _validate_source(source)
1852     _validate_ref(source, ref)
1853     bins = max(8, min(128, int(bins)))
1854     _meta, items = _radiomics_for(source, ref, bins)
1855
1856     # Stable, flattened column order: family.feature
1857     cols_keys: list[str] = []
1858     for fam, feats in items[0]["features"].items():
1859         for fname in feats:
1860             cols_keys.append(f"{fam}.{fname}")
1861     header = ["annotation_id", "label", "labels", "bi_rads", "frame", "type"] + cols_keys
1862
1863     buf = io.StringIO()
1864     writer = csv.writer(buf)
1865     writer.writerow(header)
1866     for it in items:
1867         flat = {f"{fam}.{fn}": v for fam, feats in it["features"].items() for fn, v in feats.items()}
1868         row = [
1869             it["annotation_id"], it["label"] or "", ";".join(it["labels"]),
1870             it["bi_rads"], it["frame"], it["type"],
1871             ] + [flat.get(k, "") for k in cols_keys]
1872         writer.writerow(row)
1873
1874     body = (" " + buf.getvalue()).encode("utf-8")
1875     base = Path(ref).stem or "annotations"
1876     stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
1877     return StreamingResponse(
1878         io.BytesIO(body),
1879         media_type="text/csv; charset=utf-8",
1880         headers={"Content-Disposition": f'attachment; filename="{base}_radiomics_{stamp}.csv"'},
1881     )
1882
1883
1884 @app.get("/api/stats/overview")
1885 def stats_overview(_admin: dict = Depends(auth_mod.require_role("admin", "reviewer"))):
1886     cached = _cache_get("stats")
1887     if cached is not None:
1888         return cached
1889     out: dict = {}
1890     with db_mod.get_conn() as c:
1891         out["users_total"] = c.execute("SELECT COUNT(*) FROM users").fetchone()[0]
1892         out["users_active"] = c.execute(
1893             "SELECT COUNT(*) FROM users WHERE is_active = 1"
1894         ).fetchone()[0]
1895         out["patients_total"] = c.execute("SELECT COUNT(*) FROM patients").fetchone()[0]
1896         out["records_total"] = c.execute("SELECT COUNT(*) FROM records").fetchone()[0]
1897         out["dicom_links_total"] = c.execute(

```

```

1898         "SELECT COUNT(*) FROM dicom_patient_links"
1899     ).fetchone()[0]
1900     out["history_events"] = c.execute(
1901         "SELECT COUNT(*) FROM annotation_history"
1902     ).fetchone()[0]
1903     out["worklist_total"] = c.execute(
1904         "SELECT COUNT(*) FROM worklist"
1905     ).fetchone()[0]
1906
1907     out["worklist_by_status"] = [
1908         dict(r) for r in c.execute(
1909             "SELECT status, COUNT(*) AS n FROM worklist GROUP BY status"
1910         ).fetchall()
1911     ]
1912
1913     out["history_by_action"] = [
1914         dict(r) for r in c.execute(
1915             "SELECT action, COUNT(*) AS n FROM annotation_history GROUP BY action ORDER BY n DESC"
1916         ).fetchall()
1917     ]
1918
1919     out["history_per_user"] = [
1920         dict(r) for r in c.execute(
1921             "SELECT username, COUNT(*) AS n FROM annotation_history "
1922             "WHERE username IS NOT NULL GROUP BY username ORDER BY n DESC LIMIT 20"
1923         ).fetchall()
1924     ]
1925
1926     out["history_last_30_days"] = [
1927         dict(r) for r in c.execute(
1928             "SELECT substr(ts, 1, 10) AS day, COUNT(*) AS n "
1929             "FROM annotation_history "
1930             "WHERE ts >= date('now', '-30 days') "
1931             "GROUP BY day ORDER BY day"
1932         ).fetchall()
1933     ]
1934
1935     annotation_counts: dict[str, int] = {}
1936     status_counts: dict[str, int] = {}
1937     label_counts: dict[str, int] = {}
1938     ai_origin = 0
1939     total_anns = 0
1940     review_durations: list[float] = []
1941
1942     for entry in annot_store.iter_all(ANNOT_DIR):
1943         for a in entry.get("annotations", []):
1944             total_anns += 1
1945             cb = a.get("created_by") or "?"
1946             annotation_counts[cb] = annotation_counts.get(cb, 0) + 1
1947             st = a.get("status", "draft")
1948             status_counts[st] = status_counts.get(st, 0) + 1
1949             lbl = a.get("label") or "-"
1950             label_counts[lbl] = label_counts.get(lbl, 0) + 1
1951             if a.get("ai_source"):
1952                 ai_origin += 1
1953             if a.get("created_at") and a.get("reviewed_at"):
1954                 try:
1955                     t1 = datetime.fromisoformat(a["created_at"].replace("Z", "+00:00"))
1956                     t2 = datetime.fromisoformat(a["reviewed_at"].replace("Z", "+00:00"))
1957                     review_durations.append((t2 - t1).total_seconds())
1958                 except Exception:
1959                     pass
1960
1961     out["annotations_total"] = total_anns
1962     out["annotations_by_creator"] = sorted(
1963         [{"user": k, "n": v} for k, v in annotation_counts.items()],

```

```

1964         key=lambda x: -x["n"],
1965     )[:20]
1966     out["annotations_by_status"] = [
1967         {"status": k, "n": v} for k, v in sorted(status_counts.items())
1968     ]
1969     out["annotations_by_label"] = sorted(
1970         [{"label": k, "n": v} for k, v in label_counts.items()],
1971         key=lambda x: -x["n"],
1972     )[:20]
1973     out["ai_originated_annotations"] = ai_origin
1974     if review_durations:
1975         review_durations.sort()
1976         out["review_seconds"] = {
1977             "count": len(review_durations),
1978             "median": round(review_durations[len(review_durations) // 2], 1),
1979             "p90": round(review_durations[int(len(review_durations) * 0.9)], 1),
1980             "mean": round(sum(review_durations) / len(review_durations), 1),
1981         }
1982     else:
1983         out["review_seconds"] = None
1984
1985     _cache_set("stats", out)
1986     return out
1987
1988
1989 @app.get("/api/annotations/history.csv")
1990 def export_history_csv(
1991     source: Optional[str] = None,
1992     ref: Optional[str] = None,
1993     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
1994 ):
1995     sql = "SELECT * FROM annotation_history WHERE 1=1"
1996     params: list = []
1997     if source:
1998         sql += " AND source = ?"
1999         params.append(source)
2000     if ref:
2001         sql += " AND ref = ?"
2002         params.append(ref)
2003     sql += " ORDER BY ts DESC, id DESC"
2004
2005     buf = io.StringIO()
2006     writer = csv.writer(buf)
2007     writer.writerow([
2008         "ts", "username", "action", "source", "ref", "annotation_id",
2009         "prev_status", "new_status", "prev_label", "new_label",
2010         "prev_bi_rads", "new_bi_rads", "note",
2011     ])
2012
2013     def _val(d, key):
2014         if isinstance(d, dict):
2015             v = d.get(key)
2016             return v if v is not None else ""
2017         return ""
2018
2019     with db_mod.get_conn() as c:
2020         for row in c.execute(sql, params).fetchall():
2021             d = dict(row)
2022             try:
2023                 prev = json.loads(d["prev_snapshot"]) if d.get("prev_snapshot") else {}
2024             except Exception:
2025                 prev = {}
2026             try:
2027                 new = json.loads(d["new_snapshot"]) if d.get("new_snapshot") else {}
2028             except Exception:
2029                 new = {}

```



```

2030         writer.writerow([
2031             d.get("ts") or "",
2032             d.get("username") or "",
2033             d.get("action") or "",
2034             d.get("source") or "",
2035             d.get("ref") or "",
2036             d.get("annotation_id") or "",
2037             _val(prev, "status"),
2038             _val(new, "status"),
2039             _val(prev, "label"),
2040             _val(new, "label"),
2041             _val(prev, "bi_rads"),
2042             _val(new, "bi_rads"),
2043             _val(new, "note") or _val(new, "review_note"),
2044         ])
2045
2046     body = (" " + buf.getvalue()).encode("utf-8")
2047     stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
2048     fname = f"audit_log_{stamp}.csv"
2049     return StreamingResponse(
2050         io.BytesIO(body),
2051         media_type="text/csv; charset=utf-8",
2052         headers={"Content-Disposition": f'attachment; filename="{fname}"'},
2053     )
2054
2055
2056 def _export_images() -> list[dict]:
2057     """Normalized per-image view used by YOLO/VOC/CSV/mask exporters."""
2058     out: list[dict] = []
2059     for entry in annot_store.iter_all(ANNOT_DIR):
2060         source = entry.get("source")
2061         ref = entry.get("ref")
2062         if not source or not ref:
2063             continue
2064         rows, cols = _resolve_dims(source, ref, entry.get("rows"), entry.get("cols"))
2065         out.append({
2066             "source": source,
2067             "ref": ref,
2068             "rows": rows,
2069             "cols": cols,
2070             "annotations": entry.get("annotations", []) or [],
2071         })
2072     return out
2073
2074
2075 _EXPORT_FORMATS = {
2076     "coco": ("application/json", "json"),
2077     "yolo": ("application/zip", "zip"),
2078     "voc": ("application/zip", "zip"),
2079     "csv": ("text/csv; charset=utf-8", "csv"),
2080 }
2081
2082
2083 @app.get("/api/export")
2084 def export(format: str = "coco"):
2085     fmt = format.lower()
2086     if fmt not in _EXPORT_FORMATS:
2087         raise HTTPException(400, f"unsupported format; use one of: {' ', ' '.join(_EXPORT_FORMATS)}")
2088     stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
2089     media, ext = _EXPORT_FORMATS[fmt]
2090
2091     if fmt == "coco":
2092         body = json.dumps(_build_coco(), ensure_ascii=False, indent=2).encode("utf-8")
2093         fname = f"annotations_coco_{stamp}.json"
2094     else:
2095         images = _export_images()

```

```

2096     label_names = [l["name"] for l in load_labels_config()[0]]
2097     if fmt == "yolo":
2098         body = exporters.build_yolo_zip(images, label_names)
2099         fname = f"annotations_yolo_{stamp}.zip"
2100     elif fmt == "voc":
2101         body = exporters.build_voc_zip(images)
2102         fname = f"annotations_voc_{stamp}.zip"
2103     else: # csv
2104         body = exporters.build_csv(images)
2105         fname = f"annotations_{stamp}.csv"
2106
2107     return StreamingResponse(
2108         io.BytesIO(body),
2109         media_type=media,
2110         headers={"Content-Disposition": f'attachment; filename="{fname}"'},
2111     )
2112
2113
2114 @app.get("/api/db/stats")
2115 def db_stats():
2116     with db_mod.get_conn() as c:
2117         patients = c.execute("SELECT COUNT(*) FROM patients").fetchone()[0]
2118         records = c.execute("SELECT COUNT(*) FROM records").fetchone()[0]
2119         date_min = c.execute("SELECT MIN(service_date) FROM records").fetchone()[0]
2120         date_max = c.execute("SELECT MAX(service_date) FROM records").fetchone()[0]
2121         by_code = c.execute(
2122             "SELECT exam_code, COUNT(*) AS n FROM records "
2123             "WHERE exam_code IS NOT NULL GROUP BY exam_code ORDER BY n DESC"
2124         ).fetchall()
2125     return {
2126         "patients": patients,
2127         "records": records,
2128         "service_date_min": date_min,
2129         "service_date_max": date_max,
2130         "by_exam_code": [dict(r) for r in by_code],
2131     }
2132
2133
2134 @app.get("/api/db/search")
2135 def db_search(q: str = "", limit: int = 50):
2136     q = (q or "").strip()
2137     if not q:
2138         return {"results": []}
2139     pat = f"%{q}%"
2140     with db_mod.get_conn() as c:
2141         rows = c.execute(
2142             "SELECT p.patient_id, p.first_name, p.last_name, p.sex, p.birth_date, "
2143             "      (SELECT COUNT(*) FROM records r WHERE r.patient_id=p.patient_id) AS record_count, "
2144             "      (SELECT MAX(service_date) FROM records r WHERE r.patient_id=p.patient_id) AS "
2145             "last_visit "
2146             "FROM patients p "
2147             "WHERE p.patient_id LIKE ? OR p.first_name LIKE ? OR p.last_name LIKE ? "
2148             "ORDER BY p.last_name, p.first_name LIMIT ?",
2149             (pat, pat, pat, limit),
2150         ).fetchall()
2151     return {"results": [dict(r) for r in rows]}
2152
2153
2154 @app.get("/api/db/patient/{patient_id}")
2155 def db_patient(patient_id: str):
2156     with db_mod.get_conn() as c:
2157         p = c.execute(
2158             "SELECT * FROM patients WHERE patient_id = ?", (patient_id,)
2159         ).fetchone()
2160     if not p:

```

```

2160         raise HTTPException(404, "patient not found")
2161     recs = c.execute(
2162         "SELECT * FROM records WHERE patient_id = ? "
2163         "ORDER BY service_date DESC, source_row DESC",
2164         (patient_id,),
2165     ).fetchall()
2166     return {"patient": dict(p), "records": [dict(r) for r in recs]}
2167
2168
2169 @app.get("/api/templates")
2170 def list_templates(user: dict = Depends(auth_mod.require_user)):
2171     with db_mod.get_conn() as c:
2172         rows = c.execute(
2173             "SELECT * FROM annotation_templates "
2174             "WHERE is_shared = 1 OR created_by = ? "
2175             "ORDER BY created_at DESC",
2176             (user["sub"],),
2177         ).fetchall()
2178     out = []
2179     for r in rows:
2180         d = dict(r)
2181         try:
2182             payload = json.loads(d.get("payload") or "[]")
2183         except Exception:
2184             payload = []
2185         d["annotation_count"] = len(payload)
2186         d.pop("payload", None)
2187         out.append(d)
2188     return {"templates": out, "count": len(out)}
2189
2190
2191 @app.get("/api/templates/{tpl_id}")
2192 def get_template(tpl_id: int, user: dict = Depends(auth_mod.require_user)):
2193     with db_mod.get_conn() as c:
2194         row = c.execute(
2195             "SELECT * FROM annotation_templates WHERE id = ?", (tpl_id,)
2196         ).fetchone()
2197         if not row:
2198             raise HTTPException(404, "template not found")
2199     d = dict(row)
2200     if not d.get("is_shared") and d.get("created_by") != user["sub"]:
2201         raise HTTPException(403, "private template")
2202     try:
2203         d["payload"] = json.loads(d.get("payload") or "[]")
2204     except Exception:
2205         d["payload"] = []
2206     return d
2207
2208
2209 class TemplateBody(BaseModel):
2210     name: str
2211     description: Optional[str] = None
2212     payload: list[dict]
2213     is_shared: bool = True
2214
2215
2216 @app.post("/api/templates")
2217 def create_template(
2218     body: TemplateBody,
2219     user: dict = Depends(auth_mod.require_user),
2220 ):
2221     name = (body.name or "").strip()
2222     if not name:
2223         raise HTTPException(400, "name required")
2224     if not body.payload:
2225         raise HTTPException(400, "payload empty")

```

```

2226
2227     cleaned = []
2228     for a in body.payload:
2229         c = dict(a)
2230         for k in ("id", "created_by", "created_at", "updated_by", "updated_at",
2231                 "reviewed_by", "reviewed_at", "review_note", "status"):
2232             c.pop(k, None)
2233         cleaned.append(c)
2234
2235     now = datetime.now(timezone.utc).isoformat()
2236     with db_mod.get_conn() as c:
2237         try:
2238             c.execute(
2239                 "INSERT INTO annotation_templates "
2240                 "(name, description, payload, created_by, created_at, is_shared) "
2241                 "VALUES (?, ?, ?, ?, ?, ?)",
2242                 (name, body.description, json.dumps(cleaned, ensure_ascii=False),
2243                 user["sub"], now, 1 if body.is_shared else 0),
2244             )
2245             c.commit()
2246             new_id = c.execute("SELECT last_insert_rowid()").fetchone()[0]
2247         except Exception as e:
2248             raise HTTPException(409, f"insert failed: {e}")
2249     return {"ok": True, "id": new_id, "name": name, "annotation_count": len(cleaned)}
2250
2251
2252 @app.delete("/api/templates/{tpl_id}")
2253 def delete_template(
2254     tpl_id: int,
2255     user: dict = Depends(auth_mod.require_user),
2256 ):
2257     with db_mod.get_conn() as c:
2258         row = c.execute(
2259             "SELECT created_by FROM annotation_templates WHERE id = ?", (tpl_id,)
2260         ).fetchone()
2261         if not row:
2262             raise HTTPException(404, "template not found")
2263         is_admin = user.get("role") == "admin"
2264         if not is_admin and row[0] != user["sub"]:
2265             raise HTTPException(403, "can only delete own templates")
2266         c.execute("DELETE FROM annotation_templates WHERE id = ?", (tpl_id,))
2267         c.commit()
2268     return {"ok": True}
2269
2270
2271 @app.get("/api/annotations/overview")
2272 def annotations_overview(
2273     status: Optional[str] = None,
2274     created_by: Optional[str] = None,
2275     own: bool = False,
2276     limit: int = 200,
2277     user: dict = Depends(auth_mod.require_user),
2278 ):
2279     if own and not created_by:
2280         created_by = user["sub"]
2281
2282     links: dict[tuple, dict] = {}
2283     with db_mod.get_conn() as c:
2284         for r in c.execute(
2285             "SELECT l.source, l.ref, p.patient_id, p.first_name, p.last_name, "
2286             "p.sex, p.birth_date "
2287             "FROM dicom_patient_links l "
2288             "JOIN patients p ON p.patient_id = l.patient_id"
2289         ).fetchall():
2290             d = dict(r)
2291             links[(d.pop("source"), d.pop("ref"))] = d

```

```

2292
2293     grouped: dict[tuple, dict] = {}
2294     for entry in annot_store.iter_all(ANNOT_DIR):
2295         src = entry.get("source")
2296         ref = entry.get("ref")
2297         if not src or not ref:
2298             continue
2299         anns = entry.get("annotations") or []
2300         if status:
2301             anns = [a for a in anns if a.get("status", "draft") == status]
2302         if created_by:
2303             anns = [a for a in anns if a.get("created_by") == created_by]
2304         if not anns:
2305             continue
2306
2307         key = (src, ref)
2308         statuses: dict[str, int] = {}
2309         labels: dict[str, int] = {}
2310         creators: set = set()
2311         latest = ""
2312         for a in anns:
2313             s = a.get("status", "draft")
2314             statuses[s] = statuses.get(s, 0) + 1
2315             lb = a.get("label") or "-"
2316             labels[lb] = labels.get(lb, 0) + 1
2317             if a.get("created_by"):
2318                 creators.add(a["created_by"])
2319             t = a.get("updated_at") or a.get("created_at") or ""
2320             if t > latest:
2321                 latest = t
2322
2323         grouped[key] = {
2324             "source": src,
2325             "ref": ref,
2326             "annotation_count": len(anns),
2327             "by_status": statuses,
2328             "by_label": labels,
2329             "creators": sorted(creators),
2330             "latest_ts": latest,
2331             "patient": links.get(key),
2332         }
2333
2334     items = sorted(grouped.values(), key=lambda x: x.get("latest_ts") or "", reverse=True)
2335     return {"items": items[:limit], "count": min(len(items), limit), "total": len(items)}
2336
2337
2338 @app.get("/api/db/patient/{patient_id}/overview")
2339 def db_patient_overview(patient_id: str):
2340     with db_mod.get_conn() as c:
2341         p = c.execute(
2342             "SELECT * FROM patients WHERE patient_id = ?", (patient_id,)
2343         ).fetchone()
2344         if not p:
2345             raise HTTPException(404, "patient not found")
2346         records = [dict(r) for r in c.execute(
2347             "SELECT * FROM records WHERE patient_id = ? "
2348             "ORDER BY service_date DESC, source_row DESC", (patient_id,)
2349         ).fetchall()]
2350         worklist = [dict(r) for r in c.execute(
2351             "SELECT * FROM worklist WHERE patient_id = ? "
2352             "ORDER BY study_date DESC", (patient_id,)
2353         ).fetchall()]
2354         links = [dict(r) for r in c.execute(
2355             "SELECT source, ref, confidence, confirmed_at FROM dicom_patient_links "
2356             "WHERE patient_id = ? ORDER BY confirmed_at DESC", (patient_id,)
2357         ).fetchall()]

```

```

2358
2359     dicoms = []
2360     for link in links:
2361         ann_count = annot_store.count(ANNOT_DIR, link["source"], link["ref"])
2362         ann_data = annot_store.load(ANNOT_DIR, link["source"], link["ref"])
2363         statuses: dict[str, int] = {}
2364         for a in ann_data.get("annotations", []):
2365             s = a.get("status", "draft")
2366             statuses[s] = statuses.get(s, 0) + 1
2367         dicoms.append({
2368             "source": link["source"],
2369             "ref": link["ref"],
2370             "confidence": link["confidence"],
2371             "confirmed_at": link["confirmed_at"],
2372             "annotation_count": ann_count,
2373             "annotation_statuses": statuses,
2374         })
2375
2376     timeline: list[dict] = []
2377     for r in records:
2378         if r.get("service_date"):
2379             timeline.append({
2380                 "kind": "record",
2381                 "ts": r["service_date"],
2382                 "exam_code": r.get("exam_code"),
2383                 "exam_name": r.get("exam_name"),
2384                 "diagnosis": r.get("diagnosis"),
2385                 "record_id": r["id"],
2386             })
2387     for w in worklist:
2388         if w.get("study_date"):
2389             timeline.append({
2390                 "kind": "worklist",
2391                 "ts": w["study_date"],
2392                 "modality": w.get("modality"),
2393                 "status": w.get("status"),
2394                 "assigned_to": w.get("assigned_to"),
2395                 "wl_id": w["id"],
2396             })
2397     for d in dicoms:
2398         ts = (d.get("confirmed_at") or "")[:10]
2399         timeline.append({
2400             "kind": "dicom",
2401             "ts": ts,
2402             "source": d["source"],
2403             "ref": d["ref"],
2404             "annotation_count": d["annotation_count"],
2405             "annotation_statuses": d["annotation_statuses"],
2406         })
2407     timeline.sort(key=lambda x: x.get("ts") or "", reverse=True)
2408
2409     return {
2410         "patient": dict(p),
2411         "records": records,
2412         "worklist": worklist,
2413         "dicoms": dicoms,
2414         "timeline": timeline,
2415     }
2416
2417
2418 @app.get("/api/db/record/{record_id}")
2419 def db_record(record_id: int):
2420     with db_mod.get_conn() as c:
2421         r = c.execute("SELECT * FROM records WHERE id = ?", (record_id,)).fetchone()
2422         if not r:
2423             raise HTTPException(404, "record not found")

```

```

2424     return dict(r)
2425
2426
2427 class LinkBody(BaseModel):
2428     source: str
2429     ref: str
2430     patient_id: str
2431     confidence: Optional[int] = None
2432     note: Optional[str] = None
2433
2434
2435 @app.get("/api/db/link")
2436 def db_get_link(source: str, ref: str):
2437     _validate_source(source)
2438     _validate_ref(source, ref)
2439     with db_mod.get_conn() as c:
2440         row = c.execute(
2441             "SELECT * FROM dicom_patient_links WHERE source = ? AND ref = ?",
2442             (source, ref),
2443         ).fetchone()
2444         if not row:
2445             return {"linked": False}
2446         link = dict(row)
2447         patient = c.execute(
2448             "SELECT * FROM patients WHERE patient_id = ?", (link["patient_id"],)
2449         ).fetchone()
2450         record_count = c.execute(
2451             "SELECT COUNT(*) FROM records WHERE patient_id = ?",
2452             (link["patient_id"],),
2453         ).fetchone()[0]
2454         return {
2455             "linked": True,
2456             "link": link,
2457             "patient": dict(patient) if patient else None,
2458             "record_count": record_count,
2459         }
2460
2461
2462 @app.put("/api/db/link")
2463 def db_set_link(body: LinkBody, _user: dict = Depends(auth_mod.require_user)):
2464     _validate_source(body.source)
2465     _validate_ref(body.source, body.ref)
2466     with db_mod.get_conn() as c:
2467         if not c.execute(
2468             "SELECT 1 FROM patients WHERE patient_id = ?", (body.patient_id,)
2469         ).fetchone():
2470             raise HTTPException(404, "patient not found")
2471         now = datetime.now(timezone.utc).isoformat()
2472         c.execute(
2473             "INSERT OR REPLACE INTO dicom_patient_links "
2474             "(source, ref, patient_id, confidence, note, confirmed_at) "
2475             "VALUES (?, ?, ?, ?, ?, ?)",
2476             (body.source, body.ref, body.patient_id,
2477              body.confidence, body.note, now),
2478         )
2479         c.commit()
2480     return {"ok": True, "confirmed_at": now}
2481
2482
2483 @app.get("/api/db/links")
2484 def db_list_links(
2485     q: Optional[str] = None,
2486     source: Optional[str] = None,
2487     limit: int = 200,
2488 ):
2489     where = []

```

```

2490     params: list = []
2491     if source:
2492         if source not in ("upload", "local"):
2493             raise HTTPException(400, "invalid source")
2494         where.append("l.source = ?")
2495         params.append(source)
2496     if q:
2497         q = q.strip()
2498         if q:
2499             pat = f"%{q}%"
2500             where.append(
2501                 "(p.first_name LIKE ? OR p.last_name LIKE ? OR p.patient_id LIKE ? OR l.ref LIKE ?)"
2502             )
2503             params.extend([pat, pat, pat, pat])
2504
2505     sql = (
2506         "SELECT l.source, l.ref, l.confidence, l.confirmed_at, "
2507         "p.patient_id, p.first_name, p.last_name, p.sex, p.birth_date, "
2508         "(SELECT COUNT(*) FROM records r WHERE r.patient_id = l.patient_id) AS record_count, "
2509         "(SELECT MAX(r.service_date) FROM records r WHERE r.patient_id = l.patient_id) AS last_visit "
2510         "FROM dicom_patient_links l "
2511         "JOIN patients p ON p.patient_id = l.patient_id"
2512     )
2513     if where:
2514         sql += " WHERE " + " AND ".join(where)
2515     sql += " ORDER BY l.confirmed_at DESC LIMIT ?"
2516     params.append(limit)
2517
2518     with db_mod.get_conn() as c:
2519         rows = [dict(r) for r in c.execute(sql, params).fetchall()]
2520
2521     for r in rows:
2522         r["annotation_count"] = annot_store.count(ANNOT_DIR, r["source"], r["ref"])
2523
2524     return {"links": rows, "count": len(rows)}
2525
2526
2527 @app.delete("/api/db/link")
2528 def db_delete_link(source: str, ref: str, _user: dict = Depends(auth_mod.require_user)):
2529     _validate_source(source)
2530     if not ref:
2531         raise HTTPException(400, "missing ref")
2532     with db_mod.get_conn() as c:
2533         c.execute(
2534             "DELETE FROM dicom_patient_links WHERE source = ? AND ref = ?",
2535             (source, ref),
2536         )
2537         c.commit()
2538     return {"ok": True}
2539
2540
2541 def _parse_dicom_name(name: Optional[str]) -> tuple[Optional[str], Optional[str]]:
2542     if not name:
2543         return (None, None)
2544     s = str(name).replace("^", " ").strip()
2545     parts = re.split(r"\s+", s)
2546     parts = [p for p in parts if p]
2547     if not parts:
2548         return (None, None)
2549     if len(parts) == 1:
2550         return (parts[0].upper(), None)
2551     return (parts[0].upper(), " ".join(parts[1:]).upper())
2552
2553
2554 def _norm_dicom_dob(dob: Optional[str]) -> Optional[str]:
2555     if not dob:

```



```

2556         return None
2557     s = str(dob).strip()
2558     if re.match(r"^\d{8}$", s):
2559         return f"{s[:4]}-{s[4:6]}-{s[6:8]}"
2560     m = re.match(r"^\(\d{4}-\d{2}-\d{2}\)", s)
2561     if m:
2562         return m.group(1)
2563     return None
2564
2565
2566 @app.get("/api/db/match")
2567 def db_match(
2568     patient_id: Optional[str] = None,
2569     name: Optional[str] = None,
2570     birth_date: Optional[str] = None,
2571     limit: int = 10,
2572 ):
2573     last, first = _parse_dicom_name(name)
2574     dob = _norm_dicom_dob(birth_date)
2575
2576     candidates: dict[str, dict] = {}
2577
2578     def add(row, score: int, reason: str):
2579         pid = row["patient_id"]
2580         cur = candidates.get(pid)
2581         if cur is None:
2582             candidates[pid] = {
2583                 "patient": dict(row),
2584                 "score": score,
2585                 "reasons": [reason],
2586             }
2587             return
2588         if reason not in cur["reasons"]:
2589             cur["reasons"].append(reason)
2590         if score > cur["score"]:
2591             cur["score"] = score
2592
2593     with db_mod.get_conn() as c:
2594         if patient_id:
2595             row = c.execute(
2596                 "SELECT * FROM patients WHERE patient_id = ?", (patient_id,)
2597             ).fetchone()
2598             if row:
2599                 add(row, 100, "patient_id exact")
2600
2601         if last and first and dob:
2602             for row in c.execute(
2603                 "SELECT * FROM patients WHERE last_name = ? AND first_name = ? AND birth_date = ?",
2604                 (last, first, dob),
2605             ).fetchall():
2606                 add(row, 95, "last+first+dob")
2607
2608         if last and first:
2609             for row in c.execute(
2610                 "SELECT * FROM patients WHERE last_name = ? AND first_name = ?",
2611                 (last, first),
2612             ).fetchall():
2613                 add(row, 80, "last+first")
2614
2615         if last and dob:
2616             for row in c.execute(
2617                 "SELECT * FROM patients WHERE last_name = ? AND birth_date = ?",
2618                 (last, dob),
2619             ).fetchall():
2620                 add(row, 70, "last+dob")
2621

```

```

2622         if first and dob:
2623             for row in c.execute(
2624                 "SELECT * FROM patients WHERE first_name = ? AND birth_date = ?",
2625                 (first, dob),
2626             ).fetchall():
2627                 add(row, 60, "first+dob")
2628
2629         if last:
2630             for row in c.execute(
2631                 "SELECT * FROM patients WHERE last_name = ? LIMIT 30", (last,)
2632             ).fetchall():
2633                 add(row, 30, "last only")
2634
2635         ranked = sorted(candidates.values(), key=lambda v: -v["score"])[0:limit]
2636
2637         for cand in ranked:
2638             pid = cand["patient"]["patient_id"]
2639             cand["record_count"] = c.execute(
2640                 "SELECT COUNT(*) FROM records WHERE patient_id = ?", (pid,)
2641             ).fetchone()[0]
2642             cand["last_visit"] = c.execute(
2643                 "SELECT MAX(service_date) FROM records WHERE patient_id = ?", (pid,)
2644             ).fetchone()[0]
2645
2646         return {
2647             "input": {
2648                 "patient_id": patient_id,
2649                 "name": name,
2650                 "birth_date": birth_date,
2651                 "parsed": {"last_name": last, "first_name": first, "dob": dob},
2652             },
2653             "candidates": ranked,
2654         }
2655
2656
2657 @app.get("/api/inference/status")
2658 def inference_status():
2659     ok, err = inf.is_available()
2660     info = inf.device_info()
2661     info["available"] = ok
2662     info["error"] = err if not ok else None
2663     info["models_dir"] = str(inf.MODELS_DIR)
2664     info["models"] = inf.list_models()
2665     return info
2666
2667
2668 @app.get("/api/inference/models")
2669 def inference_models():
2670     return {"models": inf.list_models(), "models_dir": str(inf.MODELS_DIR)}
2671
2672
2673 ENSEMBLE_MODEL = "__ensemble__"
2674
2675
2676 class InferenceBody(BaseModel):
2677     source: str
2678     ref: str
2679     model: str
2680     frame: int = 0
2681     conf: float = 0.25
2682     iou: float = 0.5
2683     imgsz: int = 1024
2684     wc: Optional[float] = None
2685     ww: Optional[float] = None
2686     tta: bool = False
2687     models: Optional[list[str]] = None # for ensemble; default = all available

```

```

2688
2689
2690 class BatchInferenceBody(BaseModel):
2691     items: list[dict]
2692     model: str
2693     conf: float = 0.25
2694     iou: float = 0.5
2695     imgsz: int = 1024
2696     auto_save: bool = False
2697     tta: bool = False
2698     models: Optional[list[str]] = None
2699
2700
2701 def _run_inference(png_bytes: bytes, *, model: str, conf: float, iou: float, imgsz: int,
2702                  tta: bool, models: Optional[list[str]]):
2703     """Dispatch to a single model or the WBF ensemble of several models."""
2704     if model == ENSEMBLE_MODEL:
2705         names = models or [m["name"] for m in inf.list_models()]
2706         return inf.infer_ensemble(png_bytes, names, conf=conf, iou=iou, imgsz=imgsz, tta=tta)
2707     return inf.infer_png(png_bytes, model_name=model, conf=conf, iou=iou, imgsz=imgsz, tta=tta)
2708
2709
2710 @app.post("/api/inference/batch")
2711 @limiter.limit("5/minute")
2712 async def inference_batch(
2713     request: Request,
2714     body: BatchInferenceBody,
2715     user: dict = Depends(auth_mod.require_user),
2716 ):
2717     if not body.items:
2718         raise HTTPException(400, "items empty")
2719     if len(body.items) > 50:
2720         raise HTTPException(400, "too many items (max 50 per batch)")
2721
2722     results = []
2723     username = user.get("sub") or ""
2724     now = datetime.now(timezone.utc).isoformat()
2725
2726     for it in body.items:
2727         src = it.get("source")
2728         ref = it.get("ref")
2729         if src not in ("upload", "local") or not ref:
2730             results.append({"source": src, "ref": ref, "ok": False, "error": "invalid item"})
2731             continue
2732         try:
2733             path = _resolve_dicom_path(src, ref)
2734             png_bytes = render_frame_png(path, frame=0, max_dim=2048)
2735             inf_res = _run_inference(
2736                 png_bytes, model=body.model, conf=body.conf, iou=body.iou,
2737                 imgsz=body.imgsz, tta=body.tta, models=body.models,
2738             )
2739         except Exception as e:
2740             results.append({"source": src, "ref": ref, "ok": False, "error": str(e)})
2741             continue
2742
2743     if body.auto_save and inf_res.get("detections"):
2744         existing = annot_store.load(ANNOT_DIR, src, ref)
2745         new_anns = list(existing.get("annotations") or [])
2746         for d in inf_res["detections"]:
2747             lbl = d.get("label") or "mass"
2748             new_anns.append({
2749                 "id": uuid.uuid4().hex,
2750                 "type": "bbox",
2751                 "label": lbl,
2752                 "bi_rads": "",
2753                 "frame": 0,

```

```

2754         "bbox": d["bbox"],
2755         "note": f"AI: {lbl} {d['confidence']*100:.1f}%",
2756         "ai_source": {
2757             "label": lbl,
2758             "confidence": d["confidence"],
2759             "class_id": d.get("class_id"),
2760         },
2761         "status": "draft",
2762         "created_by": username,
2763         "created_at": now,
2764         "updated_by": username,
2765         "updated_at": now,
2766     })
2767     annot_store.save(ANNOT_DIR, src, ref, {
2768         "source": src, "ref": ref,
2769         "rows": existing.get("rows"), "cols": existing.get("cols"),
2770         "annotations": new_anns,
2771     })
2772     results.append({
2773         "source": src, "ref": ref, "ok": True,
2774         "detections": len(inf_res.get("detections", [])),
2775         "saved": body.auto_save,
2776     })
2777     return {"ok": True, "results": results, "count": len(results)}
2778
2779
2780 @app.post("/api/inference/run")
2781 @limiter.limit("20/minute")
2782 def inference_run(request: Request, body: InferenceBody, _user: dict =
Depends(auth_mod.require_user)):
2783     _validate_source(body.source)
2784     _validate_ref(body.source, body.ref)
2785
2786     if body.source == "upload":
2787         path = UPLOAD_DIR / f"{body.ref}.dcm"
2788     else:
2789         path = _resolve_local(body.ref)
2790
2791     try:
2792         png_bytes = render_frame_png(
2793             path, frame=body.frame, wc=body.wc, ww=body.ww, max_dim=2048
2794         )
2795     except Exception as e:
2796         raise HTTPException(500, f"render failed: {e}")
2797
2798     try:
2799         result = _run_inference(
2800             png_bytes, model=body.model, conf=body.conf, iou=body.iou,
2801             imgsz=body.imgsz, tta=body.tta, models=body.models,
2802         )
2803     except FileNotFoundError as e:
2804         raise HTTPException(404, str(e))
2805     except RuntimeError as e:
2806         raise HTTPException(503, str(e))
2807     except Exception as e:
2808         raise HTTPException(500, f"inference failed: {e}")
2809     return result
2810
2811
2812 @app.websocket("/ws/dicom")
2813 async def ws_dicom(websocket: WebSocket, source: str, ref: str, token: str = ""):
2814     if not token:
2815         await websocket.close(code=1008)
2816         return
2817     try:
2818         payload = auth_mod.decode_token(token)

```

```

2819 except HTTPException:
2820     await websocket.close(code=1008)
2821     return
2822
2823 if source not in ("upload", "local") or not ref:
2824     await websocket.close(code=1008)
2825     return
2826
2827 username = payload.get("sub", "?")
2828 await websocket.accept()
2829 await ws_mod.rooms.join(source, ref, websocket, username)
2830 try:
2831     while True:
2832         data = await websocket.receive_json()
2833         kind = data.get("type")
2834         if kind == "ping":
2835             await websocket.send_json({"type": "pong"})
2836         elif kind == "cursor":
2837             await ws_mod.rooms.broadcast(source, ref, {
2838                 "type": "cursor",
2839                 "user": username,
2840                 "x": data.get("x"),
2841                 "y": data.get("y"),
2842             }, exclude_ws=websocket)
2843     except WebSocketDisconnect:
2844         pass
2845 except Exception:
2846     pass
2847 finally:
2848     await ws_mod.rooms.leave(source, ref, websocket, username)
2849
2850
2851 # -----
2852 # Research / radiologist verification (review_decisions table)
2853 # -----
2854
2855
2856 class ReviewDecideBody(BaseModel):
2857     status: str # "accepted" | "edited" | "rejected"
2858     final_bboxes: list[dict] | None = None # YOLO-ready bboxes the radiologist confirmed
2859     comments: str | None = None
2860
2861
2862 def _row_to_review_dict(row, include_pseudo: bool = False, include_final: bool = False) -> dict:
2863     out = {
2864         "id": row["id"],
2865         "sop_uid": row["sop_uid"],
2866         "view": row["view"] or "",
2867         "laterality": row["laterality"] or "",
2868         "status": row["status"],
2869         "reviewer": row["reviewer"] or "",
2870         "decided_at": row["decided_at"] or "",
2871         "imported_at": row["imported_at"],
2872         "source_queue": row["source_queue"] or "",
2873         "comments": row["comments"] or "",
2874         "n_pseudo": 0,
2875         "n_final": 0,
2876         "has_preview": bool(row["preprocessed_png_path"]) and
Path(row["preprocessed_png_path"]).exists(),
2877     }
2878     try:
2879         pseudo = json.loads(row["pseudo_bboxes_json"] or "[]")
2880         out["n_pseudo"] = len(pseudo)
2881         if include_pseudo:
2882             out["pseudo_bboxes"] = pseudo
2883             out["findings"] = json.loads(row["findings_json"] or "{}")

```

```

2884     except Exception:
2885         pass
2886     if row["final_bboxes_json"]:
2887         try:
2888             final = json.loads(row["final_bboxes_json"])
2889             out["n_final"] = len(final)
2890             if include_final:
2891                 out["final_bboxes"] = final
2892         except Exception:
2893             pass
2894     return out
2895
2896
2897 @app.get("/api/research/review/queue")
2898 def research_review_queue(
2899     status: str = Query("pending", pattern=r"^(pending|accepted|edited|rejected|all)$"),
2900     limit: int = Query(100, ge=1, le=500),
2901     offset: int = Query(0, ge=0),
2902     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
2903 ):
2904     sql = "SELECT * FROM review_decisions"
2905     params: list = []
2906     if status != "all":
2907         sql += " WHERE status = ?"
2908         params.append(status)
2909     sql += " ORDER BY (status = 'pending') DESC, imported_at DESC LIMIT ? OFFSET ?"
2910     params.extend([limit, offset])
2911     with db_mod.get_conn() as c:
2912         rows = c.execute(sql, params).fetchall()
2913         counts = c.execute(
2914             "SELECT status, COUNT(*) AS n FROM review_decisions GROUP BY status"
2915         ).fetchall()
2916     return {
2917         "items": [_row_to_review_dict(r) for r in rows],
2918         "counts": {r["status"]: r["n"] for r in counts},
2919         "limit": limit,
2920         "offset": offset,
2921     }
2922
2923
2924 @app.get("/api/research/review/export")
2925 def research_review_export(
2926     include_status: str = Query("accepted,edited", description="Comma-separated statuses to export"),
2927     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
2928 ):
2929     """Emit a JSONL bundle of gold-label review_decisions ready to be turned
2930     into a YOLO dataset by `app.research.coco_to_yolo` or piped into
2931     `app.research.train_tillnet --texts <reports>`.
2932     """
2933     statuses = [s.strip() for s in include_status.split(",") if s.strip()]
2934     if not statuses:
2935         raise HTTPException(status_code=400, detail="no statuses requested")
2936     placeholders = ",".join("{}" * len(statuses))
2937     with db_mod.get_conn() as c:
2938         rows = c.execute(
2939             f"SELECT * FROM review_decisions WHERE status IN ({placeholders}) "
2940             f"ORDER BY decided_at",
2941             statuses,
2942         ).fetchall()
2943
2944     def _stream():
2945         for r in rows:
2946             try:
2947                 final = json.loads(r["final_bboxes_json"] or "[ ]")
2948             except Exception:
2949                 final = []

```

```

2950         yield json.dumps({
2951             "sop_uid": r["sop_uid"],
2952             "dicom_path": r["dicom_path"] or "",
2953             "preprocessed_png": r["preprocessed_png_path"] or "",
2954             "view": r["view"] or "",
2955             "laterality": r["laterality"] or "",
2956             "status": r["status"],
2957             "reviewer": r["reviewer"] or "",
2958             "decided_at": r["decided_at"] or "",
2959             "bboxes": final,
2960         }, ensure_ascii=False) + "\n"
2961
2962     return StreamingResponse(
2963         _stream(),
2964         media_type="application/x-ndjson",
2965         headers={"Content-Disposition": "attachment; filename="gold_labels.jsonl"},
2966     )
2967
2968
2969 @app.get("/api/research/review/{review_id}")
2970 def research_review_get(
2971     review_id: int,
2972     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
2973 ):
2974     with db_mod.get_conn() as c:
2975         row = c.execute(
2976             "SELECT * FROM review_decisions WHERE id = ?", (review_id,)
2977         ).fetchone()
2978     if not row:
2979         raise HTTPException(status_code=404, detail="review item not found")
2980     return _row_to_review_dict(row, include_pseudo=True, include_final=True)
2981
2982
2983 @app.get("/api/research/review/{review_id}/preview")
2984 def research_review_preview(
2985     review_id: int,
2986     _user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
2987 ):
2988     with db_mod.get_conn() as c:
2989         row = c.execute(
2990             "SELECT preprocessed_png_path FROM review_decisions WHERE id = ?", (review_id,)
2991         ).fetchone()
2992     if not row:
2993         raise HTTPException(status_code=404, detail="review item not found")
2994     p = (row["preprocessed_png_path"] or "").strip()
2995     if not p:
2996         raise HTTPException(status_code=404, detail="no preview available")
2997     f = Path(p)
2998     if not f.exists() or not f.is_file():
2999         raise HTTPException(status_code=404, detail="preview file missing")
3000     return FileResponse(str(f), media_type="image/png")
3001
3002
3003 @app.post("/api/research/review/{review_id}/decide")
3004 def research_review_decide(
3005     review_id: int,
3006     body: ReviewDecideBody,
3007     user: dict = Depends(auth_mod.require_role("admin", "reviewer")),
3008 ):
3009     if body.status not in ("accepted", "edited", "rejected"):
3010         raise HTTPException(status_code=400, detail="invalid status")
3011     final = body.final_bboxes or []
3012     if body.status == "rejected":
3013         final = [] # ignore any sent boxes on reject
3014     final_json = json.dumps(final, ensure_ascii=False)
3015     now = datetime.now(timezone.utc).isoformat()

```

```

3016     reviewer = user.get("sub") or ""
3017     with db_mod.get_conn() as c:
3018         cur = c.execute(
3019             "UPDATE review_decisions "
3020             "SET status = ?, final_bboxes_json = ?, reviewer = ?, decided_at = ?, comments = ? "
3021             "WHERE id = ?",
3022             (body.status, final_json, reviewer, now, body.comments or "", review_id),
3023         )
3024         if cur.rowcount == 0:
3025             raise HTTPException(status_code=404, detail="review item not found")
3026         c.commit()
3027     return {"id": review_id, "status": body.status, "n_final": len(final), "decided_at": now}
3028
3029
3030 app.mount("/", StaticFiles(directory=str(STATIC_DIR), html=True), name="static")

```

app/auth.py

Fayl yo'li: app/auth.py | Qatorlar: 239 | Hajm: 7068 bayt

```

1  from __future__ import annotations
2
3  import os
4  import secrets
5  from datetime import datetime, timedelta, timezone
6  from pathlib import Path
7  from typing import Optional
8
9  import bcrypt
10 import jwt
11 from fastapi import Depends, HTTPException, Request
12 from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
13
14 from . import db as db_mod
15
16 ROLES = ("admin", "reviewer", "annotator")
17 TOKEN_TTL = timedelta(hours=12)
18 ALGORITHM = "HS256"
19
20 _SECRET_FILE = Path(__file__).resolve().parent / ".jwt_secret"
21
22
23 def _load_secret() -> str:
24     env = os.environ.get("JWT_SECRET", "").strip()
25     if env:
26         return env
27     if _SECRET_FILE.exists():
28         return _SECRET_FILE.read_text(encoding="utf-8").strip()
29     s = secrets.token_urlsafe(48)
30     _SECRET_FILE.write_text(s, encoding="utf-8")
31     return s
32
33
34 _SECRET = _load_secret()
35 _bearer = HTTPBearer(auto_error=False)
36
37
38 def hash_password(plain: str) -> str:
39     return bcrypt.hashpw(plain.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")
40
41
42 def verify_password(plain: str, hashed: str) -> bool:
43     try:
44         return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("utf-8"))

```



```

45     except (ValueError, TypeError):
46         return False
47
48
49 def issue_token(user: dict) -> tuple[str, str]:
50     now = datetime.now(timezone.utc)
51     exp = now + TOKEN_TTL
52     payload = {
53         "sub": user["username"],
54         "uid": user["id"],
55         "role": user["role"],
56         "iat": int(now.timestamp()),
57         "exp": int(exp.timestamp()),
58     }
59     return jwt.encode(payload, _SECRET, algorithm=ALGORITHM), exp.isoformat()
60
61
62 def decode_token(token: str) -> dict:
63     try:
64         return jwt.decode(token, _SECRET, algorithms=[ALGORITHM])
65     except jwt.ExpiredSignatureError:
66         raise HTTPException(401, "token expired")
67     except jwt.InvalidTokenError:
68         raise HTTPException(401, "invalid token")
69
70
71 def get_user_by_username(username: str) -> Optional[dict]:
72     with db_mod.get_conn() as c:
73         row = c.execute(
74             "SELECT * FROM users WHERE username = ?", (username,)
75         ).fetchone()
76     return dict(row) if row else None
77
78
79 def create_user(
80     username: str,
81     password: str,
82     role: str,
83     display_name: Optional[str] = None,
84     email: Optional[str] = None,
85 ) -> dict:
86     if role not in ROLES:
87         raise ValueError(f"role must be one of {ROLES}")
88     now = datetime.now(timezone.utc).isoformat()
89     with db_mod.get_conn() as c:
90         c.execute(
91             "INSERT INTO users (username, password_hash, role, display_name, email, is_active,
created_at) "
92             "VALUES (?, ?, ?, ?, ?, 1, ?)",
93             (username, hash_password(password), role, display_name, email, now),
94         )
95         c.commit()
96         row = c.execute("SELECT * FROM users WHERE username = ?", (username,)).fetchone()
97     return dict(row)
98
99
100 def update_password(username: str, new_password: str) -> bool:
101     with db_mod.get_conn() as c:
102         cur = c.execute(
103             "UPDATE users SET password_hash = ? WHERE username = ?",
104             (hash_password(new_password), username),
105         )
106         c.commit()
107     return cur.rowcount > 0
108
109

```

```

110 def update_last_login(user_id: int) -> None:
111     now = datetime.now(timezone.utc).isoformat()
112     with db_mod.get_conn() as c:
113         c.execute("UPDATE users SET last_login_at = ? WHERE id = ?", (now, user_id))
114         c.commit()
115
116
117 def public_user(row: dict) -> dict:
118     return {
119         "id": row["id"],
120         "username": row["username"],
121         "role": row["role"],
122         "display_name": row.get("display_name"),
123         "email": row.get("email"),
124         "is_active": bool(row.get("is_active", 1)),
125         "last_login_at": row.get("last_login_at"),
126         "created_at": row.get("created_at"),
127         "totp_enrolled": bool(row.get("totp_enrolled", 0)),
128     }
129
130
131 def generate_totp_secret() -> str:
132     import pyotp
133     return pyotp.random_base32()
134
135
136 def totp_provisioning_uri(username: str, secret: str, issuer: str = "MAMOGRAF") -> str:
137     import pyotp
138     return pyotp.totp.TOTP(secret).provisioning_uri(name=username, issuer_name=issuer)
139
140
141 def totp_qr_data_uri(provisioning_uri: str) -> str:
142     import io
143     import base64
144     import qrcode
145     img = qrcode.make(provisioning_uri)
146     buf = io.BytesIO()
147     img.save(buf, format="PNG")
148     return "data:image/png;base64," + base64.b64encode(buf.getvalue()).decode("ascii")
149
150
151 def verify_totp(secret: str, code: str) -> bool:
152     if not secret or not code:
153         return False
154     import pyotp
155     try:
156         return pyotp.TOTP(secret).verify(code.strip(), valid_window=1)
157     except Exception:
158         return False
159
160
161 def set_totp_secret(username: str, secret: Optional[str], enrolled: bool) -> None:
162     with db_mod.get_conn() as c:
163         c.execute(
164             "UPDATE users SET totp_secret = ?, totp_enrolled = ? WHERE username = ?",
165             (secret, 1 if enrolled else 0, username),
166         )
167         c.commit()
168
169
170 def get_setting(key: str, default: str = "") -> str:
171     with db_mod.get_conn() as c:
172         row = c.execute(
173             "SELECT value FROM system_settings WHERE key = ?", (key,)
174         ).fetchone()
175     return row[0] if row else default

```

```

176
177
178 def set_setting(key: str, value: str, updated_by: Optional[str] = None) -> None:
179     from datetime import datetime, timezone
180     now = datetime.now(timezone.utc).isoformat()
181     with db_mod.get_conn() as c:
182         c.execute(
183             "INSERT INTO system_settings (key, value, updated_by, updated_at) "
184             "VALUES (?, ?, ?, ?) "
185             "ON CONFLICT(key) DO UPDATE SET value=excluded.value, "
186             "updated_by=excluded.updated_by, updated_at=excluded.updated_at",
187             (key, value, updated_by, now),
188         )
189     c.commit()
190
191
192 def totp_required_roles() -> list[str]:
193     raw = get_setting("totp_required_roles", "")
194     return [r.strip() for r in raw.split(",") if r.strip()]
195
196
197 def needs_totp_setup(user_row: dict) -> bool:
198     if user_row.get("totp_enrolled"):
199         return False
200     return user_row.get("role") in totp_required_roles()
201
202
203 def has_any_user() -> bool:
204     with db_mod.get_conn() as c:
205         return c.execute("SELECT COUNT(*) FROM users").fetchone()[0] > 0
206
207
208 def list_users() -> list[dict]:
209     with db_mod.get_conn() as c:
210         rows = c.execute(
211             "SELECT * FROM users ORDER BY username"
212         ).fetchall()
213     return [public_user(dict(r)) for r in rows]
214
215
216 async def current_user_optional(
217     request: Request,
218     creds: Optional[HTTPAuthorizationCredentials] = Depends(_bearer),
219 ) -> Optional[dict]:
220     if creds is None:
221         return None
222     payload = decode_token(creds.credentials)
223     return payload
224
225
226 async def require_user(
227     user: Optional[dict] = Depends(current_user_optional),
228 ) -> dict:
229     if user is None:
230         raise HTTPException(401, "authentication required")
231     return user
232
233
234 def require_role(*roles: str):
235     async def _dep(user: dict = Depends(require_user)) -> dict:
236         if user.get("role") not in roles:
237             raise HTTPException(403, f"role required: {' '.join(roles)}")
238         return user
239     return _dep

```

app/db.py

Fayl yo'li: app/db.py | Qatorlar: 223 | Hajm: 5887 bayt

```
1 from __future__ import annotations
2
3 import sqlite3
4 import threading
5 from pathlib import Path
6
7 DB_PATH = Path(__file__).resolve().parent / "db.sqlite3"
8
9 SCHEMA = """
10 CREATE TABLE IF NOT EXISTS patients (
11     patient_id TEXT PRIMARY KEY,
12     first_name TEXT,
13     last_name TEXT,
14     sex TEXT,
15     birth_date TEXT
16 );
17
18 CREATE TABLE IF NOT EXISTS records (
19     id INTEGER PRIMARY KEY AUTOINCREMENT,
20     source_row INTEGER UNIQUE,
21     patient_id TEXT NOT NULL,
22     service_date TEXT,
23     approval_date TEXT,
24     exam_code TEXT,
25     exam_name TEXT,
26     report TEXT,
27     history TEXT,
28     complaints TEXT,
29     clinical_findings TEXT,
30     recommendations TEXT,
31     disease_info TEXT,
32     treatment TEXT,
33     lab_notes TEXT,
34     discharge_meds TEXT,
35     occupational TEXT,
36     admission_reason TEXT,
37     diagnosis TEXT,
38     diagnostic_procedures TEXT,
39     imported_at TEXT,
40     FOREIGN KEY(patient_id) REFERENCES patients(patient_id)
41 );
42
43 CREATE INDEX IF NOT EXISTS idx_records_patient ON records(patient_id);
44 CREATE INDEX IF NOT EXISTS idx_records_service_date ON records(service_date);
45 CREATE INDEX IF NOT EXISTS idx_records_exam_code ON records(exam_code);
46 CREATE INDEX IF NOT EXISTS idx_patients_name ON patients(last_name, first_name);
47
48 CREATE TABLE IF NOT EXISTS dicom_patient_links (
49     id INTEGER PRIMARY KEY AUTOINCREMENT,
50     source TEXT NOT NULL,
51     ref TEXT NOT NULL,
52     patient_id TEXT NOT NULL,
53     confidence INTEGER,
54     note TEXT,
55     confirmed_at TEXT,
56     UNIQUE(source, ref),
57     FOREIGN KEY(patient_id) REFERENCES patients(patient_id)
58 );
59
60 CREATE INDEX IF NOT EXISTS idx_links_patient ON dicom_patient_links(patient_id);
61
62 CREATE TABLE IF NOT EXISTS users (
```

```

63  id INTEGER PRIMARY KEY AUTOINCREMENT,
64  username TEXT UNIQUE NOT NULL,
65  password_hash TEXT NOT NULL,
66  role TEXT NOT NULL CHECK (role IN ('admin', 'reviewer', 'annotator')),
67  display_name TEXT,
68  email TEXT,
69  is_active INTEGER NOT NULL DEFAULT 1,
70  created_at TEXT,
71  last_login_at TEXT,
72  totp_secret TEXT,
73  totp_enrolled INTEGER NOT NULL DEFAULT 0
74 );
75
76 CREATE TABLE IF NOT EXISTS annotation_history (
77  id INTEGER PRIMARY KEY AUTOINCREMENT,
78  source TEXT NOT NULL,
79  ref TEXT NOT NULL,
80  annotation_id TEXT NOT NULL,
81  action TEXT NOT NULL,
82  username TEXT,
83  ts TEXT NOT NULL,
84  prev_snapshot TEXT,
85  new_snapshot TEXT
86 );
87
88 CREATE INDEX IF NOT EXISTS idx_history_ann ON annotation_history(source, ref, annotation_id);
89 CREATE INDEX IF NOT EXISTS idx_history_ts ON annotation_history(ts);
90
91 CREATE TABLE IF NOT EXISTS notifications (
92  id INTEGER PRIMARY KEY AUTOINCREMENT,
93  recipient TEXT NOT NULL,
94  ts TEXT NOT NULL,
95  read_at TEXT,
96  kind TEXT NOT NULL,
97  title TEXT,
98  body TEXT,
99  link_source TEXT,
100 link_ref TEXT,
101 link_annotation_id TEXT,
102 actor TEXT
103 );
104
105 CREATE INDEX IF NOT EXISTS idx_notif_recipient ON notifications(recipient, read_at);
106 CREATE INDEX IF NOT EXISTS idx_notif_ts ON notifications(ts);
107
108 CREATE TABLE IF NOT EXISTS worklist (
109  id INTEGER PRIMARY KEY AUTOINCREMENT,
110  patient_id TEXT NOT NULL,
111  patient_name TEXT,
112  sex TEXT,
113  dob TEXT,
114  study_date TEXT,
115  modality TEXT,
116  has_file INTEGER DEFAULT 0,
117  has_report INTEGER DEFAULT 0,
118  report_status TEXT,
119  assigned_to TEXT,
120  priority TEXT DEFAULT 'normal',
121  status TEXT DEFAULT 'pending',
122  notes TEXT,
123  imported_at TEXT,
124  source_row INTEGER,
125  UNIQUE(patient_id, study_date, modality, source_row)
126 );
127 CREATE INDEX IF NOT EXISTS idx_worklist_assigned ON worklist(assigned_to, status);
128 CREATE INDEX IF NOT EXISTS idx_worklist_patient ON worklist(patient_id);

```

```

129 CREATE INDEX IF NOT EXISTS idx_worklist_date ON worklist(study_date);
130
131 CREATE TABLE IF NOT EXISTS pacs_servers (
132     id INTEGER PRIMARY KEY AUTOINCREMENT,
133     name TEXT UNIQUE NOT NULL,
134     host TEXT NOT NULL,
135     port INTEGER NOT NULL,
136     aet TEXT NOT NULL,
137     calling_aet TEXT DEFAULT 'MAMOGRAF_SCU',
138     notes TEXT,
139     added_by TEXT,
140     added_at TEXT
141 );
142
143 CREATE TABLE IF NOT EXISTS annotation_templates (
144     id INTEGER PRIMARY KEY AUTOINCREMENT,
145     name TEXT NOT NULL,
146     description TEXT,
147     payload TEXT NOT NULL,
148     created_by TEXT,
149     created_at TEXT,
150     is_shared INTEGER NOT NULL DEFAULT 1,
151     UNIQUE(name, created_by)
152 );
153
154 CREATE INDEX IF NOT EXISTS idx_templates_shared ON annotation_templates(is_shared, created_by);
155
156 CREATE TABLE IF NOT EXISTS system_settings (
157     key TEXT PRIMARY KEY,
158     value TEXT,
159     updated_by TEXT,
160     updated_at TEXT
161 );
162
163 CREATE TABLE IF NOT EXISTS review_decisions (
164     id INTEGER PRIMARY KEY AUTOINCREMENT,
165     sop_uid TEXT NOT NULL,
166     dicom_path TEXT,
167     preprocessed_png_path TEXT,
168     view TEXT,
169     laterality TEXT,
170     findings_json TEXT,
171     pseudo_bboxes_json TEXT NOT NULL,
172     final_bboxes_json TEXT,
173     status TEXT NOT NULL DEFAULT 'pending'
174     CHECK (status IN ('pending', 'accepted', 'edited', 'rejected')),
175     reviewer TEXT,
176     decided_at TEXT,
177     comments TEXT,
178     imported_at TEXT NOT NULL,
179     source_queue TEXT,
180     UNIQUE(sop_uid, source_queue)
181 );
182
183 CREATE INDEX IF NOT EXISTS idx_review_status ON review_decisions(status);
184 CREATE INDEX IF NOT EXISTS idx_review_reviewer ON review_decisions(reviewer);
185 CREATE INDEX IF NOT EXISTS idx_review_sop ON review_decisions(sop_uid);
186 """
187
188 _init_lock = threading.Lock()
189 _initialized = False
190
191
192 def get_conn() -> sqlite3.Connection:
193     conn = sqlite3.connect(str(DB_PATH))
194     conn.row_factory = sqlite3.Row

```

```

195     conn.execute("PRAGMA foreign_keys = ON")
196     return conn
197
198
199 _MIGRATIONS = [
200     "ALTER TABLE users ADD COLUMN totp_secret TEXT",
201     "ALTER TABLE users ADD COLUMN totp_enrolled INTEGER NOT NULL DEFAULT 0",
202 ]
203
204
205 def _apply_migrations(conn) -> None:
206     for stmt in _MIGRATIONS:
207         try:
208             conn.execute(stmt)
209         except Exception:
210             pass
211
212
213 def init_db() -> None:
214     global _initialized
215     with _init_lock:
216         if _initialized:
217             return
218         DB_PATH.parent.mkdir(parents=True, exist_ok=True)
219         with get_conn() as c:
220             c.executescript(SCHEMA)
221             _apply_migrations(c)
222             c.commit()
223         _initialized = True

```

app/__init__.py

Fayl yo'li: app/__init__.py | Qatorlar: 0 | Hajm: 0 bayt

1

app/annotations.py

Fayl yo'li: app/annotations.py | Qatorlar: 74 | Hajm: 2181 bayt

```

1 from __future__ import annotations
2
3 import hashlib
4 import json
5 import threading
6 from pathlib import Path
7 from typing import Iterator
8
9 _LOCK = threading.Lock()
10
11
12 def _annot_path(annot_dir: Path, source: str, ref: str) -> Path:
13     if source == "upload":
14         key = ref
15     else:
16         key = hashlib.sha1(ref.encode("utf-8")).hexdigest()
17     return annot_dir / f"{source}__{key}.json"
18
19
20 def load(annot_dir: Path, source: str, ref: str) -> dict:
21     p = _annot_path(annot_dir, source, ref)
22     if not p.exists():
23         return {"source": source, "ref": ref, "annotations": [], "rows": None, "cols": None}

```

```

24     try:
25         data = json.loads(p.read_text(encoding="utf-8"))
26     except Exception:
27         return {"source": source, "ref": ref, "annotations": [], "rows": None, "cols": None}
28     data.setdefault("source", source)
29     data.setdefault("ref", ref)
30     data.setdefault("annotations", [])
31     return data
32
33
34 def save(annot_dir: Path, source: str, ref: str, payload: dict) -> None:
35     p = _annot_path(annot_dir, source, ref)
36     annotations = payload.get("annotations", []) or []
37
38     with _LOCK:
39         if not annotations:
40             if p.exists():
41                 p.unlink()
42             return
43         p.parent.mkdir(parents=True, exist_ok=True)
44         body = {
45             "source": source,
46             "ref": ref,
47             "rows": payload.get("rows"),
48             "cols": payload.get("cols"),
49             "annotations": annotations,
50         }
51         tmp = p.with_suffix(".json.tmp")
52         tmp.write_text(json.dumps(body, ensure_ascii=False, indent=2), encoding="utf-8")
53         tmp.replace(p)
54
55
56 def count(annot_dir: Path, source: str, ref: str) -> int:
57     p = _annot_path(annot_dir, source, ref)
58     if not p.exists():
59         return 0
60     try:
61         data = json.loads(p.read_text(encoding="utf-8"))
62         return len(data.get("annotations", []) or [])
63     except Exception:
64         return 0
65
66
67 def iter_all(annot_dir: Path) -> Iterator[dict]:
68     if not annot_dir.exists():
69         return
70     for p in sorted(annot_dir.glob("*.json")):
71         try:
72             yield json.loads(p.read_text(encoding="utf-8"))
73         except Exception:
74             continue

```

app/backup.py

Fayl yo'li: app/backup.py | Qatorlar: 162 | Hajm: 5513 bayt

```

1  """
2  Backup / restore CLI.
3
4  Yaratish:
5      python -m app.backup create
6      python -m app.backup create my_backup.tar.gz
7      python -m app.backup create --include-uploads --include-secret
8

```



```

9 Tiklash:
10     python -m app.backup restore backup_2026-05-06.tar.gz
11     python -m app.backup restore backup.tar.gz --force
12
13 Ro'yxat:
14     python -m app.backup list
15     """
16 from __future__ import annotations
17
18 import argparse
19 import shutil
20 import sys
21 import tarfile
22 import tempfile
23 from datetime import datetime
24 from pathlib import Path
25
26 APP_DIR = Path(__file__).resolve().parent
27 BACKUP_ROOT_NAME = "mamograf_backup"
28
29
30 def _items(include_uploads: bool, include_secret: bool) -> list[tuple[Path, str]]:
31     items: list[tuple[Path, str]] = []
32     db = APP_DIR / "db.sqlite3"
33     if db.exists():
34         items.append((db, "db.sqlite3"))
35     annot_dir = APP_DIR / "annotations"
36     if annot_dir.exists():
37         for p in annot_dir.glob("*.json"):
38             items.append((p, f"annotations/{p.name}"))
39     labels = APP_DIR / "labels.json"
40     if labels.exists():
41         items.append((labels, "labels.json"))
42     if include_uploads:
43         uploads = APP_DIR / "uploads"
44         if uploads.exists():
45             for p in uploads.glob("*.dcm"):
46                 items.append((p, f"uploads/{p.name}"))
47     if include_secret:
48         secret = APP_DIR / ".jwt_secret"
49         if secret.exists():
50             items.append((secret, ".jwt_secret"))
51     return items
52
53
54 def cmd_list(args):
55     items = _items(args.include_uploads, args.include_secret)
56     if not items:
57         print("(bo'sh – backup qilinadigan fayl yo'q)")
58         return
59     total = 0
60     for src, _arc in items:
61         try:
62             sz = src.stat().st_size
63         except OSError:
64             sz = 0
65         total += sz
66         print(f" {sz:>12,d}  {_arc}")
67     print(f"\nJami: {total:,} bayt ({total / 1024 / 1024:.1f} MB), {len(items)} fayl")
68
69
70 def cmd_create(args):
71     items = _items(args.include_uploads, args.include_secret)
72     if not items:
73         print("hech narsa backup qilinmaydi", file=sys.stderr)
74         sys.exit(1)

```

```

75 out_path = Path(args.output) if args.output else Path.cwd() / (
76     f"{BACKUP_ROOT_NAME}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.tar.gz"
77 )
78 with tarfile.open(out_path, "w:gz") as tar:
79     for src, arc in items:
80         tar.add(str(src), arcname=f"{BACKUP_ROOT_NAME}/{arc}")
81     sz = out_path.stat().st_size
82     print(f"yaratildi: {out_path}")
83     print(f" fayllar: {len(items)}")
84     print(f" hajm: {sz:,} bayt ({sz / 1024 / 1024:.2f} MB)")
85
86
87 def cmd_restore(args):
88     src = Path(args.input)
89     if not src.exists():
90         print(f"fayl topilmadi: {src}", file=sys.stderr)
91         sys.exit(1)
92     if not args.force:
93         existing = []
94         for cand in (APP_DIR / "db.sqlite3", APP_DIR / "annotations", APP_DIR / "uploads"):
95             if cand.exists():
96                 existing.append(str(cand.relative_to(APP_DIR.parent)))
97         if existing:
98             print("Ogohlantirish: quyidagilar mavjud:")
99             for x in existing:
100                 print(f" {x}")
101             ans = input("Davom etamizmi? Mavjud fayllar yangilanadi (yes/no): ")
102             if ans.strip().lower() not in ("yes", "y", "ha"):
103                 print("bekor qilindi")
104             return
105
106     with tempfile.TemporaryDirectory() as tmp_str:
107         tmp = Path(tmp_str)
108         with tarfile.open(src, "r:gz") as tar:
109             for m in tar.getmembers():
110                 if not m.name.startswith(f"{BACKUP_ROOT_NAME}/"):
111                     print(f" shubhali entry o'tkazib yuborildi: {m.name}", file=sys.stderr)
112                     continue
113                 if ".." in Path(m.name).parts:
114                     continue
115                 tar.extract(m, path=tmp)
116
117         root = tmp / BACKUP_ROOT_NAME
118         if not root.exists():
119             print("backup tuzilmasi noto'g'ri (root yo'q)", file=sys.stderr)
120             sys.exit(1)
121
122         moved = 0
123         for src_p in root.rglob("*"):
124             if not src_p.is_file():
125                 continue
126             rel = src_p.relative_to(root)
127             dst = APP_DIR / rel
128             dst.parent.mkdir(parents=True, exist_ok=True)
129             shutil.copy2(src_p, dst)
130             moved += 1
131         print(f"qayta tiklandi: {moved} fayl → {APP_DIR}")
132
133
134 def main():
135     ap = argparse.ArgumentParser(description="MAMOGRAF backup/restore CLI")
136     sub = ap.add_subparsers(dest="cmd", required=True)
137
138     p = sub.add_parser("list", help="backup tarkibini ko'rsatish (yaratmasdan)")
139     p.add_argument("--include-uploads", action="store_true")
140     p.add_argument("--include-secret", action="store_true")

```

```

141     p.set_defaults(func=cmd_list)
142
143     p = sub.add_parser("create", help="backup tar.gz yaratish")
144     p.add_argument("output", nargs="?", help="(ixtiyoriy) chiquvchi fayl yo'li")
145     p.add_argument("--include-uploads", action="store_true",
146                   help="yuklangan DICOM'larni ham qo'shish (katta hajm)")
147     p.add_argument("--include-secret", action="store_true",
148                   help=".jwt_secret faylini ham qo'shish (xavf – ehtiyotkorlik)")
149     p.set_defaults(func=cmd_create)
150
151     p = sub.add_parser("restore", help="tar.gz dan tiklash")
152     p.add_argument("input", help="backup tar.gz fayl yo'li")
153     p.add_argument("--force", action="store_true",
154                   help="tasdiqlashsiz tiklash (mavjud fayllarni yangilaydi)")
155     p.set_defaults(func=cmd_restore)
156
157     args = ap.parse_args()
158     args.func(args)
159
160
161 if __name__ == "__main__":
162     main()

```

app/coco_to_yolo.py

Fayl yo'li: *app/coco_to_yolo.py* | Qatorlar: 190 | Hajm: 6796 bayt

```

1  """
2  COCO JSON eksport faylini YOLO formatga aylantiradi.
3
4  MAMOGRAF Viewer tomonidan eksport qilingan `annotations_coco*.json`
5  fayldan YOLO format'da `labels/*.txt` va `images/` strukturasi yaratadi.
6
7  YOLO bbox format (har qator bitta bbox):
8      <class_id> <x_center> <y_center> <width> <height>
9
10 Hammasi normalized [0..1].
11
12 Foydalanish:
13     python -m app.coco_to_yolo annotations_coco.json --out yolo_dataset/
14     python -m app.coco_to_yolo annotations_coco.json --out dataset/ --copy-images
15     python -m app.coco_to_yolo annotations_coco.json --out dataset/ --train-val 0.8
16 """
17 from __future__ import annotations
18
19 import argparse
20 import json
21 import random
22 import shutil
23 import sys
24 from pathlib import Path
25
26
27 def convert(
28     coco_path: Path,
29     out_dir: Path,
30     copy_images: bool = False,
31     upload_dir: Path | None = None,
32     local_root: Path | None = None,
33     train_val: float = 0.0,
34     seed: int = 42,
35 ) -> dict:
36     if not coco_path.exists():
37         raise FileNotFoundError(coco_path)

```

```

38 data = json.loads(coco_path.read_text(encoding="utf-8"))
39
40 images = {img["id"]: img for img in data.get("images", [])}
41 cats = {c["id"]: c["name"] for c in data.get("categories", [])}
42 sorted_cat_ids = sorted(cats.keys())
43 cat_id_to_yolo = {cid: i for i, cid in enumerate(sorted_cat_ids)}
44
45 out_dir.mkdir(parents=True, exist_ok=True)
46 labels_dir = out_dir / "labels"
47 images_dir = out_dir / "images"
48 labels_dir.mkdir(exist_ok=True)
49 if copy_images:
50     images_dir.mkdir(exist_ok=True)
51
52 by_image: dict[int, list[dict]] = {}
53 for ann in data.get("annotations", []):
54     by_image.setdefault(ann["image_id"], []).append(ann)
55
56 written_labels = 0
57 skipped_no_dim = 0
58 copied_images = 0
59
60 image_stems: list[str] = []
61 for img_id, img in images.items():
62     stem = Path(img["file_name"]).stem or f"img_{img_id}"
63     image_stems.append(stem)
64     anns = by_image.get(img_id, [])
65     rows = []
66     w = img.get("width") or 0
67     h = img.get("height") or 0
68     if not w or not h:
69         skipped_no_dim += 1
70         continue
71     for ann in anns:
72         yc = cat_id_to_yolo.get(ann["category_id"])
73         if yc is None:
74             continue
75         bn = ann.get("bbox_normalized")
76         if bn and len(bn) == 4:
77             x_n, y_n, w_n, h_n = bn
78         else:
79             x_px, y_px, w_px, h_px = ann.get("bbox", [0, 0, 0, 0])
80             x_n = x_px / w
81             y_n = y_px / h
82             w_n = w_px / w
83             h_n = h_px / h
84         xc = x_n + w_n / 2
85         yc_norm = y_n + h_n / 2
86         rows.append(f"{yc} {xc:.6f} {yc_norm:.6f} {w_n:.6f} {h_n:.6f}")
87     (labels_dir / f"{stem}.txt").write_text("\n".join(rows), encoding="utf-8")
88     written_labels += 1
89
90 if copy_images:
91     src_path = None
92     ref = img["file_name"]
93     if img.get("source") == "upload" and upload_dir:
94         cand = upload_dir / f"{ref}.dcm"
95         if cand.exists():
96             src_path = cand
97     elif img.get("source") == "local" and local_root:
98         cand = local_root / ref
99         if cand.exists():
100             src_path = cand
101     if src_path:
102         shutil.copy2(src_path, images_dir / f"{stem}.dcm")
103         copied_images += 1

```

```

104
105     classes_file = out_dir / "classes.txt"
106     classes_file.write_text(
107         "\n".join(cats[cid] for cid in sorted_cat_ids), encoding="utf-8"
108     )
109
110     yaml_path = out_dir / "data.yaml"
111     train_path = "images/train" if train_val else "images"
112     val_path = "images/val" if train_val else "images"
113     yaml_text = (
114         f"# YOLO dataset config (avtomatik yaratilgan)\n"
115         f"path: {out_dir.resolve().as_posix()}\n"
116         f"train: {train_path}\n"
117         f"val: {val_path}\n"
118         f"names:\n"
119     )
120     for i, cid in enumerate(sorted_cat_ids):
121         yaml_text += f" {i}: {cats[cid]}\n"
122     yaml_path.write_text(yaml_text, encoding="utf-8")
123
124     split_info = {}
125     if train_val and 0 < train_val < 1:
126         random.Random(seed).shuffle(image_stems)
127         cut = int(len(image_stems) * train_val)
128         train_stems = image_stems[:cut]
129         val_stems = image_stems[cut:]
130         split_info = {"train": len(train_stems), "val": len(val_stems)}
131         for split, stems in [("train", train_stems), ("val", val_stems)]:
132             split_lbl = labels_dir.parent / "labels" / split
133             split_img = images_dir.parent / "images" / split
134             split_lbl.mkdir(parents=True, exist_ok=True)
135             split_img.mkdir(parents=True, exist_ok=True)
136             for stem in stems:
137                 lbl_src = labels_dir / f"{stem}.txt"
138                 if lbl_src.exists():
139                     shutil.move(str(lbl_src), str(split_lbl / f"{stem}.txt"))
140                 if copy_images:
141                     img_src = images_dir / f"{stem}.dcm"
142                     if img_src.exists():
143                         shutil.move(str(img_src), str(split_img / f"{stem}.dcm"))
144
145     return {
146         "out_dir": str(out_dir.resolve()),
147         "yaml": str(yaml_path),
148         "classes": list(cats[cid] for cid in sorted_cat_ids),
149         "labels_written": written_labels,
150         "images_copied": copied_images,
151         "skipped_no_dim": skipped_no_dim,
152         "split": split_info,
153     }
154
155
156 def main():
157     ap = argparse.ArgumentParser(description="MAMOGRAF COCO eksportni YOLO formatga aylantirish")
158     ap.add_argument("coco_json", help="annotations_coco_YYYYMMDD_HHMMSS.json")
159     ap.add_argument("--out", default="yolo_dataset", help="chiquvchi papka")
160     ap.add_argument("--copy-images", action="store_true",
161                     help="DICOM fayllarni dataset papkasiga ko'chirish")
162     ap.add_argument("--upload-dir", default="app/uploads", help="upload manbai")
163     ap.add_argument("--local-root", default="",
164                     help="LOCAL_DICOM_ROOT (lokal manbalar uchun)")
165     ap.add_argument("--train-val", type=float, default=0.0,
166                     help="train/val split nisbati (e.g. 0.8 = 80/20)")
167     ap.add_argument("--seed", type=int, default=42)
168     args = ap.parse_args()
169

```

```

170     coco = Path(args.coco_json)
171     if not coco.exists():
172         print(f"fayl topilmadi: {coco}", file=sys.stderr)
173         sys.exit(1)
174
175     res = convert(
176         coco,
177         Path(args.out),
178         copy_images=args.copy_images,
179         upload_dir=Path(args.upload_dir) if args.upload_dir else None,
180         local_root=Path(args.local_root) if args.local_root else None,
181         train_val=args.train_val,
182         seed=args.seed,
183     )
184     width = max(len(k) for k in res)
185     for k, v in res.items():
186         print(f" {k.ljust(width)} : {v}")
187
188
189 if __name__ == "__main__":
190     main()

```

app/deidentify.py

Fayl yo'li: app/deidentify.py | Qatorlar: 160 | Hajm: 4954 bayt

```

1  from __future__ import annotations
2
3  import io
4  from datetime import datetime
5  from pathlib import Path
6
7  import pydicom
8  from pydicom.uid import ImplicitVRLittleEndian
9
10
11  PHI_TAGS: dict[str, str] = {
12      "PatientName": "ANONYMOUS^PATIENT",
13      "PatientID": "ANON",
14      "PatientBirthDate": "19000101",
15      "PatientSex": None,
16      "PatientAge": None,
17      "PatientAddress": "",
18      "PatientTelephoneNumbers": "",
19      "PatientMotherBirthName": "",
20      "OtherPatientIDs": "",
21      "OtherPatientNames": "",
22      "EthnicGroup": "",
23      "ReferringPhysicianName": "",
24      "ReferringPhysicianAddress": "",
25      "ReferringPhysicianTelephoneNumbers": "",
26      "PerformingPhysicianName": "",
27      "OperatorsName": "",
28      "PhysiciansOfRecord": "",
29      "RequestingPhysician": "",
30      "InstitutionName": "",
31      "InstitutionAddress": "",
32      "InstitutionalDepartmentName": "",
33      "StationName": "",
34      "DeviceSerialNumber": "",
35      "AccessionNumber": "",
36      "StudyID": "",
37      "FillerOrderNumberImagingServiceRequest": "",
38      "PlacerOrderNumberImagingServiceRequest": "",

```

```

39 }
40
41 DATE_TAGS_KEEP_YEAR = (
42     "StudyDate", "SeriesDate", "AcquisitionDate", "ContentDate",
43 )
44
45
46 def detect_phi(path: Path) -> dict:
47     ds = pydicom.dcmread(str(path), stop_before_pixels=True, force=True)
48     found = []
49     for tag in PHI_TAGS:
50         if tag in ds:
51             v = ds.get(tag)
52             try:
53                 s = str(v)
54             except Exception:
55                 s = repr(v)
56             found.append({"tag": tag, "value": s[:80]})
57     for tag in DATE_TAGS_KEEP_YEAR:
58         if tag in ds:
59             try:
60                 s = str(ds.get(tag))
61             except Exception:
62                 s = ""
63             found.append({"tag": tag, "value": s, "preserves_year": True})
64     return {"file": str(path), "phi_tags_found": found, "count": len(found)}
65
66
67 def anonymize_to_bytes(path: Path) -> bytes:
68     ds = pydicom.dcmread(str(path), force=True)
69     if not getattr(ds, "file_meta", None) or not getattr(ds.file_meta, "TransferSyntaxUID", None):
70         meta = pydicom.dataset.FileMetaDataset()
71         meta.TransferSyntaxUID = ImplicitVRLittleEndian
72         ds.file_meta = meta
73
74     for tag, replacement in PHI_TAGS.items():
75         if tag in ds:
76             if replacement is None:
77                 del ds[tag]
78             else:
79                 try:
80                     ds.data_element(tag).value = replacement
81                 except Exception:
82                     pass
83
84     for tag in DATE_TAGS_KEEP_YEAR:
85         if tag in ds:
86             try:
87                 cur = str(ds.data_element(tag).value)
88                 if len(cur) >= 4 and cur[:4].isdigit():
89                     ds.data_element(tag).value = cur[:4] + "0101"
90             except Exception:
91                 pass
92
93     if "PatientIdentityRemoved" in ds:
94         ds.PatientIdentityRemoved = "YES"
95     else:
96         try:
97             ds.add_new(0x00120062, "CS", "YES")
98         except Exception:
99             pass
100     try:
101         ds.add_new(0x00120063, "LO", f"MAMOGRAF de-id {datetime.now().strftime('%Y-%m-%d')}")
102     except Exception:
103         pass
104

```

```

105     buf = io.BytesIO()
106     ds.save_as(buf, write_like_original=False)
107     return buf.getvalue()
108
109
110 def anonymize_in_place(path: Path) -> None:
111     """Anonimlashtirish + faylga qayta yozish (upload paytida)."""
112     data = anonymize_to_bytes(path)
113     path.write_bytes(data)
114
115
116 def is_deidentified(path: Path) -> bool:
117     """Fayl allaqachon anonimlashtirilganmi (`PatientIdentityRemoved=YES` belgisi)."""
118     try:
119         ds = pydicom.dcmread(str(path), stop_before_pixels=True, force=True)
120     except Exception:
121         return False
122     val = getattr(ds, "PatientIdentityRemoved", None)
123     return str(val).upper() == "YES" if val is not None else False
124
125
126 def _cli():
127     """`python -m app.deidentify <dir>` - papkadagi barcha `.dcm` fayllarni
128     anonimlashtirish (faqat hali anonim emaslarni). Mavjud (eski) yuklar uchun."""
129     import argparse, sys
130     ap = argparse.ArgumentParser(description="Deidentify all DICOM files in a directory.")
131     ap.add_argument("path", help="Papka yo'li (mas. app/uploads)")
132     ap.add_argument("--force", action="store_true", help="Anonim deb belgilanganlarni ham qayta
133     tozalash")
134     args = ap.parse_args()
135     root = Path(args.path)
136     if not root.is_dir():
137         print(f"[xato] papka topilmadi: {root}", file=sys.stderr)
138         sys.exit(2)
139
140     total = ok = skipped = failed = 0
141     for p in sorted(root.glob("*.dcm")):
142         total += 1
143         try:
144             if not args.force and is_deidentified(p):
145                 skipped += 1
146                 continue
147             anonymize_in_place(p)
148             ok += 1
149             print(f"  ✓ {p.name}")
150         except Exception as e:
151             failed += 1
152             print(f"  X {p.name}: {e}", file=sys.stderr)
153
154     print(f"\nJami: {total} | tozalandi: {ok} | o'tkazib yuborildi: {skipped} | xato: {failed}")
155     if failed:
156         sys.exit(1)
157
158
159 if __name__ == "__main__":
160     _cli()

```

app/dicom_seg.py

Fayl yo'li: app/dicom_seg.py | Qatorlar: 136 | Hajm: 4901 bayt

```

1 from __future__ import annotations
2

```



```

3 import io
4 from pathlib import Path
5 from typing import Optional
6
7 import numpy as np
8 import pydicom
9 from PIL import Image, ImageDraw
10
11
12 def _polygon_mask(rows: int, cols: int, points_norm: list[list[float]]) -> np.ndarray:
13     img = Image.new("L", (cols, rows), 0)
14     if len(points_norm) >= 3:
15         pts = [(int(round(p[0] * cols)), int(round(p[1] * rows))) for p in points_norm]
16         ImageDraw.Draw(img).polygon(pts, fill=1)
17     return np.array(img, dtype=np.uint8)
18
19
20 def _bbox_mask(rows: int, cols: int, bbox: list[float]) -> np.ndarray:
21     img = Image.new("L", (cols, rows), 0)
22     if len(bbox) == 4:
23         x0 = int(round(bbox[0] * cols))
24         y0 = int(round(bbox[1] * rows))
25         x1 = int(round((bbox[0] + bbox[2]) * cols))
26         y1 = int(round((bbox[1] + bbox[3]) * rows))
27         ImageDraw.Draw(img).rectangle([x0, y0, x1, y1], fill=1)
28     return np.array(img, dtype=np.uint8)
29
30
31 def annotations_to_seg(
32     src_path: Path,
33     annotations: list[dict],
34     rows: Optional[int] = None,
35     cols: Optional[int] = None,
36     series_description: str = "MAMOGRAF annotations SEG",
37 ) -> bytes:
38     try:
39         import highdicom as hd
40     except ImportError as e:
41         raise RuntimeError("highdicom kerak: pip install highdicom") from e
42
43     src = pydicom.dcmread(str(src_path), force=True)
44     if not getattr(src, "file_meta", None) or not getattr(src.file_meta, "TransferSyntaxUID", None):
45         from pydicom.uid import ImplicitVRLittleEndian
46         meta = pydicom.dataset.FileMetaDataset()
47         meta.TransferSyntaxUID = ImplicitVRLittleEndian
48         src.file_meta = meta
49
50     if rows is None or cols is None:
51         rows = int(getattr(src, "Rows", 0) or 0)
52         cols = int(getattr(src, "Columns", 0) or 0)
53     if not rows or not cols:
54         raise RuntimeError("source DICOM rows/cols topilmadi")
55
56     by_label: dict[str, list[dict]] = {}
57     for a in annotations:
58         if a.get("status") == "rejected":
59             continue
60         lbl = a.get("label") or "finding"
61         by_label.setdefault(lbl, []).append(a)
62     if not by_label:
63         raise RuntimeError("eksport qilnadigan annotation yo'q")
64
65     has_for_uid = bool(src.get("FrameOfReferenceUID"))
66     multi_segment = has_for_uid and len(by_label) > 1
67
68     segment_descriptions = []

```

```

69 masks_per_segment = []
70 for seg_num, (label, anns) in enumerate(sorted(by_label.items()), start=1):
71     combined = np.zeros((rows, cols), dtype=np.uint8)
72     for a in anns:
73         if a.get("type") == "polygon" and a.get("points"):
74             combined |= _polygon_mask(rows, cols, a["points"])
75         else:
76             combined |= _bbox_mask(rows, cols, a.get("bbox") or [0, 0, 0, 0])
77     if combined.sum() == 0:
78         continue
79     masks_per_segment.append((label, combined.astype(bool)))
80
81 if not masks_per_segment:
82     raise RuntimeError("hech qanday segment yaratilmadi")
83
84 if not multi_segment:
85     merged = np.zeros((rows, cols), dtype=bool)
86     labels_used = []
87     for label, m in masks_per_segment:
88         merged |= m
89         labels_used.append(label)
90     masks_per_segment = [(", ".join(labels_used[:5]), merged)]
91
92 for seg_num, (label, _) in enumerate(masks_per_segment, start=1):
93     seg_desc = hd.seg.SegmentDescription(
94         segment_number=seg_num,
95         segment_label=label,
96         segmented_property_category=hd.sr.CodedConcept(
97             "49755003", "SCT", "Morphologically Abnormal Structure"
98         ),
99         segmented_property_type=hd.sr.CodedConcept(
100             "108369006", "SCT", "Neoplasm"
101         ),
102         algorithm_type=hd.seg.SegmentAlgorithmTypeValues.MANUAL,
103         algorithm_identification=hd.AlgorithmIdentificationSequence(
104             name="MAMOGRAF",
105             version="1.0",
106             family=hd.sr.CodedConcept("113076", "DCM", "Segmentation"),
107         ),
108         tracking_uid=hd.UID(),
109         tracking_id=label,
110     )
111     segment_descriptions.append(seg_desc)
112
113 if multi_segment:
114     pixel_array = np.stack([m for _, m in masks_per_segment], axis=-1).astype(np.uint8)
115 else:
116     pixel_array = masks_per_segment[0][1]
117
118 seg = hd.seg.Segmentation(
119     source_images=[src],
120     pixel_array=pixel_array,
121     segmentation_type=hd.seg.SegmentationTypeValues.BINARY,
122     segment_descriptions=segment_descriptions,
123     series_instance_uid=hd.UID(),
124     series_number=9998,
125     sop_instance_uid=hd.UID(),
126     instance_number=1,
127     manufacturer="MAMOGRAF",
128     manufacturer_model_name="MAMOGRAF DICOM Viewer",
129     software_versions="1.0",
130     device_serial_number="0",
131     series_description=series_description,
132 )
133
134 buf = io.BytesIO()

```

```

135     seg.save_as(buf, write_like_original=False)
136     return buf.getvalue()

```

app/dicom_sr.py

Fayl yo'li: app/dicom_sr.py | Qatorlar: 186 | Hajm: 6134 bayt

```

1  from __future__ import annotations
2
3  import io
4  from datetime import datetime
5  from pathlib import Path
6  from typing import Optional
7
8  import pydicom
9  from pydicom.dataset import Dataset, FileDataset, FileMetaDataset
10 from pydicom.uid import ExplicitVRLittleEndian, generate_uid
11
12
13 COMPREHENSIVE_SR_CLASS_UID = "1.2.840.10008.5.1.4.1.1.88.33"
14 DCM_CODING_SCHEME = "DCM"
15
16
17 def _coded_concept(code: str, scheme: str, meaning: str) -> Dataset:
18     ds = Dataset()
19     ds.CodeValue = code
20     ds.CodingSchemeDesignator = scheme
21     ds.CodeMeaning = meaning
22     return ds
23
24
25 def _text_item(label: str, text: str) -> Dataset:
26     item = Dataset()
27     item.RelationshipType = "CONTAINS"
28     item.ValueType = "TEXT"
29     item.ConceptNameCodeSequence = [_coded_concept("121071", DCM_CODING_SCHEME, label)]
30     item.TextValue = text
31     return item
32
33
34 def _num_item(label: str, value: float, unit_code: str = "1") -> Dataset:
35     item = Dataset()
36     item.RelationshipType = "CONTAINS"
37     item.ValueType = "NUM"
38     item.ConceptNameCodeSequence = [_coded_concept("121070", DCM_CODING_SCHEME, label)]
39     measured = Dataset()
40     measured.NumericValue = str(value)
41     units = Dataset()
42     units.CodeValue = unit_code
43     units.CodingSchemeDesignator = "UCUM"
44     units.CodeMeaning = unit_code
45     measured.MeasurementUnitsCodeSequence = [units]
46     item.MeasuredValueSequence = [measured]
47     return item
48
49
50 def _container_item(label: str, children: list[Dataset]) -> Dataset:
51     c = Dataset()
52     c.RelationshipType = "CONTAINS"
53     c.ValueType = "CONTAINER"
54     c.ContinuityOfContent = "SEPARATE"
55     c.ConceptNameCodeSequence = [_coded_concept("121111", DCM_CODING_SCHEME, label)]
56     c.ContentSequence = children
57     return c

```

```

58
59
60 def annotations_to_sr(
61     src_path: Path,
62     annotations: list[dict],
63     rows: Optional[int] = None,
64     cols: Optional[int] = None,
65     author: Optional[str] = None,
66 ) -> bytes:
67     src = pydicom.dcmread(str(src_path), stop_before_pixels=True, force=True)
68
69     now = datetime.now()
70     sr_date = now.strftime("%Y%m%d")
71     sr_time = now.strftime("%H%M%S")
72
73     meta = FileMetaDataset()
74     meta.MediaStorageSOPClassUID = COMPREHENSIVE_SR_CLASS_UID
75     meta.MediaStorageSOPInstanceUID = generate_uid()
76     meta.TransferSyntaxUID = ExplicitVRLittleEndian
77     meta.ImplementationClassUID = generate_uid()
78     meta.ImplementationVersionName = "MAMOGRAF_SR"
79
80     sr = FileDataset(None, {}, file_meta=meta, preamble=b"\0" * 128)
81     sr.SOPClassUID = meta.MediaStorageSOPClassUID
82     sr.SOPInstanceUID = meta.MediaStorageSOPInstanceUID
83     sr.Modality = "SR"
84     sr.SeriesInstanceUID = generate_uid()
85     sr.StudyInstanceUID = src.get("StudyInstanceUID", generate_uid())
86     sr.SeriesNumber = 9999
87     sr.InstanceNumber = 1
88     sr.SeriesDescription = "MAMOGRAF annotations"
89
90     for tag in (
91         "PatientName", "PatientID", "PatientBirthDate", "PatientSex",
92         "StudyDate", "StudyTime", "AccessionNumber", "ReferringPhysicianName",
93     ):
94         if tag in src:
95             try:
96                 sr[tag] = src[tag]
97             except Exception:
98                 pass
99
100     sr.ContentDate = sr_date
101     sr.ContentTime = sr_time
102     sr.InstanceCreationDate = sr_date
103     sr.InstanceCreationTime = sr_time
104     sr.SpecificCharacterSet = "ISO_IR 192"
105
106     if author:
107         author_ds = Dataset()
108         author_ds.PersonName = author
109         sr.AuthorObserverSequence = [author_ds]
110
111     sr.CompletionFlag = "COMPLETE"
112     sr.VerificationFlag = "UNVERIFIED"
113     sr.PreliminaryFlag = "FINAL"
114
115     sr.ValueType = "CONTAINER"
116     sr.ContinuityOfContent = "SEPARATE"
117     sr.ConceptNameCodeSequence = [
118         _coded_concept("11528-7", "LN", "Radiology Report")
119     ]
120
121     findings_children: list[Dataset] = []
122     measurements_children: list[Dataset] = []
123

```

```

124 for i, a in enumerate(annotations, start=1):
125     if a.get("status") not in (None, "approved", "submitted"):
126         pass
127     label = a.get("label") or "finding"
128     bi_rads = a.get("bi_rads") or ""
129     ann_type = a.get("type") or "bbox"
130     bbox = a.get("bbox") or [0, 0, 0, 0]
131
132     if rows and cols and len(bbox) == 4:
133         x_px = round(bbox[0] * cols, 1)
134         y_px = round(bbox[1] * rows, 1)
135         w_px = round(bbox[2] * cols, 1)
136         h_px = round(bbox[3] * rows, 1)
137         geom_str = f"{ann_type} pixel x={x_px} y={y_px} w={w_px} h={h_px}"
138     else:
139         geom_str = (
140             f"{ann_type} normalized x={bbox[0]:.4f} y={bbox[1]:.4f} "
141             f"w={bbox[2]:.4f} h={bbox[3]:.4f}"
142         )
143     if ann_type == "polygon" and a.get("points"):
144         geom_str += f" · {len(a['points'])} nuqta"
145
146     text_lines = [
147         f"#{i} {label}",
148         f" status: {a.get('status', 'draft')}",
149         f" geometry: {geom_str}",
150     ]
151     if bi_rads:
152         text_lines.insert(1, f" BI-RADS: {bi_rads}")
153     if a.get("created_by"):
154         text_lines.append(f" yaratdi: {a.get('created_by')}")
155     if a.get("reviewed_by"):
156         text_lines.append(f" ko'rib chiqdi: {a.get('reviewed_by')}")
157     if a.get("review_note"):
158         text_lines.append(f" reviewer izohi: {a.get('review_note')}")
159     if a.get("note"):
160         text_lines.append(f" izoh: {a.get('note')}")
161
162     findings_children.append(_text_item("Finding", "\n".join(text_lines)))
163
164     if rows and cols and len(bbox) == 4:
165         measurements_children.append(
166             _num_item(f"#{i} {label} area px²",
167                     round(bbox[2] * cols * bbox[3] * rows, 1))
168         )
169
170     summary = _text_item(
171         "Summary",
172         f"MAMOGRAF DICOM Viewer · {len(annotations)} annotation eksport qilindi · "
173         f"{sr_date}",
174     )
175
176     sr.ContentSequence = [summary, _container_item("Findings", findings_children)]
177     if measurements_children:
178         sr.ContentSequence.append(
179             _container_item("Measurements", measurements_children)
180         )
181
182     buf = io.BytesIO()
183     sr.is_little_endian = True
184     sr.is_implicit_VR = False
185     sr.save_as(buf, write_like_original=False)
186     return buf.getvalue()

```

app/dicom_utils.py

Fayl yo'li: app/dicom_utils.py | Qatorlar: 347 | Hajm: 11891 bayt

```
1 from __future__ import annotations
2
3 import io
4 import threading
5 from collections import OrderedDict
6 from pathlib import Path
7 from typing import Optional
8
9 import numpy as np
10 import pydicom
11 from PIL import Image
12 from pydicom.uid import ImplicitVRLittleEndian
13
14
15 _PIXEL_CACHE: "OrderedDict[tuple, tuple]" = OrderedDict()
16 _PIXEL_CACHE_LOCK = threading.Lock()
17 _PIXEL_CACHE_MAX = 4
18
19
20 def _cache_get(key):
21     with _PIXEL_CACHE_LOCK:
22         v = _PIXEL_CACHE.get(key)
23         if v is not None:
24             _PIXEL_CACHE.move_to_end(key)
25         return v
26
27
28 def _cache_put(key, value):
29     with _PIXEL_CACHE_LOCK:
30         _PIXEL_CACHE[key] = value
31         _PIXEL_CACHE.move_to_end(key)
32         while len(_PIXEL_CACHE) > _PIXEL_CACHE_MAX:
33             _PIXEL_CACHE.popitem(last=False)
34
35 try:
36     from pydicom.pixels.processing import apply_modality_lut
37 except ImportError:
38     from pydicom.pixel_data_handlers.util import apply_modality_lut
39
40
41 def _ensure_transfer_syntax(ds: pydicom.Dataset) -> None:
42     fm = getattr(ds, "file_meta", None)
43     if fm is None or not getattr(fm, "TransferSyntaxUID", None):
44         meta = pydicom.dataset.FileMetaDataset()
45         meta.TransferSyntaxUID = ImplicitVRLittleEndian
46         ds.file_meta = meta
47
48
49 def get_frame_count(ds: pydicom.Dataset) -> int:
50     n = getattr(ds, "NumberOfFrames", 1) or 1
51     try:
52         return int(n)
53     except (TypeError, ValueError):
54         return 1
55
56
57 def _to_float(v) -> Optional[float]:
58     if v is None:
59         return None
60     if isinstance(v, pydicom.multival.MultiValue):
61         if len(v) == 0:
62             return None
```

```

63     v = v[0]
64     try:
65         return float(v)
66     except (TypeError, ValueError):
67         return None
68
69
70 def default_window(ds: pydicom.Dataset) -> tuple[Optional[float], Optional[float]]:
71     return _to_float(getattr(ds, "WindowCenter", None)), _to_float(getattr(ds, "WindowWidth", None))
72
73
74 def _select_frame(arr: np.ndarray, frame: int, frames: int) -> np.ndarray:
75     if frames > 1 and arr.ndim >= 3 and arr.shape[0] == frames:
76         return arr[frame]
77     return arr
78
79
80 class NoPixelDataError(ValueError):
81     def __init__(self, modality: str = "", sop_class: str = ""):
82         super().__init__(f"DICOM has no pixel data (modality={modality})")
83         self.modality = modality
84         self.sop_class = sop_class
85
86
87 def render_frame_png(
88     path: Path,
89     frame: int = 0,
90     wc: Optional[float] = None,
91     ww: Optional[float] = None,
92     invert: bool = False,
93     max_dim: int = 2048,
94 ) -> bytes:
95     try:
96         mtime = path.stat().st_mtime_ns
97     except OSError:
98         mtime = 0
99     cache_key = (str(path), mtime, frame, int(max_dim or 0))
100     cached = _cache_get(cache_key)
101     if cached is not None:
102         arr, photometric, default_wc, default_ww, is_color = cached
103     else:
104         ds = pydicom.dcmread(str(path), force=True)
105         _ensure_transfer_syntax(ds)
106         if "PixelData" not in ds:
107             raise NoPixelDataError(
108                 modality=str(getattr(ds, "Modality", "") or ""),
109                 sop_class=str(getattr(ds, "SOPClassUID", "") or ""),
110             )
111         raw = ds.pixel_array
112         frames = get_frame_count(ds)
113         raw = _select_frame(raw, frame, frames)
114         try:
115             raw = apply_modality_lut(raw, ds)
116         except Exception:
117             pass
118         is_color = raw.ndim == 3 and raw.shape[-1] in (3, 4)
119
120         if not is_color:
121             arr = np.asarray(raw, dtype=np.float32)
122             if max_dim and (arr.shape[0] > max_dim or arr.shape[1] > max_dim):
123                 pil = Image.fromarray(arr, mode="F")
124                 pil.thumbnail((max_dim, max_dim), Image.BILINEAR)
125                 arr = np.asarray(pil, dtype=np.float32)
126         else:
127             arr = np.asarray(raw, dtype=np.uint8)
128             if max_dim and (arr.shape[0] > max_dim or arr.shape[1] > max_dim):

```

```

129         mode = "RGB" if arr.shape[-1] == 3 else "RGBA"
130         pil = Image.fromarray(arr, mode=mode)
131         pil.thumbnail((max_dim, max_dim), Image.LANCZOS)
132         arr = np.asarray(pil, dtype=np.uint8)
133
134         photometric = str(getattr(ds, "PhotometricInterpretation", "MONOCHROME2"))
135         default_wc, default_ww = default_window(ds)
136         _cache_put(cache_key, (arr, photometric, default_wc, default_ww, is_color))
137
138     if is_color:
139         out = np.clip(arr, 0, 255).astype(np.uint8)
140     else:
141         if wc is None:
142             wc = default_wc
143         if ww is None:
144             ww = default_ww
145         if wc is None or ww is None or ww <= 0:
146             lo = float(np.min(arr))
147             hi = float(np.max(arr))
148             if hi <= lo:
149                 hi = lo + 1.0
150             wc = (lo + hi) / 2.0
151             ww = hi - lo
152
153         lo = wc - ww / 2.0
154         hi = wc + ww / 2.0
155         out = np.clip(arr, lo, hi)
156         out = ((out - lo) / (hi - lo) * 255.0).astype(np.uint8)
157
158         if photometric == "MONOCHROME1":
159             out = 255 - out
160         if invert:
161             out = 255 - out
162
163     if out.ndim == 2:
164         img = Image.fromarray(out, mode="L")
165     elif out.ndim == 3 and out.shape[-1] == 3:
166         img = Image.fromarray(out, mode="RGB")
167     elif out.ndim == 3 and out.shape[-1] == 4:
168         img = Image.fromarray(out, mode="RGBA")
169     else:
170         img = Image.fromarray(out)
171
172     buf = io.BytesIO()
173     img.save(buf, format="PNG", optimize=False, compress_level=1)
174     return buf.getvalue()
175
176
177 def load_frame_array(path: Path, frame: int = 0) -> tuple[np.ndarray, Optional[tuple[float, float]]]:
178     """Return the modality-LUT-applied 2D pixel array (float32, full resolution)
179     for one frame plus pixel spacing ``(row_mm, col_mm)`` if present.
180
181     Unlike :func:`render_frame_png`, this performs no windowing and no
182     downsampling, so the array aligns 1:1 with Rows×Columns – required for
183     radiomics ROI extraction.
184     """
185     ds = pydicom.dcmread(str(path), force=True)
186     _ensure_transfer_syntax(ds)
187     if "PixelData" not in ds:
188         raise NoPixelDataError(
189             modality=str(getattr(ds, "Modality", "") or ""),
190             sop_class=str(getattr(ds, "SOPClassUID", "") or ""),
191         )
192     raw = ds.pixel_array
193     frames = get_frame_count(ds)
194     raw = _select_frame(raw, frame, frames)

```



```

195     try:
196         raw = apply_modality_lut(raw, ds)
197     except Exception:
198         pass
199     if raw.ndim == 3 and raw.shape[-1] in (3, 4):
200         # Color → luminance so radiomics works on a single channel.
201         raw = raw[..., :3].astype(np.float32)
202         raw = 0.299 * raw[..., 0] + 0.587 * raw[..., 1] + 0.114 * raw[..., 2]
203     arr = np.asarray(raw, dtype=np.float32)
204
205     spacing: Optional[tuple[float, float]] = None
206     ps = getattr(ds, "PixelSpacing", None) or getattr(ds, "ImagerPixelSpacing", None)
207     if ps is not None:
208         try:
209             spacing = (float(ps[0]), float(ps[1]))
210         except (TypeError, ValueError, IndexError):
211             spacing = None
212     return arr, spacing
213
214
215 _META_KEYS = [
216     "PatientName", "PatientID", "PatientSex", "PatientBirthDate", "PatientAge",
217     "StudyDate", "StudyTime", "StudyDescription", "StudyInstanceUID",
218     "SeriesDescription", "SeriesNumber", "SeriesInstanceUID",
219     "Modality", "Manufacturer", "ManufacturerModelName",
220     "BodyPartExamined", "ViewPosition", "ImageLaterality",
221     "Rows", "Columns", "BitsAllocated", "BitsStored", "PixelRepresentation",
222     "PhotometricInterpretation",
223     "WindowCenter", "WindowWidth",
224     "RescaleSlope", "RescaleIntercept",
225     "NumberOfFrames",
226     "PixelSpacing", "ImagerPixelSpacing",
227     "InstitutionName",
228     "AcquisitionDate", "AcquisitionTime",
229     "KVP", "ExposureTime", "XRayTubeCurrent",
230     "CompressionForce", "BodyPartThickness",
231     "TransferSyntaxUID",
232 ]
233
234
235 def read_metadata(path: Path) -> dict:
236     ds = pydicom.dcmread(str(path), stop_before_pixels=True, force=True)
237     out: dict = {}
238     for k in _META_KEYS:
239         v = getattr(ds, k, None)
240         if v is None:
241             continue
242         try:
243             if isinstance(v, (int, float)):
244                 out[k] = v
245             else:
246                 out[k] = str(v)
247         except Exception:
248             out[k] = repr(v)
249
250     ts = getattr(getattr(ds, "file_meta", None), "TransferSyntaxUID", None)
251     if ts is not None:
252         out["TransferSyntaxUID"] = str(ts)
253
254     return out
255
256
257 def extract_sr_content(path: Path) -> dict:
258     ds = pydicom.dcmread(str(path), force=True)
259     _ensure_transfer_syntax(ds)
260

```

```

261 def _concept_name(item) -> str:
262     try:
263         seq = getattr(item, "ConceptNameCodeSequence", None)
264         if seq and len(seq):
265             return str(getattr(seq[0], "CodeMeaning", "") or "")
266     except Exception:
267         pass
268     return ""
269
270 def _walk(item, depth: int = 0) -> list[dict]:
271     out: list[dict] = []
272     vt = str(getattr(item, "ValueType", "") or "")
273     name = _concept_name(item)
274     node: dict = {"depth": depth, "type": vt, "name": name}
275     if vt == "TEXT":
276         node["text"] = str(getattr(item, "TextValue", "") or "")
277     elif vt == "NUM":
278         try:
279             mv = item.MeasuredValueSequence[0]
280             val = str(getattr(mv, "NumericValue", ""))
281             unit = ""
282             u_seq = getattr(mv, "MeasurementUnitsCodeSequence", None)
283             if u_seq and len(u_seq):
284                 unit = str(getattr(u_seq[0], "CodeMeaning", "") or "")
285             node["value"] = f"{val} {unit}".strip()
286         except Exception:
287             node["value"] = ""
288     elif vt == "CODE":
289         try:
290             cs = item.ConceptCodeSequence[0]
291             node["value"] = str(getattr(cs, "CodeMeaning", "") or "")
292         except Exception:
293             node["value"] = ""
294     elif vt == "DATE":
295         node["value"] = str(getattr(item, "Date", "") or "")
296     elif vt == "PNAME":
297         node["value"] = str(getattr(item, "PersonName", "") or "")
298     out.append(node)
299
300     children = getattr(item, "ContentSequence", None)
301     if children:
302         for ch in children:
303             out.extend(_walk(ch, depth + 1))
304     return out
305
306 items: list[dict] = []
307 root_name = _concept_name(ds)
308 items.append({"depth": 0, "type": "ROOT", "name": root_name or "Structured Report"})
309 for child in getattr(ds, "ContentSequence", []) or []:
310     items.extend(_walk(child, 1))
311
312 return {
313     "modality": str(getattr(ds, "Modality", "") or ""),
314     "sop_class": str(getattr(ds, "SOPClassUID", "") or ""),
315     "completion_flag": str(getattr(ds, "CompletionFlag", "") or ""),
316     "verification_flag": str(getattr(ds, "VerificationFlag", "") or ""),
317     "patient_name": str(getattr(ds, "PatientName", "") or ""),
318     "patient_id": str(getattr(ds, "PatientID", "") or ""),
319     "content_date": str(getattr(ds, "ContentDate", "") or ""),
320     "items": items,
321 }
322
323
324 def quick_summary(path: Path) -> dict:
325     try:
326         ds = pydicom.dcmread(str(path), stop_before_pixels=True, force=True)

```

```

327     except Exception as e:
328         return {"error": str(e)}
329     rows = int(getattr(ds, "Rows", 0) or 0)
330     cols = int(getattr(ds, "Columns", 0) or 0)
331     has_pixels = bool(rows and cols) and (
332         "PixelData" in ds or "Rows" in ds
333     )
334     pir = getattr(ds, "PatientIdentityRemoved", None)
335     return {
336         "patient": str(getattr(ds, "PatientName", "")),
337         "patient_id": str(getattr(ds, "PatientID", "")),
338         "modality": str(getattr(ds, "Modality", "")),
339         "study_date": str(getattr(ds, "StudyDate", "")),
340         "view": str(getattr(ds, "ViewPosition", "")),
341         "laterality": str(getattr(ds, "ImageLaterality", "")),
342         "rows": rows,
343         "cols": cols,
344         "frames": get_frame_count(ds) if has_pixels else 0,
345         "has_pixels": has_pixels,
346         "deidentified": (str(pir).upper() == "YES") if pir is not None else False,
347     }

```

app/exporters.py

Fayl yo'li: app/exporters.py | Qatorlar: 290 | Hajm: 11148 bayt

```

1  """Dataset / annotation exporters: YOLO, Pascal VOC, CSV, and segmentation masks.
2
3  All builders are pure functions that take a normalized ``images`` list and return
4  ``bytes`` (or, for CSV, ``bytes`` of UTF-8 text). An ``image`` dict looks like::
5
6      {
7          "source": "upload" | "local",
8          "ref": "<id or relative path>",
9          "rows": int,          # pixel height (0 if unknown)
10         "cols": int,          # pixel width (0 if unknown)
11         "annotations": [ {bbox|points, label, labels, type, ...}, ... ],
12     }
13
14  Bounding boxes are normalized ``[x, y, w, h]`` (top-left + size, 0..1).
15  Polygons are ``points`` = ``[[x, y], ...]`` normalized.
16
17  Multi-label: an annotation may carry ``labels: [...]``; ``label`` is the primary
18  (first) label kept for backward compatibility. ``ann_labels()`` tolerates both.
19  """
20  from __future__ import annotations
21
22  import csv
23  import io
24  import os
25  import tempfile
26  import zipfile
27  from datetime import datetime, timezone
28  from xml.sax.saxutils import escape
29
30
31  def ann_labels(a: dict) -> list[str]:
32      """Return all labels for an annotation, tolerating old single-label records."""
33      ls = a.get("labels")
34      if isinstance(ls, list) and ls:
35          return [str(x) for x in ls if x]
36      lab = a.get("label")
37      return [str(lab)] if lab else []
38

```

```

39
40 def _safe_stem(ref: str) -> str:
41     """Turn a ref (which may be a path) into a flat, filesystem-safe stem."""
42     stem = str(ref or "annotation").replace("\\", "/").split("/")[-1]
43     if stem.lower().endswith(".dcm"):
44         stem = stem[:-4]
45     out = "".join(ch if (ch.isalnum() or ch in "-_.") else "_" for ch in stem)
46     return out or "annotation"
47
48
49 # ----- #
50 # YOLO #
51 # ----- #
52 def _yolo_yaml(label_names: list[str]) -> str:
53     names = "\n".join(f" {i}: {n}" for i, n in enumerate(label_names))
54     return (
55         "# YOLO dataset config generated by MAMOGRAF\n"
56         "path: .\n"
57         "train: images\n"
58         "val: images\n"
59         f"nc: {len(label_names)}\n"
60         "names:\n"
61         f"{names}\n"
62     )
63
64
65 def build_yolo_zip(images: list[dict], label_names: list[str]) -> bytes:
66     """YOLO detection dataset: one ``labels/<stem>.txt`` per image + classes/yaml.
67
68     Each line: ``<class> <cx> <cy> <w> <h>`` (all normalized). A multi-label
69     annotation produces one line per label (shared geometry).
70     """
71     idx = {name: i for i, name in enumerate(label_names)}
72     buf = io.BytesIO()
73     with zipfile.ZipFile(buf, "w", zipfile.ZIP_DEFLATED) as z:
74         z.writestr("classes.txt", "\n".join(label_names) + "\n")
75         z.writestr("data.yaml", _yolo_yaml(label_names))
76         z.writestr("README.txt",
77             "YOLO detection format. Each labels/<image>.txt line:\n"
78             "<class_index> <x_center> <y_center> <width> <height> (normalized 0..1)\n")
79         for im in images:
80             lines: list[str] = []
81             for a in im.get("annotations", []):
82                 bbox = a.get("bbox")
83                 if not bbox or len(bbox) != 4:
84                     continue
85                 x, y, w, h = bbox
86                 cx, cy = x + w / 2.0, y + h / 2.0
87                 for lab in ann_labels(a):
88                     if lab in idx:
89                         lines.append(f"{idx[lab]} {cx:.6f} {cy:.6f} {w:.6f} {h:.6f}")
90             stem = _safe_stem(im.get("ref"))
91             z.writestr(f"labels/{stem}.txt", ("\n".join(lines) + "\n") if lines else "")
92     return buf.getvalue()
93
94
95 # ----- #
96 # Pascal VOC #
97 # ----- #
98 def _voc_xml(im: dict) -> str:
99     rows = int(im.get("rows") or 0)
100     cols = int(im.get("cols") or 0)
101     ref = im.get("ref") or "annotation"
102     stem = _safe_stem(ref)
103     objs: list[str] = []
104     for a in im.get("annotations", []):

```

```

105     bbox = a.get("bbox")
106     if not bbox or len(bbox) != 4:
107         continue
108     x, y, w, h = bbox
109     if cols and rows:
110         xmin = int(round(x * cols)); ymin = int(round(y * rows))
111         xmax = int(round((x + w) * cols)); ymax = int(round((y + h) * rows))
112     else:
113         # No pixel dims known: fall back to a 1000-unit canvas so coords stay usable.
114         xmin = int(round(x * 1000)); ymin = int(round(y * 1000))
115         xmax = int(round((x + w) * 1000)); ymax = int(round((y + h) * 1000))
116     for lab in ann_labels(a):
117         objs.append(
118             "<object>\n"
119             f"    <name>{escape(lab)}</name>\n"
120             "    <pose>Unspecified</pose>\n"
121             "    <truncated>0</truncated>\n"
122             "    <difficult>0</difficult>\n"
123             "    <bndbox>\n"
124             f"        <xmin>{xmin}</xmin>\n"
125             f"        <ymin>{ymin}</ymin>\n"
126             f"        <xmax>{xmax}</xmax>\n"
127             f"        <ymax>{ymax}</ymax>\n"
128             "    </bndbox>\n"
129             "  </object>"
130         )
131     return (
132         "<annotation>\n"
133         f"    <folder>{escape(str(im.get('source') or ''))}</folder>\n"
134         f"    <filename>{escape(stem)}.dcm</filename>\n"
135         f"    <path>{escape(str(ref))}</path>\n"
136         "    <source><database>MAMOGRAF</database></source>\n"
137         "    <size>\n"
138         f"        <width>{cols}</width>\n"
139         f"        <height>{rows}</height>\n"
140         "        <depth>1</depth>\n"
141         "    </size>\n"
142         "    <segmented>0</segmented>\n"
143         + ("\\n".join(objs) + "\\n" if objs else "")
144         + "</annotation>\n"
145     )
146
147
148 def build_voc_zip(images: list[dict]) -> bytes:
149     """Pascal VOC: one XML per image. Multi-label → multiple ``<object>`` blocks."""
150     buf = io.BytesIO()
151     with zipfile.ZipFile(buf, "w", zipfile.ZIP_DEFLATED) as z:
152         for im in images:
153             z.writestr(f"{_safe_stem(im.get('ref'))}.xml", _voc_xml(im))
154     return buf.getvalue()
155
156
157 # ----- #
158 # CSV (one row per annotation, multi-label column) #
159 # ----- #
160 CSV_HEADER = [
161     "source", "ref", "annotation_id", "type",
162     "label", "labels", "bi_rads", "status", "frame",
163     "x_norm", "y_norm", "w_norm", "h_norm",
164     "x_px", "y_px", "w_px", "h_px", "points_count",
165     "created_by", "reviewed_by", "note",
166 ]
167
168
169 def build_csv(images: list[dict]) -> bytes:
170     buf = io.StringIO()

```

```

171 w = csv.writer(buf)
172 w.writerow(CSV_HEADER)
173 for im in images:
174     rows = int(im.get("rows") or 0)
175     cols = int(im.get("cols") or 0)
176     for a in im.get("annotations", []):
177         bbox = (list(a.get("bbox") or []) + [0, 0, 0, 0])[:4]
178         x, y, wn, hn = bbox
179         labs = ann_labels(a)
180         if rows and cols:
181             px = [round(x * cols, 1), round(y * rows, 1),
182                  round(wn * cols, 1), round(hn * rows, 1)]
183         else:
184             px = ["", "", "", ""]
185         pts = a.get("points") or []
186         w.writerow([
187             im.get("source", ""), im.get("ref", ""), a.get("id", ""),
188             a.get("type", "bbox"),
189             labs[0] if labs else "", ";".join(labs),
190             a.get("bi_rads", ""), a.get("status", "draft"), a.get("frame", 0),
191             x, y, wn, hn, px[0], px[1], px[2], px[3],
192             len(pts) if a.get("type") == "polygon" else "",
193             a.get("created_by", ""), a.get("reviewed_by", ""),
194             str(a.get("note") or "").replace("\n", " ").replace("\r", " "),
195         ])
196     # BOM so Excel opens UTF-8 correctly.
197     return (" " + buf.getvalue()).encode("utf-8")
198
199
200 # ----- #
201 # Segmentation masks (PNG / NIFTI) – per image #
202 # ----- #
203 def build_label_mask(annotations: list[dict], rows: int, cols: int,
204                     label_index: dict[str, int]):
205     """Return a 2D uint8/uint16 label-indexed mask (0 = background).
206
207     On overlap the later annotation wins. Multi-label annotations are painted
208     with their primary (first) label's class index.
209     """
210     import numpy as np
211
212     if not rows or not cols:
213         raise ValueError("image dimensions unknown; cannot build mask")
214     max_cls = max(label_index.values()) if label_index else 0
215     dtype = np.uint8 if max_cls < 256 else np.uint16
216     mask = np.zeros((int(rows), int(cols)), dtype=dtype)
217
218     try:
219         import cv2
220         has_cv2 = True
221     except Exception:
222         has_cv2 = False
223     from PIL import Image, ImageDraw
224
225     if not has_cv2:
226         pil = Image.fromarray(mask)
227         draw = ImageDraw.Draw(pil)
228
229     for a in annotations:
230         labs = ann_labels(a)
231         if not labs:
232             continue
233         cls = label_index.get(labs[0])
234         if not cls:
235             continue
236         if a.get("type") == "polygon" and a.get("points"):

```

```

237     poly = [(float(px) * cols, float(py) * rows) for px, py in a["points"]]
238     if has_cv2:
239         pts = np.array([[int(round(px)), int(round(py))] for px, py in poly],
240                         dtype=np.int32)
241         cv2.fillPoly(mask, [pts], int(cls))
242     else:
243         draw.polygon([(int(round(px)), int(round(py))) for px, py in poly],
244                      fill=int(cls))
245     else:
246         bbox = a.get("bbox")
247         if not bbox or len(bbox) != 4:
248             continue
249         x, y, w, h = bbox
250         x0, y0 = int(round(x * cols)), int(round(y * rows))
251         x1, y1 = int(round((x + w) * cols)), int(round((y + h) * rows))
252         if has_cv2:
253             cv2.rectangle(mask, (x0, y0), (x1, y1), int(cls), thickness=-1)
254         else:
255             draw.rectangle([x0, y0, x1, y1], fill=int(cls))
256
257     if not has_cv2:
258         import numpy as np
259         mask = np.array(pil, dtype=dtype)
260     return mask
261
262
263 def mask_to_png_bytes(mask) -> bytes:
264     from PIL import Image
265     arr = mask
266     mode = "L" if arr.dtype.itemsize == 1 else "I;16"
267     img = Image.fromarray(arr, mode=mode)
268     bio = io.BytesIO()
269     img.save(bio, format="PNG")
270     return bio.getvalue()
271
272
273 def mask_to_nifti_bytes(mask) -> bytes:
274     import nibabel as nib
275     import numpy as np
276
277     arr = np.asarray(mask)
278     if arr.ndim == 2:
279         arr = arr[:, :, None]
280     img = nib.Nifti1Image(arr.astype(np.uint16), affine=np.eye(4))
281     with tempfile.TemporaryDirectory() as td:
282         fp = os.path.join(td, "mask.nii.gz")
283         nib.save(img, fp)
284         with open(fp, "rb") as f:
285             return f.read()
286
287
288 def mask_legend(label_index: dict[str, int]) -> dict[int, str]:
289     """Map class index → label name (useful as a sidecar JSON)."""
290     return {v: k for k, v in label_index.items()}

```

app/import_worklist.py

Fayl yo'li: app/import_worklist.py | Qatorlar: 111 | Hajm: 3438 bayt

```

1 from __future__ import annotations
2
3 import argparse
4 import csv
5 import sys

```

```

6 import time
7 from datetime import datetime, timezone
8 from pathlib import Path
9 from typing import Optional
10
11 from .db import get_conn, init_db
12
13
14 def _parse_date(s: Optional[str]) -> Optional[str]:
15     if not s:
16         return None
17     s = s.strip()
18     if not s:
19         return None
20     for fmt in ("%m/%d/%Y", "%d/%m/%Y", "%Y-%m-%d", "%Y/%m/%d"):
21         try:
22             return datetime.strptime(s, fmt).date().isoformat()
23         except ValueError:
24             continue
25     return None
26
27
28 def _parse_bool(s: Optional[str]) -> bool:
29     if not s:
30         return False
31     return s.strip().upper() in ("TRUE", "1", "YES", "Y")
32
33
34 def import_worklist(path: Path, reset: bool = False) -> dict:
35     init_db()
36     if reset:
37         with get_conn() as c:
38             c.execute("DELETE FROM worklist")
39             c.commit()
40
41     t0 = time.perf_counter()
42     inserted = 0
43     skipped = 0
44     now = datetime.now(timezone.utc).isoformat()
45
46     with path.open("r", encoding="utf-8-sig", newline="") as f:
47         reader = csv.DictReader(f)
48         rows_to_insert = []
49         for r_idx, row in enumerate(reader, start=2):
50             pid = (row.get("ID") or "").strip()
51             if not pid:
52                 skipped += 1
53                 continue
54             rows_to_insert.append((
55                 pid,
56                 (row.get("Name(DICOM)") or "").strip() or None,
57                 (row.get("Sex") or "").strip() or None,
58                 _parse_date(row.get("DOB")),
59                 _parse_date(row.get("Study Date")),
60                 (row.get("Modality") or "").strip() or None,
61                 int(_parse_bool(row.get("File"))),
62                 int(_parse_bool(row.get("Report"))),
63                 (row.get("Report Status") or "").strip() or None,
64                 None,
65                 "normal",
66                 "pending",
67                 None,
68                 now,
69                 r_idx,
70             ))
71

```



```

72     with get_conn() as c:
73         c.executemany(
74             "INSERT OR REPLACE INTO worklist "
75             "(patient_id, patient_name, sex, dob, study_date, modality, "
76             "has_file, has_report, report_status, assigned_to, priority, "
77             "status, notes, imported_at, source_row) "
78             "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
79             rows_to_insert,
80         )
81         inserted = len(rows_to_insert)
82         c.commit()
83
84     return {
85         "path": str(path),
86         "inserted_or_replaced": inserted,
87         "skipped_no_id": skipped,
88         "elapsed_s": round(time.perf_counter() - t0, 2),
89     }
90
91
92 def main():
93     ap = argparse.ArgumentParser(description="Import Worklist CSV into worklist table")
94     default_path = Path(__file__).resolve().parent.parent / ".." / "dicomfiles" /
95     "Worklist_20260220_1236_test23.csv"
96     ap.add_argument("path", nargs="?", default=str(default_path),
97                     help="path to worklist CSV (default looks in dicomfiles/)")
98     ap.add_argument("--reset", action="store_true",
99                     help="drop existing worklist rows before import")
100    args = ap.parse_args()
101    p = Path(args.path)
102    if not p.exists():
103        print(f"not found: {p}", file=sys.stderr)
104        sys.exit(1)
105    result = import_worklist(p, reset=args.reset)
106    width = max(len(k) for k in result)
107    for k, v in result.items():
108        print(f" {k.ljust(width)} : {v}")
109
110 if __name__ == "__main__":
111     main()

```

app/import_xlsx.py

Fayl yo'li: app/import_xlsx.py | Qatorlar: 198 | Hajm: 6428 bayt

```

1  from __future__ import annotations
2
3  import argparse
4  import sys
5  import time
6  from datetime import date, datetime, timedelta, timezone
7  from pathlib import Path
8
9  from openpyxl import load_workbook
10
11 from .db import DB_PATH, get_conn, init_db
12
13 XLSX_TO_DB = {
14     "hasta_adi": "first_name",
15     "soyadi": "last_name",
16     "kimlik_no": "patient_id",
17     "jins": "sex",
18     "tugilgan": "birth_date",

```

```

19     "hizmettarihi": "service_date",
20     "kod_ara": "exam_code",
21     "tetkik_ismi": "exam_name",
22     "onay_tarihi": "approval_date",
23     "Mamologiya Report": "report",
24     "hikayesi": "history",
25     "sikayeti": "complaints",
26     "klinik_bulgular": "clinical_findings",
27     "oneriler": "recommendations",
28     "hastalikhakkindabilgi": "disease_info",
29     "tedavi": "treatment",
30     "laboratuvar_notu": "lab_notes",
31     "taburcu_ilaclari": "discharge_meds",
32     "mesleki_anemnez": "occupational",
33     "gelis_nedeni": "admission_reason",
34     "tani": "diagnosis",
35     "diagnostik_uygulamalar": "diagnostic_procedures",
36 }
37
38 RECORD_FIELDS = [
39     "source_row", "patient_id",
40     "service_date", "approval_date",
41     "exam_code", "exam_name",
42     "report", "history", "complaints", "clinical_findings",
43     "recommendations", "disease_info", "treatment", "lab_notes",
44     "discharge_meds", "occupational", "admission_reason",
45     "diagnosis", "diagnostic_procedures",
46     "imported_at",
47 ]
48
49 EXCEL_EPOCH = datetime(1899, 12, 30)
50
51
52 def to_text(v):
53     if v is None:
54         return None
55     if isinstance(v, str):
56         s = v.strip()
57         return s or None
58     return str(v).strip() or None
59
60
61 def to_date(v):
62     if v is None or v == "":
63         return None
64     if isinstance(v, datetime):
65         return v.date().isoformat()
66     if isinstance(v, date):
67         return v.isoformat()
68     try:
69         s = float(v)
70     except (TypeError, ValueError):
71         return None
72     if s <= 0 or s > 80000:
73         return None
74     try:
75         return (EXCEL_EPOCH + timedelta(days=s)).date().isoformat()
76     except OverflowError:
77         return None
78
79
80 def import_xlsx(path: Path, reset: bool = False) -> dict:
81     init_db()
82     if reset:
83         with get_conn() as c:
84             c.execute("DELETE FROM records")

```

```

85         c.execute("DELETE FROM patients")
86         c.commit()
87
88     t0 = time.perf_counter()
89     wb = load_workbook(str(path), read_only=True, data_only=True)
90     ws = wb.active
91
92     rows_iter = ws.iter_rows(values_only=True)
93     header = next(rows_iter, None)
94     if header is None:
95         wb.close()
96         raise ValueError("xlsx is empty")
97     col_idx = {name: i for i, name in enumerate(header) if name}
98
99     missing_cols = [k for k in XLSX_TO_DB if k not in col_idx]
100    if missing_cols:
101        print(f"[warn] missing columns in xlsx: {missing_cols}", file=sys.stderr)
102
103    def cell(row, col_name):
104        i = col_idx.get(col_name)
105        if i is None or i >= len(row):
106            return None
107        return row[i]
108
109    patients: dict[str, tuple] = {}
110    records: list[dict] = []
111    skipped_empty = 0
112    now = datetime.now(timezone.utc).isoformat()
113
114    for r_idx, row in enumerate(rows_iter, start=2):
115        patient_id = to_text(cell(row, "kimlik_no"))
116        if not patient_id:
117            skipped_empty += 1
118            continue
119
120        if patient_id not in patients:
121            patients[patient_id] = (
122                to_text(cell(row, "hasta_adi")),
123                to_text(cell(row, "soyadi")),
124                to_text(cell(row, "jins")),
125                to_date(cell(row, "tugilgan")),
126            )
127
128        rec = {
129            "source_row": r_idx,
130            "patient_id": patient_id,
131            "service_date": to_date(cell(row, "hizmettarihi")),
132            "approval_date": to_date(cell(row, "onay_tarihi")),
133            "exam_code": to_text(cell(row, "kod_ara")),
134            "exam_name": to_text(cell(row, "tetkik_ismi")),
135            "report": to_text(cell(row, "Mamologiya Report")),
136            "history": to_text(cell(row, "hikayesi")),
137            "complaints": to_text(cell(row, "sikayeti")),
138            "clinical_findings": to_text(cell(row, "klinik_bulgular")),
139            "recommendations": to_text(cell(row, "oneriler")),
140            "disease_info": to_text(cell(row, "hastalikhakkindabilgi")),
141            "treatment": to_text(cell(row, "tedavi")),
142            "lab_notes": to_text(cell(row, "laboratuvar_notu")),
143            "discharge_meds": to_text(cell(row, "taburcu_ilaclari")),
144            "occupational": to_text(cell(row, "mesleki_anemnez")),
145            "admission_reason": to_text(cell(row, "gelis_nedeni")),
146            "diagnosis": to_text(cell(row, "tani")),
147            "diagnostic_procedures": to_text(cell(row, "diagnostik_uygulamalar")),
148            "imported_at": now,
149        }
150        records.append(rec)

```

```

151
152     wb.close()
153
154     with get_conn() as c:
155         c.executemany(
156             "INSERT OR REPLACE INTO patients "
157             "(patient_id, first_name, last_name, sex, birth_date) VALUES (?, ?, ?, ?, ?)",
158             [(pid, *vals) for pid, vals in patients.items()],
159         )
160         if records:
161             placeholders = ",".join("?" * len(RECORD_FIELDS))
162             c.executemany(
163                 f"INSERT OR REPLACE INTO records ({','.join(RECORD_FIELDS)}) "
164                 f"VALUES ({placeholders})",
165                 [(tuple(r[k] for k in RECORD_FIELDS) for r in records),
166             ]
167         )
168         c.commit()
169
170     return {
171         "path": str(path),
172         "patients_unique": len(patients),
173         "records_inserted": len(records),
174         "skipped_empty_rows": skipped_empty,
175         "elapsed_s": round(time.perf_counter() - t0, 2),
176         "db_path": str(DB_PATH),
177     }
178
179 def main():
180     ap = argparse.ArgumentParser(description="Import MamologiyaInfo.xlsx into SQLite.")
181     default_path = Path(__file__).resolve().parent.parent / "MamologiyaInfo.xlsx"
182     ap.add_argument("path", nargs="?", default=str(default_path),
183                     help="path to xlsx file (default: ../MamologiyaInfo.xlsx)")
184     ap.add_argument("--reset", action="store_true",
185                     help="drop existing rows before import")
186     args = ap.parse_args()
187     p = Path(args.path)
188     if not p.exists():
189         print(f"not found: {p}", file=sys.stderr)
190         sys.exit(1)
191     result = import_xlsx(p, reset=args.reset)
192     width = max(len(k) for k in result)
193     for k, v in result.items():
194         print(f" {k.ljust(width)} : {v}")
195
196
197 if __name__ == "__main__":
198     main()

```

app/inference.py

Fayl yo'li: app/inference.py | Qatorlar: 255 | Hajm: 8232 bayt

```

1 from __future__ import annotations
2
3 import io
4 import threading
5 from pathlib import Path
6 from typing import Optional
7
8 from PIL import Image
9
10 MODELS_DIR = Path(__file__).resolve().parent / "models"
11 MODELS_DIR.mkdir(exist_ok=True)

```

```

12
13 _model_cache: dict[str, object] = {}
14 _cache_lock = threading.Lock()
15
16
17 def is_available() -> tuple[bool, str]:
18     try:
19         import ultralytics # noqa: F401
20     except ImportError as e:
21         return False, f"ultralytics not installed: {e}"
22     return True, ""
23
24
25 def device_info() -> dict:
26     try:
27         import torch
28         cuda_ok = bool(torch.cuda.is_available())
29         return {
30             "torch": torch.__version__,
31             "cuda_available": cuda_ok,
32             "device": "cuda:0" if cuda_ok else "cpu",
33             "device_name": torch.cuda.get_device_name(0) if cuda_ok else "CPU",
34         }
35     except ImportError:
36         return {"torch": None, "cuda_available": False, "device": "cpu", "device_name": "CPU"}
37
38
39 def list_models() -> list[dict]:
40     out = []
41     for p in sorted(MODELS_DIR.glob("*.pt")):
42         try:
43             size = p.stat().st_size
44         except OSError:
45             size = 0
46         out.append({"name": p.name, "stem": p.stem, "size_bytes": size})
47     return out
48
49
50 def _resolve_model_path(name: str) -> Path:
51     safe = Path(name).name
52     if not safe.endswith(".pt"):
53         safe += ".pt"
54     p = MODELS_DIR / safe
55     if not p.exists() or not p.is_file():
56         raise FileNotFoundError(f"model weights not found: {safe}")
57     return p
58
59
60 def get_model(name: str):
61     p = _resolve_model_path(name)
62     key = str(p)
63     with _cache_lock:
64         cached = _model_cache.get(key)
65         if cached is not None:
66             return cached
67         from ultralytics import YOLO
68         model = YOLO(str(p))
69         _model_cache[key] = model
70     return model
71
72
73 def infer_png(
74     png_bytes: bytes,
75     model_name: str,
76     conf: float = 0.25,
77     iou: float = 0.5,

```

```

78     imgsz: int = 1024,
79     tta: bool = False,
80 ) -> dict:
81     ok, err = is_available()
82     if not ok:
83         raise RuntimeError(err)
84
85     model = get_model(model_name)
86
87     img = Image.open(io.BytesIO(png_bytes))
88     if img.mode == "L":
89         img = img.convert("RGB")
90     elif img.mode != "RGB":
91         img = img.convert("RGB")
92
93     img_w, img_h = img.size
94
95     results = model.predict(
96         source=img,
97         conf=conf,
98         iou=iou,
99         imgsz=imgsz,
100        augment=tta,          # test-time augmentation (multi-scale + flips)
101        verbose=False,
102    )
103
104    detections = []
105    if results:
106        r = results[0]
107        names = r.names if hasattr(r, "names") else getattr(model, "names", {})
108        boxes = getattr(r, "boxes", None)
109        if boxes is not None and len(boxes) > 0:
110            xyxy = boxes.xyxy.cpu().numpy()
111            confs = boxes.conf.cpu().numpy()
112            cls = boxes.cls.cpu().numpy().astype(int)
113            for (x1, y1, x2, y2), c, k in zip(xyxy, confs, cls):
114                w = max(0.0, float(x2 - x1))
115                h = max(0.0, float(y2 - y1))
116                detections.append({
117                    "label": str(names.get(int(k), int(k))) if isinstance(names, dict) else
118                    str(int(k)),
119                    "class_id": int(k),
120                    "confidence": float(c),
121                    "bbox": [
122                        float(x1) / img_w,
123                        float(y1) / img_h,
124                        w / img_w,
125                        h / img_h,
126                    ],
127                    "bbox_pixels": [float(x1), float(y1), w, h],
128                })
129
130    return {
131        "model": model_name,
132        "image_size": [img_w, img_h],
133        "device": device_info().get("device", "cpu"),
134        "detections": detections,
135        "params": {"conf": conf, "iou": iou, "imgsz": imgsz, "tta": tta},
136    }
137
138 # ----- #
139 # Weighted Boxes Fusion + ensemble #
140 # ----- #
141 def _iou_xyxy(a, b) -> float:
142     ax1, ay1, ax2, ay2 = a

```

```

143     bx1, by1, bx2, by2 = b
144     ix1, iy1 = max(ax1, bx1), max(ay1, by1)
145     ix2, iy2 = min(ax2, bx2), min(ay2, by2)
146     iw, ih = max(0.0, ix2 - ix1), max(0.0, iy2 - iy1)
147     inter = iw * ih
148     if inter <= 0:
149         return 0.0
150     area_a = max(0.0, ax2 - ax1) * max(0.0, ay2 - ay1)
151     area_b = max(0.0, bx2 - bx1) * max(0.0, by2 - by1)
152     return inter / (area_a + area_b - inter + 1e-9)
153
154
155 def weighted_boxes_fusion(per_source_dets: list[list[dict]], iou_thr: float = 0.55,
156                           num_sources: Optional[int] = None) -> list[dict]:
157     """Fuse detections from several sources (models or TTA passes).
158
159     Each detection is ``{label, class_id, confidence, bbox=[x,y,w,h], ...}``
160     in normalised coords. Boxes of the same label that overlap (IoU > thr) are
161     merged into one box whose coordinates are score-weighted averages and whose
162     score reflects cross-source agreement (canonical WBF).
163     """
164     if num_sources is None:
165         num_sources = len(per_source_dets)
166     num_sources = max(1, num_sources)
167
168     # Flatten to xyxy with label key.
169     items = []
170     for dets in per_source_dets:
171         for d in dets:
172             x, y, w, h = d["bbox"]
173             items.append({
174                 "label": d.get("label"),
175                 "class_id": d.get("class_id"),
176                 "score": float(d.get("confidence", 0.0)),
177                 "box": [x, y, x + w, y + h],
178             })
179     items.sort(key=lambda it: it["score"], reverse=True)
180
181     clusters: list[dict] = [] # each: {label, boxes:[...], scores:[...], fused:[xyxy]}
182     for it in items:
183         placed = False
184         for cl in clusters:
185             if cl["label"] == it["label"] and _iou_xyxy(cl["fused"], it["box"]) > iou_thr:
186                 cl["boxes"].append(it["box"])
187                 cl["scores"].append(it["score"])
188                 cl["class_id"] = it["class_id"]
189                 w = sum(cl["scores"])
190                 cl["fused"] = [
191                     sum(b[i] * s for b, s in zip(cl["boxes"], cl["scores"])) / w
192                     for i in range(4)
193                 ]
194                 placed = True
195                 break
196         if not placed:
197             clusters.append({
198                 "label": it["label"], "class_id": it["class_id"],
199                 "boxes": [it["box"]], "scores": [it["score"]], "fused": list(it["box"]),
200             })
201
202     out = []
203     for cl in clusters:
204         n = len(cl["scores"])
205         # WBF score: mean confidence rescaled by agreement across sources.
206         score = (sum(cl["scores"]) / n) * min(1.0, n / num_sources)
207         x1, y1, x2, y2 = cl["fused"]
208         out.append({

```

```

209         "label": cl["label"],
210         "class_id": cl["class_id"],
211         "confidence": float(score),
212         "bbox": [x1, y1, max(0.0, x2 - x1), max(0.0, y2 - y1)],
213         "n_models": n,
214     })
215     out.sort(key=lambda d: d["confidence"], reverse=True)
216     return out
217
218
219 def infer_ensemble(
220     png_bytes: bytes,
221     model_names: list[str],
222     conf: float = 0.25,
223     iou: float = 0.5,
224     imgsz: int = 1024,
225     tta: bool = False,
226     wbf_iou: float = 0.55,
227 ) -> dict:
228     """Run several models and fuse their detections with Weighted Boxes Fusion."""
229     ok, err = is_available()
230     if not ok:
231         raise RuntimeError(err)
232
233     per_model: list[list[dict]] = []
234     used: list[str] = []
235     img_w = img_h = 0
236     for name in model_names:
237         try:
238             r = infer_png(png_bytes, name, conf=conf, iou=iou, imgsz=imgsz, tta=tta)
239         except FileNotFoundError:
240             continue
241         used.append(name)
242         img_w, img_h = r["image_size"]
243         per_model.append(r["detections"])
244     if not used:
245         raise FileNotFoundError("no usable models for ensemble")
246
247     fused = weighted_boxes_fusion(per_model, iou_thr=wbf_iou, num_sources=len(used))
248     return {
249         "model": "ensemble:" + ",".join(used),
250         "ensemble_models": used,
251         "image_size": [img_w, img_h],
252         "device": device_info().get("device", "cpu"),
253         "detections": fused,
254         "params": {"conf": conf, "iou": iou, "imgsz": imgsz, "tta": tta, "wbf_iou": wbf_iou},
255     }

```

app/manage_users.py

Fayl yo'li: app/manage_users.py | Qatorlar: 97 | Hajm: 3170 bayt

```

1  from __future__ import annotations
2
3  import argparse
4  import getpass
5  import secrets
6  import sys
7  from pathlib import Path
8
9  from . import auth as auth_mod
10 from . import db as db_mod
11
12

```



```

13 def cmd_create(args):
14     db_mod.init_db()
15     if auth_mod.get_user_by_username(args.username):
16         print(f"error: user '{args.username}' already exists", file=sys.stderr)
17         sys.exit(1)
18     password = args.password or getpass.getpass(f"Password for {args.username}: ")
19     if not password:
20         print("error: empty password", file=sys.stderr)
21         sys.exit(1)
22     user = auth_mod.create_user(
23         username=args.username,
24         password=password,
25         role=args.role,
26         display_name=args.display_name,
27         email=args.email,
28     )
29     print(f"created user id={user['id']} username={user['username']} role={user['role']}")
30
31
32 def cmd_passwd(args):
33     db_mod.init_db()
34     if not auth_mod.get_user_by_username(args.username):
35         print(f"error: user '{args.username}' not found", file=sys.stderr)
36         sys.exit(1)
37     password = args.password or getpass.getpass(f"New password for {args.username}: ")
38     if not password:
39         print("error: empty password", file=sys.stderr)
40         sys.exit(1)
41     auth_mod.update_password(args.username, password)
42     print(f"password updated for {args.username}")
43
44
45 def cmd_rotate_secret(_args):
46     secret_file = Path(__file__).resolve().parent / ".jwt_secret"
47     new_secret = secrets.token_urlsafe(48)
48     secret_file.write_text(new_secret, encoding="utf-8")
49     print(f"yangi JWT secret yozildi: {secret_file}")
50     print("Diqqat: barcha mavjud sessiyalar bekor qilindi. Server restart qiling.")
51
52
53 def cmd_list(_args):
54     db_mod.init_db()
55     users = auth_mod.list_users()
56     if not users:
57         print("(no users)")
58         return
59     fmt = "{:>3} {:<20} {:<10} {:<25} {:<30} {"
60     print(fmt.format("id", "username", "role", "display_name", "email", "last_login"))
61     for u in users:
62         print(fmt.format(
63             u["id"], u["username"], u["role"],
64             u.get("display_name") or "", u.get("email") or "",
65             u.get("last_login_at") or "-",
66         ))
67
68
69 def main():
70     ap = argparse.ArgumentParser(description="Manage users for MAMOGRAF DICOM Viewer")
71     sub = ap.add_subparsers(dest="cmd", required=True)
72
73     p = sub.add_parser("create", help="create new user")
74     p.add_argument("username")
75     p.add_argument("--role", required=True, choices=auth_mod.ROLES)
76     p.add_argument("--display-name", dest="display_name")
77     p.add_argument("--email")
78     p.add_argument("--password", help="(optional) provide on cmdline; otherwise prompted")

```

```

79     p.set_defaults(func=cmd_create)
80
81     p = sub.add_parser("passwd", help="change user password")
82     p.add_argument("username")
83     p.add_argument("--password", help="(optional)")
84     p.set_defaults(func=cmd_passwd)
85
86     p = sub.add_parser("list", help="list all users")
87     p.set_defaults(func=cmd_list)
88
89     p = sub.add_parser("rotate-secret", help="JWT secret'ni yangilash (barcha sessiyalar bekor)")
90     p.set_defaults(func=cmd_rotate_secret)
91
92     args = ap.parse_args()
93     args.func(args)
94
95
96 if __name__ == "__main__":
97     main()

```

app/mwl_scu.py

Fayl yo'li: *app/mwl_scu.py* | Qatorlar: 228 | Hajm: 7700 bayt

```

1  """
2  Modality Worklist (MWL) SCU CLI.
3
4  Real DICOM modality worklist serveridan ish ro'yxatini olib, lokal `worklist`
5  jadvaliga import qiladi (CSV import o'rniga).
6
7  Foydalanish:
8      pip install pynetdicom
9      python -m app.mwl_scu --host 192.168.1.100 --port 4242 \
10         --aet ORTHANC --calling MAMOGRAF_SCU --modality MG \
11         --date today
12
13  Quvvatlanadigan filtrlar:
14      --modality MG | CT | MR | ...
15      --date today | YYYYMMDD | YYYYMMDD-YYYYMMDD
16      --patient-id <ID>
17      --station-aet <SCHEDULED_STATION_AE>
18
19  `pynetdicom` o'rnatilmagan bo'lsa CLI ishlaymaydi (lazy import). Production'da
20  uni alohida o'rnatish tavsiya etiladi (qariyb 30MB, CI'da kerak emas).
21  """
22  from __future__ import annotations
23
24  import argparse
25  import sys
26  from datetime import datetime, timezone
27  from typing import Optional
28
29  from .db import get_conn, init_db
30
31
32  def _parse_date_arg(s: str) -> str:
33      s = s.strip().lower()
34      if not s or s == "any":
35          return ""
36      if s == "today":
37          return datetime.now().strftime("%Y%m%d")
38      if "-" in s and len(s.replace("-", "")) == 16:
39          return s
40      if len(s) == 8 and s.isdigit():

```

```

41         return s
42     return s
43
44
45 def query(
46     host: str,
47     port: int,
48     aet: str,
49     calling_aet: str = "MAMOGRAF_SCU",
50     modality: str = "",
51     study_date: str = "",
52     patient_id: str = "",
53     station_aet: str = "",
54     timeout: int = 30,
55 ) -> list[dict]:
56     try:
57         from pydicom.dataset import Dataset
58         from pynetdicom import AE
59         from pynetdicom.sop_class import ModalityWorklistInformationFind
60     except ImportError as e:
61         raise RuntimeError(
62             "pynetdicom installed kerak: pip install pynetdicom"
63         ) from e
64
65     ae = AE(ae_title=calling_aet)
66     ae.add_requested_context(ModalityWorklistInformationFind)
67     ae.acse_timeout = timeout
68     ae.dimse_timeout = timeout
69
70     ds = Dataset()
71     ds.PatientName = ""
72     ds.PatientID = patient_id or ""
73     ds.PatientBirthDate = ""
74     ds.PatientSex = ""
75     ds.AccessionNumber = ""
76     ds.RequestedProcedureID = ""
77
78     sps = Dataset()
79     sps.Modality = modality or ""
80     sps.ScheduledStationAETitle = station_aet or ""
81     sps.ScheduledProcedureStepStartDate = study_date or ""
82     sps.ScheduledProcedureStepStartTime = ""
83     sps.ScheduledProcedureStepDescription = ""
84     sps.ScheduledPerformingPhysicianName = ""
85     ds.ScheduledProcedureStepSequence = [sps]
86
87     print(f"[mwl] connecting to {host}:{port} (AET {aet})...")
88     assoc = ae.associate(host, port, ae_title=aet)
89     if not assoc.is_established:
90         raise RuntimeError(f"failed to associate with {host}:{port} ({aet})")
91
92     found: list[dict] = []
93     try:
94         responses = assoc.send_c_find(ds, ModalityWorklistInformationFind)
95         for status, identifier in responses:
96             if status is None:
97                 continue
98             if status.Status in (0xFF00, 0xFF01) and identifier:
99                 rec = _identifier_to_record(identifier)
100                 if rec:
101                     found.append(rec)
102     finally:
103         assoc.release()
104
105     return found
106

```

```

107
108 def _identifier_to_record(identifier) -> Optional[dict]:
109     pid = str(getattr(identifier, "PatientID", "") or "").strip()
110     if not pid:
111         return None
112     name = str(getattr(identifier, "PatientName", "") or "").replace("^", " ").strip()
113     sex = str(getattr(identifier, "PatientSex", "") or "").strip()
114     dob = _format_dicom_date(getattr(identifier, "PatientBirthDate", ""))
115
116     sps_seq = getattr(identifier, "ScheduledProcedureStepSequence", None)
117     modality = ""
118     study_date = ""
119     if sps_seq:
120         sps = sps_seq[0]
121         modality = str(getattr(sps, "Modality", "") or "").strip()
122         study_date = _format_dicom_date(
123             getattr(sps, "ScheduledProcedureStepStartDate", "")
124         )
125
126     return {
127         "patient_id": pid,
128         "patient_name": name or None,
129         "sex": sex or None,
130         "dob": dob,
131         "study_date": study_date,
132         "modality": modality or None,
133         "has_file": 0,
134         "has_report": 0,
135         "report_status": None,
136     }
137
138
139 def _format_dicom_date(value) -> Optional[str]:
140     s = str(value or "").strip()
141     if len(s) == 8 and s.isdigit():
142         return f"{s[:4]}-{s[4:6]}-{s[6:8]}"
143     return s or None
144
145
146 def import_into_worklist(rows: list[dict]) -> int:
147     init_db()
148     if not rows:
149         return 0
150     now = datetime.now(timezone.utc).isoformat()
151     inserted = 0
152     with get_conn() as c:
153         for r in rows:
154             cur = c.execute(
155                 "SELECT id FROM worklist WHERE patient_id=? AND study_date=? AND modality=?",
156                 (r["patient_id"], r["study_date"], r["modality"]),
157             ).fetchone()
158             if cur:
159                 c.execute(
160                     "UPDATE worklist SET patient_name=?, sex=?, dob=?, "
161                     "has_file=?, has_report=?, report_status=?, imported_at=? "
162                     "WHERE id=?",
163                     (r["patient_name"], r["sex"], r["dob"],
164                     r["has_file"], r["has_report"], r["report_status"],
165                     now, cur[0]),
166                 )
167             else:
168                 c.execute(
169                     "INSERT INTO worklist "
170                     "(patient_id, patient_name, sex, dob, study_date, modality, "
171                     "has_file, has_report, report_status, priority, status, imported_at) "
172                     "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",

```

```

173         (r["patient_id"], r["patient_name"], r["sex"], r["dob"],
174          r["study_date"], r["modality"],
175          r["has_file"], r["has_report"], r["report_status"],
176          "normal", "pending", now),
177         )
178         inserted += 1
179     c.commit()
180     return inserted
181
182
183 def main():
184     ap = argparse.ArgumentParser(description="MWL SCU - DICOM worklist'ni server'dan olib import qilish")
185     ap.add_argument("--host", required=True, help="MWL server host/IP")
186     ap.add_argument("--port", required=True, type=int, help="MWL server portu")
187     ap.add_argument("--aet", required=True, help="MWL server AE Title (called)")
188     ap.add_argument("--calling", default="MAMOGRAF_SCU", help="bizning AE Title (calling)")
189     ap.add_argument("--modality", default="", help="masalan, MG / CT / MR")
190     ap.add_argument("--date", default="", help="today | YYYYMMDD | YYYYMMDD-YYYYMMDD")
191     ap.add_argument("--patient-id", default="", help="aniq bemor ID")
192     ap.add_argument("--station-aet", default="", help="ScheduledStationAETitle")
193     ap.add_argument("--dry-run", action="store_true", help="DB ga yozmaydi, faqat ko'rsatadi")
194     ap.add_argument("--timeout", type=int, default=30)
195     args = ap.parse_args()
196
197     try:
198         rows = query(
199             host=args.host,
200             port=args.port,
201             aet=args.aet,
202             calling_aet=args.calling,
203             modality=args.modality,
204             study_date=parse_date_arg(args.date),
205             patient_id=args.patient_id,
206             station_aet=args.station_aet,
207             timeout=args.timeout,
208         )
209     except Exception as e:
210         print(f"[xato] {e}", file=sys.stderr)
211         sys.exit(1)
212
213     print(f"[mwl] {len(rows)} ta yozuv topildi")
214     for r in rows[:10]:
215         print(f"  {r['patient_id']:<12} {r['patient_name']:<25} {r['modality']:<5} {r['study_date']}")
216     if len(rows) > 10:
217         print(f"  ... va yana {len(rows) - 10} ta")
218
219     if args.dry_run:
220         print("[dry-run] DB ga yozilmadi")
221         return
222
223     inserted = import_into_worklist(rows)
224     print(f"[mwl] worklist jadvaliga {inserted} ta yangi yozuv qo'shildi (qolgan {len(rows) - inserted} ta yangilandi)")
225
226
227 if __name__ == "__main__":
228     main()

```

app/pacs.py

Fayl yo'li: app/pacs.py | Qatorlar: 266 | Hajm: 10343 bayt

```
1 """
```

```

2 PACS DICOM SCU CLI: C-STORE (yuborish), C-FIND (qidirish), C-MOVE (olish).
3
4 Foydalanish:
5     pip install pynetdicom
6
7     # PACS'ga DICOM yuborish
8     python -m app.pacs send --host 192.168.1.10 --port 4242 --aet ORTHANC \\
9         path/to/file1.dcm path/to/file2.dcm
10
11     # Bemor bo'yicha qidirish (C-FIND)
12     python -m app.pacs query --host 192.168.1.10 --port 4242 --aet ORTHANC \\
13         --patient-id 12345 --modality MG
14
15     # Study'ni olish (C-MOVE – server bizga uzatadi)
16     python -m app.pacs fetch --host 192.168.1.10 --port 4242 --aet ORTHANC \\
17         --study-uid 1.2.3.4.5 --dest ./fetched/
18 """
19 from __future__ import annotations
20
21 import argparse
22 import sys
23 from pathlib import Path
24 from typing import Optional
25
26
27 def _import_pynet():
28     try:
29         from pydicom.dataset import Dataset
30         from pynetdicom import AE, evt, StoragePresentationContexts, debug_logger, Verification
31         from pynetdicom.sop_class import (
32             PatientRootQueryRetrieveInformationModelFind,
33             PatientRootQueryRetrieveInformationModelMove,
34             StudyRootQueryRetrieveInformationModelFind,
35             StudyRootQueryRetrieveInformationModelMove,
36             Verification as VerificationSOP,
37         )
38         return {
39             "Dataset": Dataset, "AE": AE, "evt": evt,
40             "StoragePresentationContexts": StoragePresentationContexts,
41             "debug_logger": debug_logger,
42             "Verification": VerificationSOP,
43             "PatientFind": PatientRootQueryRetrieveInformationModelFind,
44             "PatientMove": PatientRootQueryRetrieveInformationModelMove,
45             "StudyFind": StudyRootQueryRetrieveInformationModelFind,
46             "StudyMove": StudyRootQueryRetrieveInformationModelMove,
47         }
48     except ImportError as e:
49         raise RuntimeError("pynetdicom kerak: pip install pynetdicom") from e
50
51
52 def echo(host: str, port: int, aet: str,
53         calling_aet: str = "MAMOGRAF_SCU", timeout: int = 10) -> dict:
54     pn = _import_pynet()
55     ae = pn["AE"](ae_title=calling_aet)
56     ae.add_requested_context(pn["Verification"])
57     ae.acse_timeout = timeout
58     ae.dimse_timeout = timeout
59     assoc = ae.associate(host, port, ae_title=aet)
60     if not assoc.is_established:
61         raise RuntimeError(f"associate failed: {host}:{port} ({aet})")
62     try:
63         status = assoc.send_c_echo()
64         ok = status is not None and status.Status == 0x0000
65         return {"ok": ok, "status": status.Status if status else None}
66     finally:
67         assoc.release()

```

```

68
69
70 def store(host: str, port: int, aet: str, files: list[Path],
71           calling_aet: str = "MAMOGRAF SCU", timeout: int = 60) -> dict:
72     pn = _import_pynet()
73     import pydicom
74
75     ae = pn["AE"](ae_title=calling_aet)
76     for ctx in pn["StoragePresentationContexts"]:
77         ae.add_requested_context(ctx.abstract_syntax)
78     ae.acse_timeout = timeout
79     ae.dimse_timeout = timeout
80
81     sent = 0
82     failed = 0
83     print(f"[pacs] connecting to {host}:{port} (AET {aet})...")
84     assoc = ae.associate(host, port, ae_title=aet)
85     if not assoc.is_established:
86         raise RuntimeError(f"associate failed: {host}:{port} ({aet})")
87     try:
88         for path in files:
89             try:
90                 ds = pydicom.dcmread(str(path), force=True)
91                 status = assoc.send_c_store(ds)
92                 if status and status.Status == 0x0000:
93                     sent += 1
94                     print(f"  ✓ {path.name}")
95                 else:
96                     failed += 1
97                     code = status.Status if status else "no-status"
98                     print(f"  X {path.name} status=0x{code:04X}" if isinstance(code, int) else f"  X
{path.name} {code}")
99             except Exception as e:
100                 failed += 1
101                 print(f"  X {path.name}: {e}")
102     finally:
103         assoc.release()
104     return {"sent": sent, "failed": failed}
105
106
107 def query(host: str, port: int, aet: str,
108           patient_id: str = "", patient_name: str = "",
109           modality: str = "", study_date: str = "",
110           calling_aet: str = "MAMOGRAF SCU", timeout: int = 30) -> list[dict]:
111     pn = _import_pynet()
112     Dataset = pn["Dataset"]
113     ae = pn["AE"](ae_title=calling_aet)
114     ae.add_requested_context(pn["StudyFind"])
115     ae.acse_timeout = timeout
116     ae.dimse_timeout = timeout
117
118     ds = Dataset()
119     ds.QueryRetrieveLevel = "STUDY"
120     ds.PatientID = patient_id
121     ds.PatientName = patient_name
122     ds.PatientBirthDate = ""
123     ds.PatientSex = ""
124     ds.StudyInstanceUID = ""
125     ds.StudyDate = study_date
126     ds.StudyDescription = ""
127     ds.AccessionNumber = ""
128     ds.ModalitiesInStudy = modality
129
130     print(f"[pacs] C-FIND on {host}:{port}...")
131     assoc = ae.associate(host, port, ae_title=aet)
132     if not assoc.is_established:

```

```

133         raise RuntimeError(f"associate failed: {host}:{port} ({aet})")
134     results = []
135     try:
136         for status, identifier in assoc.send_c_find(ds, pn["StudyFind"]):
137             if status and status.Status in (0xFF00, 0xFF01) and identifier:
138                 results.append({
139                     "patient_id": str(getattr(identifier, "PatientID", "") or ""),
140                     "patient_name": str(getattr(identifier, "PatientName", "") or "").replace("^", "
141 ),
142                     "study_uid": str(getattr(identifier, "StudyInstanceUID", "") or ""),
143                     "study_date": str(getattr(identifier, "StudyDate", "") or ""),
144                     "modalities": str(getattr(identifier, "ModalitiesInStudy", "") or ""),
145                     "description": str(getattr(identifier, "StudyDescription", "") or ""),
146                     "accession": str(getattr(identifier, "AccessionNumber", "") or ""),
147                 })
148     finally:
149         assoc.release()
150     return results
151
152 def fetch(host: str, port: int, aet: str, study_uid: str, dest: Path,
153         calling_aet: str = "MAMOGRAF_SCU", scp_port: int = 11112,
154         timeout: int = 120) -> dict:
155     pn = _import_pynet()
156     Dataset = pn["Dataset"]
157     evt = pn["evt"]
158     dest.mkdir(parents=True, exist_ok=True)
159     received: list[str] = []
160
161     def handle_store(event):
162         ds = event.dataset
163         ds.file_meta = event.file_meta
164         sop = getattr(ds, "SOPInstanceUID", "instance")
165         out = dest / f"{sop}.dcm"
166         try:
167             ds.save_as(str(out), write_like_original=False)
168             received.append(out.name)
169             print(f" ← {out.name}")
170         except Exception as e:
171             print(f" save xato: {e}")
172         return 0xC001
173     return 0x0000
174
175     ae = pn["AE"](ae_title=calling_aet)
176     ae.add_requested_context(pn["StudyMove"])
177     for ctx in pn["StoragePresentationContexts"]:
178         ae.add_supported_context(ctx.abstract_syntax)
179     ae.acse_timeout = timeout
180     ae.dimse_timeout = timeout
181
182     handlers = [(evt.EVT_C_STORE, handle_store)]
183     print(f"[pacs] starting local SCP on port {scp_port}...")
184     scp = ae.start_server("0.0.0.0", scp_port, block=False, evt_handlers=handlers,
185                         ae_title=calling_aet)
186     try:
187         ds = Dataset()
188         ds.QueryRetrieveLevel = "STUDY"
189         ds.StudyInstanceUID = study_uid
190         print(f"[pacs] C-MOVE study={study_uid} -> {calling_aet} ...")
191         assoc = ae.associate(host, port, ae_title=aet)
192         if not assoc.is_established:
193             raise RuntimeError(f"associate failed: {host}:{port}")
194     try:
195         for status, identifier in assoc.send_c_move(ds, calling_aet, pn["StudyMove"]):
196             if status and status.Status not in (0xFF00, 0x0000):
197                 print(f" status=0x{status.Status:04X}")

```



```

198         finally:
199             assoc.release()
200     finally:
201         scp.shutdown()
202     return {"received": received, "count": len(received), "dest": str(dest)}
203
204
205 def _parse_files(args) -> list[Path]:
206     out: list[Path] = []
207     for f in args.files:
208         p = Path(f)
209         if not p.exists():
210             print(f"fail topilmadi: {p}", file=sys.stderr); sys.exit(1)
211         if p.is_dir():
212             out.extend(sorted(p.rglob("*.dcm")))
213         else:
214             out.append(p)
215     return out
216
217
218 def main():
219     ap = argparse.ArgumentParser(description="PACS DICOM SCU CLI")
220     sub = ap.add_subparsers(dest="cmd", required=True)
221
222     common = argparse.ArgumentParser(add_help=False)
223     common.add_argument("--host", required=True)
224     common.add_argument("--port", required=True, type=int)
225     common.add_argument("--aet", required=True, help="PACS server AE title")
226     common.add_argument("--calling", default="MAMOGRAF_SCU")
227     common.add_argument("--timeout", type=int, default=60)
228
229     s = sub.add_parser("send", parents=[common], help="DICOM faylni PACS'ga yuborish (C-STORE)")
230     s.add_argument("files", nargs="+", help="fayl yoki papka yo'llari")
231     s.set_defaults(func=lambda a: print(store(a.host, a.port, a.aet, _parse_files(a),
232                                     a.calling, a.timeout)))
233
234     q = sub.add_parser("query", parents=[common], help="qidirish (C-FIND)")
235     q.add_argument("--patient-id", default="")
236     q.add_argument("--patient-name", default="")
237     q.add_argument("--modality", default="")
238     q.add_argument("--study-date", default="")
239     def _q(a):
240         results = query(a.host, a.port, a.aet,
241                        a.patient_id, a.patient_name,
242                        a.modality, a.study_date,
243                        a.calling, a.timeout)
244         print(f"[pacs] {len(results)} ta study topildi")
245         for r in results[:30]:
246             print(f"  {r['patient_id']:<10} {r['patient_name']:<25} {r['study_date']:<10}
247 {r['modalities']:<10} {r['study_uid']}")
248         if len(results) > 30:
249             print(f"  ... va yana {len(results)-30}")
250     q.set_defaults(func=_q)
251
252     f = sub.add_parser("fetch", parents=[common], help="study yoki study'larni olish (C-MOVE)")
253     f.add_argument("--study-uid", required=True)
254     f.add_argument("--dest", required=True)
255     f.add_argument("--scp-port", type=int, default=11112)
256     f.set_defaults(func=lambda a: print(fetch(a.host, a.port, a.aet, a.study_uid,
257                                     Path(a.dest), a.calling, a.scp_port, a.timeout)))
258
259     args = ap.parse_args()
260     try:
261         args.func(args)
262     except RuntimeError as e:
263         print(f"[xato] {e}", file=sys.stderr); sys.exit(1)

```

```

263
264
265 if __name__ == "__main__":
266     main()

```

app/radiomics.py

Fayl yo'li: app/radiomics.py | Qatorlar: 418 | Hajm: 15875 bayt

```

1  """IBSI-inspired 2D radiomics feature extraction (self-contained).
2
3  Extracts quantitative features from a region of interest (ROI) drawn on a
4  single DICOM frame. Six feature families are computed:
5
6  * first-order (intensity statistics)
7  * shape (2D morphology)
8  * GLCM - gray-level co-occurrence matrix (texture)
9  * GLRLM - gray-level run-length matrix
10 * GLSZM - gray-level size-zone matrix
11 * NGTDM - neighbouring gray-tone difference matrix
12
13 The implementation depends only on numpy / scipy / scikit-image (no
14 PyRadiomics), so it installs cleanly on modern Python. Intensities inside the
15 ROI are discretised to a fixed number of bins for the texture matrices.
16
17 Note: research/quantitative use. Values are reproducible but this module is
18 not a certified IBSI reference implementation.
19 """
20 from __future__ import annotations
21
22 import math
23 from typing import Optional
24
25 import numpy as np
26 from scipy import ndimage
27 from skimage.draw import polygon as sk_polygon
28 from skimage.measure import label as sk_label
29 from skimage.measure import regionprops
30
31 EPS = 1e-9
32
33
34 # ----- #
35 # ROI mask #
36 # ----- #
37 def roi_mask_from_annotation(ann: dict, rows: int, cols: int) -> np.ndarray:
38     """Build a boolean ROI mask (rows×cols) from a bbox or polygon annotation.
39
40     Coordinates in annotations are normalised (0..1).
41     """
42     mask = np.zeros((rows, cols), dtype=bool)
43     if ann.get("type") == "polygon" and ann.get("points"):
44         ys = np.array([min(max(p[1], 0.0), 1.0) * (rows - 1) for p in ann["points"]])
45         xs = np.array([min(max(p[0], 0.0), 1.0) * (cols - 1) for p in ann["points"]])
46         rr, cc = sk_polygon(ys, xs, shape=(rows, cols))
47         mask[rr, cc] = True
48     else:
49         bbox = ann.get("bbox")
50         if not bbox or len(bbox) != 4:
51             return mask
52         x, y, w, h = bbox
53         x0 = int(round(max(0.0, x) * cols))
54         y0 = int(round(max(0.0, y) * rows))
55         x1 = int(round(min(1.0, x + w) * cols))

```

```

56     y1 = int(round(min(1.0, y + h) * rows))
57     x1 = max(x1, x0 + 1)
58     y1 = max(y1, y0 + 1)
59     mask[y0:min(y1, rows), x0:min(x1, cols)] = True
60     return mask
61
62
63 def _discretise(values: np.ndarray, bins: int) -> tuple[np.ndarray, float, float]:
64     mn = float(values.min())
65     mx = float(values.max())
66     if mx <= mn:
67         return np.ones_like(values, dtype=int), mn, mx
68     lvl = np.floor((values - mn) / (mx - mn) * (bins - 1)).astype(int) + 1
69     lvl = np.clip(lvl, 1, bins)
70     return lvl, mn, mx
71
72
73 # ----- #
74 # First-order #
75 # ----- #
76 def _first_order(vals: np.ndarray, bins: int, voxel_area: float) -> dict:
77     n = vals.size
78     mean = float(vals.mean())
79     var = float(vals.var())
80     std = math.sqrt(var)
81     diffs = vals - mean
82     m3 = float((diffs ** 3).mean())
83     m4 = float((diffs ** 4).mean())
84     skew = m3 / (std ** 3 + EPS)
85     kurt = m4 / (var ** 2 + EPS)
86     p10, p25, p50, p75, p90 = (float(np.percentile(vals, q)) for q in (10, 25, 50, 75, 90))
87     rng = float(vals.max() - vals.min())
88     mad = float(np.abs(diffs).mean())
89     robust = vals[(vals >= p10) & (vals <= p90)]
90     rmad = float(np.abs(robust - robust.mean()).mean()) if robust.size else 0.0
91     energy = float((vals ** 2).sum())
92     rms = math.sqrt(float((vals ** 2).mean()))
93
94     disc, _, _ = _discretise(vals, bins)
95     counts = np.bincount(disc, minlength=bins + 1)[1:]
96     p = counts / max(counts.sum(), 1)
97     nz = p[p > 0]
98     entropy = float(-(nz * np.log2(nz)).sum())
99     uniformity = float((p ** 2).sum())
100
101     return {
102         "voxel_count": float(n),
103         "mean": mean,
104         "median": p50,
105         "minimum": float(vals.min()),
106         "maximum": float(vals.max()),
107         "range": rng,
108         "variance": var,
109         "std_dev": std,
110         "skewness": skew,
111         "kurtosis": kurt,
112         "p10": p10,
113         "p90": p90,
114         "interquartile_range": p75 - p25,
115         "mean_abs_deviation": mad,
116         "robust_mean_abs_deviation": rmad,
117         "energy": energy,
118         "total_energy": energy * voxel_area,
119         "rms": rms,
120         "entropy": entropy,
121         "uniformity": uniformity,

```

```

122     }
123
124
125 # ----- #
126 # Shape (2D) #
127 # ----- #
128 def _rp(prop, *names):
129     """Read a regionprops attribute, tolerating skimage's renamed properties."""
130     for n in names:
131         try:
132             return float(getattr(prop, n))
133         except Exception:
134             continue
135     return 0.0
136
137
138 def _shape(mask: np.ndarray, spacing: tuple[float, float]) -> dict:
139     sy, sx = spacing
140     lab = mask.astype(int)
141     props = regionprops(lab)
142     if not props:
143         return {}
144     p = max(props, key=lambda r: r.area)
145     area_px = float(p.area)
146     perim_px = float(p.perimeter) if p.perimeter else 0.0
147     mean_sp = (sy + sx) / 2.0
148     circularity = (4 * math.pi * area_px) / (perim_px ** 2) if perim_px > 0 else 0.0
149     feret = _rp(p, "feret_diameter_max") * mean_sp
150     major = _rp(p, "axis_major_length", "major_axis_length")
151     minor = _rp(p, "axis_minor_length", "minor_axis_length")
152     eqdiam = _rp(p, "equivalent_diameter_area", "equivalent_diameter")
153     return {
154         "pixel_area": area_px,
155         "physical_area_mm2": area_px * sy * sx,
156         "perimeter_px": perim_px,
157         "physical_perimeter_mm": perim_px * mean_sp,
158         "circularity": float(circularity),
159         "equivalent_diameter_mm": eqdiam * mean_sp,
160         "major_axis_length_mm": major * mean_sp,
161         "minor_axis_length_mm": minor * mean_sp,
162         "elongation": float(minor / (major + EPS)),
163         "eccentricity": float(p.eccentricity),
164         "orientation_rad": float(p.orientation),
165         "extent": float(p.extent),
166         "solidity": float(p.solidity),
167         "max_feret_mm": feret,
168     }
169
170
171 # ----- #
172 # GLCM #
173 # ----- #
174 _GLCM_DIRS = [(0, 1), (-1, 1), (-1, 0), (-1, -1)]
175
176
177 def _glcm(disc: np.ndarray, ng: int) -> dict:
178     P = np.zeros((ng, ng), dtype=np.float64)
179     H, W = disc.shape
180     for dy, dx in _GLCM_DIRS:
181         ys0, ys1 = max(0, -dy), H - max(0, dy)
182         xs0, xs1 = max(0, -dx), W - max(0, dx)
183         A = disc[ys0:ys1, xs0:xs1]
184         B = disc[ys0 + dy:ys1 + dy, xs0 + dx:xs1 + dx]
185         valid = (A > 0) & (B > 0)
186         ai = A[valid] - 1
187         bi = B[valid] - 1

```

```

188     np.add.at(P, (ai, bi), 1)
189     np.add.at(P, (bi, ai), 1) # symmetric
190     total = P.sum()
191     if total <= 0:
192         return {k: 0.0 for k in ("contrast", "dissimilarity", "homogeneity",
193                                 "asm", "energy", "entropy", "correlation")}
194     P /= total
195     idx = np.arange(1, ng + 1)
196     I, J = np.meshgrid(idx, idx, indexing="ij")
197     diff = I - J
198     contrast = float((P * diff ** 2).sum())
199     dissim = float((P * np.abs(diff)).sum())
200     homog = float((P / (1.0 + diff ** 2)).sum())
201     asm = float((P ** 2).sum())
202     nz = P[P > 0]
203     entropy = float(-(nz * np.log2(nz)).sum())
204     mu_i = float((P * I).sum())
205     mu_j = float((P * J).sum())
206     sig_i = math.sqrt(float((P * (I - mu_i) ** 2).sum()))
207     sig_j = math.sqrt(float((P * (J - mu_j) ** 2).sum()))
208     corr = float((P * (I - mu_i) * (J - mu_j)).sum() / (sig_i * sig_j)) if sig_i > 0 and sig_j > 0 else
0.0
209     return {
210         "contrast": contrast,
211         "dissimilarity": dissim,
212         "homogeneity": homog,
213         "asm": asm,
214         "energy": math.sqrt(asm),
215         "entropy": entropy,
216         "correlation": corr,
217     }
218
219
220 # ----- #
221 # GLRLM #
222 # ----- #
223 def _iter_lines(disc: np.ndarray):
224     H, W = disc.shape
225     for r in range(H): # horizontal
226         yield disc[r, :]
227     for c in range(W): # vertical
228         yield disc[:, c]
229     for off in range(-H + 1, W): # main diagonal
230         yield np.diagonal(disc, offset=off)
231     flipped = disc[:, ::-1]
232     for off in range(-H + 1, W): # anti-diagonal
233         yield np.diagonal(flipped, offset=off)
234
235
236 def _glrlm(disc: np.ndarray, ng: int, np_voxels: int) -> dict:
237     max_len = max(disc.shape)
238     R = np.zeros((ng, max_len), dtype=np.float64)
239     for line in _iter_lines(disc):
240         if line.size == 0:
241             continue
242         prev = line[0]
243         run = 1
244         for v in line[1:]:
245             if v == prev and v > 0:
246                 run += 1
247             else:
248                 if prev > 0:
249                     R[prev - 1, run - 1] += 1
250                 prev = v
251                 run = 1
252         if prev > 0:

```

```

253         R[prev - 1, run - 1] += 1
254
255     nr = R.sum()
256     if nr <= 0:
257         return {k: 0.0 for k in ("short_run_emphasis", "long_run_emphasis",
258                                 "gray_level_nonuniformity", "run_length_nonuniformity",
259                                 "run_percentage", "low_gray_level_run_emphasis",
260                                 "high_gray_level_run_emphasis")}
261
262     g = np.arange(1, ng + 1)[:, None]
263     r = np.arange(1, max_len + 1)[None, :]
264     sre = float((R / r ** 2).sum() / nr)
265     lre = float((R * r ** 2).sum() / nr)
266     gln = float((R.sum(axis=1) ** 2).sum() / nr)
267     rln = float((R.sum(axis=0) ** 2).sum() / nr)
268     rp = float(nr / max(np_voxels, 1))
269     lglre = float((R / g ** 2).sum() / nr)
270     hglre = float((R * g ** 2).sum() / nr)
271     return {
272         "short_run_emphasis": sre,
273         "long_run_emphasis": lre,
274         "gray_level_nonuniformity": gln,
275         "run_length_nonuniformity": rln,
276         "run_percentage": rp,
277         "low_gray_level_run_emphasis": lglre,
278         "high_gray_level_run_emphasis": hglre,
279     }
280
281 # ----- #
282 # GLSZM #
283 # ----- #
284 def _glzm(disc: np.ndarray, ng: int, np_voxels: int) -> dict:
285     zones: dict[tuple[int, int], int] = {}
286     max_size = 1
287     struct = np.ones((3, 3), dtype=int) # 8-connectivity
288     for gl in range(1, ng + 1):
289         bw = disc == gl
290         if not bw.any():
291             continue
292         lab, n = ndimage.label(bw, structure=struct)
293         if n == 0:
294             continue
295         sizes = np.bincount(lab.ravel())[1:]
296         for s in sizes:
297             s = int(s)
298             zones[(gl, s)] = zones.get((gl, s), 0) + 1
299             max_size = max(max_size, s)
300
301     nz = sum(zones.values())
302     if nz <= 0:
303         return {k: 0.0 for k in ("small_area_emphasis", "large_area_emphasis",
304                                 "gray_level_nonuniformity", "zone_size_nonuniformity",
305                                 "zone_percentage", "low_gray_level_zone_emphasis",
306                                 "high_gray_level_zone_emphasis")}
307
308     Z = np.zeros((ng, max_size), dtype=np.float64)
309     for (gl, s), c in zones.items():
310         Z[gl - 1, s - 1] += c
311     g = np.arange(1, ng + 1)[:, None]
312     s = np.arange(1, max_size + 1)[None, :]
313     return {
314         "small_area_emphasis": float((Z / s ** 2).sum() / nz),
315         "large_area_emphasis": float((Z * s ** 2).sum() / nz),
316         "gray_level_nonuniformity": float((Z.sum(axis=1) ** 2).sum() / nz),
317         "zone_size_nonuniformity": float((Z.sum(axis=0) ** 2).sum() / nz),
318         "zone_percentage": float(nz / max(np_voxels, 1)),
319         "low_gray_level_zone_emphasis": float((Z / g ** 2).sum() / nz),

```

```

319         "high_gray_level_zone_emphasis": float((Z * g ** 2).sum() / nz),
320     }
321
322
323 # ----- #
324 # NGTDM #
325 # ----- #
326 def _ngtmdm(disc: np.ndarray, mask: np.ndarray, ng: int) -> dict:
327     kernel = np.ones((3, 3), dtype=np.float64)
328     kernel[1, 1] = 0.0
329     m = mask.astype(np.float64)
330     vals = disc.astype(np.float64) * m
331     neigh_sum = ndimage.convolve(vals, kernel, mode="constant", cval=0.0)
332     neigh_cnt = ndimage.convolve(m, kernel, mode="constant", cval=0.0)
333     valid = mask & (neigh_cnt > 0)
334     avg = np.zeros_like(vals)
335     avg[valid] = neigh_sum[valid] / neigh_cnt[valid]
336
337     s = np.zeros(ng + 1)
338     nvec = np.zeros(ng + 1)
339     gi = disc[valid]
340     diff = np.abs(gi - avg[valid])
341     np.add.at(s, gi, diff)
342     np.add.at(nvec, gi, 1.0)
343     nv = nvec.sum()
344     if nv <= 0:
345         return {k: 0.0 for k in ("coarseness", "contrast", "busyness",
346                                 "complexity", "strength")}
347     p = nvec / nv
348     present = np.where(nvec > 0)[0]
349     ngp = present.size
350
351     coarseness = 1.0 / (float((p * s).sum()) + EPS)
352
353     contrast = 0.0
354     busy_den = 0.0
355     complexity = 0.0
356     strength = 0.0
357     for i in present:
358         for j in present:
359             contrast += p[i] * p[j] * (i - j) ** 2
360             busy_den += abs(i * p[i] - j * p[j])
361             denom = p[i] + p[j]
362             if denom > 0:
363                 complexity += abs(i - j) * (p[i] * s[i] + p[j] * s[j]) / denom
364                 strength += (p[i] + p[j]) * (i - j) ** 2
365     contrast = (contrast / (ngp * (ngp - 1) + EPS)) * (float(s.sum()) / nv)
366     busyness = float((p * s).sum()) / (busy_den + EPS)
367     complexity = complexity / nv
368     strength = strength / (float(s.sum()) + EPS)
369     return {
370         "coarseness": float(coarseness),
371         "contrast": float(contrast),
372         "busyness": float(busyness),
373         "complexity": float(complexity),
374         "strength": float(strength),
375     }
376
377
378 # ----- #
379 # Public API #
380 # ----- #
381 def extract(image: np.ndarray, mask: np.ndarray,
382            spacing: Optional[tuple[float, float]] = None,
383            bins: int = 32) -> dict:
384     """Compute all feature families for one ROI.

```

```

385
386 Returns ``{"shape": {...}, "first_order": {...}, "glcm": {...},
387 "glrlm": {...}, "glszm": {...}, "ngtdm": {...}}``.
388 """
389 if image.shape != mask.shape:
390     raise ValueError(f"image {image.shape} and mask {mask.shape} shape mismatch")
391 if not mask.any():
392     raise ValueError("empty ROI mask")
393 sp = spacing if spacing else (1.0, 1.0)
394
395 ys, xs = np.where(mask)
396 y0, y1 = ys.min(), ys.max() + 1
397 x0, x1 = xs.min(), xs.max() + 1
398 sub_img = image[y0:y1, x0:x1]
399 sub_mask = mask[y0:y1, x0:x1]
400 roi_vals = sub_img[sub_mask].astype(np.float64)
401
402 disc_vals, _, _ = _discretise(roi_vals, bins)
403 disc = np.zeros(sub_img.shape, dtype=int)
404 disc[sub_mask] = disc_vals
405 np_voxels = int(sub_mask.sum())
406
407 return {
408     "shape": _shape(mask, sp),
409     "first_order": _first_order(roi_vals, bins, sp[0] * sp[1]),
410     "glcm": _glcm(disc, bins),
411     "glrlm": _glrlm(disc, bins, np_voxels),
412     "glszm": _glszm(disc, bins, np_voxels),
413     "ngtdm": _ngtdm(disc, sub_mask, bins),
414 }
415
416
417 def feature_count(result: dict) -> int:
418     return sum(len(v) for v in result.values())

```

app/ws.py

Fayl yo'li: app/ws.py | Qatorlar: 61 | Hajm: 1831 bayt

```

1 from __future__ import annotations
2
3 import asyncio
4 from collections import defaultdict
5 from typing import Optional
6
7 from fastapi import WebSocket
8
9
10 class RoomManager:
11     def __init__(self):
12         self._rooms: dict[str, set[tuple[WebSocket, str]]] = defaultdict(set)
13         self._lock = asyncio.Lock()
14
15     @staticmethod
16     def room_id(source: str, ref: str) -> str:
17         return f"{source}::{ref}"
18
19     async def join(self, source: str, ref: str, ws: WebSocket, username: str):
20         rid = self.room_id(source, ref)
21         async with self._lock:
22             self._rooms[rid].add((ws, username))
23         await self.broadcast_presence(source, ref)
24
25     async def leave(self, source: str, ref: str, ws: WebSocket, username: str):

```



```

26         rid = self.room_id(source, ref)
27         async with self._lock:
28             self._rooms[rid].discard((ws, username))
29             if not self._rooms[rid]:
30                 del self._rooms[rid]
31         await self.broadcast_presence(source, ref)
32
33     def members(self, source: str, ref: str) -> list[str]:
34         rid = self.room_id(source, ref)
35         return sorted({u for _, u in self._rooms.get(rid, set())})
36
37     async def broadcast(
38         self,
39         source: str,
40         ref: str,
41         message: dict,
42         exclude_ws: Optional[WebSocket] = None,
43     ):
44         rid = self.room_id(source, ref)
45         recipients = list(self._rooms.get(rid, set()))
46         for ws, _ in recipients:
47             if ws is exclude_ws:
48                 continue
49             try:
50                 await ws.send_json(message)
51             except Exception:
52                 pass
53
54     async def broadcast_presence(self, source: str, ref: str):
55         await self.broadcast(source, ref, {
56             "type": "presence",
57             "users": self.members(source, ref),
58         })
59
60
61 rooms = RoomManager()

```

3. Frontend — veb UI (app/static/)

app/static/index.html

Fayl yo'li: app/static/index.html | Qatorlar: 325 | Hajm: 13991 bayt

```
1 <!doctype html>
2 <html lang="uz">
3 <head>
4 <meta charset="utf-8" />
5 <meta name="viewport" content="width=device-width, initial-scale=1" />
6 <title>MAMOGRAF DICOM Viewer</title>
7 <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100
100'%3E%3Ctext y='.9em' font-size='90'%3E%3C/text%3E%3C/svg%3E" />
8 <link rel="stylesheet" href="/style.css" />
9 </head>
10 <body>
11 <header>
12 <button class="mobile-toggle" id="mobileSidebarBtn" title="Chap panel">≡</button>
13 <h1>MAMOGRAF</h1>
14 <div class="hint" id="status">tayyor</div>
15 <button class="mobile-toggle" id="mobileMetaBtn" title="O'ng panel">☰</button>
16 <div class="user-bar" id="userBar">
17 <span id="userInfo" hidden></span>
18 <span class="bell-wrap" id="bellWrap" hidden>
19 <button id="bellBtn" title="Bildirishnomalar">🔔<span class="bell-badge" id="bellBadge"
hidden>0</span></button>
20 <div class="bell-dropdown" id="bellDropdown" hidden></div>
21 </span>
22 <button id="overviewBtn" hidden title="Annotatsiya qilingan DICOM'lar (kart ko'rinish)">📄</button>
23 <button id="auditBtn" hidden title="Audit timeline">📅</button>
24 <button id="reviewBtn" hidden title="Tadqiqot review (pseudo-bbox tasdiqlash)">🔍</button>
25 <button id="statsBtn" hidden title="Statistika">📊</button>
26 <button id="pacsBtn" hidden title="PACS serverlar">🖨️</button>
27 <button id="totpBtn" hidden title="2FA sozlash">🔒</button>
28 <button id="adminBtn" hidden title="Foydalanuvchilarni boshqarish">👤</button>
29 <button id="logoutBtn" hidden title="Chiqish">🚪</button>
30 <button id="loginBtn" hidden>Kirish</button>
31 </div>
32 </header>
33
34 <div class="modal" id="loginModal" hidden>
35 <div class="modal-card">
36 <h2>Kirish</h2>
37 <form id="loginForm">
38 <label>Foydalanuvchi nomi
39 <input type="text" id="loginUsername" autocomplete="username" required autofocus />
40 </label>
41 <label>Parol
42 <input type="password" id="loginPassword" autocomplete="current-password" required />
43 </label>
44 <label id="loginTotpRow" hidden>2FA kodi (6 raqam)
45 <input type="text" id="loginTotp" inputmode="numeric" maxlength="6" autocomplete="one-time-
code" />
46 </label>
47 <div class="modal-err" id="loginErr"></div>
48 <div class="modal-actions">
49 <button type="submit" id="loginSubmit">Kirish</button>
50 </div>
51 </form>
```

```

52     </div>
53 </div>
54
55 <div class="modal" id="totpModal" hidden>
56   <div class="modal-card">
57     <div class="modal-head">
58       <h2>🔑 Ikki faktor autentifikatsiya</h2>
59       <button class="modal-close" id="totpCloseBtn">X</button>
60     </div>
61     <div class="modal-body" id="totpBody"></div>
62   </div>
63 </div>
64
65 <div class="modal" id="historyModal" hidden>
66   <div class="modal-card wide">
67     <div class="modal-head">
68       <h2>📅 Annotation tarix</h2>
69       <button class="modal-close" id="historyCloseBtn">X</button>
70     </div>
71     <div class="modal-body" id="historyBody"></div>
72   </div>
73 </div>
74
75 <div class="modal" id="radiomicsModal" hidden>
76   <div class="modal-card wide">
77     <div class="modal-head">
78       <h2>🇮🇹 Radiomika tahlili</h2>
79       <button class="modal-close" id="radiomicsCloseBtn">X</button>
80     </div>
81     <div class="modal-body" id="radiomicsBody"></div>
82   </div>
83 </div>
84
85 <div class="modal" id="adminModal" hidden>
86   <div class="modal-card wide">
87     <div class="modal-head">
88       <h2>👤 Foydalanuvchilarni boshqarish</h2>
89       <button class="modal-close" id="adminCloseBtn">X</button>
90     </div>
91     <div class="modal-body" id="adminBody"></div>
92   </div>
93 </div>
94
95 <div class="modal" id="statsModal" hidden>
96   <div class="modal-card wide">
97     <div class="modal-head">
98       <h2>🇮🇹 Statistika</h2>
99       <button class="modal-close" id="statsCloseBtn">X</button>
100    </div>
101    <div class="modal-body" id="statsBody"></div>
102  </div>
103 </div>
104
105 <div class="modal" id="auditModal" hidden>
106   <div class="modal-card wide">
107     <div class="modal-head">
108       <h2>📋 Audit timeline</h2>
109       <button class="modal-close" id="auditCloseBtn">X</button>
110     </div>
111     <div class="modal-body" id="auditBody"></div>
112   </div>
113 </div>
114
115 <div class="modal" id="overviewModal" hidden>

```

```

116 <div class="modal-card wide">
117   <div class="modal-head">
118     <h2>📁 Annotatsiyalangan DICOM'lar</h2>
119     <button class="modal-close" id="overviewCloseBtn">✕</button>
120   </div>
121   <div class="modal-body" id="overviewBody"></div>
122 </div>
123 </div>
124
125 <div class="modal" id="pacsModal" hidden>
126   <div class="modal-card wide">
127     <div class="modal-head">
128       <h2>📁 PACS</h2>
129       <button class="modal-close" id="pacsCloseBtn">✕</button>
130     </div>
131     <div class="modal-body" id="pacsBody"></div>
132   </div>
133 </div>
134
135 <div class="modal" id="patientModal" hidden>
136   <div class="modal-card wide">
137     <div class="modal-head">
138       <h2>👤 Bemor ko'rinishi</h2>
139       <button class="modal-close" id="patientCloseBtn">✕</button>
140     </div>
141     <div class="modal-body" id="patientBody"></div>
142   </div>
143 </div>
144
145 <div class="modal" id="reviewModal" hidden>
146   <div class="modal-card wide">
147     <div class="modal-head">
148       <h2>🔍 Tadqiqot review – pseudo-bbox tasdiqlash</h2>
149       <button class="modal-close" id="reviewCloseBtn">✕</button>
150     </div>
151     <div class="modal-body" id="reviewBody"></div>
152   </div>
153 </div>
154
155 <div class="modal" id="deidModal" hidden>
156   <div class="modal-card wide">
157     <div class="modal-head">
158       <h2>🔑 De-identifikatsiya</h2>
159       <button class="modal-close" id="deidCloseBtn">✕</button>
160     </div>
161     <div class="modal-body" id="deidBody"></div>
162   </div>
163 </div>
164
165 <div class="layout">
166   <aside class="sidebar">
167     <div class="tabs">
168       <button class="tab active" data-tab="uploads">Yuklangan</button>
169       <button class="tab" data-tab="local" id="localTabBtn" hidden>Lokal</button>
170       <button class="tab" data-tab="links">🔗 Bog'langan</button>
171       <button class="tab" data-tab="dashboard">📊 Mening ishim</button>
172       <button class="tab" data-tab="worklist">📋 Worklist</button>
173     </div>
174
175     <section class="pane" data-pane="uploads">
176       <div class="upload-zone" id="dropZone">
177         <p>DICOM faylni shu yerga tashlang yoki tanlang</p>
178         <input type="file" id="fileInput" multiple accept=".dcm,.dicom,application/dicom,*/*" hidden />
179         <input type="file" id="folderInput" multiple webkitdirectory directory hidden />

```

```

180     <div class="btn-row">
181         <button id="pickFiles">Fayl tanlash</button>
182         <button id="pickFolder">Papka tanlash</button>
183     </div>
184 </div>
185 <ul class="file-list" id="uploadList"></ul>
186 </section>
187
188 <section class="pane" data-pane="local" hidden>
189     <div class="crumb" id="crumb"></div>
190     <ul class="file-list" id="localList"></ul>
191 </section>
192
193 <section class="pane" data-pane="dashboard" hidden>
194     <div class="dash-controls">
195         <select id="dashStatus">
196             <option value="">Barcha statuslar</option>
197             <option value="draft">Drafts</option>
198             <option value="submitted">Submitted (kutmoqda)</option>
199             <option value="approved">Approved</option>
200             <option value="rejected">Rejected</option>
201         </select>
202         <label class="own-toggle"><input type="checkbox" id="dashOwn" /> faqat meniki</label>
203         <button id="dashRefresh" title="Yangilash">🔄</button>
204     </div>
205     <div class="dash-meta" id="dashMeta"></div>
206     <ul class="file-list" id="dashList"></ul>
207 </section>
208
209 <section class="pane" data-pane="worklist" hidden>
210     <div class="dash-controls">
211         <select id="wlStatus">
212             <option value="">Barcha statuslar</option>
213             <option value="pending">Pending</option>
214             <option value="in_progress">In progress</option>
215             <option value="done">Done</option>
216             <option value="skipped">Skipped</option>
217         </select>
218         <label class="own-toggle"><input type="checkbox" id="wlOwn" checked /> meniki</label>
219         <button id="wlRefresh" title="Yangilash">🔄</button>
220     </div>
221     <div class="dash-meta" id="wlMeta"></div>
222     <ul class="file-list" id="wlList"></ul>
223 </section>
224
225 <section class="pane" data-pane="links" hidden>
226     <div class="link-search-bar">
227         <input type="text" id="linksSearch" placeholder="Bemor / ID / fayl yo'li" />
228         <button id="linksRefresh" title="Yangilash">🔄</button>
229     </div>
230     <div class="links-meta" id="linksMeta"></div>
231     <ul class="file-list" id="linksList"></ul>
232 </section>
233 </aside>
234
235 <main class="viewer">
236     <div class="toolbar">
237         <button id="fitBtn" title="Ekranga moslash (F)">Fit</button>
238         <button id="oneToOneBtn" title="100%">1:1</button>
239         <button id="invertBtn" title="Inversiya">Invert</button>
240         <button id="resetWlBtn" title="Standart W/L">W/L reset</button>
241         <span class="sep"></span>
242         <button id="toolSelect" class="tool active" title="Tanlash (V)">📁 Select</button>
243         <button id="toolBbox" class="tool" title="Bbox chizish (B)">📏 BBox</button>
244         <button id="toolPoly" class="tool" title="Poligon chizish (P) – bosish bilan nuqta qo'yish,
dblclic yoki Enter – yopish, Esc – bekor">📐 Poly</button>

```

```

245     <select id="labelSelect" title="Label (yangi annotatsiya uchun) – qo'shimcha label'larni o'ng
panelidan belgilang"></select>
246     <select id="biradsSelect" title="BI-RADS"><option value="">BI-RADS –</option></select>
247     <span class="sep"></span>
248     <select id="aiModelSelect" title="AI model" hidden></select>
249     <button id="aiRunBtn" title="AI tahlil yugurtirish" hidden>&img alt="AI icon" data-bbox="655 138 675 150"/> AI tahlil</button>
250     <span class="ai-threshold" id="aiThresholdWrap" hidden>
251         <label>><span id="aiConfLabel">25</span>%</label>
252         <input type="range" id="aiConfSlider" min="10" max="95" step="5" value="25" />
253     </span>
254     <label id="aiTtaWrap" title="Test-time augmentation – ko'p-masshtab + flip; aniqlikni oshiradi,
sekinroq" hidden>
255         <input type="checkbox" id="aiTtaToggle" /> TTA
256     </label>
257     <button id="aiAcceptAllBtn" title="Barcha takliflarni qabul qilish" hidden>✓ Hammasini
qabul</button>
258     <button id="aiClearBtn" title="AI takliflarini tozalash" hidden>✕</button>
259     <span class="sep"></span>
260     <label>Frame <input type="range" id="frameSlider" min="0" max="0" value="0" disabled /></label>
261     <span id="frameLabel">0 / 0</span>
262     <span class="zoomLbl" id="zoomLbl">100%</span>
263     <span class="save-status" id="saveStatus"></span>
264     <button id="saveBtn" title="Annotatsiyalarni saqlash (Ctrl+S)" disabled>&img alt="Save icon" data-bbox="745 358 765 370"/> Saqlash</button>
265     <button id="exportBtn" title="Barcha annotatsiyalarni COCO JSON sifatida yuklab olish">↓
COCO</button>
266     <button id="exportYoloBtn" title="Barcha annotatsiyalarni YOLO (.txt) dataset (ZIP) sifatida
yuklab olish">↓ YOLO</button>
267     <button id="exportVocBtn" title="Barcha annotatsiyalarni Pascal VOC (XML, ZIP) sifatida yuklab
olish">↓ VOC</button>
268     <button id="exportCsvBtn" title="Barcha annotatsiyalarni CSV jadval (har annotatsiya bir qator,
ko'p-label ustun bilan) sifatida yuklab olish">↓ CSV</button>
269     <button id="deidBtn" title="PHI tahlil va anonimlashtirilgan DICOM yuklab olish" hidden>&img alt="De-ID icon" data-bbox="885 478 905 490"/> De-
ID</button>
270     <button id="srBtn" title="Annotatsiyalarni DICOM-SR sifatida yuklab olish" hidden>&img alt="SR icon" data-bbox="815 505 835 517"/> SR</button>
271     <button id="segBtn" title="Annotatsiyalarni DICOM-SEG (pixel mask) sifatida yuklab olish"
hidden>&img alt="SEG icon" data-bbox="200 532 220 544"/> SEG</button>
272     <button id="maskPngBtn" title="Joriy rasm uchun segmentatsiya maskasi – PNG (label-indexed)"
hidden>&img alt="Mask PNG icon" data-bbox="200 559 220 571"/> Mask PNG</button>
273     <button id="maskNiftiBtn" title="Joriy rasm uchun segmentatsiya maskasi – NIFTI (.nii.gz)"
hidden>&img alt="Mask NIFTI icon" data-bbox="200 586 220 598"/> NIFTI</button>
274     <button id="radiomicsBtn" title="Tanlangan annotatsiya uchun radiomika tahlili (ROI bo'yicha 60
ta belgi)" hidden>&img alt="Radiomika icon" data-bbox="280 613 300 625"/> Radiomika</button>
275     <button id="cStoreBtn" title="DICOM'ni PACS serverga yuborish" hidden>&img alt="PACS icon" data-bbox="735 628 755 640"/> PACS</button>
276     <select id="tplSelect" title="Annotation template'ni qo'llash" hidden></select>
277     <button id="tplSaveBtn" title="Joriy annotatsiyalarni template sifatida saqlash" hidden>&img alt="Save icon" data-bbox="865 655 885 667"/>
Tpl</button>
278     <span class="presence" id="presencePill" hidden title="Real vaqtda hamkorlar"></span>
279 </div>
280
281 <div class="canvas-wrap" id="canvasWrap">
282     <div class="canvas" id="canvas">
283         <div class="empty" id="emptyState">Chap paneldan DICOM tanlang yoki yuklang.</div>
284         <div class="img-stack" id="imgStack" hidden>
285             <img id="dicomImg" alt="" draggable="false" />
286             <svg id="annoSvg" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 1 1"
preserveAspectRatio="none"></svg>
287         </div>
288         <div class="sr-view" id="srView" hidden></div>
289     </div>
290 </div>
291
292 <div class="wl-bar">
293     <div class="wl-group">
294         <label>WC <input type="number" id="wcInput" step="1" /></label>

```

```

295     <input type="range" id="wcSlider" min="-2000" max="6000" value="0" />
296   </div>
297   <div class="wl-group">
298     <label>WW <input type="number" id="wwInput" step="1" min="1" /></label>
299     <input type="range" id="wwSlider" min="1" max="6000" value="1" />
300   </div>
301 </div>
302 </main>
303
304 <aside class="meta-pane">
305   <div class="tabs right">
306     <button class="tab active" data-rtab="meta">Metadata</button>
307     <button class="tab" data-rtab="anno">Annotatsiyalar <span class="badge"
id="annoCount">0</span></button>
308     <button class="tab" data-rtab="report">Hisobot <span class="badge" id="reportBadge"
hidden>0</span></button>
309   </div>
310   <div class="rpane" data-rpane="meta">
311     <table class="meta" id="metaTable"><tbody></tbody></table>
312   </div>
313   <div class="rpane" data-rpane="anno" hidden>
314     <div class="anno-list" id="annoList"></div>
315     <div class="anno-empty" id="annoEmpty">Annotatsiyalar yo'q. <b>BBox</b> tugmasini bosing yoki
<b>B</b> bosing va rasm ustida sichqonchani sudrang.</div>
316   </div>
317   <div class="rpane" data-rpane="report" hidden>
318     <div id="reportPane"></div>
319   </div>
320 </aside>
321 </div>
322
323 <script src="/app.js"></script>
324 </body>
325 </html>

```

app/static/style.css

Fayl yo'li: app/static/style.css | Qatorlar: 1064 | Hajm: 36907 bayt

```

1  :root {
2    --bg: #0e1116;
3    --panel: #161b22;
4    --panel-2: #1f2630;
5    --border: #2a313c;
6    --text: #e6edf3;
7    --muted: #8a96a4;
8    --accent: #2f81f7;
9    --accent-2: #1f6feb;
10   --danger: #f85149;
11   --warn: #ffbb00;
12   --ok: #28d97f;
13 }
14 * { box-sizing: border-box; }
15 html, body { height: 100%; margin: 0; }
16 body {
17   font-family: ui-sans-serif, system-ui, "Segoe UI", Roboto, Arial, sans-serif;
18   background: var(--bg);
19   color: var(--text);
20   font-size: 13px;
21   overflow: hidden;
22 }
23 header {
24   display: flex; align-items: center; justify-content: space-between;
25   padding: 8px 14px;

```

```

26   background: var(--panel);
27   border-bottom: 1px solid var(--border);
28 }
29 header h1 { font-size: 14px; margin: 0; letter-spacing: 0.5px; }
30 .hint { color: var(--muted); font-size: 12px; flex: 1; text-align: center; }
31
32 .user-bar { display: flex; gap: 6px; align-items: center; }
33 .user-bar #userInfo {
34   color: var(--text); font-size: 12px;
35   padding: 3px 8px; background: var(--panel-2); border-radius: 9px;
36   border: 1px solid var(--border);
37 }
38 .user-bar #userInfo .role {
39   font-size: 10px; color: var(--accent); margin-left: 4px;
40   text-transform: uppercase;
41 }
42
43 .bell-wrap { position: relative; }
44 .bell-wrap button {
45   background: var(--panel-2); border: 1px solid var(--border); color: var(--text);
46   padding: 4px 8px; border-radius: 4px; font-size: 14px; cursor: pointer;
47   position: relative;
48 }
49 .bell-wrap button:hover { border-color: var(--accent); }
50 .bell-badge {
51   position: absolute; top: -4px; right: -4px;
52   background: var(--danger); color: #fff;
53   border-radius: 9px; padding: 0 5px; font-size: 9px; font-weight: 700;
54   min-width: 14px; text-align: center;
55   border: 1px solid var(--bg);
56 }
57 .bell-dropdown {
58   position: absolute; top: calc(100% + 4px); right: 0;
59   width: 360px; max-height: 480px; overflow: auto;
60   background: var(--panel); border: 1px solid var(--border); border-radius: 6px;
61   box-shadow: 0 8px 32px rgba(0,0,0,0.5);
62   z-index: 100;
63 }
64 .bell-dropdown .head {
65   display: flex; justify-content: space-between; align-items: center;
66   padding: 8px 12px; border-bottom: 1px solid var(--border);
67   font-size: 12px; font-weight: 600;
68 }
69 .bell-dropdown .head button {
70   background: transparent; border: none; color: var(--accent);
71   font-size: 11px; cursor: pointer; padding: 0;
72 }
73 .bell-dropdown .head button:hover { text-decoration: underline; }
74 .bell-dropdown .empty {
75   padding: 24px 12px; text-align: center; color: var(--muted); font-size: 12px;
76 }
77 .notif-row {
78   padding: 8px 12px; border-bottom: 1px solid var(--border);
79   cursor: pointer; font-size: 12px;
80 }
81 .notif-row:hover { background: var(--panel-2); }
82 .notif-row.unread { background: rgba(47,129,247,0.06); border-left: 3px solid var(--accent); }
83 .notif-row .title { font-weight: 600; margin-bottom: 2px; }
84 .notif-row .meta { color: var(--muted); font-size: 10px; }
85 .notif-row .body { color: var(--muted); font-size: 11px; margin-top: 2px; }
86 .notif-kind {
87   display: inline-block; padding: 1px 6px; font-size: 9px;
88   border-radius: 3px; font-weight: 700; text-transform: uppercase;
89   margin-right: 4px;
90 }
91 .notif-kind.submitted { background: rgba(255,176,0,0.18); color: var(--warn); }

```



```

92 .notif-kind.approved { background: rgba(40,217,127,0.18); color: var(--ok); }
93 .notif-kind.rejected { background: rgba(248,81,73,0.18); color: var(--danger); }
94
95 .modal {
96   position: fixed; inset: 0; background: rgba(0,0,0,0.7);
97   display: flex; align-items: center; justify-content: center;
98   z-index: 1000;
99 }
100 .modal[hidden] { display: none; }
101 [hidden] { display: none !important; }
102 .modal-card {
103   background: var(--panel); border: 1px solid var(--border);
104   border-radius: 8px; padding: 22px 24px; min-width: 320px;
105   box-shadow: 0 8px 32px rgba(0,0,0,0.5);
106 }
107 .modal-card h2 { margin: 0 0 14px; font-size: 16px; }
108 .modal-card form { display: flex; flex-direction: column; gap: 10px; }
109 .modal-card label {
110   display: flex; flex-direction: column; gap: 4px;
111   color: var(--muted); font-size: 12px;
112 }
113 .modal-card input[type="text"],
114 .modal-card input[type="password"] {
115   background: var(--panel-2); border: 1px solid var(--border);
116   color: var(--text); padding: 7px 10px; border-radius: 4px; font-size: 13px;
117 }
118 .modal-card input:focus { outline: none; border-color: var(--accent); }
119 .modal-err {
120   min-height: 16px; color: var(--danger); font-size: 11px;
121 }
122 .modal-actions { display: flex; justify-content: flex-end; gap: 6px; }
123 .modal-actions button {
124   padding: 7px 14px; font-size: 13px;
125   background: var(--accent); border-color: var(--accent); color: #fff;
126 }
127 .modal-actions button:hover { background: var(--accent-2); }
128
129 .modal-card.wide { min-width: 640px; max-width: 90vw; max-height: 80vh; display: flex; flex-direction:
column; }
130 .modal-head { display: flex; align-items: center; justify-content: space-between; margin-bottom: 12px;
}
131 .modal-head h2 { margin: 0; font-size: 16px; }
132 .modal-close {
133   background: transparent; border: none; color: var(--muted);
134   font-size: 18px; cursor: pointer; padding: 0 6px;
135 }
136 .modal-close:hover { color: var(--text); }
137 .modal-body { overflow: auto; flex: 1; }
138
139 /* Status badges */
140 .status-pill {
141   display: inline-block; padding: 1px 7px; border-radius: 9px;
142   font-size: 10px; font-weight: 600; margin-left: 4px;
143   text-transform: uppercase; letter-spacing: 0.3px;
144   border: 1px solid;
145 }
146 .status-draft { background: rgba(138,150,164,0.12); color: var(--muted); border-color:
rgba(138,150,164,0.4); }
147 .status-submitted { background: rgba(255,176,0,0.15); color: var(--warn); border-color:
rgba(255,176,0,0.4); }
148 .status-approved { background: rgba(40,217,127,0.15); color: var(--ok); border-color:
rgba(40,217,127,0.4); }
149 .status-rejected { background: rgba(248,81,73,0.15); color: var(--danger); border-color:
rgba(248,81,73,0.4); }
150
151 .anno-row .audit { color: var(--muted); font-size: 10px; margin-top: 2px; }

```

```

152 .anno-row .status-actions { display: flex; gap: 4px; flex-wrap: wrap; margin-top: 4px; }
153 .anno-row .status-btn {
154   padding: 2px 7px; font-size: 10px; border-radius: 3px;
155   background: var(--panel-2); border: 1px solid var(--border); color: var(--text);
156   cursor: pointer;
157 }
158 .anno-row .status-btn:hover { border-color: var(--accent); }
159 .anno-row .status-btn.submit { color: var(--warn); border-color: rgba(255,176,0,0.4); }
160 .anno-row .status-btn.approve { color: var(--ok); border-color: rgba(40,217,127,0.4); }
161 .anno-row .status-btn.reject { color: var(--danger); border-color: rgba(248,81,73,0.4); }
162 .anno-row .status-btn.history { color: var(--muted); }
163 .anno-row .review-note {
164   font-size: 11px; color: var(--danger); font-style: italic;
165   background: rgba(248,81,73,0.08); padding: 3px 6px; margin-top: 3px; border-radius: 3px;
166 }
167
168 /* History modal */
169 .hist-row {
170   padding: 8px 10px; border-bottom: 1px solid var(--border);
171   font-size: 12px;
172 }
173 .hist-row:last-child { border-bottom: none; }
174 .hist-row .row-head {
175   display: flex; gap: 8px; align-items: center;
176 }
177 .hist-row .action-pill {
178   padding: 1px 7px; font-size: 10px; border-radius: 3px; font-weight: 600;
179   text-transform: uppercase;
180 }
181 .hist-row .action-create { background: rgba(40,217,127,0.18); color: var(--ok); }
182 .hist-row .action-update { background: rgba(47,129,247,0.18); color: var(--accent); }
183 .hist-row .action-delete { background: rgba(248,81,73,0.18); color: var(--danger); }
184 .hist-row .action-status { background: rgba(255,176,0,0.18); color: var(--warn); }
185 .hist-row .ts { color: var(--muted); font-size: 11px; }
186 .hist-row .who { font-weight: 600; }
187 .hist-row pre {
188   margin: 6px 0 0; padding: 6px 8px;
189   font-family: ui-monospace, monospace; font-size: 10px;
190   background: var(--panel-2); border-radius: 3px;
191   white-space: pre-wrap; word-break: break-word;
192   max-height: 120px; overflow: auto;
193 }
194
195 /* Admin user table */
196 .user-table {
197   width: 100%; border-collapse: collapse; font-size: 12px;
198 }
199 .user-table th, .user-table td {
200   padding: 6px 10px; text-align: left;
201   border-bottom: 1px solid var(--border);
202 }
203 .user-table th { color: var(--muted); font-weight: 600; font-size: 11px; text-transform: uppercase; }
204 .user-table tr.inactive { opacity: 0.5; }
205 .user-table .role-pill {
206   padding: 1px 7px; border-radius: 3px; font-size: 10px;
207   background: var(--panel-2); border: 1px solid var(--border);
208 }
209 .user-table .role-admin { color: var(--accent); border-color: rgba(47,129,247,0.4); background:
210   rgba(47,129,247,0.08); }
210 .user-table .role-reviewer { color: var(--ok); border-color: rgba(40,217,127,0.4); background:
211   rgba(40,217,127,0.08); }
211 .user-table .row-actions { display: flex; gap: 4px; flex-wrap: wrap; }
212 .user-table .row-actions button {
213   padding: 2px 6px; font-size: 11px;
214 }
215 .admin-create-form {

```

```

216 margin-top: 14px; padding: 12px; background: var(--panel-2);
217 border: 1px solid var(--border); border-radius: 4px;
218 display: grid; grid-template-columns: 1fr 1fr 1fr 1fr; gap: 8px;
219 }
220 .admin-create-form label { display: flex; flex-direction: column; gap: 2px; font-size: 11px; color:
var(--muted); }
221 .admin-create-form input, .admin-create-form select {
222   background: var(--bg); border: 1px solid var(--border);
223   color: var(--text); padding: 4px 6px; border-radius: 3px; font-size: 12px;
224 }
225 .admin-create-form .actions { grid-column: 1/-1; display: flex; justify-content: flex-end; gap: 6px; }
226 .admin-err { color: var(--danger); font-size: 11px; padding: 4px 0; }
227
228 .layout {
229   display: grid;
230   grid-template-columns: 280px 1fr 340px;
231   height: calc(100vh - 38px);
232 }
233
234 aside.sidebar, aside.meta-pane {
235   background: var(--panel);
236   border-right: 1px solid var(--border);
237   overflow: auto;
238   display: flex;
239   flex-direction: column;
240 }
241 aside.meta-pane {
242   border-right: none;
243   border-left: 1px solid var(--border);
244   padding: 0;
245 }
246
247 .tabs { display: flex; border-bottom: 1px solid var(--border); }
248 .tabs.right { background: var(--panel); }
249 .tab {
250   flex: 1;
251   background: transparent;
252   color: var(--muted);
253   border: none;
254   padding: 9px 0;
255   cursor: pointer;
256   font-size: 12px;
257 }
258 .tab.active { color: var(--text); border-bottom: 2px solid var(--accent); }
259 .badge {
260   display: inline-block; min-width: 18px; padding: 1px 5px; margin-left: 4px;
261   background: var(--panel-2); border: 1px solid var(--border); border-radius: 9px; font-size: 10px;
262 }
263 .anno-pill {
264   display: inline-block; min-width: 16px; padding: 0 6px; margin-left: 4px;
265   background: rgba(40,217,127,0.18); color: var(--ok);
266   border: 1px solid rgba(40,217,127,0.4); border-radius: 9px;
267   font-size: 10px; font-weight: 600;
268 }
269
270 .pane { padding: 10px; }
271 .upload-zone {
272   border: 1px dashed var(--border);
273   border-radius: 6px;
274   padding: 14px;
275   text-align: center;
276   background: var(--panel-2);
277 }
278 .upload-zone.drag { border-color: var(--accent); background: rgba(47,129,247,0.08); }
279 .upload-zone p { margin: 0 0 10px; color: var(--muted); }
280 .btn-row { display: flex; gap: 6px; justify-content: center; }

```

```

281
282 button {
283     background: var(--panel-2);
284     border: 1px solid var(--border);
285     color: var(--text);
286     padding: 6px 10px;
287     border-radius: 4px;
288     cursor: pointer;
289     font-size: 12px;
290 }
291 button:hover { border-color: var(--accent); }
292 button:disabled { opacity: 0.5; cursor: not-allowed; }
293 button.tool.active { background: var(--accent); border-color: var(--accent); color: #fff; }
294
295 /* Manual save button: highlighted while there are unsaved changes. */
296 #saveBtn.pending {
297     border-color: var(--warn);
298     color: var(--warn);
299     font-weight: 600;
300 }
301 #saveBtn.disabled { opacity: 0.5; cursor: default; }
302
303 /* Radiomics modal */
304 .radiomics-controls { display: flex; gap: 8px; align-items: center; flex-wrap: wrap; margin-bottom:
10px; }
305 .radiomics-info { flex: 1; min-width: 180px; font-size: 12px; color: var(--muted); }
306 .radiomics-info b { color: var(--text); }
307 .radiomics-table { width: 100%; border-collapse: collapse; margin: 2px 0 10px; font-size: 12px; }
308 .radiomics-table td { padding: 3px 8px; border-bottom: 1px solid var(--border); }
309 .radiomics-table td.val { text-align: right; font-variant-numeric: tabular-nums; color: var(--text); }
310 .radiomics-table td:first-child { color: var(--muted); }
311 button#invertBtn.active { background: var(--accent); border-color: var(--accent); color: #fff; }
312
313 select {
314     background: var(--panel-2);
315     color: var(--text);
316     border: 1px solid var(--border);
317     padding: 5px 6px;
318     border-radius: 4px;
319     font-size: 12px;
320 }
321
322 /* --- Multi-select label dropdown --- */
323 .ms { position: relative; display: inline-block; vertical-align: middle; }
324 .ms .ms-toggle {
325     background: var(--panel-2);
326     color: var(--text);
327     border: 1px solid var(--border);
328     padding: 5px 22px 5px 8px;
329     border-radius: 4px;
330     font-size: 12px;
331     cursor: pointer;
332     min-width: 90px;
333     max-width: 220px;
334     text-align: left;
335     white-space: nowrap;
336     overflow: hidden;
337     text-overflow: ellipsis;
338     position: relative;
339 }
340 .ms .ms-toggle::after {
341     content: "▼";
342     position: absolute;
343     right: 7px; top: 50%;
344     transform: translateY(-50%);
345     color: var(--muted);

```

```

346   font-size: 10px;
347 }
348 .ms .ms-toggle.has-sel { border-color: var(--accent); }
349 .ms .ms-panel {
350   position: absolute;
351   z-index: 60;
352   top: calc(100% + 4px);
353   left: 0;
354   min-width: 180px;
355   max-height: 280px;
356   overflow-y: auto;
357   background: var(--panel);
358   border: 1px solid var(--border);
359   border-radius: 6px;
360   box-shadow: 0 8px 24px rgba(0,0,0,0.45);
361   padding: 4px;
362 }
363 .ms .ms-panel[hidden] { display: none; }
364 .ms .ms-opt {
365   display: flex;
366   align-items: center;
367   gap: 8px;
368   padding: 6px 8px;
369   border-radius: 4px;
370   cursor: pointer;
371   font-size: 12px;
372   color: var(--text);
373   user-select: none;
374 }
375 .ms .ms-opt:hover { background: var(--panel-2); }
376 .ms .ms-opt input { margin: 0; accent-color: var(--accent); cursor: pointer; }
377 .ms .ms-opt .ms-dot {
378   width: 10px; height: 10px; border-radius: 2px; flex-shrink: 0;
379   border: 1px solid rgba(0,0,0,0.4);
380 }
381 .ms .ms-opt .ms-name { flex: 1; }
382 /* Compact variant used inside annotation rows */
383 .ms.ms-compact .ms-toggle { min-width: 70px; max-width: 160px; padding: 3px 18px 3px 6px; font-size:
11px; }
384 .anno-row .info .ms { margin-right: 4px; vertical-align: middle; }
385 .anno-row .info .lbl { display: flex; flex-wrap: wrap; align-items: center; gap: 4px; }
386
387 .ai-threshold {
388   display: inline-flex; align-items: center; gap: 6px;
389   padding: 2px 8px;
390   background: var(--panel-2); border: 1px solid var(--border); border-radius: 4px;
391   color: var(--muted); font-size: 11px;
392 }
393 .ai-threshold input[type="range"] { width: 80px; }
394 .ai-threshold label { color: var(--text); }
395
396 .file-list { list-style: none; padding: 0; margin: 8px 0 0; }
397 .file-list li {
398   padding: 8px 10px;
399   border-radius: 4px;
400   cursor: pointer;
401   border: 1px solid transparent;
402   margin-bottom: 4px;
403 }
404 .file-list li:hover { background: var(--panel-2); }
405 .file-list li.active { background: rgba(47,129,247,0.15); border-color: var(--accent); }
406 .file-list li .name { font-weight: 600; }
407 .file-list li .meta { color: var(--muted); font-size: 11px; margin-top: 2px; }
408 .file-list li .row { display: flex; justify-content: space-between; align-items: center; }
409 .file-list .dir { color: var(--accent); }
410 .file-list .dir::before { content: "📁 "; }

```

```

411 .file-list .file::before { content: "📁 "; }
412 .file-list .delete {
413     float: right; background: transparent; border: none; color: var(--muted); cursor: pointer; padding:
0 4px;
414 }
415 .file-list .delete:hover { color: var(--danger); }
416
417 .crumb { padding: 8px 10px; color: var(--muted); font-size: 12px; word-break: break-all; }
418
419 .dash-controls {
420     display: flex; gap: 6px; align-items: center;
421     padding: 8px 10px; border-bottom: 1px solid var(--border);
422     flex-wrap: wrap;
423 }
424 .dash-controls select {
425     flex: 1; min-width: 140px;
426 }
427 .dash-controls .own-toggle {
428     display: inline-flex; align-items: center; gap: 4px;
429     color: var(--muted); font-size: 11px; cursor: pointer;
430 }
431 .dash-meta { padding: 6px 10px; color: var(--muted); font-size: 11px; }
432 .dash-row .name::before { content: "📄 "; }
433 .dash-row .name { display: flex; align-items: center; gap: 4px; }
434 .dash-row { padding: 8px 10px; }
435
436 .wl-priority {
437     font-size: 9px; padding: 1px 5px; border-radius: 3px; font-weight: 700;
438     text-transform: uppercase; margin-left: 4px;
439 }
440 .wl-priority.urgent { background: rgba(248,81,73,0.18); color: var(--danger); }
441 .wl-priority.high { background: rgba(255,176,0,0.18); color: var(--warn); }
442 .wl-priority.normal { background: var(--panel-2); color: var(--muted); }
443 .wl-priority.low { background: var(--panel-2); color: var(--muted); opacity: 0.5; }
444 .wl-status-pending { color: var(--muted); }
445 .wl-status-in_progress { color: var(--accent); }
446 .wl-status-done { color: var(--ok); }
447 .wl-status-skipped { color: var(--muted); opacity: 0.5; text-decoration: line-through; }
448
449 .patient-overview { padding: 8px 0; }
450 .patient-overview .pat-head {
451     padding: 8px 12px; background: var(--panel-2); border-bottom: 1px solid var(--border);
452     margin-bottom: 8px; border-radius: 4px;
453 }
454 .patient-overview .pat-head .name { font-size: 16px; font-weight: 600; }
455 .patient-overview .pat-head .meta { color: var(--muted); font-size: 11px; margin-top: 2px; }
456 .patient-overview .timeline-item {
457     display: flex; gap: 12px; padding: 8px 12px;
458     border-left: 2px solid var(--border); margin-left: 12px;
459     position: relative;
460 }
461 .patient-overview .timeline-item::before {
462     content: ""; width: 8px; height: 8px; border-radius: 50%;
463     background: var(--accent); position: absolute; left: -5px; top: 14px;
464     border: 2px solid var(--bg);
465 }
466 .patient-overview .timeline-item.kind-record::before { background: var(--ok); }
467 .patient-overview .timeline-item.kind-worklist::before { background: var(--warn); }
468 .patient-overview .timeline-item.kind-dicom::before { background: var(--accent); }
469 .patient-overview .timeline-item .ts {
470     color: var(--muted); font-size: 11px; min-width: 90px;
471 }
472 .patient-overview .timeline-item .body { flex: 1; font-size: 12px; }
473 .patient-overview .timeline-item.dicom-row { cursor: pointer; }
474 .patient-overview .timeline-item.dicom-row:hover { background: var(--panel-2); }
475

```

```

476 .deid-section { padding: 8px 0; }
477 .deid-section h3 { font-size: 12px; color: var(--muted); margin: 12px 0 4px; text-transform:
uppercase; }
478 .deid-table { width: 100%; border-collapse: collapse; font-size: 11px; }
479 .deid-table td { padding: 4px 8px; border-bottom: 1px solid var(--border); }
480 .deid-table td:first-child { color: var(--muted); width: 35%; }
481 .deid-table tr.preserves { background: rgba(40,217,127,0.06); }
482 .deid-actions { display: flex; gap: 6px; margin-top: 12px; }
483 .deid-actions button { padding: 8px 14px; }
484
485 /* Stats dashboard */
486 .stats-grid {
487   display: grid;
488   grid-template-columns: repeat(auto-fill, minmax(140px, 1fr));
489   gap: 8px;
490   margin-bottom: 14px;
491 }
492 .stats-card {
493   background: var(--panel-2); border: 1px solid var(--border); border-radius: 6px;
494   padding: 10px;
495 }
496 .stats-card .lbl {
497   font-size: 10px; color: var(--muted); text-transform: uppercase; letter-spacing: 0.5px;
498 }
499 .stats-card .num {
500   font-size: 22px; font-weight: 700; color: var(--text); margin-top: 4px;
501 }
502 .stats-section { margin-top: 14px; }
503 .stats-section h3 {
504   font-size: 12px; color: var(--muted); text-transform: uppercase;
505   letter-spacing: 0.5px; margin: 0 0 6px;
506 }
507 .stats-bar-row {
508   display: grid; grid-template-columns: 140px 1fr 50px; gap: 8px;
509   align-items: center; padding: 4px 0;
510   font-size: 12px;
511 }
512 .stats-bar-row .lbl { color: var(--text); white-space: nowrap; overflow: hidden; text-overflow:
ellipsis; }
513 .stats-bar-row .bar {
514   height: 14px; background: var(--accent); border-radius: 3px;
515   min-width: 2px;
516 }
517 .stats-bar-row .bar.green { background: var(--ok); }
518 .stats-bar-row .bar.yellow { background: var(--warn); }
519 .stats-bar-row .bar.red { background: var(--danger); }
520 .stats-bar-row .num { text-align: right; color: var(--muted); }
521 .stats-spark {
522   display: flex; align-items: flex-end; gap: 2px; height: 50px;
523   padding: 4px 0; border-bottom: 1px solid var(--border);
524 }
525 .stats-spark .day {
526   flex: 1; min-width: 4px;
527   background: var(--accent); border-radius: 2px 2px 0 0;
528 }
529 .stats-spark .day:hover { background: var(--accent-2); }
530
531 /* Annotations overview cards */
532 .ov-grid {
533   display: grid;
534   grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
535   gap: 10px;
536 }
537 .ov-card {
538   background: var(--panel-2); border: 1px solid var(--border); border-radius: 6px;
539   padding: 10px; cursor: pointer; transition: border-color 0.15s;

```

```

540 }
541 .ov-card:hover { border-color: var(--accent); }
542 .ov-card .ov-name { font-weight: 600; font-size: 13px; }
543 .ov-card .ov-meta { color: var(--muted); font-size: 11px; margin-top: 2px; }
544 .ov-card .ov-stats { display: flex; gap: 6px; margin-top: 6px; flex-wrap: wrap; }
545 .ov-card .ov-pill {
546   padding: 1px 6px; border-radius: 9px; font-size: 10px; font-weight: 600;
547 }
548 .ov-pill.draft { background: rgba(138,150,164,0.18); color: var(--muted); }
549 .ov-pill.submitted { background: rgba(255,176,0,0.18); color: var(--warn); }
550 .ov-pill.approved { background: rgba(40,217,127,0.18); color: var(--ok); }
551 .ov-pill.rejected { background: rgba(248,81,73,0.18); color: var(--danger); }
552 .ov-card .ov-labels {
553   margin-top: 6px; color: var(--muted); font-size: 10px;
554   white-space: nowrap; overflow: hidden; text-overflow: ellipsis;
555 }
556 .ov-controls {
557   display: flex; gap: 6px; align-items: center;
558   margin-bottom: 12px; flex-wrap: wrap;
559 }
560 .ov-meta-bar { color: var(--muted); font-size: 11px; margin-bottom: 8px; }
561
562 /* ===== Responsive ===== */
563
564 .mobile-toggle {
565   display: none;
566   background: var(--panel-2); border: 1px solid var(--border); color: var(--text);
567   padding: 4px 10px; border-radius: 4px; cursor: pointer; font-size: 16px;
568 }
569
570 @media (max-width: 1024px) {
571   .layout {
572     grid-template-columns: 240px 1fr 280px;
573   }
574   .toolbar { gap: 4px; padding: 4px 6px; }
575   .toolbar button, .toolbar select { padding: 4px 6px; font-size: 11px; }
576 }
577
578 @media (max-width: 768px) {
579   body { font-size: 12px; }
580   header { padding: 6px 8px; }
581   header h1 { font-size: 12px; }
582   .hint { font-size: 10px; }
583
584   .mobile-toggle { display: inline-block; }
585
586   .layout {
587     display: block;
588     height: auto;
589     min-height: calc(100vh - 38px);
590   }
591   .layout > aside.sidebar,
592   .layout > aside.meta-pane {
593     display: none;
594     width: 100%;
595     max-height: 50vh;
596     border: none;
597     border-bottom: 1px solid var(--border);
598   }
599   .layout > aside.sidebar.open,
600   .layout > aside.meta-pane.open {
601     display: flex;
602   }
603   main.viewer {
604     height: calc(100vh - 38px);
605     min-height: 360px;

```



```

606 }
607 .toolbar { flex-wrap: wrap; }
608 .ai-threshold input[type="range"] { width: 60px; }
609 .wl-bar { grid-template-columns: 1fr; gap: 6px; }
610 .modal-card.wide { min-width: auto; width: 95vw; }
611 .admin-create-form { grid-template-columns: 1fr 1fr; }
612 .stats-grid { grid-template-columns: repeat(2, 1fr); }
613 .stats-bar-row { grid-template-columns: 80px 1fr 40px; }
614 .canvas-wrap { touch-action: none; }
615 button, select, input[type="text"], input[type="password"] {
616     min-height: 32px;
617 }
618 .file-list li { padding: 10px 8px; }
619 .anno-row { padding: 10px 12px; }
620 .anno-row.x, .anno-row .status-btn { min-height: 28px; padding: 4px 8px; }
621 }
622
623 @media (max-width: 480px) {
624     .toolbar { font-size: 10px; }
625     .toolbar button { padding: 4px 5px; }
626     .toolbar .sep { display: none; }
627     .ai-threshold { display: none; }
628     .save-status, .zoomLbl { display: none; }
629     .modal-card { padding: 14px 12px; }
630     .modal-card.wide { width: 98vw; padding: 10px; }
631     .admin-create-form { grid-template-columns: 1fr; }
632     .stats-grid { grid-template-columns: 1fr 1fr; }
633     .meta-pane .tab { font-size: 11px; padding: 7px 0; }
634 }
635
636 .link-search-bar {
637     display: flex; gap: 6px; padding: 8px 10px; border-bottom: 1px solid var(--border);
638 }
639 .link-search-bar input {
640     flex: 1;
641     background: var(--panel-2); border: 1px solid var(--border); color: var(--text);
642     padding: 4px 8px; border-radius: 3px; font-size: 12px;
643 }
644 .links-meta { padding: 6px 10px; color: var(--muted); font-size: 11px; }
645 .file-list .link-row .name::before { content: "🔗 "; }
646 .file-list .link-row { padding: 6px 10px; }
647
648 main.viewer {
649     display: flex; flex-direction: column;
650     background: #000;
651     position: relative;
652     overflow: hidden;
653 }
654 .toolbar {
655     display: flex; align-items: center; gap: 6px;
656     padding: 6px 10px;
657     background: var(--panel);
658     border-bottom: 1px solid var(--border);
659     flex-wrap: wrap;
660 }
661 .toolbar .sep { width: 1px; height: 18px; background: var(--border); margin: 0 4px; }
662 .toolbar input[type="range"] { vertical-align: middle; }
663 .toolbar label { color: var(--muted); font-size: 12px; display: inline-flex; align-items: center; gap:
4px; }
664 .zoomLbl { color: var(--muted); font-size: 12px; margin-left: auto; }
665 .presence {
666     display: inline-flex; gap: 4px; align-items: center;
667     padding: 2px 8px; background: rgba(40,217,127,0.12);
668     border: 1px solid rgba(40,217,127,0.4); border-radius: 9px;
669     color: var(--ok); font-size: 11px; font-weight: 600;
670 }

```

```

671 .presence::before { content: "• "; font-size: 10px; }
672
673 .img-stack svg .remote-cursor {
674     fill: rgba(0,0,0,0.85);
675     stroke-width: 2;
676     vector-effect: non-scaling-stroke;
677     pointer-events: none;
678 }
679 .img-stack svg .remote-cursor-label {
680     font-family: ui-sans-serif, system-ui;
681     font-weight: 600;
682     paint-order: stroke;
683     stroke: rgba(0,0,0,0.85);
684     stroke-width: 4;
685     stroke-linejoin: round;
686     pointer-events: none;
687 }
688 .save-status { color: var(--muted); font-size: 12px; min-width: 60px; text-align: right; }
689 .save-status.dirty { color: var(--warn); }
690 .save-status.saved { color: var(--ok); }
691 .save-status.err { color: var(--danger); }
692
693 .canvas-wrap {
694     flex: 1;
695     overflow: hidden;
696     position: relative;
697     background: #000;
698     cursor: grab;
699 }
700 .canvas-wrap.dragging { cursor: grabbing; }
701 .canvas-wrap.drawing { cursor: crosshair; }
702
703 .canvas {
704     position: absolute;
705     inset: 0;
706     display: flex;
707     align-items: center;
708     justify-content: center;
709 }
710 .empty { color: var(--muted); position: absolute; }
711
712 .sr-view {
713     position: absolute; inset: 0;
714     overflow: auto; padding: 30px 50px;
715     background: var(--bg); color: var(--text);
716     font-family: ui-sans-serif, system-ui, "Segoe UI", sans-serif;
717 }
718 .sr-view .sr-head {
719     background: var(--panel); border: 1px solid var(--border);
720     padding: 16px; border-radius: 6px; margin-bottom: 18px;
721 }
722 .sr-view .sr-head h2 {
723     margin: 0 0 6px; color: var(--accent); font-size: 16px;
724 }
725 .sr-view .sr-head .meta { color: var(--muted); font-size: 12px; }
726 .sr-view .sr-flag {
727     display: inline-block; padding: 2px 8px; margin-left: 6px;
728     background: var(--panel-2); border: 1px solid var(--border);
729     border-radius: 9px; font-size: 10px; text-transform: uppercase;
730 }
731 .sr-view .sr-item {
732     padding: 6px 0; border-bottom: 1px solid var(--border);
733 }
734 .sr-view .sr-item:last-child { border-bottom: none; }
735 .sr-view .sr-item .lbl {
736     font-weight: 600; color: var(--muted); font-size: 11px;

```

```

737 text-transform: uppercase; letter-spacing: 0.4px;
738 margin-bottom: 2px;
739 }
740 .sr-view .sr-item .val {
741   white-space: pre-wrap; word-break: break-word;
742   font-size: 12px; line-height: 1.5;
743 }
744 .sr-view .sr-item.depth-1 { padding-left: 0; }
745 .sr-view .sr-item.depth-2 { padding-left: 18px; }
746 .sr-view .sr-item.depth-3 { padding-left: 36px; }
747 .sr-view .sr-item.depth-4 { padding-left: 54px; }
748
749 .img-stack {
750   position: relative;
751   transform-origin: center center;
752   line-height: 0;
753 }
754 .img-stack img {
755   display: block;
756   max-width: none; max-height: none;
757   user-select: none;
758   -webkit-user-drag: none;
759   image-rendering: -webkit-optimize-contrast;
760 }
761 .img-stack svg {
762   position: absolute;
763   inset: 0;
764   width: 100%;
765   height: 100%;
766   pointer-events: none;
767   overflow: visible;
768 }
769 .img-stack svg .box {
770   fill-opacity: 0.10;
771   stroke-width: 2;
772   vector-effect: non-scaling-stroke;
773   pointer-events: all;
774   cursor: pointer;
775 }
776 .img-stack svg .box.selected {
777   stroke-width: 3;
778   filter: drop-shadow(0 0 4px rgba(255,255,255,0.6));
779 }
780 .img-stack svg .preview {
781   fill: none;
782   stroke: #ffd24d;
783   stroke-width: 2;
784   vector-effect: non-scaling-stroke;
785   stroke-dasharray: 6 4;
786   pointer-events: none;
787 }
788 .img-stack svg .suggestion {
789   fill: rgba(255,210,77,0.10);
790   stroke: #ffd24d;
791   stroke-width: 2;
792   vector-effect: non-scaling-stroke;
793   stroke-dasharray: 8 4;
794   pointer-events: all;
795   cursor: pointer;
796 }
797 .img-stack svg .suggestion:hover {
798   fill: rgba(255,210,77,0.22);
799   stroke-width: 3;
800 }
801 .img-stack svg .sug-label {
802   font-family: ui-monospace, "Cascadia Mono", Menlo, monospace;

```

```

803     fill: #ffd24d;
804     paint-order: stroke;
805     stroke: rgba(0,0,0,0.85);
806     stroke-width: 4;
807     stroke-linejoin: round;
808     pointer-events: none;
809     dominant-baseline: text-after-edge;
810 }
811 .img-stack svg .handle {
812     fill: #fff;
813     stroke: #000;
814     stroke-width: 1;
815     vector-effect: non-scaling-stroke;
816     pointer-events: all;
817 }
818 .img-stack svg .handle:hover { fill: #ffd24d; }
819 .img-stack svg .handle.midpoint { fill: #28d97f; }
820 .img-stack svg .handle.midpoint:hover { fill: #ffd24d; }
821
822 .img-stack svg .poly {
823     fill-opacity: 0.10;
824     stroke-width: 2;
825     vector-effect: non-scaling-stroke;
826     pointer-events: all;
827     cursor: pointer;
828 }
829 .img-stack svg .poly.selected {
830     stroke-width: 3;
831     filter: drop-shadow(0 0 4px rgba(255,255,255,0.6));
832 }
833 .img-stack svg .poly-preview {
834     fill: rgba(255,210,77,0.10);
835     stroke: #ffd24d;
836     stroke-width: 2;
837     vector-effect: non-scaling-stroke;
838     stroke-dasharray: 6 4;
839     pointer-events: none;
840 }
841 .img-stack svg .poly-vertex {
842     fill: #ffd24d;
843     stroke: #000;
844     stroke-width: 1;
845     vector-effect: non-scaling-stroke;
846     pointer-events: all;
847 }
848 .img-stack.drawing-poly svg { pointer-events: all; cursor: crosshair; }
849 .img-stack.drawing-poly svg .box,
850 .img-stack.drawing-poly svg .poly { pointer-events: none; }
851 .img-stack svg .label {
852     font-family: ui-monospace, "Cascadia Mono", Menlo, monospace;
853     fill: #fff;
854     paint-order: stroke;
855     stroke: rgba(0,0,0,0.85);
856     stroke-width: 4;
857     stroke-linejoin: round;
858     pointer-events: none;
859     dominant-baseline: text-after-edge;
860 }
861 .img-stack.drawing svg { pointer-events: all; cursor: crosshair; }
862 .img-stack.drawing svg .box { pointer-events: none; }
863
864 .wl-bar {
865     display: grid;
866     grid-template-columns: 1fr 1fr;
867     gap: 12px;
868     padding: 8px 12px;

```

```

869     background: var(--panel);
870     border-top: 1px solid var(--border);
871 }
872 .wl-group { display: flex; align-items: center; gap: 8px; }
873 .wl-group label { color: var(--muted); font-size: 12px; display: flex; align-items: center; gap: 6px;
}
874 .wl-group input[type="number"] {
875     width: 80px;
876     background: var(--panel-2);
877     border: 1px solid var(--border);
878     color: var(--text);
879     padding: 3px 6px;
880     border-radius: 3px;
881     font-size: 12px;
882 }
883 .wl-group input[type="range"] { flex: 1; }
884
885 .rpane { padding: 0; overflow: auto; }
886 .meta { width: 100%; border-collapse: collapse; font-size: 12px; }
887 .meta td { padding: 4px 6px; vertical-align: top; word-break: break-all; }
888 .meta td:first-child { color: var(--muted); width: 40%; }
889 .meta tr:nth-child(odd) { background: var(--panel-2); }
890
891 .anno-list { display: flex; flex-direction: column; }
892 .anno-empty { padding: 16px 14px; color: var(--muted); font-size: 12px; line-height: 1.5; }
893 .anno-empty kbd {
894     background: var(--panel-2); border: 1px solid var(--border); border-bottom-width: 2px;
895     border-radius: 3px; padding: 1px 5px; font-family: inherit; font-size: 11px;
896 }
897 .anno-row {
898     padding: 8px 12px;
899     border-bottom: 1px solid var(--border);
900     display: flex; gap: 8px; align-items: center;
901     cursor: pointer;
902 }
903 .anno-row:hover { background: var(--panel-2); }
904 .anno-row.selected { background: rgba(47,129,247,0.18); }
905 .anno-row .swatch { width: 12px; height: 12px; border-radius: 2px; flex-shrink: 0; border: 1px solid
rgba(0,0,0,0.4); }
906 .anno-row .info { flex: 1; min-width: 0; }
907 .anno-row .info .lbl { font-weight: 600; }
908 .anno-row .info .meta-row { color: var(--muted); font-size: 11px; margin-top: 1px; }
909 .anno-row .info select {
910     font-size: 11px;
911     padding: 1px 3px;
912     margin-right: 4px;
913 }
914 .anno-row .x { background: transparent; border: none; color: var(--muted); cursor: pointer; padding:
2px 6px; font-size: 14px; }
915 .anno-row .x:hover { color: var(--danger); }
916
917 /* Report pane */
918 .report-input {
919     padding: 10px 12px; border-bottom: 1px solid var(--border);
920     background: var(--panel-2);
921     color: var(--muted); font-size: 11px;
922     word-break: break-all;
923 }
924 .report-input b { color: var(--text); }
925 .report-empty {
926     padding: 16px 14px; color: var(--muted); font-size: 12px; line-height: 1.6;
927 }
928 .report-section-title {
929     padding: 8px 12px;
930     color: var(--muted); font-size: 11px; text-transform: uppercase; letter-spacing: 0.5px;
931     border-bottom: 1px solid var(--border);

```

```

932 }
933 .report-candidate {
934   display: block; width: calc(100% - 16px);
935   margin: 6px 8px;
936   padding: 8px 10px;
937   text-align: left;
938   background: var(--panel-2);
939   border: 1px solid var(--border);
940   border-radius: 4px;
941   color: var(--text);
942   font-size: 12px;
943 }
944 .report-candidate:hover { border-color: var(--accent); }
945 .cand-actions { display: flex; gap: 6px; margin-top: 8px; }
946 .cand-btn {
947   flex: 1;
948   background: var(--bg);
949   border: 1px solid var(--border);
950   color: var(--text);
951   padding: 5px 8px;
952   border-radius: 3px;
953   font-size: 11px;
954   cursor: pointer;
955 }
956 .cand-btn:hover { border-color: var(--accent); }
957 .cand-btn.link {
958   background: rgba(47,129,247,0.18);
959   border-color: rgba(47,129,247,0.4);
960   color: var(--accent);
961   font-weight: 600;
962 }
963 .cand-btn.link:hover { background: rgba(47,129,247,0.30); }
964 .report-candidate .top { display: flex; justify-content: space-between; align-items: center; gap: 6px; }
965 .report-candidate .cand-dismiss {
966   background: transparent; border: none; color: var(--muted);
967   cursor: pointer; padding: 2px 6px; font-size: 14px;
968   line-height: 1; flex-shrink: 0;
969 }
970 .report-candidate .cand-dismiss:hover { color: var(--danger); background: rgba(248,81,73,0.1); border-radius: 3px; }
971 .report-candidate .name { font-weight: 600; }
972 .report-candidate .score {
973   font-size: 10px; padding: 1px 6px; border-radius: 9px;
974   background: rgba(47,129,247,0.18); color: var(--accent);
975   border: 1px solid rgba(47,129,247,0.4);
976 }
977 .report-candidate .score.high { background: rgba(40,217,127,0.18); color: var(--ok); border-color: rgba(40,217,127,0.4); }
978 .report-candidate .score.med { background: rgba(255,176,0,0.18); color: var(--warn); border-color: rgba(255,176,0,0.4); }
979 .report-candidate .meta-line { color: var(--muted); font-size: 11px; margin-top: 2px; }
980
981 .link-banner {
982   display: flex; align-items: center; justify-content: space-between; gap: 8px;
983   padding: 8px 12px;
984   background: rgba(40,217,127,0.12);
985   border-bottom: 1px solid rgba(40,217,127,0.4);
986   color: var(--ok);
987   font-size: 12px;
988 }
989 .link-banner.linked b { color: var(--ok); }
990 .link-action {
991   background: var(--panel-2);
992   border: 1px solid var(--border);
993   color: var(--muted);

```

```

994 padding: 3px 8px;
995 border-radius: 3px;
996 font-size: 11px;
997 cursor: pointer;
998 }
999 .link-action:hover { color: var(--text); border-color: var(--accent); }
1000 .link-action.primary {
1001   background: rgba(47,129,247,0.18);
1002   border-color: rgba(47,129,247,0.4);
1003   color: var(--accent);
1004   margin-top: 6px;
1005   font-weight: 600;
1006 }
1007 .link-action.primary:hover { background: rgba(47,129,247,0.30); }
1008
1009 .report-patient {
1010   padding: 10px 12px;
1011   border-bottom: 1px solid var(--border);
1012 }
1013 .report-patient .name { font-size: 14px; font-weight: 600; }
1014 .report-patient .id { color: var(--muted); font-size: 11px; margin-top: 2px; }
1015 .report-patient .switch {
1016   margin-top: 6px; background: transparent; border: none;
1017   color: var(--accent); cursor: pointer; padding: 0; font-size: 11px;
1018 }
1019 .report-patient .switch:hover { text-decoration: underline; }
1020
1021 .record-card {
1022   border-bottom: 1px solid var(--border);
1023 }
1024 .record-card > summary {
1025   list-style: none;
1026   padding: 8px 12px; cursor: pointer;
1027   background: var(--panel-2);
1028   font-size: 12px;
1029   display: flex; gap: 8px; align-items: center;
1030 }
1031 .record-card > summary::-webkit-details-marker { display: none; }
1032 .record-card > summary::before {
1033   content: "▶"; color: var(--muted); font-size: 9px; transition: transform 0.15s;
1034 }
1035 .record-card[open] > summary::before { transform: rotate(90deg); }
1036 .record-card > summary:hover { background: rgba(47,129,247,0.08); }
1037 .record-card > summary .date { font-weight: 600; color: var(--accent); }
1038 .record-card > summary .code { color: var(--muted); font-size: 10px; padding: 1px 5px; background:
var(--bg); border-radius: 3px; }
1039 .record-body { padding: 6px 12px 10px; }
1040 .record-section { margin: 8px 0 0; }
1041 .record-section-label {
1042   color: var(--muted); font-size: 10px; text-transform: uppercase; letter-spacing: 0.5px;
1043   font-weight: 600; margin-bottom: 3px;
1044 }
1045 .record-section pre {
1046   margin: 0; padding: 6px 8px;
1047   white-space: pre-wrap; word-break: break-word;
1048   font-family: ui-monospace, "Cascadia Mono", Menlo, monospace;
1049   font-size: 11px; line-height: 1.45;
1050   color: var(--text); background: var(--panel-2);
1051   border-radius: 3px;
1052   max-height: 240px; overflow: auto;
1053 }
1054
1055 .report-search {
1056   padding: 8px 12px; border-bottom: 1px solid var(--border);
1057   display: flex; gap: 6px;
1058 }

```

```

1059 .report-search input {
1060   flex: 1;
1061   background: var(--panel-2); border: 1px solid var(--border);
1062   color: var(--text); padding: 4px 8px; border-radius: 3px; font-size: 12px;
1063 }
1064 .report-search button { padding: 4px 10px; }

```

app/static/app.js

Fayl yo'li: app/static/app.js | Qatorlar: 4921 | Hajm: 172700 bayt

```

1  'use strict';
2
3  const $ = (id) => document.getElementById(id);
4  const SVG_NS = 'http://www.w3.org/2000/svg';
5
6  const state = {
7    current: null,
8    meta: null,
9    frame: 0,
10   frames: 1,
11   wc: null, ww: null,
12   defaultWc: null, defaultWw: null,
13   invert: false,
14   scale: 1, tx: 0, ty: 0,
15   fitScale: 1,
16   localEnabled: false,
17   tool: 'select',
18   annotations: [],
19   selectedId: null,
20   labels: [],
21   birads: [],
22   saving: false,
23   dirty: false,
24   aiAvailable: false,
25   aiModels: [],
26   suggestions: [],
27   allSuggestions: [],
28   aiThreshold: 0.25,
29 };
30
31 function setStatus(msg) { $('status').textContent = msg; }
32
33 const TOKEN_KEY = 'mamograf_jwt';
34 function getToken() { return localStorage.getItem(TOKEN_KEY); }
35 function setToken(t) { if (t) localStorage.setItem(TOKEN_KEY, t); else
localStorage.removeItem(TOKEN_KEY); }
36
37 async function api(url, opts) {
38   opts = opts || {};
39   const headers = new Headers(opts.headers || {});
40   const tok = getToken();
41   if (tok && !headers.has('Authorization')) headers.set('Authorization', `Bearer ${tok}`);
42   const res = await fetch(url, { ...opts, headers });
43   if (res.status === 401) {
44     setToken(null);
45     state._user = null;
46     updateUserBar();
47     if (state.authRequired) showLoginModal('Sessiya tugadi. Qayta kiring.');
```



```

53   }
54   return res;
55 }
56 async function apiJson(url, opts) { return (await api(url, opts)).json(); }
57
58 function el(name, attrs, children) {
59   const e = document.createElementNS(SVG_NS, name);
60   if (attrs) for (const [k, v] of Object.entries(attrs)) e.setAttribute(k, v);
61   if (children) for (const c of children) e.appendChild(c);
62   return e;
63 }
64
65 function clamp01(v) { return Math.max(0, Math.min(1, v)); }
66 function uid() { return 'b' + Date.now().toString(36) + Math.random().toString(36).slice(2, 6); }
67 function colorFor(label) {
68   const m = state.labels.find(l => l.name === label);
69   return m ? m.color : '#ff5050';
70 }
71
72 function refSource() {
73   if (!state.current) return null;
74   return state.current.kind === 'upload' ? 'upload' : 'local';
75 }
76 function refValue() {
77   if (!state.current) return null;
78   return state.current.kind === 'upload' ? state.current.id : state.current.path;
79 }
80
81 function imageUrl() {
82   if (!state.current) return '';
83   const params = new URLSearchParams();
84   params.set('frame', state.frame);
85   if (state.wc !== null) params.set('wc', state.wc);
86   if (state.ww !== null) params.set('ww', state.ww);
87   if (state.invert) params.set('invert', 'true');
88   if (state.current.kind === 'upload') {
89     return `/api/files/${encodeURIComponent(state.current.id)}/image?${params}`;
90   }
91   params.set('path', state.current.path);
92   return `/api/local/image?${params}`;
93 }
94
95 async function loadConfig() {
96   const cfg = await apiJson('/api/config');
97   state.localEnabled = !!cfg.local_browsing;
98   state.authRequired = !!cfg.auth_required;
99   if (state.localEnabled) $('localTabBtn').hidden = false;
100 }
101
102 function showLoginModal(errMsg) {
103   $('loginErr').textContent = errMsg || '';
104   $('loginModal').hidden = false;
105   setTimeout(() => $('loginUsername').focus(), 50);
106 }
107 function hideLoginModal() {
108   $('loginModal').hidden = true;
109   $('loginErr').textContent = '';
110 }
111
112 function updateUserBar() {
113   const info = $('userInfo');
114   const logout = $('logoutBtn');
115   const login = $('loginBtn');
116   const admin = $('adminBtn');
117   const stats = $('statsBtn');
118   const bell = $('bellWrap');

```

```

119   if (state._user) {
120     const name = state._user.display_name || state._user.username;
121     info.innerHTML = `${name} <span class="role">${state._user.role}</span>`;
122     info.hidden = false;
123     logout.hidden = false;
124     login.hidden = true;
125     admin.hidden = state._user.role !== 'admin';
126     stats.hidden = !(state._user.role === 'admin' || state._user.role === 'reviewer');
127     $('pacsBtn').hidden = !(state._user.role === 'admin' || state._user.role === 'reviewer');
128     $('overviewBtn').hidden = false;
129     $('auditBtn').hidden = !(state._user.role === 'admin' || state._user.role === 'reviewer');
130     $('reviewBtn').hidden = !(state._user.role === 'admin' || state._user.role === 'reviewer');
131     $('totpBtn').hidden = false;
132     $('totpBtn').textContent = state._user.totp_enrolled ? '🔑' : '🔒';
133     $('totpBtn').title = state._user.totp_enrolled
134       ? "2FA yoqilgan – boshqarish"
135       : "2FA o'chirilgan – yoqish tavsiya etiladi";
136     bell.hidden = false;
137   } else {
138     info.hidden = true;
139     logout.hidden = true;
140     admin.hidden = true;
141     stats.hidden = true;
142     $('pacsBtn').hidden = true;
143     $('overviewBtn').hidden = true;
144     $('auditBtn').hidden = true;
145     $('reviewBtn').hidden = true;
146     $('totpBtn').hidden = true;
147     bell.hidden = true;
148     login.hidden = !state.authRequired;
149   }
150 }
151
152 function isAnnotator() { return state._user && state._user.role === 'annotator'; }
153 function isReviewer() { return state._user && (state._user.role === 'reviewer' || state._user.role ===
'admin'); }
154 function isAdmin() { return state._user && state._user.role === 'admin'; }
155 function ownAnn(a) { return state._user && a.created_by === state._user.username; }
156 function canEditAnn(a) {
157   if (!state._user) return false;
158   if (a.status === 'approved' && isAnnotator()) return false;
159   if (isAnnotator() && a.created_by && !ownAnn(a)) return false;
160   return true;
161 }
162
163 async function attemptResumeSession() {
164   if (!getToken()) return false;
165   try {
166     const me = await apiJson('/api/auth/me');
167     state._user = me;
168     updateUserBar();
169     return true;
170   } catch (e) {
171     setToken(null);
172     state._user = null;
173     return false;
174   }
175 }
176
177 async function doLogin(username, password, totpCode) {
178   const body = { username, password };
179   if (totpCode) body.totp_code = totpCode;
180   const res = await fetch('/api/auth/login', {
181     method: 'POST',
182     headers: { 'content-type': 'application/json' },
183     body: JSON.stringify(body),

```

```

184 });
185 if (!res.ok) {
186   let parsed = null;
187   try { parsed = await res.json(); } catch (e) {}
188   const detail = parsed && parsed.detail;
189   if (res.status === 401 && detail && typeof detail === 'object' && detail.error ===
'totp_required') {
190     const err = new Error('TOTP kodi kerak');
191     err.totp_required = true;
192     throw err;
193   }
194   if (res.status === 401) throw new Error("Noto'g'ri foydalanuvchi/parol/kod");
195   throw new Error(typeof detail === 'string' ? detail : `xato: ${res.status}`);
196 }
197 const data = await res.json();
198 setToken(data.token);
199 state._user = data.user;
200 updateUserBar();
201 return data;
202 }
203
204 function doLogout() {
205   setToken(null);
206   state._user = null;
207   updateUserBar();
208   if (state.authRequired) showLoginModal();
209 }
210
211 (function bindAuthUi() {
212   const form = $('#loginForm');
213   form.addEventListener('submit', async (e) => {
214     e.preventDefault();
215     const u = $('#loginUsername').value.trim();
216     const p = $('#loginPassword').value;
217     const t = $('#loginTotp').value.trim();
218     if (!u || !p) return;
219     const submit = $('#loginSubmit');
220     submit.disabled = true;
221     try {
222       await doLogin(u, p, t || null);
223       hideLoginModal();
224       $('#loginPassword').value = '';
225       $('#loginTotp').value = '';
226       $('#loginTotpRow').hidden = true;
227       await loadAuthenticatedAssets();
228       setStatus(`xush kelibsiz, ${state._user.display_name || state._user.username}`);
229     } catch (err) {
230       if (err.totp_required) {
231         $('#loginTotpRow').hidden = false;
232         $('#loginErr').textContent = 'Authenticator kodingizni kiriting';
233         setTimeout(() => $('#loginTotp').focus(), 50);
234       } else {
235         $('#loginErr').textContent = err.message;
236       }
237     } finally {
238       submit.disabled = false;
239     }
240   });
241   $('#logoutBtn').addEventListener('click', doLogout);
242   $('#loginBtn').addEventListener('click', () => showLoginModal());
243   $('#adminBtn').addEventListener('click', openAdminModal);
244   $('#adminCloseBtn').addEventListener('click', () => { $('#adminModal').hidden = true; });
245   $('#reviewBtn').addEventListener('click', openReviewModal);
246   $('#reviewCloseBtn').addEventListener('click', () => { $('#reviewModal').hidden = true; });
247 })();
248

```

```

249 // ===== Dashboard =====
250
251 async function loadDashboard() {
252   if (!state._user) return;
253   const status = $('dashStatus').value;
254   const own = $('dashOwn').checked;
255   const params = new URLSearchParams();
256   if (status) params.set('status', status);
257   if (own) params.set('own', 'true');
258   params.set('limit', '500');
259   let data;
260   try {
261     data = await apiJson(`/api/annotations/list?${params}`);
262   } catch (e) {
263     setStatus("dashboard xatosi: " + e.message);
264     return;
265   }
266   $('dashMeta').textContent = `${data.count} / ${data.total} ta annotatsiya`;
267   const ul = $('dashList');
268   ul.innerHTML = '';
269   for (const it of data.items) {
270     const li = document.createElement('li');
271     li.className = 'dash-row';
272     const top = document.createElement('div');
273     top.className = 'row';
274     const left = document.createElement('div');
275
276     const name = document.createElement('div');
277     name.className = 'name';
278     const lbl = it.label || '-';
279     const typeIcon = it.type === 'polygon' ? '●' : '□';
280     const nameTxt = document.createElement('span');
281     nameTxt.textContent = `${typeIcon} ${lbl}`;
282     nameTxt.style.fontWeight = '600';
283     name.appendChild(nameTxt);
284     if (it.bi_rads) {
285       const br = document.createElement('span');
286       br.className = 'badge';
287       br.textContent = it.bi_rads;
288       name.appendChild(br);
289     }
290     const sp = document.createElement('span');
291     sp.className = `status-pill status-${it.status}`;
292     sp.textContent = it.status;
293     name.appendChild(sp);
294     left.appendChild(name);
295
296     const sub = document.createElement('div');
297     sub.className = 'meta';
298     if (it.patient) {
299       sub.textContent = `${it.patient.last_name || ''} ${it.patient.first_name || ''} (ID
300       ${it.patient.patient_id})`;
301     } else {
302       sub.textContent = it.ref;
303     }
304     left.appendChild(sub);
305
306     const sub2 = document.createElement('div');
307     sub2.className = 'meta';
308     sub2.style.opacity = '0.7';
309     const updated = (it.updated_at || '').slice(0, 16).replace('T', ' ');
310     const reviewer = it.reviewed_by ? ` · ${it.status === 'approved' ? '✓' : 'X'}${it.reviewed_by}` :
311     '';
312     sub2.textContent = `${it.created_by || '-'} · ${updated}${reviewer}`;
313     left.appendChild(sub2);
314   }

```

```

313     if (it.review_note && it.status === 'rejected') {
314         const note = document.createElement('div');
315         note.className = 'review-note';
316         note.textContent = `« ${it.review_note} »`;
317         note.style.marginTop = '3px';
318         left.appendChild(note);
319     }
320
321     top.appendChild(left);
322     li.appendChild(top);
323     li.addEventListener('click', () => openAnnotationFromDashboard(it));
324     ul.appendChild(li);
325 }
326 }
327
328 async function openAnnotationFromDashboard(it) {
329     if (it.source === 'upload') {
330         await openItem({ kind: 'upload', id: it.ref });
331     } else {
332         await openItem({ kind: 'local', path: it.ref });
333     }
334     setTimeout(() => {
335         state.selectedId = it.annotation_id;
336         renderSvg();
337         renderAnnoList();
338         document.querySelectorAll('.meta-pane .tab').forEach(b => b.classList.remove('active'));
339         document.querySelector('.meta-pane .tab[data-rtab="anno"]')?.classList.add('active');
340         document.querySelectorAll('.meta-pane .rpane').forEach(p => p.hidden = p.dataset.rpane !==
'anno');
341     }, 600);
342 }
343
344 (function bindDashboardUi() {
345     $('dashStatus').addEventListener('change', loadDashboard);
346     $('dashOwn').addEventListener('change', loadDashboard);
347     $('dashRefresh').addEventListener('click', loadDashboard);
348 })();
349
350 // ===== Notifications =====
351
352 let _notifPollTimer = null;
353
354 async function refreshNotifications() {
355     if (!state._user) return;
356     try {
357         const data = await apiJson('/api/notifications?limit=20');
358         state._notifs = data.notifications || [];
359         state._notifUnread = data.unread_count || 0;
360         const badge = $('bellBadge');
361         if (state._notifUnread > 0) {
362             badge.textContent = state._notifUnread > 99 ? '99+' : state._notifUnread;
363             badge.hidden = false;
364         } else {
365             badge.hidden = true;
366         }
367     } catch (e) {
368         // non-fatal
369     }
370 }
371
372 function renderBellDropdown() {
373     const dd = $('bellDropdown');
374     dd.innerHTML = '';
375     const head = document.createElement('div');
376     head.className = 'head';
377     head.innerHTML = `<span>Bildirishnomalar</span>`;

```

```

378 if (state._notifUnread > 0) {
379   const all = document.createElement('button');
380   all.textContent = "Hammasini o'qildi qilish";
381   all.addEventListener('click', async (e) => {
382     e.stopPropagation();
383     try {
384       await api('/api/notifications/mark-read', {
385         method: 'POST',
386         headers: { 'content-type': 'application/json' },
387         body: JSON.stringify({ all: true }),
388       });
389       await refreshNotifications();
390       renderBellDropdown();
391     } catch (err) {}
392   });
393   head.appendChild(all);
394 }
395 dd.appendChild(head);
396
397 if (!state._notifs || state._notifs.length === 0) {
398   const empty = document.createElement('div');
399   empty.className = 'empty';
400   empty.textContent = "Bildirishnomalar yo'q";
401   dd.appendChild(empty);
402   return;
403 }
404
405 for (const n of state._notifs) {
406   const row = document.createElement('div');
407   row.className = 'notif-row' + (n.read_at ? '' : ' unread');
408   const head2 = document.createElement('div');
409   head2.className = 'title';
410   const k = document.createElement('span');
411   k.className = `notif-kind ${n.kind}`;
412   k.textContent = n.kind;
413   head2.appendChild(k);
414   const t = document.createElement('span');
415   t.textContent = n.title || '';
416   head2.appendChild(t);
417   row.appendChild(head2);
418
419   if (n.body) {
420     const b = document.createElement('div');
421     b.className = 'body';
422     b.textContent = n.body;
423     row.appendChild(b);
424   }
425   const m = document.createElement('div');
426   m.className = 'meta';
427   m.textContent = (n.ts || '').slice(0, 16).replace('T', ' ') + (n.actor ? ` · ${n.actor}` : '');
428   row.appendChild(m);
429
430   row.addEventListener('click', async () => {
431     if (!n.read_at) {
432       try {
433         await api('/api/notifications/mark-read', {
434           method: 'POST',
435           headers: { 'content-type': 'application/json' },
436           body: JSON.stringify({ ids: [n.id] }),
437         });
438       } catch (e) {}
439     }
440     $('bellDropdown').hidden = true;
441     if (n.link_source && n.link_ref) {
442       await openAnnotationFromDashboard({
443         source: n.link_source,

```

```

444         ref: n.link_ref,
445         annotation_id: n.link_annotation_id,
446     });
447     }
448     await refreshNotifications();
449     });
450     dd.appendChild(row);
451     }
452 }
453
454 (function bindNotifUi() {
455     $('bellBtn').addEventListener('click', async (e) => {
456         e.stopPropagation();
457         const dd = $('bellDropdown');
458         if (dd.hidden) {
459             await refreshNotifications();
460             renderBellDropdown();
461             dd.hidden = false;
462         } else {
463             dd.hidden = true;
464         }
465     });
466     document.addEventListener('click', (e) => {
467         const dd = $('bellDropdown');
468         if (!dd.hidden && !dd.contains(e.target) && e.target !== $('bellBtn')) {
469             dd.hidden = true;
470         }
471     });
472 })();
473
474 function startNotifPolling() {
475     if (_notifPollTimer) clearInterval(_notifPollTimer);
476     _notifPollTimer = setInterval(refreshNotifications, 30000);
477 }
478
479 async function openAdminModal() {
480     $('adminModal').hidden = false;
481     await renderAdminUsers();
482 }
483
484 async function renderAdminUsers() {
485     const body = $('adminBody');
486     body.innerHTML = '<div class="report-empty">Yuklanmoqda...</div>';
487     let data;
488     try {
489         data = await apiJson('/api/auth/users');
490     } catch (e) {
491         body.innerHTML = `<div class="admin-err">${e.message}</div>`;
492         return;
493     }
494     body.innerHTML = '';
495
496     let settings = { totp_required_roles: [] };
497     try { settings = await apiJson('/api/settings'); } catch (e) {}
498
499     const settingsBar = document.createElement('div');
500     settingsBar.style.cssText = 'background:var(--panel-2);border:1px solid var(--border);border-
radius:4px;padding:10px;margin-bottom:10px';
501     settingsBar.innerHTML = `
502         <div style="font-size:11px;color:var(--muted);text-transform:uppercase;margin-bottom:6px">TOTP
majburiy rollar</div>
503         <div style="display:flex;gap:14px;align-items:center">
504             ${[['admin', 'reviewer', 'annotator'].map(r => `
505                 <label style="display:flex;align-items:center;gap:4px;font-size:12px">
506                     <input type="checkbox" data-totp-role="${r}" ${settings.totp_required_roles.includes(r) ?
'checked' : ''}>`

```

```

507     ${r}
508   </label>
509   `).join('')}
510 </div>
511 `;
512 body.appendChild(settingsBar);
513 settingsBar.querySelectorAll('input[data-totp-role]').forEach(cb => {
514   cb.addEventListener('change', async () => {
515     const roles = [];
516     settingsBar.querySelectorAll('input[data-totp-role]').forEach(c => {
517       if (c.checked) roles.push(c.dataset.totpRole);
518     });
519     try {
520       await api('/api/settings', {
521         method: 'PATCH',
522         headers: { 'content-type': 'application/json' },
523         body: JSON.stringify({ totp_required_roles: roles }),
524       });
525     } catch (e) { alert(e.message); cb.checked = !cb.checked; }
526   });
527 });
528
529 const auditBar = document.createElement('div');
530 auditBar.style.cssText = 'display:flex; justify-content:flex-end; gap:6px; margin-bottom:10px;';
531 const auditBtn = document.createElement('button');
532 auditBtn.textContent = '↓ Audit log CSV';
533 auditBtn.title = 'Barcha annotation_history yozuvlarini CSV sifatida tushirish';
534 auditBtn.addEventListener('click', async () => {
535   const tok = getToken();
536   const res = await fetch('/api/annotations/history.csv', {
537     headers: { 'Authorization': `Bearer ${tok}` },
538   });
539   if (!res.ok) { alert('audit eksport xatosi: ' + res.status); return; }
540   const blob = await res.blob();
541   const url = URL.createObjectURL(blob);
542   const a = document.createElement('a');
543   a.href = url;
544   a.download = `audit_log_${new Date().toISOString().slice(0,10)}.csv`;
545   document.body.appendChild(a);
546   a.click();
547   a.remove();
548   URL.revokeObjectURL(url);
549 });
550 auditBar.appendChild(auditBtn);
551 body.appendChild(auditBar);
552 const table = document.createElement('table');
553 table.className = 'user-table';
554 table.innerHTML = `<thead><tr>
555   <th>username</th><th>role</th><th>display name</th><th>email</th>
556   <th>last login</th><th>active</th><th>amallar</th>
557 </tr></thead>`;
558 const tbody = document.createElement('tbody');
559 for (const u of data.users) {
560   const tr = document.createElement('tr');
561   if (!u.is_active) tr.className = 'inactive';
562   tr.innerHTML = `
563     <td><b>${u.username}</b></td>
564     <td><span class="role-pill role-${u.role}">${u.role}</span></td>
565     <td>${u.display_name || ''}</td>
566     <td>${u.email || ''}</td>
567     <td style="font-size:10px; color: var(--muted)">${(u.last_login_at || '-').slice(0,
19).replace('T', ' ')}</td>
568     <td>${u.is_active ? '√' : 'X'}</td>
569   `;
570   const actions = document.createElement('td');
571   actions.className = 'row-actions';

```



```

572
573 const roleSel = document.createElement('select');
574 roleSel.title = 'role';
575 for (const r of ['admin', 'reviewer', 'annotator']) {
576   const o = document.createElement('option');
577   o.value = r; o.textContent = r;
578   if (u.role === r) o.selected = true;
579   roleSel.appendChild(o);
580 }
581 roleSel.addEventListener('change', async () => {
582   try {
583     await api(`/api/auth/users/${encodeURIComponent(u.username)}`, {
584       method: 'PATCH',
585       headers: { 'content-type': 'application/json' },
586       body: JSON.stringify({ role: roleSel.value }),
587     });
588     await renderAdminUsers();
589   } catch (e) {
590     alert("Role o'zgartirib bo'lmadi: " + e.message);
591     roleSel.value = u.role;
592   }
593 });
594 actions.appendChild(roleSel);
595
596 const toggleBtn = document.createElement('button');
597 toggleBtn.textContent = u.is_active ? 'Disable' : 'Enable';
598 toggleBtn.addEventListener('click', async () => {
599   try {
600     await api(`/api/auth/users/${encodeURIComponent(u.username)}`, {
601       method: 'PATCH',
602       headers: { 'content-type': 'application/json' },
603       body: JSON.stringify({ is_active: !u.is_active }),
604     });
605     await renderAdminUsers();
606   } catch (e) { alert(e.message); }
607 });
608 actions.appendChild(toggleBtn);
609
610 const pwdBtn = document.createElement('button');
611 pwdBtn.textContent = '🔑 Reset';
612 pwdBtn.addEventListener('click', async () => {
613   const np = prompt(`Yangi parol (${u.username} uchun, min 4):`, '');
614   if (np === null) return;
615   if (np.length < 4) { alert('parol kalta'); return; }
616   try {
617     await api(`/api/auth/users/${encodeURIComponent(u.username)}/reset-password`, {
618       method: 'POST',
619       headers: { 'content-type': 'application/json' },
620       body: JSON.stringify({ new_password: np }),
621     });
622     alert(`${u.username} parolini yangilandi`);
623   } catch (e) { alert(e.message); }
624 });
625 actions.appendChild(pwdBtn);
626
627 if (u.username !== state._user.username) {
628   const delBtn = document.createElement('button');
629   delBtn.textContent = '🗑️';
630   delBtn.style.color = 'var(--danger)';
631   delBtn.title = "O'chirish";
632   delBtn.addEventListener('click', async () => {
633     if (!confirm(`${u.username} o'chirilsinmi?`)) return;
634     try {
635       await api(`/api/auth/users/${encodeURIComponent(u.username)}`, { method: 'DELETE' });
636       await renderAdminUsers();
637     } catch (e) { alert(e.message); }

```

```

638     });
639     actions.appendChild(delBtn);
640 }
641
642 tr.appendChild(actions);
643 tbody.appendChild(tr);
644 }
645 table.appendChild(tbody);
646 body.appendChild(table);
647
648 const form = document.createElement('div');
649 form.className = 'admin-create-form';
650 form.innerHTML = `
651     <label>Username <input type="text" id="newUsername" /></label>
652     <label>Role
653         <select id="newRole">
654             <option value="annotator">annotator</option>
655             <option value="reviewer">reviewer</option>
656             <option value="admin">admin</option>
657         </select>
658     </label>
659     <label>Display name <input type="text" id="newDisplayName" /></label>
660     <label>Email <input type="text" id="newEmail" /></label>
661     <label>Password <input type="password" id="newPassword" /></label>
662     <div class="admin-err" id="newErr"></div>
663     <div class="actions"><button id="newSubmit">+ Yaratish</button></div>
664 `;
665 body.appendChild(form);
666 $('newSubmit').addEventListener('click', async () => {
667     const username = $('newUsername').value.trim();
668     const password = $('newPassword').value;
669     const role = $('newRole').value;
670     const display_name = $('newDisplayName').value.trim() || null;
671     const email = $('newEmail').value.trim() || null;
672     if (!username || !password) {
673         $('newErr').textContent = "username va parol kerak"; return;
674     }
675     if (password.length < 4) {
676         $('newErr').textContent = "parol kalta (min 4)"; return;
677     }
678     try {
679         await api('/api/auth/users', {
680             method: 'POST',
681             headers: { 'content-type': 'application/json' },
682             body: JSON.stringify({ username, password, role, display_name, email }),
683         });
684         await renderAdminUsers();
685     } catch (e) {
686         $('newErr').textContent = e.message;
687     }
688 });
689 }
690
691 async function loadAiStatus() {
692     let s;
693     try {
694         s = await apiJson('/api/inference/status');
695     } catch (e) {
696         state.aiAvailable = false; return;
697     }
698     state.aiAvailable = !!s.available && (s.models || []).length > 0;
699     state.aiModels = s.models || [];
700     state.aiDevice = s.device_name || s.device;
701     state.aiInstalled = !!s.available;
702     const sel = $('aiModelSelect');
703     sel.innerHTML = '';

```

```

704 for (const m of state.aiModels) {
705     const o = document.createElement('option');
706     o.value = m.name;
707     o.textContent = m.stem;
708     sel.appendChild(o);
709 }
710 // Ensemble (Weighted Boxes Fusion of all models) – only meaningful with ≥2 models.
711 if (state.aiModels.length >= 2) {
712     const o = document.createElement('option');
713     o.value = '__ensemble__';
714     o.textContent = '🚀 Ensemble (barcha modellar · WBF)';
715     sel.appendChild(o);
716 }
717 if (state.aiAvailable) {
718     sel.hidden = false;
719     $('aiRunBtn').hidden = false;
720     $('aiThresholdWrap').hidden = false;
721     $('aiTtaWrap').hidden = false;
722     $('aiRunBtn').title = `AI tahlil (${state.aiDevice})`;
723 } else if (state.aiInstalled) {
724     setStatus(`AI tayyor, lekin model topilmadi (app/models/*.pt qo'shing)`);
725 }
726 }
727
728 function closeAllMultiSelects() {
729     document.querySelectorAll('.ms .ms-panel').forEach(p => { p.hidden = true; });
730 }
731 document.addEventListener('click', closeAllMultiSelects);
732
733 // Reusable checkbox dropdown for picking one OR several labels.
734 // opts: { selected: string[], onChange(arr), compact?: bool, placeholder?: string }
735 function buildLabelMultiSelect(opts) {
736     const host = document.createElement('span');
737     host.className = 'ms' + (opts.compact ? ' ms-compact' : '');
738     let selected = [...(opts.selected || [])];
739
740     const toggle = document.createElement('button');
741     toggle.type = 'button';
742     toggle.className = 'ms-toggle';
743
744     const panel = document.createElement('div');
745     panel.className = 'ms-panel';
746     panel.hidden = true;
747
748     function renderToggle() {
749         if (selected.length === 0) {
750             toggle.textContent = opts.placeholder || 'Label tanlang';
751             toggle.classList.remove('has-sel');
752         } else {
753             toggle.textContent = selected.join(', ');
754             toggle.classList.add('has-sel');
755         }
756         toggle.title = toggle.textContent;
757     }
758
759     function buildOptions() {
760         panel.innerHTML = '';
761         for (const l of state.labels) {
762             const opt = document.createElement('label');
763             opt.className = 'ms-opt';
764             const cb = document.createElement('input');
765             cb.type = 'checkbox';
766             cb.checked = selected.includes(l.name);
767             const dot = document.createElement('span');
768             dot.className = 'ms-dot';
769             dot.style.background = l.color;

```

```

770     const name = document.createElement('span');
771     name.className = 'ms-name';
772     name.textContent = l.name;
773     opt.appendChild(cb); opt.appendChild(dot); opt.appendChild(name);
774     cb.addEventListener('change', () => {
775         if (cb.checked) {
776             if (!selected.includes(l.name)) selected.push(l.name);
777         } else {
778             selected = selected.filter(x => x !== l.name);
779         }
780         renderToggle();
781         if (opts.onChange) opts.onChange([...selected]);
782     });
783     opt.addEventListener('click', e => e.stopPropagation());
784     panel.appendChild(opt);
785 }
786 }
787
788 toggle.addEventListener('click', e => {
789     e.stopPropagation();
790     const willOpen = panel.hidden;
791     closeAllMultiSelects();
792     if (willOpen) { buildOptions(); panel.hidden = false; }
793 });
794
795 host._msSet = (arr) => { selected = [...(arr || [])]; renderToggle(); };
796 host._msGet = () => [...selected];
797
798 renderToggle();
799 host.appendChild(toggle);
800 host.appendChild(panel);
801 return host;
802 }
803
804 async function loadLabels() {
805     const data = await apiJson('/api/labels');
806     state.labels = data.labels || [];
807     state.birads = data.birads || [];
808
809     // Toolbar keeps the classic single <select> (one label for the next new
810     // annotation). Multiple labels are added per-annotation in the right panel.
811     const sel = $('labelSelect');
812     sel.innerHTML = '';
813     for (const l of state.labels) {
814         const o = document.createElement('option');
815         o.value = l.name; o.textContent = l.name;
816         o.style.color = l.color;
817         sel.appendChild(o);
818     }
819
820     const bsel = $('biradsSelect');
821     for (const b of state.birads) {
822         const o = document.createElement('option');
823         o.value = b; o.textContent = `BI-RADS ${b}`;
824         bsel.appendChild(o);
825     }
826 }
827
828 async function refreshUploads() {
829     const data = await apiJson('/api/files');
830     renderFileList($('uploadList'), data.files.map(f => ({
831         kind: 'upload', id: f.id, label: fileLabel(f), sub: fileSub(f),
832         removable: true, annoCount: f.annotation_count || 0,
833     })));
834 }
835 function fileLabel(f) {

```

```

836   const parts = [];
837   if (f.patient) parts.push(f.patient);
838   if (f.modality) parts.push(f.modality);
839   return parts.join(' · ') || f.id;
840 }
841 function fileSub(f) {
842   const parts = [];
843   if (f.laterality) parts.push(f.laterality);
844   if (f.view) parts.push(f.view);
845   if (f.study_date) parts.push(f.study_date);
846   if (f.rows && f.cols) parts.push(`${f.cols}×${f.rows}`);
847   return parts.join(' · ');
848 }
849
850 function renderFileList(ul, items) {
851   ul.innerHTML = '';
852   const batchable = items.some(i => i.kind === 'upload' || i.kind === 'local');
853   if (batchable && state._user && state.aiAvailable) {
854     const bar = document.createElement('div');
855     bar.style.cssText = 'padding:6px 10px; border-bottom:1px solid var(--border); display:flex;
gap:6px; align-items:center;';
856     bar.innerHTML = `
857       <span style="font-size:10px; color:var(--muted)">tanlangan: <span
id="batchCount_${ul.id}">0</span></span></span>
858       <button class="batch-ai-btn" data-list="${ul.id}" style="font-size:11px; padding:3px 8px;"
disabled><img alt="AI icon" data-bbox="215 395 230 405"/> AI batch</button>
859     `;
860     ul.appendChild(bar);
861   }
862   for (const it of items) {
863     const li = document.createElement('li');
864     li.className = it.kind === 'dir' ? 'dir' : 'file';
865     li.dataset.kind = it.kind;
866     if (it.id) li.dataset.id = it.id;
867     if (it.path != null) li.dataset.path = it.path;
868
869     const top = document.createElement('div');
870     top.className = 'row';
871
872     if ((it.kind === 'upload' || it.kind === 'local') && state._user) {
873       const cb = document.createElement('input');
874       cb.type = 'checkbox';
875       cb.className = 'batch-cb';
876       cb.style.cssText = 'margin-right:6px;';
877       cb.addEventListener('click', e => e.stopPropagation());
878       cb.addEventListener('change', () => updateBatchCount(ul));
879       top.appendChild(cb);
880     }
881
882     const left = document.createElement('div');
883     const name = document.createElement('div');
884     name.className = 'name';
885     name.textContent = it.label;
886     if (it.annoCount && it.annoCount > 0) {
887       const pill = document.createElement('span');
888       pill.className = 'anno-pill';
889       pill.textContent = it.annoCount;
890       pill.title = `${it.annoCount} annotatsiya`;
891       name.appendChild(pill);
892     }
893     left.appendChild(name);
894     if (it.sub) {
895       const sub = document.createElement('div');
896       sub.className = 'meta';
897       sub.textContent = it.sub;
898       left.appendChild(sub);

```

```

899     }
900     top.appendChild(left);
901
902     if (it.removable) {
903         const x = document.createElement('button');
904         x.className = 'delete';
905         x.textContent = 'X';
906         x.title = "O'chirish";
907         x.addEventListener('click', async (e) => {
908             e.stopPropagation();
909             await api(`/api/files/${encodeURIComponent(it.id)}`, { method: 'DELETE' });
910             if (state.current && state.current.id === it.id) clearViewer();
911             refreshUploads();
912         });
913         top.appendChild(x);
914     }
915
916     li.appendChild(top);
917     li.addEventListener('click', () => onItemClick(it, li));
918     ul.appendChild(li);
919 }
920 }
921
922 function updateBatchCount(ul) {
923     const checked = ul.querySelectorAll('.batch-cb:checked');
924     const count = checked.length;
925     const span = ul.querySelector(`#batchCount_${ul.id}`);
926     if (span) span.textContent = count;
927     const btn = ul.querySelector('.batch-ai-btn');
928     if (btn) btn.disabled = count === 0;
929 }
930
931 document.addEventListener('click', async (e) => {
932     if (!e.target.classList.contains('batch-ai-btn')) return;
933     const ulId = e.target.dataset.list;
934     const ul = document.getElementById(ulId);
935     if (!ul) return;
936     const items = [];
937     ul.querySelectorAll('.batch-cb:checked').forEach(cb => {
938         const li = cb.closest('li');
939         if (!li) return;
940         if (li.dataset.kind === 'upload') items.push({ source: 'upload', ref: li.dataset.id });
941         else if (li.dataset.kind === 'local') items.push({ source: 'local', ref: li.dataset.path });
942     });
943     if (!items.length) return;
944     const model = $('aiModelSelect').value;
945     if (!model) { alert("AI model tanlang (toolbar'dan)"); return; }
946     if (!confirm(`${items.length} ta DICOM'ga ${model} bilan AI batch ishga tushirilsinmi?\n\nNatijalar avtomatik annotation sifatida saqlanadi.`)) return;
947     e.target.disabled = true;
948     e.target.textContent = '🕒 ishlamoqda...';
949     try {
950         const r = await apiJson('/api/inference/batch', {
951             method: 'POST',
952             headers: { 'content-type': 'application/json' },
953             body: JSON.stringify({ items, model, conf: state.aiThreshold || 0.25, auto_save: true,
954                                 tta: !($('aiTtaToggle') && $('aiTtaToggle').checked) }),
955         });
956         const ok = r.results.filter(x => x.ok).length;
957         const det = r.results.reduce((s, x) => s + (x.detections || 0), 0);
958         alert(`Batch tugadi:\n/ ${ok}/${items.length} muvaffaqiyat\n${det} ta detection saqlandi`);
959         refreshActiveList();
960     } catch (err) {
961         alert("xato: " + err.message);
962     } finally {
963         e.target.disabled = false;

```

```

964     e.target.textContent = '🤖 AI batch';
965   }
966 });
967
968 async function onItemClick(it, li) {
969   if (it.kind === 'dir') { loadLocalDir(it.path); return; }
970   document.querySelectorAll('.file-list li.active').forEach(x => x.classList.remove('active'));
971   li.classList.add('active');
972   await openItem(it);
973 }
974
975 async function openItem(it) {
976   // Safety net: persist unsaved work before leaving the current image so
977   // manual-save mode never silently loses annotations on navigation.
978   if (state.dirty && state.current) {
979     try { await saveAnnotations(); } catch (e) { /* keep navigating */ }
980   }
981   setStatus("o'qilmoqda...");
982   state.current = { kind: it.kind, id: it.id, path: it.path };
983   state.frame = 0;
984   state.wc = null; state.ww = null;
985   state.invert = false;
986   state.scale = 1; state.tx = 0; state.ty = 0;
987   state.annotations = [];
988   state.selectedId = null;
989   state.suggestions = [];
990   state.allSuggestions = [];
991   $('aiClearBtn').hidden = true;
992   $('aiAcceptAllBtn').hidden = true;
993
994   let metaUrl;
995   if (it.kind === 'upload') metaUrl = `/api/files/${encodeURIComponent(it.id)}/metadata`;
996   else metaUrl = `/api/local/metadata?path=${encodeURIComponent(it.path)}`;
997
998   const meta = await apiJson(metaUrl);
999   state.meta = meta;
1000   renderMeta(meta);
1001
1002   state.defaultWc = parseFirstNumber(meta.WindowCenter);
1003   state.defaultWw = parseFirstNumber(meta.WindowWidth);
1004   state.frames = parseInt(meta.NumberOfFrames || '1', 10) || 1;
1005   applyDefaults();
1006   configureFrameSlider();
1007   configureWLSliders(meta);
1008
1009   await loadAnnotations();
1010   state.dirty = false;
1011   setSaveStatus('', '');
1012   updateSaveBtn();
1013   loadReport().catch(() => {});
1014   await loadImage(true);
1015   $('deidBtn').hidden = false;
1016   $('srBtn').hidden = false;
1017   $('segBtn').hidden = false;
1018   $('maskPngBtn').hidden = false;
1019   $('maskNiftiBtn').hidden = false;
1020   $('radiomicsBtn').hidden = false;
1021   $('cStoreBtn').hidden = false;
1022   $('tplSelect').hidden = false;
1023   $('tplSaveBtn').hidden = false;
1024   refreshTemplates().catch(() => {});
1025   wsConnect();
1026   setStatus('tayyor');
1027 }
1028
1029 const RECORD_SECTIONS = [

```

```

1030 ['report', 'Mammografiya hisoboti'],
1031 ['complaints', 'Shikoyatlari'],
1032 ['history', 'Anamnesis morbi'],
1033 ['clinical_findings', 'Klinik ko'rinish'],
1034 ['recommendations', 'Tavsiyalar'],
1035 ['diagnosis', 'Tashxis'],
1036 ['treatment', 'Davolash / oilaviy anamnez'],
1037 ['lab_notes', 'Laboratoriya / UTT / gistologiya'],
1038 ['disease_info', 'Status praesens'],
1039 ['occupational', 'Status localis'],
1040 ['admission_reason', 'Kelish sababi'],
1041 ['diagnostic_procedures', 'Dagnostik amaliyot'],
1042 ['discharge_meds', 'Chiqish dorilari'],
1043 ];
1044
1045 function _dismissedKey() {
1046   if (!state.current) return null;
1047   return `mamograf_dismissed_${refSource()}__${refValue()}`;
1048 }
1049 function getDismissed() {
1050   const k = _dismissedKey();
1051   if (!k) return new Set();
1052   try {
1053     const raw = localStorage.getItem(k);
1054     return new Set(raw ? JSON.parse(raw) : []);
1055   } catch (e) { return new Set(); }
1056 }
1057 function addDismissed(patientId) {
1058   const k = _dismissedKey();
1059   if (!k) return;
1060   const s = getDismissed();
1061   s.add(patientId);
1062   try { localStorage.setItem(k, JSON.stringify([...s])); } catch (e) {}
1063 }
1064 function clearDismissed() {
1065   const k = _dismissedKey();
1066   if (k) try { localStorage.removeItem(k); } catch (e) {}
1067 }
1068
1069 async function loadReport() {
1070   if (!state.meta) return;
1071   state._linked = null;
1072
1073   const refS = refSource();
1074   const refV = refValue();
1075   if (refS && refV) {
1076     try {
1077       const linkData = await apiJson(`/api/db/link?source=${refS}&ref=${encodeURIComponent(refV)}`);
1078       if (linkData.linked && linkData.patient) {
1079         state._linked = linkData;
1080         state._matchInput = {
1081           patient_id: state.meta.PatientID || '',
1082           name: state.meta.PatientName || '',
1083           birth_date: state.meta.PatientBirthDate || '',
1084           parsed: {},
1085         };
1086         await loadAndRenderPatient(linkData.patient.patient_id, /*append*/false);
1087         return;
1088       }
1089     } catch (e) {
1090       // ignore – fall through to fuzzy match
1091     }
1092   }
1093
1094   const params = new URLSearchParams();
1095   if (state.meta.PatientID) params.set('patient_id', state.meta.PatientID);

```



```

1096 if (state.meta.PatientName) params.set('name', state.meta.PatientName);
1097 if (state.meta.PatientBirthDate) params.set('birth_date', state.meta.PatientBirthDate);
1098 let data;
1099 try {
1100   data = await apiJson(`/api/db/match?${params}`);
1101 } catch (e) {
1102   renderReportError(e.message);
1103   return;
1104 }
1105 state._matchInput = data.input;
1106 const dismissed = getDismissed();
1107 data.candidates = (data.candidates || []).filter(c => !dismissed.has(c.patient.patient_id));
1108 if (data.candidates.length === 1 && data.candidates[0].score >= 80) {
1109   renderReportInputBar(data.input);
1110   await loadAndRenderPatient(data.candidates[0].patient.patient_id, /*append*/true);
1111 } else if (data.candidates.length > 0) {
1112   renderReportCandidates(data);
1113 } else {
1114   renderReportEmpty(data.input);
1115 }
1116 }
1117
1118 async function saveLink(patientId, confidence) {
1119   await api('/api/db/link', {
1120     method: 'PUT',
1121     headers: { 'content-type': 'application/json' },
1122     body: JSON.stringify({
1123       source: refSource(),
1124       ref: refValue(),
1125       patient_id: patientId,
1126       confidence: Math.round(Number(confidence) || 0),
1127     }),
1128   });
1129 }
1130
1131 async function unlinkPatient() {
1132   if (!state.current) return;
1133   if (!confirm("Bemor bilan aloqani uzasizmi?")) return;
1134   try {
1135     await api(
1136       `/api/db/link?source=${refSource()}&ref=${encodeURIComponent(refValue())}`,
1137       { method: 'DELETE' },
1138     );
1139     state._linked = null;
1140     await loadReport();
1141   } catch (e) {
1142     setStatus('uzish xatosi: ' + e.message);
1143   }
1144 }
1145
1146 function renderReportError(msg) {
1147   const pane = $('reportPane');
1148   pane.innerHTML = `

153


```

```

1161 function renderReportCandidates(data) {
1162   renderReportInputBar(data.input);
1163   const pane = $('reportPane');
1164   const title = document.createElement('div');
1165   title.className = 'report-section-title';
1166   const dismissedCount = getDismissed().size;
1167   title.innerHTML = `${data.candidates.length} ta nomzod` +
1168     (dismissedCount > 0
1169       ? ` <a href="#" id="restoreDismissed" style="font-size:10px; color:var(--accent); margin-left:6px; text-decoration:none">+${dismissedCount} yashirilgan </a>`
1170       : '');
1171   pane.appendChild(title);
1172   const restore = pane.querySelector('#restoreDismissed');
1173   if (restore) restore.addEventListener('click', (e) => { e.preventDefault(); clearDismissed(); loadReport(); });
1174   for (const c of data.candidates) {
1175     pane.appendChild(buildCandidateBtn(c));
1176   }
1177   appendManualSearch(pane);
1178 }
1179
1180 function buildCandidateBtn(c) {
1181   const p = c.patient;
1182   const btn = document.createElement('div');
1183   btn.className = 'report-candidate';
1184   const scoreCls = c.score >= 80 ? 'high' : (c.score >= 60 ? 'med' : '');
1185   const top = document.createElement('div');
1186   top.className = 'top';
1187   const nm = document.createElement('div');
1188   nm.className = 'name';
1189   nm.textContent = `${p.last_name || ''} ${p.first_name || ''}.trim() || '-';
1190   const sc = document.createElement('div');
1191   sc.className = `score ${scoreCls}`;
1192   sc.textContent = `${c.score}%`;
1193   const dismissBtn = document.createElement('button');
1194   dismissBtn.className = 'cand-dismiss';
1195   dismissBtn.textContent = 'X';
1196   dismissBtn.title = "Bu nomzodni ro'yxatdan olib tashlash";
1197   dismissBtn.addEventListener('click', (e) => {
1198     e.stopPropagation();
1199     addDismissed(p.patient_id);
1200     btn.remove();
1201     const list = document.querySelectorAll('#reportPane .report-candidate');
1202     if (list.length === 0) {
1203       const pane = $('reportPane');
1204       const empty = document.createElement('div');
1205       empty.className = 'report-empty';
1206       empty.innerHTML = `Barcha nomzodlar yashirildi. <a href="#" id="restoreDismissed">Ko'rsatish</a>`;
1207       pane.appendChild(empty);
1208       const link = pane.querySelector('#restoreDismissed');
1209       if (link) link.addEventListener('click', (ev) => { ev.preventDefault(); clearDismissed(); loadReport(); });
1210     }
1211   });
1212   top.append(nm, sc, dismissBtn);
1213   btn.appendChild(top);
1214   const meta1 = document.createElement('div');
1215   meta1.className = 'meta-line';
1216   meta1.textContent = `ID ${p.patient_id} · ${p.sex || '-'} · DOB ${p.birth_date || '-'}`;
1217   btn.appendChild(meta1);
1218   const meta2 = document.createElement('div');
1219   meta2.className = 'meta-line';
1220   meta2.textContent = `${c.record_count} yozuv${c.last_visit ? ' · oxirgi ' + c.last_visit : ''} · ${c.reasons || []}.join(', ')`;
1221   btn.appendChild(meta2);

```

```

1222
1223   const actions = document.createElement('div');
1224   actions.className = 'cand-actions';
1225   const previewBtn = document.createElement('button');
1226   previewBtn.className = 'cand-btn';
1227   previewBtn.textContent = '🔍 Yozuvlarni ko\'rish';
1228   previewBtn.addEventListener('click', (e) => {
1229     e.stopPropagation();
1230     loadAndRenderPatient(p.patient_id, /*append*/false);
1231   });
1232   const linkBtn = document.createElement('button');
1233   linkBtn.className = 'cand-btn link';
1234   linkBtn.textContent = '🔗 Bog\'lash';
1235   linkBtn.title = 'Shu DICOM bilan shu bemorni doimiy bog\'lash';
1236   linkBtn.addEventListener('click', async (e) => {
1237     e.stopPropagation();
1238     try {
1239       await saveLink(p.patient_id, c.score);
1240       await loadReport();
1241     } catch (err) {
1242       setStatus('bog\'lash xatosi: ' + err.message);
1243     }
1244   });
1245   actions.append(previewBtn, linkBtn);
1246   btn.appendChild(actions);
1247   return btn;
1248 }
1249
1250 function renderReportEmpty(input) {
1251   const pane = $('reportPane');
1252   pane.innerHTML = '';
1253   renderReportInputBar(input);
1254   const empty = document.createElement('div');
1255   empty.className = 'report-empty';
1256   empty.innerHTML = `Mos hisobot topilmadi.<br>Quyida ID yoki ism bo'yicha qo'lda qidirishingiz
mumkin.`;
1257   pane.appendChild(empty);
1258   appendManualSearch(pane);
1259 }
1260
1261 function appendManualSearch(pane) {
1262   const wrap = document.createElement('div');
1263   wrap.className = 'report-search';
1264   const inp = document.createElement('input');
1265   inp.type = 'text';
1266   inp.placeholder = "ID yoki ism (masalan, 14173 yoki SHUKUROVA)";
1267   const btn = document.createElement('button');
1268   btn.textContent = 'Qidirish';
1269   const run = async () => {
1270     const q = inp.value.trim();
1271     if (!q) return;
1272     try {
1273       const data = await apiJson(`/api/db/search?q=${encodeURIComponent(q)}&limit=15`);
1274       renderManualResults(data, q);
1275     } catch (e) { renderReportError(e.message); }
1276   };
1277   btn.addEventListener('click', run);
1278   inp.addEventListener('keydown', (e) => { if (e.key === 'Enter') run(); });
1279   wrap.append(inp, btn);
1280   pane.appendChild(wrap);
1281 }
1282
1283 function renderManualResults(data, q) {
1284   const pane = $('reportPane');
1285   let title = pane.querySelector('.report-section-title.manual');
1286   if (!title) {

```

```

1287     title = document.createElement('div');
1288     title.className = 'report-section-title manual';
1289     pane.appendChild(title);
1290 }
1291 title.textContent = `Qidiruv natijalari "${q}" - ${data.results.length}`;
1292 let listWrap = pane.querySelector('.manual-results');
1293 if (listWrap) listWrap.remove();
1294 listWrap = document.createElement('div');
1295 listWrap.className = 'manual-results';
1296 for (const r of data.results) {
1297     const c = {
1298         patient: r,
1299         score: 0,
1300         reasons: ['qo\lda qidiruv'],
1301         record_count: r.record_count || 0,
1302         last_visit: r.last_visit,
1303     };
1304     listWrap.appendChild(buildCandidateBtn(c));
1305 }
1306 pane.appendChild(listWrap);
1307 }
1308
1309 async function loadAndRenderPatient(patientId, append) {
1310     let data;
1311     try {
1312         data = await apiJson(`/api/db/patient/${encodeURIComponent(patientId)}`);
1313     } catch (e) {
1314         renderReportError(e.message);
1315         return;
1316     }
1317     const pane = $('reportPane');
1318     if (!append) {
1319         pane.innerHTML = '';
1320         if (state._matchInput) renderReportInputBar(state._matchInput);
1321     }
1322
1323     const isLinked = state._linked && state._linked.patient && state._linked.patient.patient_id ===
patientId;
1324
1325     if (isLinked) {
1326         const banner = document.createElement('div');
1327         banner.className = 'link-banner linked';
1328         const txt = document.createElement('div');
1329         const lk = state._linked.link || {};
1330         const conf = lk.confidence != null ? ` · ${lk.confidence}%` : '';
1331         const at = lk.confirmed_at ? ` · ${lk.confirmed_at.slice(0, 10)}` : '';
1332         txt.innerHTML = `🔗 <b>Bog'langan</b>${lk.confidence}${lk.confirmed_at}`;
1333         const unlink = document.createElement('button');
1334         unlink.className = 'link-action';
1335         unlink.textContent = '🔗 Aloqani uzish';
1336         unlink.addEventListener('click', unlinkPatient);
1337         banner.append(txt, unlink);
1338         pane.appendChild(banner);
1339     }
1340
1341     const head = document.createElement('div');
1342     head.className = 'report-patient';
1343     const p = data.patient;
1344     const nm = document.createElement('div');
1345     nm.className = 'name';
1346     nm.textContent = `${p.last_name || ''} ${p.first_name || ''}`.trim() || '-';
1347     const id = document.createElement('div');
1348     id.className = 'id';
1349     id.textContent = `ID ${p.patient_id} · ${p.sex || '-'} · DOB ${p.birth_date || '-'} ·
${data.records.length} yozuv`;
1350     head.append(nm, id);

```

```

1351
1352   if (!isLinked) {
1353     const linkHere = document.createElement('button');
1354     linkHere.className = 'link-action primary';
1355     linkHere.textContent = "🔗 Shu bemorga doimiy bog'lash";
1356     linkHere.addEventListener('click', async () => {
1357       try {
1358         await saveLink(p.patient_id, null);
1359         await loadReport();
1360       } catch (e) { setStatus("bog'lash xatosi: " + e.message); }
1361     });
1362     head.appendChild(linkHere);
1363   }
1364
1365   const sw = document.createElement('button');
1366   sw.className = 'switch';
1367   sw.textContent = '← Boshqa bemorni tanlash';
1368   sw.addEventListener('click', () => loadReport());
1369   head.appendChild(sw);
1370   pane.appendChild(head);
1371
1372   for (let i = 0; i < data.records.length; i++) {
1373     pane.appendChild(buildRecordCard(data.records[i], i === 0));
1374   }
1375
1376   const badge = $('reportBadge');
1377   badge.textContent = data.records.length;
1378   badge.hidden = data.records.length === 0;
1379 }
1380
1381 function buildRecordCard(r, openByDefault) {
1382   const det = document.createElement('details');
1383   det.className = 'record-card';
1384   if (openByDefault) det.open = true;
1385
1386   const sum = document.createElement('summary');
1387   const dt = document.createElement('span');
1388   dt.className = 'date';
1389   dt.textContent = r.service_date || '-';
1390   sum.appendChild(dt);
1391   if (r.exam_code) {
1392     const code = document.createElement('span');
1393     code.className = 'code';
1394     code.textContent = r.exam_code;
1395     sum.appendChild(code);
1396   }
1397   if (r.exam_name) {
1398     const en = document.createElement('span');
1399     en.textContent = r.exam_name;
1400     sum.appendChild(en);
1401   }
1402   det.appendChild(sum);
1403
1404   const body = document.createElement('div');
1405   body.className = 'record-body';
1406   let any = false;
1407   for (const [key, label] of RECORD_SECTIONS) {
1408     const v = r[key];
1409     if (!v) continue;
1410     any = true;
1411     const sec = document.createElement('div');
1412     sec.className = 'record-section';
1413     const lbl = document.createElement('div');
1414     lbl.className = 'record-section-label';
1415     lbl.textContent = label;
1416     const pre = document.createElement('pre');

```

```

1417     pre.textContent = v;
1418     sec.appendChild(lbl, pre);
1419     body.appendChild(sec);
1420 }
1421 if (!any) {
1422     const empty = document.createElement('div');
1423     empty.className = 'report-empty';
1424     empty.style.padding = '8px 0 0';
1425     empty.textContent = 'Bo\'sh yozuv';
1426     body.appendChild(empty);
1427 }
1428 det.appendChild(body);
1429 return det;
1430 }
1431
1432 async function loadAnnotations() {
1433     if (!state.current) return;
1434     try {
1435         const data = await
apiJson(`/api/annotations?source=${refSource()}&ref=${encodeURIComponent(refValue())}`);
1436         state.annotations = (data.annotations || []).map(normalizeAnn);
1437     } catch (e) {
1438         state.annotations = [];
1439     }
1440     renderSvg();
1441     renderAnnoList();
1442 }
1443
1444 function normalizeAnn(a) {
1445     const out = Object.assign({}, a);
1446     if (!out.id) out.id = uid();
1447     if (!out.bbox) out.bbox = [0, 0, 0, 0];
1448     if (typeof out.frame !== 'number') out.frame = 0;
1449     // Multi-label support: `labels` is the source of truth, `label` stays as the
1450     // primary (first) label for backward compatibility (color, export, stats).
1451     if (Array.isArray(out.labels) && out.labels.length) {
1452         out.label = out.labels[0];
1453     } else {
1454         out.labels = out.label ? [out.label] : [];
1455     }
1456     return out;
1457 }
1458
1459 // Read the primary + full label set for an annotation, tolerating old records.
1460 function annLabels(a) {
1461     if (Array.isArray(a.labels) && a.labels.length) return a.labels;
1462     return a.label ? [a.label] : [];
1463 }
1464 function annLabelText(a) {
1465     const ls = annLabels(a);
1466     return ls.length ? ls.join(' + ') : '-';
1467 }
1468
1469 // Primary label chosen in the toolbar for a new annotation (always one).
1470 // Extra labels are added afterwards via the per-annotation editor.
1471 function pendingLabels() {
1472     const sel = $('#labelSelect');
1473     const v = sel ? sel.value : '';
1474     return v ? [v] : [state.labels[0]?.name || 'mass'];
1475 }
1476
1477 function parseFirstNumber(v) {
1478     if (v == null) return null;
1479     if (typeof v === 'number') return v;
1480     const s = String(v).trim();
1481     const m = s.match(/-?\d+(\.\d+)?/);

```

```

1482     return m ? parseFloat(m[0]) : null;
1483 }
1484
1485 function applyDefaults() {
1486     state.wc = state.defaultWc;
1487     state.ww = state.defaultWw;
1488     $('wcInput').value = state.wc != null ? Math.round(state.wc) : '';
1489     $('wwInput').value = state.ww != null ? Math.round(state.ww) : '';
1490     if (state.wc != null) $('wcSlider').value = clampRange($('wcSlider'), state.wc);
1491     if (state.ww != null) $('wwSlider').value = clampRange($('wwSlider'), state.ww);
1492 }
1493
1494 function clampRange(input, v) {
1495     const min = parseFloat(input.min), max = parseFloat(input.max);
1496     return Math.max(min, Math.min(max, v));
1497 }
1498
1499 function configureFrameSlider() {
1500     const s = $('frameSlider');
1501     s.min = 0;
1502     s.max = Math.max(0, state.frames - 1);
1503     s.value = 0;
1504     s.disabled = state.frames <= 1;
1505     $('frameLabel').textContent = `1 / ${state.frames}`;
1506 }
1507
1508 function configureWLSliders(meta) {
1509     const bits = parseInt(meta.BitsStored || '12', 10);
1510     const signed = String(meta.PixelRepresentation || '0') === '1';
1511     let lo, hi;
1512     if (signed) { lo = -(1 << (bits - 1)); hi = (1 << (bits - 1)) - 1; }
1513     else { lo = 0; hi = (1 << bits) - 1; }
1514     const wcS = $('wcSlider'); wcS.min = lo; wcS.max = hi;
1515     if (state.wc != null) wcS.value = clampRange(wcS, state.wc);
1516     const wwS = $('wwSlider'); wwS.min = 1; wwS.max = hi - lo;
1517     if (state.ww != null) wwS.value = clampRange(wwS, state.ww);
1518 }
1519
1520 function renderMeta(meta) {
1521     const tbody = $('metaTable').querySelector('tbody');
1522     tbody.innerHTML = '';
1523     const order = [
1524         'PatientName', 'PatientID', 'PatientSex', 'PatientAge', 'PatientBirthDate',
1525         'StudyDate', 'StudyDescription', 'SeriesDescription',
1526         'Modality', 'BodyPartExamined', 'ImageLaterality', 'ViewPosition',
1527         'Rows', 'Columns', 'BitsStored', 'NumberOfFrames', 'PhotometricInterpretation',
1528         'WindowCenter', 'WindowWidth', 'RescaleSlope', 'RescaleIntercept',
1529         'PixelSpacing', 'ImagerPixelSpacing', 'BodyPartThickness', 'CompressionForce',
1530         'KVP', 'ExposureTime', 'XRayTubeCurrent',
1531         'Manufacturer', 'ManufacturerModelName', 'InstitutionName',
1532         'StudyInstanceUID', 'SeriesInstanceUID', 'TransferSyntaxUID',
1533     ];
1534     const seen = new Set();
1535     const addRow = (k, v) => {
1536         if (v === undefined || v === null || v === '') return;
1537         const tr = document.createElement('tr');
1538         const td1 = document.createElement('td'); td1.textContent = k;
1539         const td2 = document.createElement('td'); td2.textContent = v;
1540         tr.append(td1, td2); tbody.appendChild(tr);
1541     };
1542     for (const k of order) { if (k in meta) { addRow(k, meta[k]); seen.add(k); } }
1543     for (const [k, v] of Object.entries(meta)) { if (!seen.has(k)) addRow(k, v); }
1544 }
1545
1546 function clearViewer() {
1547     wsClose();

```

```

1548 state.current = null;
1549 state.meta = null;
1550 state.annotations = [];
1551 state.selectedId = null;
1552 state.suggestions = [];
1553 state._matchInput = null;
1554 $('imgStack').hidden = true;
1555 $('srView').hidden = true;
1556 $('srView').innerHTML = '';
1557 $('emptyState').style.display = '';
1558 $('dicomImg').removeAttribute('src');
1559 $('metaTable').querySelector('tbody').innerHTML = '';
1560 $('reportPane').innerHTML = '';
1561 $('reportBadge').hidden = true;
1562 $('aiClearBtn').hidden = true;
1563 $('deidBtn').hidden = true;
1564 $('srBtn').hidden = true;
1565 $('segBtn').hidden = true;
1566 $('maskPngBtn').hidden = true;
1567 $('maskNiftiBtn').hidden = true;
1568 $('radiomicsBtn').hidden = true;
1569 $('cStoreBtn').hidden = true;
1570 $('tplSelect').hidden = true;
1571 $('tplSaveBtn').hidden = true;
1572 renderSvg();
1573 renderAnnoList();
1574 }
1575
1576 let _imgLoadToken = 0;
1577 async function loadImage(fitAfter) {
1578   if (!state.current) return;
1579   const token = ++_imgLoadToken;
1580   const url = imageUrl();
1581
1582   try {
1583     const probe = await fetch(url, { method: 'HEAD' });
1584     if (probe.status === 422 || (state.meta && (state.meta.Modality === 'SR' || (state.meta.Modality
1585 || '').toUpperCase() === 'SR'))) {
1586       if (probe.status === 422 || state.meta?.Modality === 'SR') {
1587         await renderSrView();
1588         return;
1589       }
1590     } catch (e) {
1591       // fall through to img-tag flow
1592     }
1593
1594     const img = $('dicomImg');
1595     img.onload = async () => {
1596       if (token !== _imgLoadToken) return;
1597       if (img.naturalWidth === 0) {
1598         // still failed - try SR fallback
1599         const r = await fetch(url);
1600         if (r.status === 422) { await renderSrView(); return; }
1601         setStatus("rasmni o'qib bo'lmadi");
1602         return;
1603       }
1604       $('srView').hidden = true;
1605       $('emptyState').style.display = 'none';
1606       $('imgStack').hidden = false;
1607       syncSvgViewBox();
1608       if (fitAfter) fitToScreen();
1609       else applyTransform();
1610       renderSvg();
1611     };
1612     img.onerror = async () => {

```



```

1613     if (token !== _imgLoadToken) return;
1614     try {
1615         const r = await fetch(url);
1616         if (r.status === 422) {
1617             await renderSrView();
1618             return;
1619         }
1620     } catch (e) {}
1621     setStatus("rasmni o'qib bo'lmadi");
1622 };
1623 img.src = url;
1624 }
1625
1626 async function renderSrView() {
1627     $('imgStack').hidden = true;
1628     $('emptyState').style.display = 'none';
1629     const view = $('srView');
1630     view.hidden = false;
1631     view.innerHTML = '<div class="empty">SR matni yuklanmoqda...</div>';
1632
1633     let data = null;
1634     try {
1635         if (state.current.kind === 'upload') {
1636             data = await apiJson(`/api/files/${encodeURIComponent(state.current.id)}/sr-content`);
1637         } else {
1638             data = await apiJson(`/api/local/sr-content?path=${encodeURIComponent(state.current.path)}`);
1639         }
1640     } catch (e) {
1641         const m = state.meta || {};
1642         view.innerHTML = `
1643             <div class="sr-head">
1644                 <h2>  ${m.Modality || 'Structured Report'} – bu DICOM rasm emas</h2>
1645                 <div class="meta">
1646                     Bemor: <b>${m.PatientName || '-'}</b> · ID ${m.PatientID || '-'}
1647                     <span class="sr-flag">Modality: ${m.Modality || '-'}</span>
1648                 </div>
1649             </div>
1650             <div class="empty" style="padding-top:14px">
1651                 SR matn parserga ulanmadi (server eski kodda bo'lishi mumkin).<br>
1652                 <b>Server qayta ishga tushirilsin</b> (run.bat ni yopib qayta oching) keyin Ctrl+F5.
1653             </div>
1654             <div class="sr-item depth-1"><div class="lbl">Xom xato</div><div
class="val">${e.message}</div></div>
1655         `;
1656         return;
1657     }
1658
1659     view.innerHTML = '';
1660     const head = document.createElement('div');
1661     head.className = 'sr-head';
1662     const title = document.createElement('h2');
1663     title.textContent = '  ' + (data.items?.[0]?.name || 'Structured Report');
1664     head.appendChild(title);
1665     const meta = document.createElement('div');
1666     meta.className = 'meta';
1667     meta.innerHTML = `Bemor: <b>${data.patient_name || '-'}</b> · ID ${data.patient_id || '-'} ·
    ${data.content_date || ''}` +
    (data.completion_flag ? `<span class="sr-flag">${data.completion_flag}</span>` : '') +
    (data.verification_flag ? `<span class="sr-flag">${data.verification_flag}</span>` : '') +
    `<span class="sr-flag">Modality: ${data.modality || '-'}</span>`;
1671     head.appendChild(meta);
1672     view.appendChild(head);
1673
1674     for (const it of (data.items || []).slice(1)) {
1675         const div = document.createElement('div');
1676         div.className = `sr-item depth-${Math.min(it.depth || 1, 4)}`;

```

```

1677     const lbl = document.createElement('div');
1678     lbl.className = 'lbl';
1679     lbl.textContent = `${it.type} || ''${it.name ? ' ' + it.name : ''}`;
1680     div.appendChild(lbl);
1681     const val = document.createElement('div');
1682     val.className = 'val';
1683     val.textContent = it.text || it.value || '';
1684     div.appendChild(val);
1685     view.appendChild(div);
1686 }
1687
1688 setStatus(`  SR - ${data.modality} || 'Structured Report'`);
1689 }
1690
1691 function syncSvgViewBox() {
1692     const img = $('dicomImg');
1693     const svg = $('annoSvg');
1694     if (!img.naturalWidth) return;
1695     svg.setAttribute('viewBox', `0 0 ${img.naturalWidth} ${img.naturalHeight}`);
1696     svg.setAttribute('width', img.naturalWidth);
1697     svg.setAttribute('height', img.naturalHeight);
1698 }
1699
1700 let _wlDebounce;
1701 function scheduleReload() {
1702     clearTimeout(_wlDebounce);
1703     _wlDebounce = setTimeout(() => loadImage(false), 150);
1704 }
1705
1706 function fitToScreen() {
1707     const wrap = $('canvasWrap');
1708     const img = $('dicomImg');
1709     if (!img.naturalWidth) return;
1710     const sx = wrap.clientWidth / img.naturalWidth;
1711     const sy = wrap.clientHeight / img.naturalHeight;
1712     const s = Math.min(sx, sy) * 0.98;
1713     state.scale = s; state.fitScale = s;
1714     state.tx = 0; state.ty = 0;
1715     applyTransform();
1716 }
1717 function oneToOne() { state.scale = 1; state.tx = 0; state.ty = 0; applyTransform(); }
1718 let _lastScale = 1;
1719 function applyTransform() {
1720     $('imgStack').style.transform = `translate(${state.tx}px, ${state.ty}px) scale(${state.scale})`;
1721     $('zoomLbl').textContent = `${Math.round(state.scale * 100)}%`;
1722     if (state.current && state.selectedId && Math.abs(state.scale - _lastScale) > 1e-3) {
1723         _lastScale = state.scale;
1724         renderSvg();
1725     } else {
1726         _lastScale = state.scale;
1727     }
1728 }
1729
1730 // pan & zoom
1731 (() => {
1732     const wrap = $('canvasWrap');
1733     let dragging = false, lastX = 0, lastY = 0;
1734
1735     let touchMode = null;
1736     let lastTouchX = 0, lastTouchY = 0;
1737     let lastPinchDist = 0;
1738     let lastPinchCx = 0, lastPinchCy = 0;
1739
1740     function dist(t1, t2) {
1741         const dx = t2.clientX - t1.clientX;
1742         const dy = t2.clientY - t1.clientY;

```

```

1743     return Math.hypot(dx, dy);
1744 }
1745
1746 wrap.addEventListener('touchstart', (e) => {
1747     if (e.touches.length === 1) {
1748         touchMode = 'pan';
1749         lastTouchX = e.touches[0].clientX;
1750         lastTouchY = e.touches[0].clientY;
1751     } else if (e.touches.length >= 2) {
1752         touchMode = 'pinch';
1753         lastPinchDist = dist(e.touches[0], e.touches[1]);
1754         lastPinchCx = (e.touches[0].clientX + e.touches[1].clientX) / 2;
1755         lastPinchCy = (e.touches[0].clientY + e.touches[1].clientY) / 2;
1756     }
1757     e.preventDefault();
1758 }, { passive: false });
1759
1760 wrap.addEventListener('touchmove', (e) => {
1761     if (touchMode === 'pan' && e.touches.length === 1) {
1762         const t = e.touches[0];
1763         state.tx += t.clientX - lastTouchX;
1764         state.ty += t.clientY - lastTouchY;
1765         lastTouchX = t.clientX;
1766         lastTouchY = t.clientY;
1767         applyTransform();
1768         e.preventDefault();
1769     } else if (touchMode === 'pinch' && e.touches.length >= 2) {
1770         const d = dist(e.touches[0], e.touches[1]);
1771         if (lastPinchDist > 0) {
1772             const factor = d / lastPinchDist;
1773             const newScale = Math.max(0.05, Math.min(40, state.scale * factor));
1774             const cx = (e.touches[0].clientX + e.touches[1].clientX) / 2;
1775             const cy = (e.touches[0].clientY + e.touches[1].clientY) / 2;
1776             const rect = wrap.getBoundingClientRect();
1777             const px = cx - rect.left - rect.width / 2;
1778             const py = cy - rect.top - rect.height / 2;
1779             state.tx = px - (px - state.tx) * (newScale / state.scale);
1780             state.ty = py - (py - state.ty) * (newScale / state.scale);
1781             state.tx += cx - lastPinchCx;
1782             state.ty += cy - lastPinchCy;
1783             state.scale = newScale;
1784             applyTransform();
1785         }
1786         lastPinchDist = d;
1787         lastPinchCx = (e.touches[0].clientX + e.touches[1].clientX) / 2;
1788         lastPinchCy = (e.touches[0].clientY + e.touches[1].clientY) / 2;
1789         e.preventDefault();
1790     }
1791 }, { passive: false });
1792
1793 wrap.addEventListener('touchend', (e) => {
1794     if (e.touches.length === 0) {
1795         touchMode = null;
1796     } else if (e.touches.length === 1) {
1797         touchMode = 'pan';
1798         lastTouchX = e.touches[0].clientX;
1799         lastTouchY = e.touches[0].clientY;
1800     }
1801 });
1802
1803 wrap.addEventListener('mousedown', (e) => {
1804     if (state.tool === 'bbox' && e.target.id === 'annoSvg') return;
1805     if (e.button !== 0 && e.button !== 1) return;
1806     dragging = true; lastX = e.clientX; lastY = e.clientY;
1807     wrap.classList.add('dragging');
1808 });

```

```

1809 window.addEventListener('mousemove', (e) => {
1810     if (!dragging) return;
1811     state.tx += e.clientX - lastX;
1812     state.ty += e.clientY - lastY;
1813     lastX = e.clientX; lastY = e.clientY;
1814     applyTransform();
1815 });
1816 window.addEventListener('mouseup', () => { dragging = false; wrap.classList.remove('dragging'); });
1817 wrap.addEventListener('wheel', (e) => {
1818     e.preventDefault();
1819     const factor = Math.exp(-e.deltaY * 0.0015);
1820     const newScale = Math.max(0.05, Math.min(40, state.scale * factor));
1821     const rect = wrap.getBoundingClientRect();
1822     const cx = e.clientX - rect.left - rect.width / 2;
1823     const cy = e.clientY - rect.top - rect.height / 2;
1824     state.tx = cx - (cx - state.tx) * (newScale / state.scale);
1825     state.ty = cy - (cy - state.ty) * (newScale / state.scale);
1826     state.scale = newScale;
1827     applyTransform();
1828 }, { passive: false });
1829 })();
1830
1831 // upload - parallel + progress
1832 const UPLOAD_CONCURRENCY = 3;
1833
1834 function uploadOne(file, onProgress) {
1835     return new Promise((resolve, reject) => {
1836         const xhr = new XMLHttpRequest();
1837         xhr.open('POST', '/api/upload');
1838         const tok = getToken();
1839         if (tok) xhr.setRequestHeader('Authorization', `Bearer ${tok}`);
1840         xhr.upload.onprogress = (e) => {
1841             if (e.lengthComputable && onProgress) onProgress(e.loaded, e.total);
1842         };
1843         xhr.onload = () => {
1844             if (xhr.status === 401) { setToken(null); state._user = null; updateUserBar(); reject(new
Error('401')); return; }
1845             if (xhr.status >= 200 && xhr.status < 300) {
1846                 try { resolve(JSON.parse(xhr.responseText)); }
1847                 catch (e) { resolve({}); }
1848             } else reject(new Error(`HTTP ${xhr.status}: ${xhr.responseText}`));
1849         };
1850         xhr.onerror = () => reject(new Error('network error'));
1851         xhr.onabort = () => reject(new Error('aborted'));
1852         const fd = new FormData();
1853         fd.append('files', file, file.name);
1854         xhr.send(fd);
1855     });
1856 }
1857
1858 (() => {
1859     const dz = $('#dropZone');
1860     const inp = $('#fileInput');
1861     const finp = $('#folderInput');
1862
1863     const upload = async (filesIn) => {
1864         if (!filesIn || filesIn.length === 0) return;
1865         const files = Array.from(filesIn);
1866         const total = files.length;
1867         let done = 0;
1868         let failed = 0;
1869         const totalBytes = files.reduce((s, f) => s + f.size, 0);
1870         let totalUploaded = 0;
1871         const fileProgress = new Map();
1872         const t0 = performance.now();
1873

```

```

1874     const fmtSize = (n) => {
1875         if (n < 1024) return `${n} B`;
1876         if (n < 1024 * 1024) return `${(n / 1024).toFixed(0)} KB`;
1877         return `${(n / 1024 / 1024).toFixed(1)} MB`;
1878     };
1879
1880     const refreshStatus = () => {
1881         const sumLoaded = [...fileProgress.values()].reduce((s, v) => s + v, 0) + totalUploaded;
1882         const pct = totalBytes ? (sumLoaded * 100 / totalBytes) : 0;
1883         const elapsed = (performance.now() - t0) / 1000;
1884         const rate = elapsed > 0.5 ? sumLoaded / elapsed : 0;
1885         const rateStr = rate > 0 ? ` · ${fmtSize(rate)}/s` : '';
1886         const etaStr = rate > 0 ? ` · ETA ${Math.max(0, Math.round((totalBytes - sumLoaded) / rate))}s`
1887         : '';
1888         setStatus(`📁 ${done}/${total} · ${pct.toFixed(0)}%${rateStr}${etaStr}`);
1889     };
1890
1891     const queue = files.slice();
1892     const worker = async () => {
1893         while (queue.length) {
1894             const f = queue.shift();
1895             try {
1896                 await uploadOne(f, (loaded, _t) => {
1897                     fileProgress.set(f, loaded);
1898                     refreshStatus();
1899                 });
1900                 totalUploaded += f.size;
1901                 fileProgress.delete(f);
1902                 done++;
1903             } catch (e) {
1904                 failed++;
1905                 fileProgress.delete(f);
1906                 console.error('upload', f.name, e.message);
1907             }
1908             refreshStatus();
1909         }
1910     };
1911
1912     refreshStatus();
1913     const workers = [];
1914     for (let i = 0; i < Math.min(UPLOAD_CONCURRENCY, files.length); i++) {
1915         workers.push(worker());
1916     }
1917     await Promise.all(workers);
1918
1919     await refreshUploads();
1920     const elapsed = ((performance.now() - t0) / 1000).toFixed(1);
1921     if (failed) setStatus(`✓ ${done}/${total} yuklandi · ${failed} xato (${elapsed}s`);
1922     else setStatus(`✓ ${done} ta fayl yuklandi (${elapsed}s`);
1923 };
1924
1925 dz.addEventListener('dragover', (e) => { e.preventDefault(); dz.classList.add('drag'); });
1926 dz.addEventListener('dragleave', () => dz.classList.remove('drag'));
1927 dz.addEventListener('drop', (e) => {
1928     e.preventDefault();
1929     dz.classList.remove('drag');
1930     upload(e.dataTransfer.files);
1931 });
1932 $('pickFiles').addEventListener('click', () => inp.click());
1933 $('pickFolder').addEventListener('click', () => finp.click());
1934 inp.addEventListener('change', (e) => upload(e.target.files));
1935 finp.addEventListener('change', (e) => upload(e.target.files));
1936 }());
1937 // left tabs
1938 document.querySelectorAll('.sidebar .tab').forEach(btn => {

```

```

1939 btn.addEventListener('click', () => {
1940     document.querySelectorAll('.sidebar .tab').forEach(b => b.classList.remove('active'));
1941     btn.classList.add('active');
1942     const t = btn.dataset.tab;
1943     document.querySelectorAll('.sidebar .pane').forEach(p => p.hidden = p.dataset.pane !== t);
1944     if (t === 'local' && state.localEnabled) loadLocalDir('');
1945     if (t === 'links') loadLinks('');
1946     if (t === 'dashboard') loadDashboard();
1947     if (t === 'worklist') loadWorklist();
1948 });
1949 });
1950
1951 // ===== Worklist =====
1952
1953 async function loadWorklist() {
1954     if (!state._user) return;
1955     const status = $('wlStatus').value;
1956     const own = $('wlOwn').checked;
1957     const params = new URLSearchParams();
1958     if (status) params.set('status', status);
1959     if (own) params.set('own', 'true');
1960     let data;
1961     try {
1962         data = await apiJson(`/api/worklist?${params}`);
1963     } catch (e) {
1964         setStatus('worklist xatosi: ' + e.message);
1965         return;
1966     }
1967     $('wlMeta').textContent = `${data.count} ta yozuv`;
1968     const ul = $('wlList');
1969     ul.innerHTML = '';
1970     for (const it of data.items) {
1971         const li = document.createElement('li');
1972         li.className = 'dash-row';
1973         const left = document.createElement('div');
1974
1975         const name = document.createElement('div');
1976         name.className = 'name';
1977         const ttl = document.createElement('span');
1978         ttl.style.fontWeight = '600';
1979         ttl.textContent = it.patient_name || it.patient_id;
1980         name.appendChild(ttl);
1981         if (it.priority && it.priority !== 'normal') {
1982             const pr = document.createElement('span');
1983             pr.className = `wl-priority ${it.priority}`;
1984             pr.textContent = it.priority;
1985             name.appendChild(pr);
1986         }
1987         const sp = document.createElement('span');
1988         sp.className = `wl-status-${it.status}`;
1989         sp.textContent = it.status;
1990         sp.style.fontSize = '11px';
1991         sp.style.marginLeft = '4px';
1992         name.appendChild(sp);
1993         left.appendChild(name);
1994
1995         const sub = document.createElement('div');
1996         sub.className = 'meta';
1997         const ass = it.assigned_to ? `→ ${it.assigned_to}` : 'tayinlanmagan';
1998         sub.textContent = `ID ${it.patient_id} · ${it.modality || '-'} · ${it.study_date || '-'} ·
1999         ${ass}`;
2000         left.appendChild(sub);
2001
2002         if (it.last_name && state._user.role !== 'annotator') {
2003             const sub2 = document.createElement('div');
2004             sub2.className = 'meta';

```

```

2004     sub2.style.opacity = '0.7';
2005     sub2.textContent = `xlsx: ${it.last_name} ${it.first_name || ''} (${it.record_count} yozuv)`;
2006     left.appendChild(sub2);
2007 }
2008
2009 const top = document.createElement('div');
2010 top.className = 'row';
2011 top.appendChild(left);
2012 li.appendChild(top);
2013
2014 if (state._user.role === 'admin' || state._user.role === 'reviewer') {
2015     li.addEventListener('contextmenu', (e) => {
2016         e.preventDefault();
2017         promptWorklistEdit(it);
2018     });
2019     li.title = "Right-click: tayinlash/status o'zgartirish";
2020 }
2021 li.addEventListener('click', () => {
2022     if (it.last_name) showPatientOverview(it.patient_id);
2023 });
2024 ul.appendChild(li);
2025 }
2026 }
2027
2028 async function promptWorklistEdit(it) {
2029     const mode = prompt(
2030         `${it.patient_name || it.patient_id}` uchun:\n\n` +
2031         `1 - assign\n` +
2032         `2 - priority (low/normal/high/urgent)\n` +
2033         `3 - status (pending/in_progress/done/skipped)\n` +
2034         `Tanlang:`, ''
2035     );
2036     if (!mode) return;
2037     let body = {};
2038     if (mode === '1') {
2039         const u = prompt('Foydalanuvchi nomi (bo\'sh - bekor qilish):', it.assigned_to || '');
2040         if (u === null) return;
2041         body.assigned_to = u || '';
2042     } else if (mode === '2') {
2043         const p = prompt('Priority:', it.priority || 'normal');
2044         if (!p) return;
2045         body.priority = p;
2046     } else if (mode === '3') {
2047         const s = prompt('Status:', it.status || 'pending');
2048         if (!s) return;
2049         body.status = s;
2050     } else return;
2051     try {
2052         await api(`/api/worklist/${it.id}`, {
2053             method: 'PATCH',
2054             headers: { 'content-type': 'application/json' },
2055             body: JSON.stringify(body),
2056         });
2057         await loadWorklist();
2058     } catch (e) { alert(e.message); }
2059 }
2060
2061 (function bindWorklistUi() {
2062     $('wlStatus').addEventListener('change', loadWorklist);
2063     $('wlOwn').addEventListener('change', loadWorklist);
2064     $('wlRefresh').addEventListener('click', loadWorklist);
2065 })();
2066
2067 // ===== Patient overview modal =====
2068
2069 async function showPatientOverview(patientId) {

```

```

2070 let data;
2071 try {
2072   data = await apiJson(`/api/db/patient/${encodeURIComponent(patientId)}/overview`);
2073 } catch (e) {
2074   alert('xatosi: ' + e.message);
2075   return;
2076 }
2077 const body = $('patientBody');
2078 const p = data.patient;
2079 body.innerHTML = '';
2080 const wrap = document.createElement('div');
2081 wrap.className = 'patient-overview';
2082
2083 const head = document.createElement('div');
2084 head.className = 'pat-head';
2085 const nm = document.createElement('div');
2086 nm.className = 'name';
2087 nm.textContent = `${p.last_name || ''} ${p.first_name || ''}`.trim() || '-';
2088 head.appendChild(nm);
2089 const meta = document.createElement('div');
2090 meta.className = 'meta';
2091 meta.textContent = `ID ${p.patient_id} · ${p.sex || '-'} · DOB ${p.birth_date || '-'} · ` +
2092   `${data.records.length} yozuv · ${data.dicoms.length} bog'langan DICOM · ${data.worklist.length}
worklist`;
2093 head.appendChild(meta);
2094 wrap.appendChild(head);
2095
2096 if (!data.timeline.length) {
2097   const empty = document.createElement('div');
2098   empty.className = 'report-empty';
2099   empty.textContent = "Timeline bo'sh.";
2100   wrap.appendChild(empty);
2101   body.appendChild(wrap);
2102   $('patientModal').hidden = false;
2103   return;
2104 }
2105
2106 for (const t of data.timeline) {
2107   const item = document.createElement('div');
2108   item.className = `timeline-item kind-${t.kind}`;
2109   if (t.kind === 'dicom') item.classList.add('dicom-row');
2110   const ts = document.createElement('div');
2111   ts.className = 'ts';
2112   ts.textContent = t.ts || '-';
2113   const bd = document.createElement('div');
2114   bd.className = 'body';
2115   if (t.kind === 'record') {
2116     bd.innerHTML = ` <b>${t.exam_code || '-'}</b> ${t.exam_name || ''}` +
2117       `(t.diagnosis ? ` - <i>${(t.diagnosis || '').slice(0,100)}</i>` : '')`;
2118   } else if (t.kind === 'worklist') {
2119     bd.innerHTML = ` ${t.modality || '-'} · status: <b>${t.status}</b>` +
2120       `(t.assigned_to ? ` · ${t.assigned_to}` : '')`;
2121   } else if (t.kind === 'dicom') {
2122     const sCounts = Object.entries(t.annotation_statuses || {}).map(([k,v]) => `${k}:${v}`).join('');
2123     bd.innerHTML = ` <b>${t.ref}</b> · ${t.annotation_count} ann ${sCounts ? `(${sCounts})` : ''}`;
2124   }
2125   item.addEventListener('click', () => {
2126     $('patientModal').hidden = true;
2127     if (t.source === 'upload') openItem({ kind: 'upload', id: t.ref });
2128     else openItem({ kind: 'local', path: t.ref });
2129   });
2130   item.appendChild(ts, bd);
2131   wrap.appendChild(item);
2132 }

```



```

2133 body.appendChild(wrap);
2134 $('patientModal').hidden = false;
2135 }
2136
2137 $('patientCloseBtn').addEventListener('click', () => { $('patientModal').hidden = true; });
2138
2139 // ===== De-identification modal =====
2140
2141 async function openDeidModal() {
2142   if (!state.current) return;
2143   const body = $('deidBody');
2144   body.innerHTML = '<div class="report-empty">PHI maydonlar tekshirilmoqda...</div>';
2145   $('deidModal').hidden = false;
2146   let data;
2147   try {
2148     data = await apiJson(
2149       `/api/deidentify/detect?source=${refSource()}&ref=${encodeURIComponent(refValue())}`
2150     );
2151   } catch (e) {
2152     body.innerHTML = '<div class="admin-err">${e.message}</div>`;
2153     return;
2154   }
2155   body.innerHTML = '';
2156
2157   const sec = document.createElement('div');
2158   sec.className = 'deid-section';
2159   const h3 = document.createElement('h3');
2160   h3.textContent = `${data.count} ta PHI maydon topildi`;
2161   sec.appendChild(h3);
2162
2163   const table = document.createElement('table');
2164   table.className = 'deid-table';
2165   for (const item of data.phi_tags_found) {
2166     const tr = document.createElement('tr');
2167     if (item.preserves_year) tr.classList.add('preserves');
2168     tr.innerHTML = `<td>${item.tag}${item.preserves_year ? ' (yil saqlanadi)' : ''}</td>` +
2169       `<td><code>${(item.value || '').replace(/[<>]/g, '_')}</code></td>`;
2170     table.appendChild(tr);
2171   }
2172   sec.appendChild(table);
2173   body.appendChild(sec);
2174
2175   const note = document.createElement('div');
2176   note.className = 'report-empty';
2177   note.style.fontSize = '11px';
2178   note.textContent = 'Anonimlashtirilgan DICOM yuklab olinganda: ism/ID/manzil tozalanadi, sana -
yilni saqlab, sanai 01.01 ga moslashtiriladi, "PatientIdentityRemoved=YES" yoziladi.';
2179   body.appendChild(note);
2180
2181   const actions = document.createElement('div');
2182   actions.className = 'deid-actions';
2183   const dl = document.createElement('button');
2184   dl.textContent = '↓ Anonimlashtirilgan DICOM yuklab olish';
2185   dl.addEventListener('click', async () => {
2186     const tok = getToken();
2187     const res = await fetch(
2188       `/api/deidentify/download?source=${refSource()}&ref=${encodeURIComponent(refValue())}`,
2189       { headers: { 'Authorization': `Bearer ${tok}` } },
2190     );
2191     if (!res.ok) { alert('xato: ' + res.status); return; }
2192     const blob = await res.blob();
2193     const url = URL.createObjectURL(blob);
2194     const a = document.createElement('a');
2195     a.href = url;
2196     const stem = (refValue() || 'anon').split('/').pop().replace(/\.dcm$/i, '');
2197     a.download = `${stem}_anon.dcm`;

```

```

2198     document.body.appendChild(a);
2199     a.click();
2200     a.remove();
2201     URL.revokeObjectURL(url);
2202 });
2203 actions.appendChild(dl);
2204 body.appendChild(actions);
2205 }
2206
2207 // ===== Research / radiologist verification (review_decisions) =====
2208
2209 async function openReviewModal() {
2210     $('reviewModal').hidden = false;
2211     await renderReviewQueue('pending');
2212 }
2213
2214 async function renderReviewQueue(filter) {
2215     const body = $('reviewBody');
2216     body.innerHTML = '<div class="report-empty">Yuklanmoqda...</div>';
2217     let data;
2218     try {
2219         data = await apiJson(`/api/research/review/queue?status=${encodeURIComponent(filter)}&limit=200`);
2220     } catch (e) {
2221         body.innerHTML = '<div class="admin-err">${e.message}</div>';
2222         return;
2223     }
2224     body.innerHTML = '';
2225
2226     const counts = data.counts || {};
2227     const tabBar = document.createElement('div');
2228     tabBar.style.cssText = 'display:flex;gap:6px;margin-bottom:12px;flex-wrap:wrap;align-items:center';
2229     const tabs = [
2230         ['pending', `🕒 Kutmoqda (${counts.pending || 0})`],
2231         ['accepted', `✓ Qabul (${counts.accepted || 0})`],
2232         ['edited', `✍ Tahrirlangan (${counts.edited || 0})`],
2233         ['rejected', `✗ Rad etilgan (${counts.rejected || 0})`],
2234         ['all', `Barchasi`],
2235     ];
2236     for (const [key, label] of tabs) {
2237         const b = document.createElement('button');
2238         b.textContent = label;
2239         b.style.cssText = 'padding:6px 10px;font-size:12px;'
2240             + (key === filter ? 'background:var(--accent);color:#fff;font-weight:bold' : '');
2241         b.addEventListener('click', () => renderReviewQueue(key));
2242         tabBar.appendChild(b);
2243     }
2244
2245     const exportBtn = document.createElement('button');
2246     exportBtn.textContent = '↓ Gold labels JSONL';
2247     exportBtn.title = 'Tasdiqlangan + tahrirlangan barcha review yozuvlarini eksport qilish';
2248     exportBtn.style.cssText = 'margin-left:auto;padding:6px 10px;font-size:12px';
2249     exportBtn.addEventListener('click', async () => {
2250         const tok = getToken();
2251         const res = await fetch('/api/research/review/export?include_status=accepted,edited', {
2252             headers: { 'Authorization': `Bearer ${tok}` },
2253         });
2254         if (!res.ok) { alert('eksport xatosi: ' + res.status); return; }
2255         const blob = await res.blob();
2256         const url = URL.createObjectURL(blob);
2257         const a = document.createElement('a');
2258         a.href = url;
2259         a.download = `gold_labels_${new Date().toISOString().slice(0,10)}.jsonl`;
2260         document.body.appendChild(a); a.click(); a.remove();
2261         URL.revokeObjectURL(url);
2262     });
2263     tabBar.appendChild(exportBtn);

```

```

2264 body.appendChild(tabBar);
2265
2266 if (!data.items || !data.items.length) {
2267   const empty = document.createElement('div');
2268   empty.className = 'report-empty';
2269   empty.textContent = `${filter} statusida review yo'q.`;
2270   body.appendChild(empty);
2271   return;
2272 }
2273
2274 const table = document.createElement('table');
2275 table.className = 'user-table';
2276 table.style.fontSize = '12px';
2277 table.innerHTML = `<thead><tr>
2278   <th>SOP UID</th><th>Ko'rinish</th><th>Pseudo</th><th>Status</th>
2279   <th>Reviewer</th><th>Imported</th><th>Amallar</th>
2280 </tr></thead>`;
2281 const tbody = document.createElement('tbody');
2282 for (const it of data.items) {
2283   const tr = document.createElement('tr');
2284   const sopShort = (it.sop_uid || '').slice(-20);
2285   const view = `${it.laterality}-${it.view}`;
2286   tr.innerHTML = `
2287     <td title="${it.sop_uid}"><code style="font-size:10px">...${sopShort}</code></td>
2288     <td>${view}</td>
2289     <td>${it.n_pseudo} ${it.n_final ? `> <b>${it.n_final}</b>` : ''}</td>
2290     <td><span class="role-pill role-${it.status === 'accepted' || it.status === 'edited' ? 'admin' :
it.status === 'rejected' ? 'annotator' : 'reviewer'}">${it.status}</span></td>
2291     <td>${it.reviewer || '-'}</td>
2292     <td style="font-size:10px;color:var(--muted)">${(it.imported_at || '').slice(0,16).replace('T','
')}}</td>
2293   `;
2294   const actions = document.createElement('td');
2295   actions.style.cssText = 'display:flex;gap:4px;flex-wrap:wrap';
2296
2297   const inspectBtn = document.createElement('button');
2298   inspectBtn.textContent = '🔍 Ko'rish';
2299   inspectBtn.style.cssText = 'padding:3px 8px;font-size:11px';
2300   inspectBtn.addEventListener('click', () => openReviewItem(it.id));
2301   actions.appendChild(inspectBtn);
2302
2303   if (it.status === 'pending') {
2304     const acceptBtn = document.createElement('button');
2305     acceptBtn.textContent = '✓';
2306     acceptBtn.title = 'Pseudo-bboxlarni o'zgartirmasdan qabul qilish';
2307     acceptBtn.style.cssText = 'padding:3px 8px;font-size:11px;background:#2a7e2a;color:#fff';
2308     acceptBtn.addEventListener('click', async () => {
2309       if (!confirm('Pseudo-bboxlarni gold deb belgilaysizmi?')) return;
2310       await decideReview(it.id, 'accepted', null, '');
2311       await renderReviewQueue(filter);
2312     });
2313     actions.appendChild(acceptBtn);
2314
2315     const rejectBtn = document.createElement('button');
2316     rejectBtn.textContent = 'X';
2317     rejectBtn.title = 'Bu element uchun bbox emas (rad etish)';
2318     rejectBtn.style.cssText = 'padding:3px 8px;font-size:11px;background:#a02020;color:#fff';
2319     rejectBtn.addEventListener('click', async () => {
2320       const reason = prompt('Rad etish sababi (ixtiyoriy):') || '';
2321       await decideReview(it.id, 'rejected', null, reason);
2322       await renderReviewQueue(filter);
2323     });
2324     actions.appendChild(rejectBtn);
2325   }
2326
2327   tr.appendChild(actions);

```

```

2328     tbody.appendChild(tr);
2329   }
2330   table.appendChild(tbody);
2331   body.appendChild(table);
2332 }
2333
2334 async function decideReview(reviewId, status, finalBboxes, comments) {
2335   try {
2336     await api(`/api/research/review/${reviewId}/decide`, {
2337       method: 'POST',
2338       headers: { 'content-type': 'application/json' },
2339       body: JSON.stringify({
2340         status,
2341         final_bboxes: finalBboxes,
2342         comments: comments || '',
2343       }),
2344     });
2345   } catch (e) {
2346     alert('decide xatosi: ' + e.message);
2347   }
2348 }
2349
2350 async function openReviewItem(reviewId) {
2351   let item;
2352   try {
2353     item = await apiJson(`/api/research/review/${reviewId}`);
2354   } catch (e) {
2355     alert('item yuklash xatosi: ' + e.message);
2356     return;
2357   }
2358   const body = $('reviewBody');
2359   body.innerHTML = '';
2360
2361   const back = document.createElement('button');
2362   back.textContent = '← Roʻyxatga qaytish';
2363   back.style.cssText = 'padding:6px 10px;margin-bottom:12px;font-size:12px';
2364   back.addEventListener('click', () => renderReviewQueue('pending'));
2365   body.appendChild(back);
2366
2367   const wrap = document.createElement('div');
2368   wrap.style.cssText = 'display:grid;grid-template-columns:1fr 320px;gap:16px;align-items:start';
2369
2370   const left = document.createElement('div');
2371   left.style.cssText = 'position:relative;background:#000;border:1px solid var(--border);min-height:300px';
2372   if (item.has_preview) {
2373     const img = document.createElement('img');
2374     img.style.cssText = 'display:block;max-width:100%;height:auto';
2375     img.id = 'reviewPreviewImg';
2376     const tok = getToken();
2377     const r = await fetch(`/api/research/review/${reviewId}/preview`, {
2378       headers: { 'Authorization': `Bearer ${tok}` },
2379     });
2380     if (r.ok) {
2381       const blob = await r.blob();
2382       img.src = URL.createObjectURL(blob);
2383       img.onload = () => {
2384         // overlay pseudo bboxes as SVG
2385         const svg = document.createElementNS('http://www.w3.org/2000/svg', 'svg');
2386         svg.setAttribute('viewBox', `0 0 ${img.naturalWidth} ${img.naturalHeight}`);
2387         svg.style.cssText = 'position:absolute;top:0;left:0;width:100%;height:100%;pointer-events:none';
2388         for (const b of (item.pseudo_bboxes || [])) {
2389           const rect = document.createElementNS('http://www.w3.org/2000/svg', 'rect');
2390           rect.setAttribute('x', b.x0); rect.setAttribute('y', b.y0);
2391           rect.setAttribute('width', Math.max(0, b.x1 - b.x0));

```

```

2392     rect.setAttribute('height', Math.max(0, b.y1 - b.y0));
2393     rect.setAttribute('fill', 'none');
2394     rect.setAttribute('stroke', '#ffb000');
2395     rect.setAttribute('stroke-width', '4');
2396     rect.setAttribute('stroke-dasharray', '8,4');
2397     svg.appendChild(rect);
2398     const txt = document.createElementNS('http://www.w3.org/2000/svg', 'text');
2399     txt.setAttribute('x', b.x0 + 6); txt.setAttribute('y', b.y0 + 22);
2400     txt.setAttribute('fill', '#ffb000');
2401     txt.setAttribute('font-size', '20');
2402     txt.textContent = `cls=${b.cls} c=${(b.confidence || 0).toFixed(2)}`;
2403     svg.appendChild(txt);
2404   }
2405   left.appendChild(svg);
2406   };
2407   left.appendChild(img);
2408   } else {
2409     left.innerHTML = '<div style="padding:18px;color:#888">Preview unavailable</div>';
2410   }
2411   } else {
2412     left.innerHTML = '<div style="padding:18px;color:#888">Preview unavailable</div>';
2413   }
2414
2415   const right = document.createElement('div');
2416   right.innerHTML = `
2417     <div style="font-size:11px;color:var(--muted);margin-bottom:4px">SOP UID</div>
2418     <code style="display:block;font-size:10px;word-break:break-all;background:var(--panel-
2419 2);padding:4px;border-radius:3px;margin-bottom:10px">${item.sop_uid}</code>
2419     <div style="font-size:11px;color:var(--muted)">Ko'rinish</div>
2420     <div style="margin-bottom:10px"><b>${item.laterality} - ${item.view}</b> · status:
2421     <b>${item.status}</b></div>
2421     <div style="font-size:11px;color:var(--muted)">Findings (matn)</div>
2422     <pre style="font-size:11px;background:var(--panel-2);padding:6px;border-radius:3px;max-
2423 height:140px;overflow:auto;margin-bottom:10px">${JSON.stringify(item.findings || {}, null, 2)}</pre>
2423     <div style="font-size:11px;color:var(--muted)">Pseudo-bboxlar (${(item.pseudo_bboxes ||
2424 []).length})</div>
2424     <pre style="font-size:11px;background:var(--panel-2);padding:6px;border-radius:3px;max-
2425 height:160px;overflow:auto;margin-bottom:10px">${JSON.stringify(item.pseudo_bboxes || [], null, 2)}</pre>
2425   `;
2426
2427   const decideBar = document.createElement('div');
2428   decideBar.style.cssText = 'display:flex;flex-direction:column;gap:6px';
2429   if (item.status === 'pending' || item.status === 'rejected') {
2430     const acc = document.createElement('button');
2431     acc.textContent = '✓ Qabul (pseudo-ni gold)';
2432     acc.style.cssText = 'padding:8px;background:#2a7e2a;color:#fff;font-weight:bold';
2433     acc.addEventListener('click', async () => {
2434       if (!confirm('Pseudo-bboxlarni gold deb belgilaysizmi?')) return;
2435       await decideReview(reviewId, 'accepted', item.pseudo_bboxes || [], '');
2436       await renderReviewQueue('pending');
2437     });
2438     decideBar.appendChild(acc);
2439   }
2440   if (item.status === 'pending' || item.status === 'accepted') {
2441     const rej = document.createElement('button');
2442     rej.textContent = 'X Rad etish';
2443     rej.style.cssText = 'padding:8px;background:#a02020;color:#fff';
2444     rej.addEventListener('click', async () => {
2445       const reason = prompt('Rad etish sababi (ixtiyoriy):') || '';
2446       await decideReview(reviewId, 'rejected', null, reason);
2447       await renderReviewQueue('pending');
2448     });
2449     decideBar.appendChild(rej);
2450   }
2451   right.appendChild(decideBar);
2452

```

```

2453 wrap.appendChild(left);
2454 wrap.appendChild(right);
2455 body.appendChild(wrap);
2456 }
2457
2458 async function openStatsModal() {
2459   $('statsModal').hidden = false;
2460   const body = $('statsBody');
2461   body.innerHTML = '<div class="report-empty">Yuklanmoqda...</div>';
2462   let s;
2463   try {
2464     s = await apiJson('/api/stats/overview');
2465   } catch (e) {
2466     body.innerHTML = '<div class="admin-err">${e.message}</div>';
2467     return;
2468   }
2469   body.innerHTML = '';
2470
2471   const grid = document.createElement('div');
2472   grid.className = 'stats-grid';
2473   const cards = [
2474     ['Foydalanuvchilar', `${s.users_active} / ${s.users_total}`],
2475     ['Bemorlar', s.patients_total],
2476     ['Yozuvlar', s.records_total],
2477     ['Bog\'langan DICOM', s.dicom_links_total],
2478     ['Annotatsiyalar', s.annotations_total],
2479     ['AI dan kelgan', s.ai_originated_annotations],
2480     ['Worklist', s.worklist_total],
2481     ['Audit hodisalar', s.history_events],
2482   ];
2483   for (const [lbl, num] of cards) {
2484     const c = document.createElement('div');
2485     c.className = 'stats-card';
2486     c.innerHTML = '<div class="lbl">${lbl}</div><div class="num">${num}</div>';
2487     grid.appendChild(c);
2488   }
2489   body.appendChild(grid);
2490
2491   if (s.review_seconds) {
2492     const sec = document.createElement('div');
2493     sec.className = 'stats-section';
2494     sec.innerHTML = '<h3>Review davomiyligi (sekund)</h3>' +
2495       '<div style="display:flex;gap:14px;color:var(--muted);font-size:12px">' +
2496       '<span>n=${s.review_seconds.count}</span>' +
2497       '<span>median=${s.review_seconds.median}s</span>' +
2498       '<span>p90=${s.review_seconds.p90}s</span>' +
2499       '<span>mean=${s.review_seconds.mean}s</span>' +
2500       '</div>';
2501     body.appendChild(sec);
2502   }
2503
2504   body.appendChild(buildBarSection('Annotatsiyalar status bo\'yicha', s.annotations_by_status,
2505     'status', statusBarColor));
2506   body.appendChild(buildBarSection('Top label\'lar', s.annotations_by_label.slice(0, 10), 'label'));
2507   body.appendChild(buildBarSection('Top yaratuvchilar', s.annotations_by_creator.slice(0, 10),
2508     'user'));
2509   body.appendChild(buildBarSection('Worklist statuslar', s.worklist_by_status, 'status'));
2510   body.appendChild(buildBarSection('Audit action turlari', s.history_by_action, 'action'));
2511   body.appendChild(buildBarSection('Top audit yaratuvchilar', s.history_per_user.slice(0, 10),
2512     'username'));
2513
2514   if (s.history_last_30_days && s.history_last_30_days.length) {
2515     const sec = document.createElement('div');
2516     sec.className = 'stats-section';
2517     sec.innerHTML = '<h3>Oxirgi 30 kun audit faolligi</h3>';
2518     const max = Math.max(...s.history_last_30_days.map(d => d.n));

```

```

2516     const spark = document.createElement('div');
2517     spark.className = 'stats-spark';
2518     for (const d of s.history_last_30_days) {
2519         const bar = document.createElement('div');
2520         bar.className = 'day';
2521         bar.style.height = `${Math.max(2, Math.round(d.n * 100 / max))}%`;
2522         bar.title = `${d.day}: ${d.n}`;
2523         spark.appendChild(bar);
2524     }
2525     sec.appendChild(spark);
2526     body.appendChild(sec);
2527 }
2528 }
2529
2530 function statusBarColor(label) {
2531     return ({ approved: 'green', submitted: 'yellow', rejected: 'red', done: 'green', in_progress:
'yellow' })[label] || '';
2532 }
2533
2534 function buildBarSection(title, items, key, colorFn) {
2535     const sec = document.createElement('div');
2536     sec.className = 'stats-section';
2537     sec.innerHTML = `<h3>${title}</h3>`;
2538     if (!items || !items.length) {
2539         sec.innerHTML += '<div class="report-empty" style="padding:6px 0">— bo\'sh</div>';
2540         return sec;
2541     }
2542     const max = Math.max(...items.map(i => i.n)) || 1;
2543     for (const it of items) {
2544         const lbl = it[key] || '?';
2545         const row = document.createElement('div');
2546         row.className = 'stats-bar-row';
2547         const bar = document.createElement('div');
2548         bar.className = 'bar';
2549         if (colorFn) {
2550             const c = colorFn(lbl);
2551             if (c) bar.classList.add(c);
2552         }
2553         bar.style.width = `${Math.round(it.n * 100 / max)}%`;
2554         row.innerHTML = `<div class="lbl" title="${lbl}">${lbl}</div>`;
2555         row.appendChild(bar);
2556         const num = document.createElement('div');
2557         num.className = 'num';
2558         num.textContent = it.n;
2559         row.appendChild(num);
2560         sec.appendChild(row);
2561     }
2562     return sec;
2563 }
2564
2565 function toggleMobile(selector) {
2566     const el = document.querySelector(selector);
2567     if (!el) return;
2568     el.classList.toggle('open');
2569 }
2570 $('mobileSidebarBtn').addEventListener('click', () => toggleMobile('aside.sidebar'));
2571 $('mobileMetaBtn').addEventListener('click', () => toggleMobile('aside.meta-pane'));
2572
2573 $('statsBtn').addEventListener('click', openStatsModal);
2574 $('statsCloseBtn').addEventListener('click', () => { $('statsModal').hidden = true; });
2575
2576 // ===== PACS modal =====
2577
2578 let _pacsSelectedSrv = null;
2579 let _pacsLastQuery = null;
2580

```

```

2581 async function openPacsModal() {
2582   $('pacsModal').hidden = false;
2583   await renderPacsServers();
2584 }
2585
2586 async function renderPacsServers() {
2587   const body = $('pacsBody');
2588   body.innerHTML = '<div class="report-empty">Yuklanmoqda...</div>';
2589   let data;
2590   try { data = await apiJson('/api/pacs/servers'); }
2591   catch (e) { body.innerHTML = '<div class="admin-err">${e.message}</div>'; return; }
2592   body.innerHTML = '';
2593
2594   const left = document.createElement('div');
2595   left.style.cssText = 'display:flex; flex-direction:column; gap:6px; margin-bottom:12px;';
2596   const h = document.createElement('h3');
2597   h.style.cssText = 'font-size:12px; color:var(--muted); text-transform:uppercase; margin:0;';
2598   h.textContent = 'Serverlar';
2599   left.appendChild(h);
2600
2601   for (const s of data.servers) {
2602     const row = document.createElement('div');
2603     row.style.cssText = 'display:flex; gap:6px; padding:6px 8px; background:var(--panel-2); border:1px
solid var(--border); border-radius:4px; align-items:center; cursor:pointer;';
2604     if (_pacsSelectedSrv && _pacsSelectedSrv.id === s.id) {
2605       row.style.borderColor = 'var(--accent)';
2606     }
2607     const info = document.createElement('div');
2608     info.style.flex = '1';
2609     info.innerHTML = '<b>${s.name}</b> <span style="color:var(--muted);font-
size:11px">${s.host}:${s.port} (${s.aet})</span>';
2610     row.appendChild(info);
2611
2612     const echoBtn = document.createElement('button');
2613     echoBtn.textContent = '🔊 Echo';
2614     echoBtn.addEventListener('click', async (e) => {
2615       e.stopPropagation();
2616       try {
2617         const r = await apiJson(`/api/pacs/servers/${s.id}/echo`, { method: 'POST' });
2618         alert(r.ok ? 'Echo OK ✓' : `Echo xato: ${r.status}`);
2619       } catch (err) { alert(err.message); }
2620     });
2621     row.appendChild(echoBtn);
2622
2623     if (state._user.role === 'admin') {
2624       const del = document.createElement('button');
2625       del.textContent = '🗑️'; del.style.color = 'var(--danger)';
2626       del.addEventListener('click', async (e) => {
2627         e.stopPropagation();
2628         if (!confirm(`'${s.name}' o'chirilsinmi?`)) return;
2629         try {
2630           await api(`/api/pacs/servers/${s.id}`, { method: 'DELETE' });
2631           if (_pacsSelectedSrv && _pacsSelectedSrv.id === s.id) _pacsSelectedSrv = null;
2632           await renderPacsServers();
2633         } catch (err) { alert(err.message); }
2634       });
2635       row.appendChild(del);
2636     }
2637
2638     row.addEventListener('click', () => {
2639       _pacsSelectedSrv = s;
2640       renderPacsServers();
2641     });
2642
2643     left.appendChild(row);
2644   }

```



```

2645
2646   if (state._user.role === 'admin' || state._user.role === 'reviewer') {
2647     const form = document.createElement('div');
2648     form.style.cssText = 'display:grid; grid-template-columns: 1fr 1fr 80px 1fr 1fr; gap:6px; margin-
2649     top:8px;';
2650     form.innerHTML = `
2651       <input type="text" id="pNewName" placeholder="nomi" />
2652       <input type="text" id="pNewHost" placeholder="host/IP" />
2653       <input type="number" id="pNewPort" placeholder="4242" min="1" max="65535" />
2654       <input type="text" id="pNewAet" placeholder="AET" />
2655       <button id="pNewSubmit">+ Qo'shish</button>
2656     `;
2657     left.appendChild(form);
2658   }
2659   body.appendChild(left);
2660
2661   const submit = $('pNewSubmit');
2662   if (submit) {
2663     submit.addEventListener('click', async () => {
2664       const name = $('pNewName').value.trim();
2665       const host = $('pNewHost').value.trim();
2666       const port = parseInt($('pNewPort').value, 10);
2667       const aet = $('pNewAet').value.trim();
2668       if (!name || !host || !aet || !port) { alert('Hamma maydonlar kerak'); return; }
2669       try {
2670         await api('/api/pacs/servers', {
2671           method: 'POST',
2672           headers: { 'content-type': 'application/json' },
2673           body: JSON.stringify({ name, host, port, aet }),
2674         });
2675         await renderPacsServers();
2676       } catch (e) { alert(e.message); }
2677     });
2678   }
2679
2680   if (_pacsSelectedSrv) {
2681     const right = document.createElement('div');
2682     right.style.cssText = 'border-top: 1px solid var(--border); padding-top:12px;';
2683     const h2 = document.createElement('h3');
2684     h2.style.cssText = 'font-size:12px; color:var(--muted); text-transform:uppercase; margin:0 0 6px';
2685     h2.textContent = `Qidirish - ${_pacsSelectedSrv.name}`;
2686     right.appendChild(h2);
2687
2688     const form = document.createElement('div');
2689     form.style.cssText = 'display:grid; grid-template-columns: 1fr 1fr 1fr 1fr 100px; gap:6px;';
2690     form.innerHTML = `
2691       <input type="text" id="pqId" placeholder="Patient ID" />
2692       <input type="text" id="pqName" placeholder="Patient name" />
2693       <input type="text" id="pqMod" placeholder="Modality (MG)" value="MG" />
2694       <input type="text" id="pqDate" placeholder="StudyDate YYYYMMDD" />
2695       <button id="pqSubmit">🔍 Qidirish</button>
2696     `;
2697     right.appendChild(form);
2698
2699     const results = document.createElement('div');
2700     results.id = 'pacsResults';
2701     results.style.cssText = 'margin-top:10px; max-height:300px; overflow:auto;';
2702     if (_pacsLastQuery) {
2703       renderPacsResults(results, _pacsLastQuery);
2704     }
2705     right.appendChild(results);
2706
2707     body.appendChild(right);
2708
2709     $('pqSubmit').addEventListener('click', async () => {
2710       const params = {

```

```

2710     patient_id: $('pqId').value.trim(),
2711     patient_name: $('pqName').value.trim(),
2712     modality: $('pqMod').value.trim(),
2713     study_date: $('pqDate').value.trim(),
2714   });
2715   results.innerHTML = '<div class="report-empty">Qidirilmoqda...</div>';
2716   try {
2717     const data = await apiJson(`/api/pacs/servers/${_pacsSelectedSrv.id}/query`, {
2718       method: 'POST',
2719       headers: { 'content-type': 'application/json' },
2720       body: JSON.stringify(params),
2721     });
2722     _pacsLastQuery = data.results;
2723     renderPacsResults(results, data.results);
2724   } catch (e) {
2725     results.innerHTML = `<div class="admin-err">${e.message}</div>`;
2726   }
2727   });
2728 }
2729 }
2730
2731 function renderPacsResults(container, results) {
2732   container.innerHTML = '';
2733   if (!results || !results.length) {
2734     container.innerHTML = '<div class="report-empty">- natija yo\'q</div>';
2735     return;
2736   }
2737   const meta = document.createElement('div');
2738   meta.style.cssText = 'color:var(--muted); font-size:11px; margin-bottom:6px';
2739   meta.textContent = `${results.length} ta study`;
2740   container.appendChild(meta);
2741
2742   const table = document.createElement('table');
2743   table.className = 'user-table';
2744   table.innerHTML = `<thead><tr>
2745     <th>Patient ID</th><th>Name</th><th>Date</th><th>Modality</th><th>Description</th><th></th>
2746   </tr></thead>`;
2747   const tbody = document.createElement('tbody');
2748   for (const r of results) {
2749     const tr = document.createElement('tr');
2750     tr.innerHTML = `
2751       <td>${r.patient_id || ''}</td>
2752       <td>${r.patient_name || ''}</td>
2753       <td>${r.study_date || ''}</td>
2754       <td>${r.modalities || ''}</td>
2755       <td style="font-size:10px; color:var(--muted)">${(r.description || '').slice(0,40)}</td>
2756     `;
2757     const tdAct = document.createElement('td');
2758     const btn = document.createElement('button');
2759     btn.textContent = '→ Worklist';
2760     btn.title = "Worklist'ga qo'shish";
2761     btn.addEventListener('click', async () => {
2762       const dt = r.study_date || '';
2763       const formatted = dt.length === 8 ? `${dt.slice(0,4)}-${dt.slice(4,6)}-${dt.slice(6,8)}` : dt;
2764       try {
2765         await api('/api/pacs/to-worklist', {
2766           method: 'POST',
2767           headers: { 'content-type': 'application/json' },
2768           body: JSON.stringify({
2769             patient_id: r.patient_id,
2770             patient_name: r.patient_name,
2771             study_date: formatted,
2772             modality: (r.modalities || '').split('\\').shift() || '',
2773           }),
2774         });
2775         btn.textContent = '✓ qo\'shildi';

```

```

2776         btn.disabled = true;
2777     } catch (e) { alert(e.message); }
2778     });
2779     tdAct.appendChild(btn);
2780     tr.appendChild(tdAct);
2781     tbody.appendChild(tr);
2782 }
2783 table.appendChild(tbody);
2784 container.appendChild(table);
2785 }
2786
2787 $('pacsBtn').addEventListener('click', openPacsModal);
2788 $('pacsCloseBtn').addEventListener('click', () => { $('pacsModal').hidden = true; });
2789
2790 // ===== TOTP modal =====
2791
2792 async function openTotpModal() {
2793     $('totpModal').hidden = false;
2794     await renderTotp();
2795 }
2796
2797 async function renderTotp() {
2798     const body = $('totpBody');
2799     if (state._user.totp_enrolled) {
2800         body.innerHTML = `
2801             <div class="report-empty" style="padding:0 0 8px">
2802                 2FA hozir <b style="color:var(--ok)">yoqilgan</b>. Har kirishda authenticator kodi so'raladi.
2803             </div>
2804             <h3 style="font-size:12px;color:var(--muted);text-transform:uppercase;margin:14px 0 6px">O'chirish</h3>
2805             <div style="display:flex;gap:6px">
2806                 <input type="password" id="totpDisablePwd" placeholder="parolingiz" style="flex:1;
background:var(--panel-2);border:1px solid var(--border);color:var(--text);padding:6px;border-radius:3px"/>
2807                 <button id="totpDisableBtn" style="color:var(--danger)">2FA ni o'chirish</button>
2808             </div>
2809             <div class="modal-err" id="totpErr"></div>
2810         `;
2811         $('totpDisableBtn').addEventListener('click', async () => {
2812             const pwd = $('totpDisablePwd').value;
2813             if (!pwd) { $('totpErr').textContent = 'parol kerak'; return; }
2814             try {
2815                 await api('/api/auth/totp/disable', {
2816                     method: 'POST',
2817                     headers: { 'content-type': 'application/json' },
2818                     body: JSON.stringify({ password: pwd }),
2819                 });
2820                 state._user.totp_enrolled = false;
2821                 updateUserBar();
2822                 await renderTotp();
2823             } catch (e) {
2824                 $('totpErr').textContent = e.message;
2825             }
2826         });
2827         return;
2828     }
2829
2830     body.innerHTML = '<div class="report-empty">QR kod tayyorlanmoqda...</div>';
2831     let setup;
2832     try {
2833         setup = await apiJson('/api/auth/totp/setup', { method: 'POST' });
2834     } catch (e) {
2835         body.innerHTML = `<div class="admin-err">${e.message}</div>`;
2836         return;
2837     }
2838
2839     body.innerHTML = `

```

```

2840     <p style="margin:0 0 10px;font-size:12px;color:var(--muted)">
2841         Telefonigizdagi authenticator (Google Authenticator, Authy, 1Password, ...) 'da bu QR kodni
skanerlang
2842         yoki secret'ni qo'lda kiriting:
2843     </p>
2844     <div style="display:flex;gap:14px;align-items:flex-start">
2845         
2846         <div style="flex:1">
2847             <div style="font-size:10px;color:var(--muted);text-transform:uppercase">Secret (manual)</div>
2848             <code style="display:block;background:var(--panel-2);padding:6px;border-radius:3px;font-
size:11px;word-break:break-all;margin-top:4px">${setup.secret}</code>
2849             <div style="font-size:10px;color:var(--muted);margin-top:10px">App'dagi 6 raqamli kodni
kiriting:</div>
2850             <input type="text" id="totpVerifyCode" inputmode="numeric" maxlength="6"
2851                 style="width:120px; background:var(--panel-2); border:1px solid var(--border);
color:var(--text); padding:8px; font-size:18px; letter-spacing:4px; border-radius:3px; margin-top:6px"/>
2852             <button id="totpVerifyBtn" style="display:block;margin-top:8px">✓ Yoqish</button>
2853             <div class="modal-err" id="totpErr"></div>
2854         </div>
2855     </div>
2856 `;
2857 $('totpVerifyBtn').addEventListener('click', async () => {
2858     const code = $('totpVerifyCode').value.trim();
2859     if (code.length !== 6) { $('totpErr').textContent = '6 raqamli kod kerak'; return; }
2860     try {
2861         await api('/api/auth/totp/verify', {
2862             method: 'POST',
2863             headers: { 'content-type': 'application/json' },
2864             body: JSON.stringify({ code }),
2865         });
2866         state._user.totp_enrolled = true;
2867         updateUserBar();
2868         await renderTotp();
2869         setStatus('✓ 2FA yoqildi');
2870     } catch (e) {
2871         $('totpErr').textContent = e.message;
2872     }
2873 });
2874 }
2875
2876 $('totpBtn').addEventListener('click', openTotpModal);
2877 $('totpCloseBtn').addEventListener('click', () => { $('totpModal').hidden = true; });
2878
2879 // ===== Annotated DICOMS overview =====
2880
2881 async function openOverviewModal() {
2882     $('overviewModal').hidden = false;
2883     await renderOverview();
2884 }
2885
2886 async function renderOverview(filter) {
2887     const body = $('overviewBody');
2888     body.innerHTML = '<div class="report-empty">Yuklanmoqda...</div>';
2889     const params = new URLSearchParams();
2890     if (filter && filter.status) params.set('status', filter.status);
2891     if (filter && filter.own) params.set('own', 'true');
2892     let data;
2893     try {
2894         data = await apiJson(`/api/annotations/overview?${params}&limit=300`);
2895     } catch (e) {
2896         body.innerHTML = `<div class="admin-err">${e.message}</div>`;
2897         return;
2898     }
2899     body.innerHTML = '';
2900

```

```

2901 const ctrl = document.createElement('div');
2902 ctrl.className = 'ov-controls';
2903 const sel = document.createElement('select');
2904 sel.innerHTML = `
2905   <option value="">Barcha statuslar</option>
2906   <option value="draft">Drafts</option>
2907   <option value="submitted">Submitted</option>
2908   <option value="approved">Approved</option>
2909   <option value="rejected">Rejected</option>
2910 `;
2911 if (filter && filter.status) sel.value = filter.status;
2912 sel.addEventListener('change', () => renderOverview({ status: sel.value, own: ownChk.checked }));
2913
2914 const ownChk = document.createElement('input');
2915 ownChk.type = 'checkbox';
2916 ownChk.checked = !(filter && filter.own);
2917 ownChk.addEventListener('change', () => renderOverview({ status: sel.value, own: ownChk.checked }));
2918 const ownLbl = document.createElement('label');
2919 ownLbl.style.cssText = 'display:flex; gap:4px; align-items:center; font-size:12px; color:var(--muted)';
2920 ownLbl.append(ownChk, document.createTextNode(' faqat meniki'));
2921
2922 ctrl.appendChild(sel);
2923 ctrl.appendChild(ownLbl);
2924 body.appendChild(ctrl);
2925
2926 const meta = document.createElement('div');
2927 meta.className = 'ov-meta-bar';
2928 meta.textContent = `${data.count} / ${data.total} ta annotatsiyalangan DICOM`;
2929 body.appendChild(meta);
2930
2931 if (!data.items.length) {
2932   const empty = document.createElement('div');
2933   empty.className = 'report-empty';
2934   empty.textContent = "Topilmadi.";
2935   body.appendChild(empty);
2936   return;
2937 }
2938
2939 const grid = document.createElement('div');
2940 grid.className = 'ov-grid';
2941 for (const it of data.items) {
2942   const card = document.createElement('div');
2943   card.className = 'ov-card';
2944
2945   const name = document.createElement('div');
2946   name.className = 'ov-name';
2947   if (it.patient) {
2948     name.textContent = `${it.patient.last_name || ''} ${it.patient.first_name || ''}`.trim();
2949   } else {
2950     name.textContent = it.ref.split('/').pop();
2951   }
2952   card.appendChild(name);
2953
2954   const m1 = document.createElement('div');
2955   m1.className = 'ov-meta';
2956   if (it.patient) {
2957     m1.textContent = `ID ${it.patient.patient_id} · ${it.patient.sex || '-'} · DOB
2958     ${it.patient.birth_date || '-'}`;
2959   } else {
2960     m1.textContent = `${it.source}: ${it.ref}`;
2961   }
2962   card.appendChild(m1);
2963
2964   const m2 = document.createElement('div');
2965   m2.className = 'ov-meta';

```

```

2965     m2.style.opacity = '0.7';
2966     m2.textContent = `${it.annotation_count} ann · ${it.creators || []}.join(', ') || '-'} ·
${it.latest_ts || ''}.slice(0,10)}`;
2967     card.appendChild(m2);
2968
2969     const stats = document.createElement('div');
2970     stats.className = 'ov-stats';
2971     for (const [s, n] of Object.entries(it.by_status)) {
2972         const p = document.createElement('span');
2973         p.className = `ov-pill ${s}`;
2974         p.textContent = `${s}: ${n}`;
2975         stats.appendChild(p);
2976     }
2977     card.appendChild(stats);
2978
2979     if (Object.keys(it.by_label).length) {
2980         const labels = document.createElement('div');
2981         labels.className = 'ov-labels';
2982         labels.textContent = Object.entries(it.by_label)
2983             .map(([k, v]) => `${k}×${v}`).join(', ');
2984         card.appendChild(labels);
2985     }
2986
2987     card.addEventListener('click', () => {
2988         $('overviewModal').hidden = true;
2989         if (it.source === 'upload') openItem({ kind: 'upload', id: it.ref });
2990         else openItem({ kind: 'local', path: it.ref });
2991     });
2992     grid.appendChild(card);
2993 }
2994 body.appendChild(grid);
2995 }
2996
2997 $('overviewBtn').addEventListener('click', openOverviewModal);
2998 $('overviewCloseBtn').addEventListener('click', () => { $('overviewModal').hidden = true; });
2999
3000 // ===== Annotation templates =====
3001
3002 async function refreshTemplates() {
3003     if (!state._user) return;
3004     try {
3005         const data = await apiJson('/api/templates');
3006         state._templates = data.templates || [];
3007         const sel = $('tplSelect');
3008         sel.innerHTML = '<option value="">📁 Template...</option>';
3009         for (const t of state._templates) {
3010             const o = document.createElement('option');
3011             o.value = t.id;
3012             o.textContent = `${t.name} (${t.annotation_count})`;
3013             sel.appendChild(o);
3014         }
3015     } catch (e) {}
3016 }
3017
3018 $('tplSelect').addEventListener('change', async (e) => {
3019     const id = e.target.value;
3020     if (!id) return;
3021     try {
3022         const t = await apiJson(`/api/templates/${id}`);
3023         if (!t.payload || !t.payload.length) { setStatus('template bo\'sh'); return; }
3024         const now = new Date().toISOString();
3025         let added = 0;
3026         for (const a of t.payload) {
3027             const fresh = JSON.parse(JSON.stringify(a));
3028             fresh.id = uid();
3029             fresh.frame = state.frame;

```

```

3030     fresh.created_at = now;
3031     fresh.updated_at = now;
3032     fresh.note = (fresh.note ? fresh.note + ' ' : '') + `[tpl: ${t.name}]`;
3033     state.annotations.push(fresh);
3034     added++;
3035   }
3036   state.selectedId = state.annotations[state.annotations.length - 1].id;
3037   renderSvg(); renderAnnoList();
3038   scheduleAutosave();
3039   setStatus(`✓ ${added} ann (template "${t.name}")`);
3040 } catch (err) {
3041   setStatus('template xato: ' + err.message);
3042 }
3043 e.target.value = '';
3044 });
3045
3046 $('tplSaveBtn').addEventListener('click', async () => {
3047   if (!state.annotations.length) { alert("hech narsa yo'q"); return; }
3048   const name = prompt("Template nomi:", `tpl_${new Date().toISOString().slice(0,10)}`);
3049   if (!name) return;
3050   const desc = prompt("Tavsif (ixtiyoriy):", "") || null;
3051   const isShared = confirm("Hammaga ko'rinsinmi? (Cancel = faqat menga)");
3052   try {
3053     const r = await apiJson('/api/templates', {
3054       method: 'POST',
3055       headers: { 'content-type': 'application/json' },
3056       body: JSON.stringify({
3057         name, description: desc, is_shared: isShared,
3058         payload: state.annotations,
3059       }),
3060     });
3061     setStatus(`✓ template "${name}" saqlandi (${r.annotation_count} ann)`);
3062     await refreshTemplates();
3063   } catch (e) { alert(e.message); }
3064 });
3065
3066 // ===== C-STORE =====
3067
3068 async function pickPacsServer() {
3069   let data;
3070   try { data = await apiJson('/api/pacs/servers'); }
3071   catch (e) { alert(e.message); return null; }
3072   if (!data.servers || !data.servers.length) {
3073     alert("PACS server qo'shilmagan. 🚫 PACS modal orqali qo'shing.");
3074     return null;
3075   }
3076   if (data.servers.length === 1) return data.servers[0];
3077   const names = data.servers.map((s, i) => `${i + 1}. ${s.name} (${s.host}:${s.port})`).join('\n');
3078   const choice = prompt(`Qaysi PACS serverga yuborish?\n${names}\n\nRaqam:`, "1");
3079   const idx = parseInt(choice, 10) - 1;
3080   return data.servers[idx] || null;
3081 }
3082
3083 $('cStoreBtn').addEventListener('click', async () => {
3084   if (!state.current) return;
3085   const srv = await pickPacsServer();
3086   if (!srv) return;
3087   if (!confirm(`${srv.name} (${srv.aet}@${srv.host}:${srv.port}) serverga joriy DICOM yuborilsinmi?`)) return;
3088   setStatus(`🚀 ${srv.name} ga yuborilmoqda...`);
3089   try {
3090     const r = await apiJson(`/api/pacs/servers/${srv.id}/store`, {
3091       method: 'POST',
3092       headers: { 'content-type': 'application/json' },
3093       body: JSON.stringify({
3094         items: [{ source: refSource(), ref: refValue() }],

```

```

3095     }},
3096   });
3097   setStatus(`✓ ${r.sent}/${r.total} yuborildi (${r.failed} xato)`);
3098 } catch (e) {
3099   setStatus("PACS xato: " + e.message);
3100 }
3101 });
3102
3103 // ===== Audit timeline =====
3104
3105 async function openAuditModal() {
3106   $('auditModal').hidden = false;
3107   await renderAuditTimeline();
3108 }
3109
3110 async function renderAuditTimeline(filter) {
3111   const body = $('auditBody');
3112   body.innerHTML = '<div class="report-empty">Yuklanmoqda...</div>';
3113   const params = new URLSearchParams();
3114   if (filter && filter.username) params.set('username', filter.username);
3115   if (filter && filter.action) params.set('action', filter.action);
3116   params.set('limit', '300');
3117   let data;
3118   try { data = await apiJson(`/api/audit/timeline?${params}`); }
3119   catch (e) { body.innerHTML = '<div class="admin-err">${e.message}</div>'; return; }
3120   body.innerHTML = '';
3121
3122   const ctrl = document.createElement('div');
3123   ctrl.className = 'ov-controls';
3124   const userInp = document.createElement('input');
3125   userInp.placeholder = 'username';
3126   userInp.value = (filter && filter.username) || '';
3127   userInp.style.cssText = 'background:var(--panel-2);border:1px solid var(--border);color:var(--text);padding:4px 8px;border-radius:3px';
3128   const actSel = document.createElement('select');
3129   actSel.innerHTML = `
3130     <option value="">Barcha amallar</option>
3131     <option value="create">create</option>
3132     <option value="update">update</option>
3133     <option value="delete">delete</option>
3134     <option value="status">status</option>
3135   `;
3136   if (filter && filter.action) actSel.value = filter.action;
3137   const apply = document.createElement('button');
3138   apply.textContent = '🔍';
3139   apply.addEventListener('click', () => renderAuditTimeline({
3140     username: userInp.value.trim(),
3141     action: actSel.value,
3142   }));
3143   ctrl.append(userInp, actSel, apply);
3144   body.appendChild(ctrl);
3145
3146   const meta = document.createElement('div');
3147   meta.className = 'ov-meta-bar';
3148   meta.textContent = `${data.count} ta hodisa (eng so'nggi)`;
3149   body.appendChild(meta);
3150
3151   if (!data.events.length) {
3152     const empty = document.createElement('div');
3153     empty.className = 'report-empty';
3154     empty.textContent = "Hodisalar yo'q";
3155     body.appendChild(empty);
3156     return;
3157   }
3158   let lastDay = '';
3159   for (const ev of data.events) {

```



```

3160     const day = (ev.ts || '').slice(0, 10);
3161     if (day !== lastDay) {
3162         const sep = document.createElement('div');
3163         sep.style.cssText = 'margin:12px 0 4px; color:var(--accent); font-weight:600; font-size:12px';
3164         sep.textContent = day;
3165         body.appendChild(sep);
3166         lastDay = day;
3167     }
3168     const row = document.createElement('div');
3169     row.className = 'hist-row';
3170     row.style.borderLeft = '3px solid var(--border)';
3171     row.style.paddingLeft = '10px';
3172     row.innerHTML = `<div class="row-head">
3173         <span class="action-pill action-${ev.action}">${ev.action}</span>
3174         <span class="who">${ev.username || '-'}</span>
3175         <span class="ts">${(ev.ts || '').slice(11, 19)}</span>
3176         <span class="ts" style="opacity:0.7">${ev.ref}</span>
3177     </div>`;
3178     if (ev.action === 'status' && ev.prev_snapshot && ev.new_snapshot) {
3179         const t = document.createElement('div');
3180         t.style.cssText = 'margin-top:4px; font-size:11px';
3181         const note = ev.new_snapshot.note ? ` (${ev.new_snapshot.note})` : '';
3182         t.innerHTML = `<b>${ev.prev_snapshot.status}</b> → <b>${ev.new_snapshot.status}</b>${note}`;
3183         row.appendChild(t);
3184     }
3185     body.appendChild(row);
3186 }
3187 }
3188
3189 $('auditBtn').addEventListener('click', openAuditModal);
3190 $('auditCloseBtn').addEventListener('click', () => { $('auditModal').hidden = true; });
3191
3192 $('deidBtn').addEventListener('click', openDeidModal);
3193 $('deidCloseBtn').addEventListener('click', () => { $('deidModal').hidden = true; });
3194
3195 async function downloadDicomExport(kind, suffix, label) {
3196     if (!state.current) return;
3197     const tok = getToken();
3198     setStatus(`${label} yaratilmoqda...`);
3199     const res = await fetch(
3200         `/api/export/${kind}?source=${refSource()}&ref=${encodeURIComponent(refValue())}`,
3201         { headers: { 'Authorization': `Bearer ${tok}` } },
3202     );
3203     if (!res.ok) {
3204         if (res.status === 404) alert("Annotatsiyalar yo'q – avval annotatsiya qiling");
3205         else if (res.status === 503) {
3206             const err = await res.text();
3207             alert(`Server ${kind} eksportni qo'llab-quvvatlamayapti: \n${err}`);
3208         } else alert(`${label} xato: ` + res.status);
3209         setStatus('');
3210         return;
3211     }
3212     const blob = await res.blob();
3213     const url = URL.createObjectURL(blob);
3214     const a = document.createElement('a');
3215     a.href = url;
3216     const stem = (refValue() || 'anon').split('/').pop().replace(/\.dcm$/i, '');
3217     a.download = `${stem}_${suffix}.dcm`;
3218     document.body.appendChild(a);
3219     a.click();
3220     a.remove();
3221     URL.revokeObjectURL(url);
3222     setStatus(`${label} yuklab olindi`);
3223 }
3224
3225 $('srBtn').addEventListener('click', () => downloadDicomExport('dicom-sr', 'sr', 'DICOM-SR'));

```

```

3226 $('segBtn').addEventListener('click', () => downloadDicomExport('dicom-seg', 'seg', 'DICOM-SEG'));
3227
3228 async function downloadMaskExport(format, ext, label) {
3229   if (!state.current) return;
3230   const tok = getToken();
3231   setStatus(`${label} yaratilmoqda...`);
3232   const res = await fetch(
3233     `/api/export/mask?source=${refSource()}&ref=${encodeURIComponent(refValue())}&format=${format}`,
3234     { headers: { 'Authorization': `Bearer ${tok}` } },
3235   );
3236   if (!res.ok) {
3237     if (res.status === 404) alert("Annotatsiyalar yo'q – avval annotatsiya qiling");
3238     else if (res.status === 422) alert('Rasm o\'lchamlari noma'lum – maska yaratib bo'lmadi');
3239     else alert(`${label} xato: ` + res.status);
3240     setStatus('');
3241     return;
3242   }
3243   const legend = res.headers.get('X-Mask-Legend');
3244   const blob = await res.blob();
3245   const url = URL.createObjectURL(blob);
3246   const a = document.createElement('a');
3247   a.href = url;
3248   const stem = (refValue() || 'anon').split('/').pop().replace(/\.dcm$/i, '');
3249   a.download = `${stem}_${ext}`;
3250   document.body.appendChild(a);
3251   a.click();
3252   a.remove();
3253   URL.revokeObjectURL(url);
3254   setStatus(`${label} yuklab olindi${legend ? ' · legend: ' + legend : ''}`);
3255 }
3256 $('maskPngBtn').addEventListener('click', () => downloadMaskExport('png', 'mask.png', 'Mask PNG'));
3257 $('maskNiftiBtn').addEventListener('click', () => downloadMaskExport('nifti', 'mask.nii.gz', 'Mask
NIFTI'));
3258
3259 (function bindLinksUi() {
3260   const inp = $('linksSearch');
3261   let timer = null;
3262   inp.addEventListener('input', () => {
3263     clearTimeout(timer);
3264     timer = setTimeout(() => loadLinks(inp.value), 300);
3265   });
3266   inp.addEventListener('keydown', (e) => { if (e.key === 'Enter') loadLinks(inp.value); });
3267   $('linksRefresh').addEventListener('click', () => loadLinks(inp.value));
3268 })();
3269
3270 // right tabs
3271 document.querySelectorAll('.meta-pane .tab').forEach(btn => {
3272   btn.addEventListener('click', () => {
3273     document.querySelectorAll('.meta-pane .tab').forEach(b => b.classList.remove('active'));
3274     btn.classList.add('active');
3275     const t = btn.dataset.rtab;
3276     document.querySelectorAll('.meta-pane .rpane').forEach(p => p.hidden = p.dataset.rpane !== t);
3277   });
3278 });
3279
3280 let _currentLocalDir = '';
3281
3282 function refreshActiveList() {
3283   const uploadsActive = !document.querySelector('.sidebar [data-pane="uploads"]').hidden;
3284   if (uploadsActive) {
3285     refreshUploads().catch(() => {});
3286   } else if (state.localEnabled) {
3287     loadLocalDir(_currentLocalDir).catch(() => {});
3288   }
3289 }
3290

```

```

3291 async function loadLinks(q) {
3292   setStatus("bog'langanlar olinmoqda...");
3293   try {
3294     const params = new URLSearchParams();
3295     if (q) params.set('q', q);
3296     params.set('limit', '500');
3297     const data = await apiJson(`/api/db/links?${params}`);
3298     $('linksMeta').textContent = `${data.count} ta bog'langan DICOM` + (q ? ` ("${q}")` : '');
3299     const ul = $('linksList');
3300     ul.innerHTML = '';
3301     for (const r of data.links) {
3302       const li = document.createElement('li');
3303       li.className = 'link-row';
3304       const top = document.createElement('div');
3305       top.className = 'row';
3306       const left = document.createElement('div');
3307       const name = document.createElement('div');
3308       name.className = 'name';
3309       name.textContent = `${r.last_name || ''} ${r.first_name || ''}`.trim() || '-';
3310       if (r.annotation_count > 0) {
3311         const pill = document.createElement('span');
3312         pill.className = 'anno-pill';
3313         pill.textContent = r.annotation_count;
3314         pill.title = `${r.annotation_count} annotatsiya`;
3315         name.appendChild(pill);
3316       }
3317       left.appendChild(name);
3318       const sub = document.createElement('div');
3319       sub.className = 'meta';
3320       const conf = r.confidence != null ? `${r.confidence}%` : 'qo'lda';
3321       sub.textContent = `ID ${r.patient_id} · DOB ${r.birth_date || '-'} · ${r.record_count} yozuv · oxirgi ${r.last_visit || '-'} · ${conf}`;
3322       left.appendChild(sub);
3323       const sub2 = document.createElement('div');
3324       sub2.className = 'meta';
3325       sub2.textContent = `${r.source}: ${r.ref}`;
3326       sub2.style.opacity = '0.7';
3327       left.appendChild(sub2);
3328       top.appendChild(left);
3329       li.appendChild(top);
3330       li.addEventListener('click', () => openLinkedRef(r));
3331       ul.appendChild(li);
3332     }
3333     setStatus('tayyor');
3334   } catch (e) {
3335     setStatus("xato: " + e.message);
3336   }
3337 }
3338
3339 async function openLinkedRef(r) {
3340   document.querySelectorAll('.file-list li.active').forEach(x => x.classList.remove('active'));
3341   if (r.source === 'upload') {
3342     await openItem({ kind: 'upload', id: r.ref });
3343   } else {
3344     await openItem({ kind: 'local', path: r.ref });
3345   }
3346 }
3347
3348 async function loadLocalDir(subdir) {
3349   setStatus("ro'yxat olinmoqda...");
3350   _currentLocalDir = subdir || '';
3351   try {
3352     const data = await apiJson(`/api/local/list?subdir=${encodeURIComponent(subdir)}`);
3353     $('crumb').textContent = subdir ? '/' + subdir : '/';
3354     const items = [];
3355     if (subdir) items.push({ kind: 'dir', label: '..', path: data.parent });

```

```

3356     for (const d of data.dirs) items.push({ kind: 'dir', label: d.name, path: d.path });
3357     for (const f of data.files) items.push({
3358       kind: 'local', label: f.name, sub: humanSize(f.size),
3359       path: f.path, annoCount: f.annotation_count || 0,
3360     });
3361     renderFileList($('localList'), items);
3362     setStatus('tayyor');
3363   } catch (e) {
3364     setStatus('xato: ' + e.message);
3365   }
3366 }
3367 function humanSize(n) {
3368   if (!n) return '';
3369   const u = ['B', 'KB', 'MB', 'GB'];
3370   let i = 0; while (n >= 1024 && i < u.length - 1) { n /= 1024; i++; }
3371   return `${n.toFixed(i ? 1 : 0)} ${u[i]}`;
3372 }
3373
3374 // toolbar buttons
3375 $('fitBtn').addEventListener('click', fitToScreen);
3376 $('oneToOneBtn').addEventListener('click', oneToOne);
3377 $('invertBtn').addEventListener('click', () => {
3378   state.invert = !state.invert;
3379   $('invertBtn').classList.toggle('active', state.invert);
3380   loadImage(false);
3381 });
3382 $('resetWlBtn').addEventListener('click', () => {
3383   state.wc = state.defaultWc; state.ww = state.defaultWw;
3384   applyDefaults();
3385   loadImage(false);
3386 });
3387
3388 // frame slider
3389 $('frameSlider').addEventListener('input', (e) => {
3390   state.frame = parseInt(e.target.value, 10) || 0;
3391   $('frameLabel').textContent = `${state.frame + 1} / ${state.frames}`;
3392   loadImage(false);
3393   renderSvg();
3394 });
3395
3396 // W/L bindings
3397 (function bindWl() {
3398   $('wcSlider').addEventListener('input', (e) => {
3399     state.wc = parseFloat(e.target.value); $('wcInput').value = e.target.value; scheduleReload();
3400   });
3401   $('wwSlider').addEventListener('input', (e) => {
3402     state.ww = parseFloat(e.target.value); $('wwInput').value = e.target.value; scheduleReload();
3403   });
3404   $('wcInput').addEventListener('change', (e) => {
3405     state.wc = parseFloat(e.target.value);
3406     $('wcSlider').value = clampRange($('wcSlider'), state.wc);
3407     scheduleReload();
3408   });
3409   $('wwInput').addEventListener('change', (e) => {
3410     state.ww = parseFloat(e.target.value);
3411     $('wwSlider').value = clampRange($('wwSlider'), state.ww);
3412     scheduleReload();
3413   });
3414 })();
3415
3416 window.addEventListener('resize', () => {
3417   if (Math.abs(state.scale - state.fitScale) < 1e-3) fitToScreen();
3418 });
3419
3420 // ===== ANNOTATION TOOLS =====
3421

```

```

3422 function setTool(t) {
3423   if (state.tool === 'poly' && t !== 'poly') cancelPolyDraw();
3424   state.tool = t;
3425   $('toolSelect').classList.toggle('active', t === 'select');
3426   $('toolBbox').classList.toggle('active', t === 'bbox');
3427   $('toolPoly').classList.toggle('active', t === 'poly');
3428   const stack = $('imgStack');
3429   stack.classList.toggle('drawing', t === 'bbox');
3430   stack.classList.toggle('drawing-poly', t === 'poly');
3431   $('canvasWrap').classList.toggle('drawing', t === 'bbox' || t === 'poly');
3432 }
3433 $('toolSelect').addEventListener('click', () => setTool('select'));
3434 $('toolBbox').addEventListener('click', () => setTool('bbox'));
3435 $('toolPoly').addEventListener('click', () => setTool('poly'));
3436
3437 function svgViewSize() {
3438   const svg = $('annoSvg');
3439   const vb = svg.viewBox.baseVal;
3440   return { w: vb.width || 1, h: vb.height || 1 };
3441 }
3442 function svgPointPx(e) {
3443   const svg = $('annoSvg');
3444   const pt = svg.createSVGPoint();
3445   pt.x = e.clientX; pt.y = e.clientY;
3446   const ctm = svg.getScreenCTM();
3447   if (!ctm) return { x: 0, y: 0 };
3448   return pt.matrixTransform(ctm.inverse());
3449 }
3450 function svgPointNorm(e) {
3451   const p = svgPointPx(e);
3452   const { w, h } = svgViewSize();
3453   return { x: p.x / w, y: p.y / h };
3454 }
3455
3456 // Polygon drawing
3457 const polyDraw = { points: [], cursor: null };
3458
3459 function cancelPolyDraw() {
3460   polyDraw.points = [];
3461   polyDraw.cursor = null;
3462   renderSvg();
3463 }
3464
3465 function finalizePolyDraw() {
3466   if (polyDraw.points.length < 3) {
3467     cancelPolyDraw();
3468     return;
3469   }
3470   const labels = pendingLabels();
3471   const label = labels[0];
3472   const birads = $('biradsSelect').value || '';
3473   const pts = polyDraw.points.map(p => [clamp01(p[0]), clamp01(p[1])]);
3474   const xs = pts.map(p => p[0]);
3475   const ys = pts.map(p => p[1]);
3476   const minX = Math.min(...xs), minY = Math.min(...ys);
3477   const maxX = Math.max(...xs), maxY = Math.max(...ys);
3478   const ann = {
3479     id: uid(),
3480     type: 'polygon',
3481     label,
3482     labels,
3483     bi_rads: birads,
3484     note: '',
3485     frame: state.frame,
3486     points: pts,
3487     bbox: [minX, minY, maxX - minX, maxY - minY],

```

```

3488     created_at: new Date().toISOString(),
3489     updated_at: new Date().toISOString(),
3490   };
3491   state.annotations.push(ann);
3492   state.selectedId = ann.id;
3493   cancelPolyDraw();
3494   renderAnnoList();
3495   scheduleAutosave();
3496 }
3497
3498 function onPolySvgClick(e) {
3499   if (state.tool !== 'poly') return;
3500   if (e.detail === 2) return;
3501   e.preventDefault(); e.stopPropagation();
3502   const p = svgPointNorm(e);
3503   polyDraw.points.push([p.x, p.y]);
3504   renderSvg();
3505 }
3506
3507 function onPolySvgMove(e) {
3508   const p = svgPointNorm(e);
3509   if (state.current && _ws && _ws.readyState === 1) {
3510     if (p.x >= 0 && p.x <= 1 && p.y >= 0 && p.y <= 1) {
3511       sendCursor(p.x, p.y);
3512     }
3513   }
3514   if (state.tool !== 'poly' || polyDraw.points.length === 0) return;
3515   polyDraw.cursor = [p.x, p.y];
3516   renderSvg();
3517 }
3518
3519 function onPolyDbClick(e) {
3520   if (state.tool !== 'poly') return;
3521   e.preventDefault(); e.stopPropagation();
3522   finalizePolyDraw();
3523 }
3524
3525 (() => {
3526   const svg = $('annoSvg');
3527   svg.addEventListener('click', onPolySvgClick);
3528   svg.addEventListener('mousemove', onPolySvgMove);
3529   svg.addEventListener('dblclick', onPolyDbClick);
3530
3531   svg.addEventListener('mousedown', (e) => {
3532     if (state.tool !== 'bbox' || e.button !== 0) return;
3533     e.preventDefault(); e.stopPropagation();
3534     const start = svgPointNorm(e);
3535     const { w: VW, h: VH } = svgViewSize();
3536     const preview = el('rect', {
3537       class: 'preview',
3538       x: start.x * VW, y: start.y * VH, width: 0, height: 0,
3539     });
3540     svg.appendChild(preview);
3541
3542     const onMove = (ev) => {
3543       const cur = svgPointNorm(ev);
3544       const x = Math.min(start.x, cur.x);
3545       const y = Math.min(start.y, cur.y);
3546       const w = Math.abs(cur.x - start.x);
3547       const h = Math.abs(cur.y - start.y);
3548       preview.setAttribute('x', x * VW);
3549       preview.setAttribute('y', y * VH);
3550       preview.setAttribute('width', w * VW);
3551       preview.setAttribute('height', h * VH);
3552     };
3553     const onUp = (ev) => {

```

```

3554     document.removeEventListener('mousemove', onMove);
3555     document.removeEventListener('mouseup', onUp);
3556     preview.remove();
3557     const cur = svgPointNorm(ev);
3558     const x = clamp01(Math.min(start.x, cur.x));
3559     const y = clamp01(Math.min(start.y, cur.y));
3560     const w = Math.min(1 - x, Math.abs(cur.x - start.x));
3561     const h = Math.min(1 - y, Math.abs(cur.y - start.y));
3562     if (w < 0.005 || h < 0.005) return;
3563     addBox(x, y, w, h);
3564   };
3565   document.addEventListener('mousemove', onMove);
3566   document.addEventListener('mouseup', onUp);
3567   });
3568   })();
3569
3570   function addBox(x, y, w, h) {
3571     const labels = pendingLabels();
3572     const label = labels[0];
3573     const birads = $('biradsSelect').value || '';
3574     const ann = {
3575       id: uid(),
3576       type: 'bbox',
3577       label,
3578       labels,
3579       bi_rads: birads,
3580       note: '',
3581       frame: state.frame,
3582       bbox: [x, y, w, h],
3583       created_at: new Date().toISOString(),
3584       updated_at: new Date().toISOString(),
3585     };
3586     state.annotations.push(ann);
3587     state.selectedId = ann.id;
3588     renderSvg(); renderAnnoList();
3589     scheduleAutosave();
3590   }
3591
3592   function selectAnn(id) {
3593     state.selectedId = id;
3594     renderSvg();
3595     renderAnnoList();
3596   }
3597
3598   function deleteAnn(id) {
3599     state.annotations = state.annotations.filter(a => a.id !== id);
3600     if (state.selectedId === id) state.selectedId = null;
3601     renderSvg(); renderAnnoList();
3602     scheduleAutosave();
3603   }
3604
3605   function updateAnn(id, patch, opts = {}) {
3606     const a = state.annotations.find(x => x.id === id);
3607     if (!a) return;
3608     Object.assign(a, patch);
3609     a.updated_at = new Date().toISOString();
3610     renderSvg();
3611     if (!opts.skipList) renderAnnoList();
3612     scheduleAutosave();
3613   }
3614
3615   function renderSvg() {
3616     const svg = $('annoSvg');
3617     while (svg.firstChild) svg.removeChild(svg.firstChild);
3618     if (!state.current) return;
3619     const { w: VW, h: VH } = svgViewSize();

```

```

3620 const fontSize = Math.max(10, Math.round(VH * 0.012));
3621
3622 let selectedAnn = null;
3623 for (const a of state.annotations) {
3624   if (a.frame !== state.frame) continue;
3625   const color = colorFor(a.label);
3626   const isSel = a.id === state.selectedId;
3627   if (isSel) selectedAnn = a;
3628
3629   let labelX, labelY;
3630   if (a.type === 'polygon' && a.points && a.points.length >= 3) {
3631     const ptsAttr = a.points.map(p => `${p[0] * VW},${p[1] * VH}`).join(' ');
3632     const poly = el('polygon', {
3633       class: 'poly' + (isSel ? ' selected' : ''),
3634       points: ptsAttr,
3635       stroke: color,
3636       fill: color,
3637     });
3638     poly.dataset.id = a.id;
3639     poly.addEventListener('mousedown', (e) => onPolyAnnMouseDown(e, a));
3640     svg.appendChild(poly);
3641     labelX = Math.min(...a.points.map(p => p[0])) * VW;
3642     labelY = Math.min(...a.points.map(p => p[1])) * VH - fontSize * 0.3;
3643   } else {
3644     const [bx, by, bw, bh] = a.bbox;
3645     const rect = el('rect', {
3646       class: 'box' + (isSel ? ' selected' : ''),
3647       x: bx * VW, y: by * VH, width: bw * VW, height: bh * VH,
3648       stroke: color,
3649       fill: color,
3650     });
3651     rect.dataset.id = a.id;
3652     rect.addEventListener('mousedown', (e) => onBoxMouseDown(e, a));
3653     svg.appendChild(rect);
3654     labelX = bx * VW;
3655     labelY = by * VH - fontSize * 0.3;
3656   }
3657
3658   const txt = el('text', {
3659     class: 'label',
3660     x: labelX, y: labelY,
3661     'font-size': fontSize,
3662   });
3663   txt.textContent = annLabelText(a) + (a.bi_rads ? ` · BI-RADS ${a.bi_rads}` : '');
3664   svg.appendChild(txt);
3665 }
3666
3667 if (selectedAnn && state.tool !== 'bbox' && state.tool !== 'poly') {
3668   if (selectedAnn.type === 'polygon') {
3669     renderPolygonHandles(selectedAnn);
3670   } else {
3671     renderHandles(selectedAnn);
3672   }
3673 }
3674
3675 if (state.tool === 'poly' && polyDraw.points.length > 0) {
3676   const allPts = polyDraw.cursor
3677     ? [...polyDraw.points, polyDraw.cursor]
3678     : polyDraw.points;
3679   const ptsAttr = allPts.map(p => `${p[0] * VW},${p[1] * VH}`).join(' ');
3680   const preview = el('polyline', {
3681     class: 'poly-preview',
3682     points: ptsAttr,
3683   });
3684   svg.appendChild(preview);
3685   for (const p of polyDraw.points) {

```



```

3686     const c = el('circle', {
3687       class: 'poly-vertex',
3688       cx: p[0] * VW, cy: p[1] * VH, r: 4 / Math.max(state.scale, 0.05),
3689     });
3690     svg.appendChild(c);
3691   }
3692 }
3693
3694 for (const s of state.suggestions) {
3695   if (s.frame !== state.frame) continue;
3696   const [bx, by, bw, bh] = s.bbox;
3697   const rect = el('rect', {
3698     class: 'suggestion',
3699     x: bx * VW, y: by * VH, width: bw * VW, height: bh * VH,
3700   });
3701   rect.dataset.sid = s.id;
3702   rect.addEventListener('click', (e) => onSuggestionClick(e, s));
3703   svg.appendChild(rect);
3704   const txt = el('text', {
3705     class: 'sug-label',
3706     x: bx * VW, y: by * VH - fontSize * 0.3,
3707     'font-size': fontSize,
3708   });
3709   txt.textContent = `🔍 ${s.label} ${((s.confidence * 100).toFixed(0))}%`;
3710   svg.appendChild(txt);
3711 }
3712 }
3713
3714 const HANDLE_DIRS = ['nw', 'n', 'ne', 'e', 'se', 's', 'sw', 'w'];
3715 const HANDLE_CURSORS = {
3716   nw: 'nwse-resize', se: 'nwse-resize',
3717   ne: 'nesw-resize', sw: 'nesw-resize',
3718   n: 'ns-resize', s: 'ns-resize',
3719   e: 'ew-resize', w: 'ew-resize',
3720 };
3721
3722 function renderHandles(ann) {
3723   const svg = $('annoSvg');
3724   const { w: VW, h: VH } = svgViewSize();
3725   const [bx, by, bw, bh] = ann.bbox;
3726   const r = 6 / Math.max(state.scale, 0.05);
3727   const positions = {
3728     nw: [bx, by],          n: [bx + bw / 2, by],          ne: [bx + bw, by],
3729     w: [bx, by + bh / 2],  e: [bx + bw, by + bh / 2],
3730     sw: [bx, by + bh],     s: [bx + bw / 2, by + bh],     se: [bx + bw, by + bh],
3731   };
3732   for (const dir of HANDLE_DIRS) {
3733     const [px, py] = positions[dir];
3734     const c = el('circle', {
3735       class: 'handle',
3736       cx: px * VW, cy: py * VH, r,
3737     });
3738     c.style.cursor = HANDLE_CURSORS[dir];
3739     c.addEventListener('mousedown', (e) => onHandleMouseDown(e, ann, dir));
3740     svg.appendChild(c);
3741   }
3742 }
3743
3744 function onHandleMouseDown(e, ann, dir) {
3745   if (state.tool === 'bbox') return;
3746   e.preventDefault(); e.stopPropagation();
3747   selectAnn(ann.id);
3748
3749   const start = svgPointNorm(e);
3750   const orig = ann.bbox.slice();
3751   const minSize = 0.005;

```

```

3752 let moved = false;
3753
3754 const onMove = (ev) => {
3755     const cur = svgPointNorm(ev);
3756     const dx = cur.x - start.x;
3757     const dy = cur.y - start.y;
3758     if (Math.abs(dx) + Math.abs(dy) < 0.0005) return;
3759     moved = true;
3760
3761     let x = orig[0], y = orig[1], w = orig[2], h = orig[3];
3762     const right = orig[0] + orig[2];
3763     const bottom = orig[1] + orig[3];
3764
3765     if (dir.includes('w')) {
3766         let nx = clamp01(orig[0] + dx);
3767         if (right - nx < minSize) nx = right - minSize;
3768         x = nx;
3769         w = right - nx;
3770     }
3771     if (dir.includes('e')) {
3772         let nw = orig[2] + dx;
3773         nw = Math.max(minSize, Math.min(1 - orig[0], nw));
3774         w = nw;
3775     }
3776     if (dir.includes('n')) {
3777         let ny = clamp01(orig[1] + dy);
3778         if (bottom - ny < minSize) ny = bottom - minSize;
3779         y = ny;
3780         h = bottom - ny;
3781     }
3782     if (dir.includes('s')) {
3783         let nh = orig[3] + dy;
3784         nh = Math.max(minSize, Math.min(1 - orig[1], nh));
3785         h = nh;
3786     }
3787
3788     ann.bbox = [x, y, w, h];
3789     renderSvg();
3790 };
3791 const onUp = () => {
3792     document.removeEventListener('mousemove', onMove);
3793     document.removeEventListener('mouseup', onUp);
3794     if (moved) { ann.updated_at = new Date().toISOString(); scheduleAutosave(); }
3795     renderAnnoList();
3796 };
3797 document.addEventListener('mousemove', onMove);
3798 document.addEventListener('mouseup', onUp);
3799 }
3800
3801 function recomputePolyBbox(ann) {
3802     const xs = ann.points.map(p => p[0]);
3803     const ys = ann.points.map(p => p[1]);
3804     const minX = Math.min(...xs), minY = Math.min(...ys);
3805     const maxX = Math.max(...xs), maxY = Math.max(...ys);
3806     ann.bbox = [minX, minY, maxX - minX, maxY - minY];
3807 }
3808
3809 function renderPolygonHandles(ann) {
3810     const svg = $('annoSvg');
3811     const { w: VW, h: VH } = svgViewSize();
3812     const r = 6 / Math.max(state.scale, 0.05);
3813     for (let i = 0; i < ann.points.length; i++) {
3814         const [px, py] = ann.points[i];
3815         const c = el('circle', {
3816             class: 'handle',
3817             cx: px * VW, cy: py * VH, r,

```

```

3818 });
3819 c.style.cursor = 'move';
3820 c.addEventListener('mousedown', (e) => onPolyVertexDown(e, ann, i));
3821 c.addEventListener('contextmenu', (e) => onPolyVertexRightClick(e, ann, i));
3822 svg.appendChild(c);
3823 }
3824 for (let i = 0; i < ann.points.length; i++) {
3825   const a1 = ann.points[i];
3826   const a2 = ann.points[(i + 1) % ann.points.length];
3827   const mx = (a1[0] + a2[0]) / 2;
3828   const my = (a1[1] + a2[1]) / 2;
3829   const m = el('circle', {
3830     class: 'handle midpoint',
3831     cx: mx * VW, cy: my * VH, r: r * 0.6,
3832     'fill-opacity': 0.5,
3833   });
3834   m.style.cursor = 'crosshair';
3835   m.style.fill = '#28d97f';
3836   m.title = "Bosish: shu chetiga yangi nuqta qo'shish";
3837   m.addEventListener('mousedown', (e) => onPolyEdgeMidDown(e, ann, i, mx, my));
3838   svg.appendChild(m);
3839 }
3840 }
3841
3842 function onPolyVertexDown(e, ann, i) {
3843   if (e.button !== 0) return;
3844   e.preventDefault(); e.stopPropagation();
3845   selectAnn(ann.id);
3846   const start = svgPointNorm(e);
3847   const orig = ann.points[i].slice();
3848   let moved = false;
3849   const onMove = (ev) => {
3850     const cur = svgPointNorm(ev);
3851     const dx = cur.x - start.x;
3852     const dy = cur.y - start.y;
3853     if (Math.abs(dx) + Math.abs(dy) < 0.0005) return;
3854     moved = true;
3855     ann.points[i] = [clamp01(orig[0] + dx), clamp01(orig[1] + dy)];
3856     recomputePolyBbox(ann);
3857     renderSvg();
3858   };
3859   const onUp = () => {
3860     document.removeEventListener('mousemove', onMove);
3861     document.removeEventListener('mouseup', onUp);
3862     if (moved) { ann.updated_at = new Date().toISOString(); scheduleAutosave(); }
3863     renderAnnoList();
3864   };
3865   document.addEventListener('mousemove', onMove);
3866   document.addEventListener('mouseup', onUp);
3867 }
3868
3869 function onPolyVertexRightClick(e, ann, i) {
3870   e.preventDefault(); e.stopPropagation();
3871   if (ann.points.length <= 3) {
3872     setStatus("Polygon kamida 3 nuqtaga ega bo'lishi kerak");
3873     return;
3874   }
3875   ann.points.splice(i, 1);
3876   recomputePolyBbox(ann);
3877   ann.updated_at = new Date().toISOString();
3878   renderSvg();
3879   renderAnnoList();
3880   scheduleAutosave();
3881 }
3882
3883 function onPolyEdgeMidDown(e, ann, edgeIdx, mx, my) {

```

```

3884   if (e.button !== 0) return;
3885   e.preventDefault(); e.stopPropagation();
3886   selectAnn(ann.id);
3887   ann.points.splice(edgeIdx + 1, 0, [mx, my]);
3888   recomputePolyBbox(ann);
3889   const newIdx = edgeIdx + 1;
3890   const start = svgPointNorm(e);
3891   const orig = ann.points[newIdx].slice();
3892   let moved = false;
3893   const onMove = (ev) => {
3894     const cur = svgPointNorm(ev);
3895     const dx = cur.x - start.x;
3896     const dy = cur.y - start.y;
3897     moved = true;
3898     ann.points[newIdx] = [clamp01(orig[0] + dx), clamp01(orig[1] + dy)];
3899     recomputePolyBbox(ann);
3900     renderSvg();
3901   };
3902   const onUp = () => {
3903     document.removeEventListener('mousemove', onMove);
3904     document.removeEventListener('mouseup', onUp);
3905     if (moved || true) {
3906       ann.updated_at = new Date().toISOString();
3907       scheduleAutosave();
3908     }
3909     renderAnnoList();
3910   };
3911   document.addEventListener('mousemove', onMove);
3912   document.addEventListener('mouseup', onUp);
3913 }
3914
3915 function onPolyAnnMouseDown(e, a) {
3916   if (state.tool === 'bbox' || state.tool === 'poly') return;
3917   e.preventDefault(); e.stopPropagation();
3918   selectAnn(a.id);
3919
3920   const start = svgPointNorm(e);
3921   const orig = a.points.map(p => p.slice());
3922   let moved = false;
3923
3924   const onMove = (ev) => {
3925     const cur = svgPointNorm(ev);
3926     const dx = cur.x - start.x;
3927     const dy = cur.y - start.y;
3928     if (Math.abs(dx) + Math.abs(dy) < 0.001) return;
3929     moved = true;
3930     const xs = orig.map(p => p[0] + dx);
3931     const ys = orig.map(p => p[1] + dy);
3932     let shiftX = 0, shiftY = 0;
3933     const minX = Math.min(...xs), maxX = Math.max(...xs);
3934     const minY = Math.min(...ys), maxY = Math.max(...ys);
3935     if (minX < 0) shiftX = -minX;
3936     else if (maxX > 1) shiftX = 1 - maxX;
3937     if (minY < 0) shiftY = -minY;
3938     else if (maxY > 1) shiftY = 1 - maxY;
3939     a.points = orig.map(p => [p[0] + dx + shiftX, p[1] + dy + shiftY]);
3940     const xs2 = a.points.map(p => p[0]);
3941     const ys2 = a.points.map(p => p[1]);
3942     a.bbox = [
3943       Math.min(...xs2), Math.min(...ys2),
3944       Math.max(...xs2) - Math.min(...xs2),
3945       Math.max(...ys2) - Math.min(...ys2),
3946     ];
3947     renderSvg();
3948   };
3949   const onUp = () => {

```

```

3950     document.removeEventListener('mousemove', onMove);
3951     document.removeEventListener('mouseup', onUp);
3952     if (moved) { a.updated_at = new Date().toISOString(); scheduleAutosave(); }
3953     renderAnnoList();
3954 };
3955 document.addEventListener('mousemove', onMove);
3956 document.addEventListener('mouseup', onUp);
3957 }
3958
3959 function onBoxMouseDown(e, a) {
3960     if (state.tool === 'bbox') return;
3961     e.preventDefault(); e.stopPropagation();
3962     selectAnn(a.id);
3963
3964     const startNorm = svgPointNorm(e);
3965     const orig = a.bbox.slice();
3966     let moved = false;
3967
3968     const onMove = (ev) => {
3969         const cur = svgPointNorm(ev);
3970         const dx = cur.x - startNorm.x;
3971         const dy = cur.y - startNorm.y;
3972         if (Math.abs(dx) + Math.abs(dy) < 0.001) return;
3973         moved = true;
3974         let nx = orig[0] + dx;
3975         let ny = orig[1] + dy;
3976         nx = Math.min(Math.max(0, nx), 1 - orig[2]);
3977         ny = Math.min(Math.max(0, ny), 1 - orig[3]);
3978         a.bbox = [nx, ny, orig[2], orig[3]];
3979         renderSvg();
3980     };
3981     const onUp = () => {
3982         document.removeEventListener('mousemove', onMove);
3983         document.removeEventListener('mouseup', onUp);
3984         if (moved) { a.updated_at = new Date().toISOString(); scheduleAutosave(); }
3985         renderAnnoList();
3986     };
3987     document.addEventListener('mousemove', onMove);
3988     document.addEventListener('mouseup', onUp);
3989 }
3990
3991 function renderAnnoList() {
3992     const list = $('annoList');
3993     list.innerHTML = '';
3994     const visible = state.annotations.filter(a => a.frame === state.frame);
3995     $('annoCount').textContent = state.annotations.length;
3996     $('annoEmpty').style.display = state.annotations.length ? 'none' : '';
3997
3998     for (const a of visible) {
3999         const row = document.createElement('div');
4000         row.className = 'anno-row' + (a.id === state.selectedId ? ' selected' : '');
4001         row.addEventListener('click', () => selectAnn(a.id));
4002
4003         const sw = document.createElement('div');
4004         sw.className = 'swatch';
4005         sw.style.background = colorFor(a.label);
4006         row.appendChild(sw);
4007
4008         const info = document.createElement('div');
4009         info.className = 'info';
4010
4011         const labelSel = buildLabelMultiSelect({
4012             selected: annLabels(a),
4013             compact: true,
4014             placeholder: 'Label',
4015             onChange: (arr) => {

```

```

4016     const labels = arr.length ? arr : [];
4017     // skipList: keep the dropdown open so several labels can be toggled in a row;
4018     // refresh the swatch + canvas in place instead of rebuilding the whole list.
4019     updateAnn(a.id, { labels, label: labels[0] || '' }, { skipList: true });
4020     sw.style.background = colorFor(labels[0]);
4021   },
4022 });
4023 labelSel.addEventListener('click', e => e.stopPropagation());
4024
4025 const biSel = document.createElement('select');
4026 const empty = document.createElement('option');
4027 empty.value = ''; empty.textContent = 'BI-RADS -';
4028 biSel.appendChild(empty);
4029 for (const b of state.birads) {
4030   const o = document.createElement('option');
4031   o.value = b; o.textContent = b;
4032   if (b === a.bi_rads) o.selected = true;
4033   biSel.appendChild(o);
4034 }
4035 biSel.addEventListener('click', e => e.stopPropagation());
4036 biSel.addEventListener('change', e => updateAnn(a.id, { bi_rads: e.target.value }));
4037
4038 const head = document.createElement('div');
4039 head.className = 'lbl';
4040 head.appendChild(labelSel);
4041 head.appendChild(biSel);
4042 const status = a.status || 'draft';
4043 const sp = document.createElement('span');
4044 sp.className = `status-pill status-${status}`;
4045 sp.textContent = status;
4046 head.appendChild(sp);
4047 info.appendChild(head);
4048
4049 const audit = document.createElement('div');
4050 audit.className = 'audit';
4051 const created = a.created_by ? `${a.created_by}` : '-';
4052 const updated = a.updated_by && a.updated_by !== a.created_by ? ` · oxirgi: ${a.updated_by}` : '';
4053 const reviewed = a.reviewed_by ? ` · ${status === 'approved' ? '✓' : 'X'} ${a.reviewed_by}` : '';
4054 audit.textContent = `Yaratdi: ${created}${updated}${reviewed}`;
4055 info.appendChild(audit);
4056
4057 if (a.review_note && status === 'rejected') {
4058   const note = document.createElement('div');
4059   note.className = 'review-note';
4060   note.textContent = `Reviewer izohi: ${a.review_note}`;
4061   info.appendChild(note);
4062 }
4063
4064 const sa = document.createElement('div');
4065 sa.className = 'status-actions';
4066 if (status === 'draft') {
4067   if (ownAnn(a) || isReviewer()) {
4068     sa.appendChild(makeStatusBtn('▲ Submit', 'submit', () => changeStatus(a.id, 'submitted')));
4069   }
4070 } else if (status === 'submitted') {
4071   if (isReviewer()) {
4072     sa.appendChild(makeStatusBtn('✓ Approve', 'approve', () => changeStatus(a.id, 'approved')));
4073     sa.appendChild(makeStatusBtn('X Reject', 'reject', () => promptRejectStatus(a.id)));
4074   }
4075   if (ownAnn(a)) {
4076     sa.appendChild(makeStatusBtn('▼ Recall', '', () => changeStatus(a.id, 'draft')));
4077   }
4078 } else if (status === 'approved') {
4079   if (isReviewer()) {
4080     sa.appendChild(makeStatusBtn('🔒 Reopen', '', () => changeStatus(a.id, 'draft')));
4081   }

```

```

4082     } else if (status === 'rejected') {
4083         if (ownAnn(a) || isReviewer()) {
4084             sa.appendChild(makeStatusBtn('🔒 Reopen', '', () => changeStatus(a.id, 'draft')));
4085         }
4086     }
4087     sa.appendChild(makeStatusBtn('📜 Tarix', 'history', () => showHistoryFor(a.id)));
4088     sa.appendChild(makeStatusBtn('📊 Radiomika', 'history', () => showRadiomicsFor(a.id)));
4089     info.appendChild(sa);
4090
4091     const meta = document.createElement('div');
4092     meta.className = 'meta-row';
4093     const [x, y, w, h] = a.bbox;
4094     const W = state.meta && state.meta.Columns ? parseInt(state.meta.Columns, 10) : null;
4095     const H = state.meta && state.meta.Rows ? parseInt(state.meta.Rows, 10) : null;
4096     const typeIcon = a.type === 'polygon' ? `● ${a.points?.length || 0} nuqta` : '';
4097     if (W && H) {
4098         meta.textContent = `${typeIcon} ${Math.round(x * W)}, ${Math.round(y * H)} - ${Math.round(w *
4099 W)}x${Math.round(h * H)} px`;
4100     } else {
4101         meta.textContent = `${typeIcon} ${((x*100).toFixed(1))%, ${((y*100).toFixed(1))}%} -
4102 ${((w*100).toFixed(1))}%x${((h*100).toFixed(1))}%`;
4103     }
4104     info.appendChild(meta);
4105     row.appendChild(info);
4106
4107     const xBtn = document.createElement('button');
4108     xBtn.className = 'x';
4109     xBtn.textContent = 'X';
4110     xBtn.title = "O'chirish";
4111     xBtn.addEventListener('click', e => { e.stopPropagation(); deleteAnn(a.id); });
4112     row.appendChild(xBtn);
4113
4114     list.appendChild(row);
4115 }
4116
4117 let _saveTimer = null;
4118 // Manual-save mode: drawing/editing only marks the work as unsaved.
4119 // The user persists explicitly via the 💾 Saqlash button (or Ctrl+S).
4120 function scheduleAutosave() {
4121     state.dirty = true;
4122     setSaveStatus('dirty', '● saqlanmagan');
4123     updateSaveBtn();
4124 }
4125 function setSaveStatus(cls, text) {
4126     const node = $('saveStatus');
4127     node.className = 'save-status ' + (cls || '');
4128     node.textContent = text || '';
4129 }
4130 function updateSaveBtn() {
4131     const b = $('saveBtn');
4132     if (!b) return;
4133     b.disabled = !(state.dirty && state.current);
4134     b.classList.toggle('pending', !!state.dirty);
4135 }
4136
4137 function makeStatusBtn(text, cls, onClick) {
4138     const b = document.createElement('button');
4139     b.className = 'status-btn ' + cls;
4140     b.textContent = text;
4141     b.addEventListener('click', (e) => { e.stopPropagation(); onClick(); });
4142     return b;
4143 }
4144
4145 async function changeStatus(annotationId, newStatus, note) {

```

```

4145 if (!state.current) return;
4146 try {
4147     const body = {
4148         source: refSource(),
4149         ref: refValue(),
4150         annotation_id: annotationId,
4151         status: newStatus,
4152     };
4153     if (note) body.note = note;
4154     const res = await apiJson('/api/annotations/status', {
4155         method: 'POST',
4156         headers: { 'content-type': 'application/json' },
4157         body: JSON.stringify(body),
4158     });
4159     const a = state.annotations.find(x => x.id === annotationId);
4160     if (a && res.snapshot) Object.assign(a, res.snapshot);
4161     renderSvg();
4162     renderAnnoList();
4163     setStatus(`status: ${newStatus}`);
4164     refreshNotifications().catch(() => {});
4165 } catch (e) {
4166     setStatus('status xatosi: ' + e.message);
4167 }
4168 }
4169
4170 function promptRejectStatus(annotationId) {
4171     const note = prompt("Rejection sababi (ixtiyoriy):", "");
4172     if (note === null) return;
4173     changeStatus(annotationId, 'rejected', note);
4174 }
4175
4176 async function showHistoryFor(annotationId) {
4177     if (!state.current) return;
4178     const params = new URLSearchParams({
4179         source: refSource(),
4180         ref: refValue(),
4181     });
4182     if (annotationId) params.set('annotation_id', annotationId);
4183     let data;
4184     try {
4185         data = await apiJson(`/api/annotations/history?${params}`);
4186     } catch (e) {
4187         setStatus("tarix xatosi: " + e.message);
4188         return;
4189     }
4190     const body = $('historyBody');
4191     body.innerHTML = '';
4192     if (!data.history.length) {
4193         body.innerHTML = '<div class="report-empty">Tarix bo\'sh.</div>';
4194     } else {
4195         for (const h of data.history) {
4196             const row = document.createElement('div');
4197             row.className = 'hist-row';
4198             const head = document.createElement('div');
4199             head.className = 'row-head';
4200             const ap = document.createElement('span');
4201             ap.className = `action-pill action-${h.action}`;
4202             ap.textContent = h.action;
4203             const who = document.createElement('span');
4204             who.className = 'who';
4205             who.textContent = h.username || '-';
4206             const ts = document.createElement('span');
4207             ts.className = 'ts';
4208             ts.textContent = (h.ts || '').replace('T', ' ').slice(0, 19);
4209             const aid = document.createElement('span');
4210             aid.className = 'ts';

```



```

4211     aid.textContent = '· ' + (h.annotation_id || '').slice(0, 12);
4212     head.append(ap, who, ts, aid);
4213     row.appendChild(head);
4214
4215     if (h.action === 'status') {
4216         const txt = document.createElement('div');
4217         const oldS = h.prev_snapshot && h.prev_snapshot.status;
4218         const newS = h.new_snapshot && h.new_snapshot.status;
4219         const note = h.new_snapshot && h.new_snapshot.note;
4220         txt.style.marginTop = '4px';
4221         txt.innerHTML = `<b>${oldS || '-'}</b> → <b>${newS || '-'}</b>` + (note ? `
<i>("${note}")</i>` : '');
4222         row.appendChild(txt);
4223     } else if (h.action !== 'create' && h.prev_snapshot && h.new_snapshot) {
4224         const summary = annotationDiffSummary(h.prev_snapshot, h.new_snapshot);
4225         if (summary) {
4226             const txt = document.createElement('div');
4227             txt.style.marginTop = '4px';
4228             txt.style.fontSize = '11px';
4229             txt.textContent = summary;
4230             row.appendChild(txt);
4231         }
4232     } else if (h.action === 'create' && h.new_snapshot) {
4233         const txt = document.createElement('div');
4234         txt.style.marginTop = '4px';
4235         txt.style.fontSize = '11px';
4236         txt.textContent = `${h.new_snapshot.type || 'bbox'} · ${h.new_snapshot.label || ''}`;
4237         row.appendChild(txt);
4238     }
4239     body.appendChild(row);
4240 }
4241 }
4242 $('historyModal').hidden = false;
4243 }
4244
4245 function annotationDiffSummary(prev, next) {
4246     const changed = [];
4247     const keys = ['label', 'bi_rads', 'note', 'frame'];
4248     for (const k of keys) {
4249         if (JSON.stringify(prev[k]) !== JSON.stringify(next[k])) {
4250             changed.push(`${k}: ${prev[k] ?? '-'} → ${next[k] ?? '-'}`);
4251         }
4252     }
4253     if (JSON.stringify(prev.bbox) !== JSON.stringify(next.bbox)) changed.push('bbox o\'zgardi');
4254     if (JSON.stringify(prev.points) !== JSON.stringify(next.points)) {
4255         const pp = (prev.points || []).length;
4256         const np = (next.points || []).length;
4257         changed.push(`points: ${pp} → ${np}`);
4258     }
4259     return changed.join(', ');
4260 }
4261
4262 $('historyCloseBtn').addEventListener('click', () => { $('historyModal').hidden = true; });
4263 $('radiomicsCloseBtn').addEventListener('click', () => { $('radiomicsModal').hidden = true; });
4264
4265 // Toolbar button: radiomics for the selected annotation (or the only one).
4266 $('radiomicsBtn').addEventListener('click', () => {
4267     const visible = state.annotations.filter(a => a.frame === state.frame);
4268     if (!visible.length) {
4269         alert("Bu kadrda annotatsiya yo'q. Avval BBox yoki Poly bilan ROI chizing.");
4270         return;
4271     }
4272     let id = state.selectedId;
4273     if (!id || !visible.some(a => a.id === id)) {
4274         if (visible.length === 1) id = visible[0].id;
4275         else { alert("Avval annotatsiyani tanlang (ro'yxatdan yoki rasm ustida bosib)."); return; }

```

```

4276     }
4277     showRadiomicsFor(id);
4278   });
4279
4280   const RADIOMICS_FAMILIES = [
4281     ['shape', 'Shakl (2D morfologiya)'],
4282     ['first_order', 'First-order (intensivlik)'],
4283     ['glcm', 'GLCM (tekstura – co-occurrence)'],
4284     ['glrlm', 'GLRLM (run-length)'],
4285     ['glszm', 'GLSZM (size-zone)'],
4286     ['ngtdm', 'NGTDM (neighbouring tone)'],
4287   ];
4288
4289   function fmtFeature(v) {
4290     if (typeof v !== 'number' || !isFinite(v)) return String(v);
4291     if (v !== 0 && (Math.abs(v) >= 1e4 || Math.abs(v) < 1e-3)) return v.toExponential(3);
4292     return v.toFixed(4);
4293   }
4294
4295   async function showRadiomicsFor(annotationId, bins) {
4296     if (!state.current) return;
4297     // Radiomics runs on the saved copy; persist first in manual-save mode.
4298     if (state.dirty) { try { await saveAnnotations(); } catch (e) { /* try anyway */ } }
4299     bins = bins || 32;
4300     const body = $('radiomicsBody');
4301     body.innerHTML = '<div class="report-empty">Hisoblanmoqda...</div>';
4302     $('radiomicsModal').hidden = false;
4303     try {
4304       const res = await fetch(
4305         `/api/radiomics?source=${refSource()}&ref=${encodeURIComponent(refValue())}` +
4306         `&annotation_id=${encodeURIComponent(annotationId)}&bins=${bins}`,
4307         { headers: { 'Authorization': `Bearer ${getToken()}` } },
4308       );
4309       if (!res.ok) {
4310         const hint = res.status === 404 ? " (avval annotatsiyani saqlang)" :
4311           res.status === 422 ? " (ROI yoki rasm o'lchami yaroqsiz)" : '';
4312         body.innerHTML = `<div class="report-empty">Xato: ${res.status}${hint}</div>`;
4313         return;
4314       }
4315       renderRadiomics(body, await res.json(), annotationId);
4316     } catch (e) {
4317       body.innerHTML = `<div class="report-empty">Tarmoq xatosi: ${e.message}</div>`;
4318     }
4319   }
4320
4321   function renderRadiomics(body, data, annotationId) {
4322     const item = (data.results || [])[0];
4323     body.innerHTML = '';
4324     if (!item) { body.innerHTML = '<div class="report-empty">Natija yo'q.</div>'; return; }
4325
4326     const ctl = document.createElement('div');
4327     ctl.className = 'radiomics-controls';
4328     const info = document.createElement('div');
4329     info.className = 'radiomics-info';
4330     const sp = item.spacing_mm ? `${item.spacing_mm.map(s => s.toFixed(3)).join('x')} mm/px`
4331       : "piksel o'lchami yo'q – birliklar px";
4332     info.innerHTML = `Label: <b>${(item.labels || [item.label]).join(' + ')}</b> · ` +
4333       `tur: ${item.type} · ${sp} · bins: ${data.bins}`;
4334     ctl.appendChild(info);
4335
4336     const binSel = document.createElement('select');
4337     for (const b of [16, 32, 64, 128]) {
4338       const o = document.createElement('option');
4339       o.value = b; o.textContent = `${b} bins`;
4340       if (b === data.bins) o.selected = true;
4341       binSel.appendChild(o);

```

```

4342 }
4343 binSel.title = 'Tekstura uchun intensivlik darajalari soni';
4344 binSel.addEventListener('change', () => showRadiomicsFor(annotationId, parseInt(binSel.value, 10)));
4345 ctl.appendChild(binSel);
4346
4347 const csvBtn = document.createElement('button');
4348 csvBtn.textContent = '↓ CSV (barcha ROI)';
4349 csvBtn.title = 'Shu rasmdagi barcha annotatsiyalar radiomikasini CSV qilib yuklash';
4350 csvBtn.addEventListener('click', () => downloadRadiomicsCsv(data.bins));
4351 ctl.appendChild(csvBtn);
4352 body.appendChild(ctl);
4353
4354 for (const [fam, title] of RADIOMICS_FAMILIES) {
4355     const feats = item.features[fam];
4356     if (!feats) continue;
4357     const h = document.createElement('div');
4358     h.className = 'report-section-title';
4359     h.textContent = `${title} · ${Object.keys(feats).length}`;
4360     body.appendChild(h);
4361     const tbl = document.createElement('table');
4362     tbl.className = 'radiomics-table';
4363     for (const [k, v] of Object.entries(feats)) {
4364         const tr = document.createElement('tr');
4365         const td1 = document.createElement('td'); td1.textContent = k;
4366         const td2 = document.createElement('td'); td2.className = 'val'; td2.textContent =
4367         fmtFeature(v);
4368         tr.appendChild(td1); tr.appendChild(td2);
4369         tbl.appendChild(tr);
4370     }
4371     body.appendChild(tbl);
4372 }
4373
4374 async function downloadRadiomicsCsv(bins) {
4375     try {
4376         const res = await fetch(
4377             `/api/radiomics/csv?source=${refSource()}&ref=${encodeURIComponent(refValue())}&bins=${bins ||
4378             32}`,
4379             { headers: { 'Authorization': `Bearer ${getToken()}` } },
4380         );
4381         if (!res.ok) { alert('CSV xato: ' + res.status); return; }
4382         const blob = await res.blob();
4383         const url = URL.createObjectURL(blob);
4384         const a = document.createElement('a');
4385         a.href = url;
4386         const stem = (refValue() || 'anon').split('/').pop().replace(/\.dcm$/i, '');
4387         a.download = `${stem}_radiomics.csv`;
4388         document.body.appendChild(a); a.click(); a.remove();
4389         URL.revokeObjectURL(url);
4390     } catch (e) {
4391         alert('CSV xato: ' + e.message);
4392     }
4393 }
4394
4395 async function saveAnnotations() {
4396     if (!state.current) return;
4397     if (state.saving) {
4398         _saveTimer = setTimeout(saveAnnotations, 200);
4399         return;
4400     }
4401     state.saving = true;
4402     setSaveStatus('dirty', 'saqlanmoqda...');
4403     try {
4404         const body = {
4405             source: refSource(),
4406             ref: refValue(),

```

```

4406     annotations: state.annotations,
4407     rows: parseInt(state.meta?.Rows || '0', 10) || null,
4408     cols: parseInt(state.meta?.Columns || '0', 10) || null,
4409   });
4410   await api('/api/annotations', {
4411     method: 'PUT',
4412     headers: { 'content-type': 'application/json' },
4413     body: JSON.stringify(body),
4414   });
4415   state.dirty = false;
4416   setSaveStatus('saved', '✓ saqlandi');
4417   updateSaveBtn();
4418   setTimeout(() => { if (!state.dirty) setSaveStatus('', ''); }, 1500);
4419   refreshActiveList();
4420 } catch (e) {
4421   setSaveStatus('err', 'xato!');
4422   updateSaveBtn();
4423 } finally {
4424   state.saving = false;
4425 }
4426 }
4427
4428 async function runInference() {
4429   if (!state.current || !state.aiAvailable) return;
4430   const model = $('aiModelSelect').value;
4431   if (!model) return;
4432   const btn = $('aiRunBtn');
4433   btn.disabled = true;
4434   const prev = btn.textContent;
4435   btn.textContent = '🖨 ishlayapti...';
4436   setStatus(`AI tahlil (${model}) – bir necha soniya kuting...`);
4437   try {
4438     const body = {
4439       source: refSource(),
4440       ref: refValue(),
4441       model,
4442       frame: state.frame,
4443       conf: 0.10,
4444       iou: 0.5,
4445       imgsz: 1024,
4446       wc: state.wc,
4447       ww: state.ww,
4448       tta: !!( $('aiTtaToggle') && $('aiTtaToggle').checked ),
4449     };
4450     const res = await apiJson('/api/inference/run', {
4451       method: 'POST',
4452       headers: { 'content-type': 'application/json' },
4453       body: JSON.stringify(body),
4454     });
4455     state.allSuggestions = (res.detections || []).map((d, i) => ({
4456       id: 'sug_' + Date.now() + '_' + i,
4457       frame: state.frame,
4458       label: d.label,
4459       confidence: d.confidence,
4460       bbox: d.bbox,
4461       class_id: d.class_id,
4462     }));
4463     applyAiThreshold();
4464     setStatus(`AI: ${state.allSuggestions.length} ta taklif (${res.device}) – ko'rsatilmoqda:
4465     ${state.suggestions.length}`);
4466   } catch (e) {
4467     setStatus('AI xato: ' + e.message);
4468   } finally {
4469     btn.disabled = false;
4470     btn.textContent = prev;
4471   }

```

```

4471 }
4472
4473 function applyAiThreshold() {
4474   state.suggestions = state.allSuggestions.filter(s => s.confidence >= state.aiThreshold);
4475   renderSvg();
4476   $('aiClearBtn').hidden = state.allSuggestions.length === 0;
4477   $('aiAcceptAllBtn').hidden = state.suggestions.length === 0;
4478   $('aiAcceptAllBtn').textContent = `✓ ${state.suggestions.length} taklifni qabul`;
4479 }
4480
4481 function clearSuggestions() {
4482   state.allSuggestions = [];
4483   state.suggestions = [];
4484   renderSvg();
4485   $('aiClearBtn').hidden = true;
4486   $('aiAcceptAllBtn').hidden = true;
4487 }
4488
4489 function acceptAllSuggestions() {
4490   if (state.suggestions.length === 0) return;
4491   if (!confirm(`${state.suggestions.length} ta AI taklifini annotatsiyaga aylantirasizmi?`)) return;
4492   for (const s of state.suggestions) {
4493     let label = s.label;
4494     if (!state.labels.find(l => l.name === label)) {
4495       label = state.labels[0]?.name || 'mass';
4496     }
4497     state.annotations.push({
4498       id: uid(),
4499       type: 'bbox',
4500       label,
4501       labels: [label],
4502       bi_rads: '',
4503       note: `AI: ${s.label} ${((s.confidence * 100).toFixed(1))}%`,
4504       frame: state.frame,
4505       bbox: s.bbox.slice(),
4506       created_at: new Date().toISOString(),
4507       updated_at: new Date().toISOString(),
4508       ai_source: { label: s.label, confidence: s.confidence, class_id: s.class_id },
4509     });
4510   }
4511   const acceptedIds = new Set(state.suggestions.map(s => s.id));
4512   state.allSuggestions = state.allSuggestions.filter(s => !acceptedIds.has(s.id));
4513   applyAiThreshold();
4514   renderAnnoList();
4515   scheduleAutosave();
4516   setStatus(`${acceptedIds.size} ta taklif qabul qilindi`);
4517 }
4518
4519 function onSuggestionClick(e, s) {
4520   e.preventDefault(); e.stopPropagation();
4521   const proceed = window.confirm(
4522     `AI taklifini qabul qilasizmi?\n\n` +
4523     `Label: ${s.label}\n` +
4524     `Confidence: ${((s.confidence * 100).toFixed(1))}%\n\n` +
4525     `Annotatsiyaga aylantiriladi.`
4526   );
4527   if (!proceed) return;
4528   let label = s.label;
4529   if (!state.labels.find(l => l.name === label)) {
4530     label = state.labels[0]?.name || 'mass';
4531   }
4532   const ann = {
4533     id: uid(),
4534     type: 'bbox',
4535     label,
4536     labels: [label],

```

```

4537     bi_rads: '',
4538     note: `AI: ${s.label} ${((s.confidence * 100).toFixed(1))}%`,
4539     frame: state.frame,
4540     bbox: s.bbox.slice(),
4541     created_at: new Date().toISOString(),
4542     updated_at: new Date().toISOString(),
4543     ai_source: { label: s.label, confidence: s.confidence, class_id: s.class_id },
4544   });
4545   state.annotations.push(ann);
4546   state.suggestions = state.suggestions.filter(x => x.id !== s.id);
4547   state.selectedId = ann.id;
4548   $('aiClearBtn').hidden = state.suggestions.length === 0;
4549   renderSvg(); renderAnnoList();
4550   scheduleAutosave();
4551 }
4552
4553 $('aiRunBtn').addEventListener('click', runInference);
4554 $('aiClearBtn').addEventListener('click', clearSuggestions);
4555 $('aiAcceptAllBtn').addEventListener('click', acceptAllSuggestions);
4556 $('aiConfSlider').addEventListener('input', (e) => {
4557   state.aiThreshold = parseFloat(e.target.value) / 100;
4558   $('aiConfLabel').textContent = e.target.value;
4559   applyAiThreshold();
4560 });
4561
4562 function downloadDatasetExport(fmt) {
4563   const a = document.createElement('a');
4564   a.href = `/api/export?format=${encodeURIComponent(fmt)}`;
4565   a.download = '';
4566   document.body.appendChild(a);
4567   a.click();
4568   a.remove();
4569 }
4570 $('exportBtn').addEventListener('click', () => downloadDatasetExport('coco'));
4571 $('exportYoloBtn').addEventListener('click', () => downloadDatasetExport('yolo'));
4572 $('exportVocBtn').addEventListener('click', () => downloadDatasetExport('voc'));
4573 $('exportCsvBtn').addEventListener('click', () => downloadDatasetExport('csv'));
4574
4575 // Manual save: button + Ctrl/Cmd+S, with an unsaved-changes guard on unload.
4576 $('saveBtn').addEventListener('click', () => { if (state.dirty) saveAnnotations(); });
4577 window.addEventListener('keydown', (e) => {
4578   if ((e.ctrlKey || e.metaKey) && (e.key === 's' || e.key === 'S')) {
4579     e.preventDefault();
4580     if (state.dirty) saveAnnotations();
4581   }
4582 });
4583 window.addEventListener('beforeunload', (e) => {
4584   if (state.dirty) { e.preventDefault(); e.returnValue = ''; }
4585 });
4586
4587 function _zoomBy(factor) {
4588   const wrap = $('canvasWrap');
4589   const newScale = Math.max(0.05, Math.min(40, state.scale * factor));
4590   state.tx = state.tx * (newScale / state.scale);
4591   state.ty = state.ty * (newScale / state.scale);
4592   state.scale = newScale;
4593   applyTransform();
4594 }
4595
4596 function _selectableAnns() {
4597   return state.annotations
4598     .filter(a => a.frame === state.frame)
4599     .sort((a, b) => {
4600       const ay = (a.bbox?.[1] || 0), by = (b.bbox?.[1] || 0);
4601       if (ay !== by) return ay - by;
4602       return (a.bbox?.[0] || 0) - (b.bbox?.[0] || 0);

```

```

4603     });
4604   }
4605
4606   function _cycleAnn(delta) {
4607     const list = _selectableAnns();
4608     if (!list.length) return;
4609     const idx = state.selectedId ? list.findIndex(a => a.id === state.selectedId) : -1;
4610     const next = idx === -1 ? (delta > 0 ? 0 : list.length - 1) : (idx + delta + list.length) %
list.length;
4611     state.selectedId = list[next].id;
4612     renderSvg();
4613     renderAnnoList();
4614   }
4615
4616   function _quickStatus(transition) {
4617     if (!state.selectedId) {
4618       setStatus("avval annotatsiya tanlang");
4619       return;
4620     }
4621     const a = state.annotations.find(x => x.id === state.selectedId);
4622     if (!a) return;
4623     if (transition === 'submit') changeStatus(a.id, 'submitted');
4624     else if (transition === 'approve') changeStatus(a.id, 'approved');
4625     else if (transition === 'reject') promptRejectStatus(a.id);
4626     else if (transition === 'recall') changeStatus(a.id, 'draft');
4627   }
4628
4629   window.addEventListener('keydown', (e) => {
4630     if (e.target.matches('input, textarea, select')) return;
4631
4632     if ((e.ctrlKey || e.metaKey) && (e.key === 'c' || e.key === 'C')) {
4633       copySelectedAnns(); e.preventDefault(); return;
4634     }
4635     if ((e.ctrlKey || e.metaKey) && (e.key === 'v' || e.key === 'V')) {
4636       pasteAnns(); e.preventDefault(); return;
4637     }
4638
4639     if (e.key === 'Delete' || e.key === 'Backspace') {
4640       if (state.selectedId) { deleteAnn(state.selectedId); e.preventDefault(); }
4641     } else if (e.key === 'v' || e.key === 'V') {
4642       setTool('select');
4643     } else if (e.key === 'b' || e.key === 'B') {
4644       setTool('bbox');
4645     } else if (e.key === 'p' || e.key === 'P') {
4646       setTool('poly');
4647     } else if (e.key === 'f' || e.key === 'F') {
4648       fitToScreen();
4649     } else if (e.key === 'i' || e.key === 'I') {
4650       state.invert = !state.invert;
4651       $('invertBtn').classList.toggle('active', state.invert);
4652       loadImage(false);
4653     } else if (e.key === '+' || e.key === '=') {
4654       _zoomBy(1.2); e.preventDefault();
4655     } else if (e.key === '-' || e.key === '_') {
4656       _zoomBy(1 / 1.2); e.preventDefault();
4657     } else if (e.key === '0') {
4658       oneToOne();
4659     } else if (e.key === 'j' || e.key === 'J' || e.key === 'ArrowDown') {
4660       _cycleAnn(+1); e.preventDefault();
4661     } else if (e.key === 'k' || e.key === 'K' || e.key === 'ArrowUp') {
4662       _cycleAnn(-1); e.preventDefault();
4663     } else if (e.key === 'ArrowLeft' && state.frames > 1) {
4664       if (state.frame > 0) {
4665         state.frame--; $('frameSlider').value = state.frame;
4666         $('frameLabel').textContent = `${state.frame + 1} / ${state.frames}`;
4667         loadImage(false); renderSvg();

```

```

4668     }
4669     e.preventDefault();
4670   } else if (e.key === 'ArrowRight' && state.frames > 1) {
4671     if (state.frame < state.frames - 1) {
4672       state.frame++; $('frameSlider').value = state.frame;
4673       $('frameLabel').textContent = `${state.frame + 1} / ${state.frames}`;
4674       loadImage(false); renderSvg();
4675     }
4676     e.preventDefault();
4677   } else if (e.key === 'y' || e.key === 'Y') {
4678     _quickStatus('approve'); e.preventDefault();
4679   } else if (e.key === 'n' || e.key === 'N') {
4680     _quickStatus('reject'); e.preventDefault();
4681   } else if (e.key === 's' || e.key === 'S') {
4682     _quickStatus('submit'); e.preventDefault();
4683   } else if (e.key === 'r' || e.key === 'R') {
4684     _quickStatus('recall'); e.preventDefault();
4685   } else if (e.key === '?') {
4686     showHotkeysModal();
4687   } else if (e.key === 'Enter' && state.tool === 'poly') {
4688     e.preventDefault();
4689     finalizePolyDraw();
4690   } else if (e.key === 'Escape') {
4691     if (state.tool === 'poly' && polyDraw.points.length > 0) {
4692       cancelPolyDraw();
4693     } else {
4694       state.selectedId = null; renderSvg(); renderAnnoList();
4695     }
4696   }
4697 });
4698
4699 const CLIPBOARD_KEY = 'mamograf_ann_clipboard';
4700
4701 function copySelectedAnns() {
4702   if (!state.annotations.length) {
4703     setStatus("nusxa olishga annotatsiya yo'q");
4704     return;
4705   }
4706   let toCopy;
4707   if (state.selectedId) {
4708     toCopy = state.annotations.filter(a => a.id === state.selectedId);
4709   } else {
4710     toCopy = state.annotations.filter(a => a.frame === state.frame);
4711   }
4712   if (!toCopy.length) {
4713     setStatus("nusxa olishga annotatsiya yo'q");
4714     return;
4715   }
4716   const stripped = toCopy.map(a => {
4717     const c = JSON.parse(JSON.stringify(a));
4718     delete c.id;
4719     delete c.created_by; delete c.created_at;
4720     delete c.updated_by; delete c.updated_at;
4721     delete c.reviewed_by; delete c.reviewed_at; delete c.review_note;
4722     delete c.status;
4723     return c;
4724   });
4725   try {
4726     localStorage.setItem(CLIPBOARD_KEY, JSON.stringify({
4727       ts: new Date().toISOString(),
4728       anns: stripped,
4729     }));
4730     setStatus(`✓ ${stripped.length} annotatsiya nusxalandi`);
4731   } catch (e) {
4732     setStatus("nusxa olish xatosi");
4733   }

```



```

4734 }
4735
4736 function pasteAnns() {
4737   if (!state.current) return;
4738   const raw = localStorage.getItem(CLIPBOARD_KEY);
4739   if (!raw) { setStatus("clipboard bo'sh"); return; }
4740   let data;
4741   try { data = JSON.parse(raw); } catch (e) { setStatus("clipboard buzilgan"); return; }
4742   const anns = data.anns || [];
4743   if (!anns.length) { setStatus("clipboard bo'sh"); return; }
4744   let added = 0;
4745   const now = new Date().toISOString();
4746   for (const a of anns) {
4747     const fresh = JSON.parse(JSON.stringify(a));
4748     fresh.id = uid();
4749     fresh.frame = state.frame;
4750     fresh.created_at = now;
4751     fresh.updated_at = now;
4752     fresh.note = (fresh.note ? fresh.note + " " : "") + "[paste]";
4753     state.annotations.push(fresh);
4754     added++;
4755   }
4756   state.selectedId = state.annotations[state.annotations.length - 1].id;
4757   renderSvg();
4758   renderAnnoList();
4759   scheduleAutosave();
4760   setStatus(`✓ ${added} annotatsiya qo'yildi`);
4761 }
4762
4763 function showHotkeysModal() {
4764   const lines = [
4765     'Tool: V=Select B=BBox P=Polygon',
4766     'Zoom: + = , - _ , 0 (1:1) , F (fit) , I (invert)',
4767     'Frame: ← → (multi-frame DICOM)',
4768     'Annotatsiya: J/↓ next, K/↑ prev, Delete o\'chirish',
4769     'Status: S=Submit Y=Approve N=Reject R=Recall',
4770     'Clipboard: Ctrl+C (nusxa) Ctrl+V (qo\'yish)',
4771     'Polygon: Enter (yopish), Esc (bekor)',
4772     '? : shu yordam',
4773   ];
4774   alert(lines.join('\n'));
4775 }
4776
4777 async function loadAuthenticatedAssets() {
4778   await loadLabels();
4779   await loadAiStatus();
4780   await refreshUploads();
4781   await refreshNotifications();
4782   startNotifPolling();
4783   setTool('select');
4784   if (state._user && state._user.totp_setup_required) {
4785     setStatus("⚠ Sizning rolingiz uchun 2FA majburiy. Iltimos, sozlang.");
4786     setTimeout(() => openTotpModal(), 600);
4787   }
4788 }
4789
4790 // ===== WebSocket presence/sync =====
4791
4792 let _ws = null;
4793 let _wsRetryTimer = null;
4794 const _remoteCursors = new Map(); // user → {x, y, lastSeen}
4795 let _cursorThrottleTs = 0;
4796 const CURSOR_COLORS = ['#ff5050', '#ffb000', '#00c4ff', '#a070ff', '#28d97f', '#ff80b4', '#ffe44d'];
4797 function colorForUser(name) {
4798   let h = 0;
4799   for (let i = 0; i < name.length; i++) h = (h * 31 + name.charCodeAt(i)) & 0xffff;

```

```

4800     return CURSOR_COLORS[h % CURSOR_COLORS.length];
4801 }
4802
4803 function wsClose() {
4804     if (_ws) {
4805         try { _ws.close(); } catch (e) {}
4806         _ws = null;
4807     }
4808     if (_wsRetryTimer) { clearTimeout(_wsRetryTimer); _wsRetryTimer = null; }
4809     state._presence = [];
4810     $('presencePill').hidden = true;
4811 }
4812
4813 function sendCursor(x, y) {
4814     if (!_ws || _ws.readyState !== 1) return;
4815     const now = performance.now();
4816     if (now - _cursorThrottleTs < 50) return;
4817     _cursorThrottleTs = now;
4818     try {
4819         _ws.send(JSON.stringify({ type: 'cursor', x, y }));
4820     } catch (e) {}
4821 }
4822
4823 function renderRemoteCursors() {
4824     const svg = $('annoSvg');
4825     svg.querySelectorAll('.remote-cursor, .remote-cursor-label').forEach(el => el.remove());
4826     if (!_remoteCursors.size) return;
4827     const { w: VW, h: VH } = svgViewSize();
4828     const fontSize = Math.max(10, Math.round(VH * 0.012));
4829     const r = 6 / Math.max(state.scale, 0.05);
4830     const now = Date.now();
4831     for (const [user, c] of _remoteCursors.entries()) {
4832         if (now - c.lastSeen > 5000) { _remoteCursors.delete(user); continue; }
4833         if (c.x == null || c.y == null) continue;
4834         const color = colorForUser(user);
4835         const dot = el('circle', {
4836             class: 'remote-cursor',
4837             cx: c.x * VW, cy: c.y * VH, r,
4838             stroke: color, fill: color,
4839         });
4840         svg.appendChild(dot);
4841         const txt = el('text', {
4842             class: 'remote-cursor-label',
4843             x: c.x * VW + r * 1.5, y: c.y * VH - r * 0.5,
4844             'font-size': fontSize, fill: color,
4845         });
4846         txt.textContent = user;
4847         svg.appendChild(txt);
4848     }
4849 }
4850
4851 setInterval(() => {
4852     if (!_remoteCursors.size) return;
4853     const now = Date.now();
4854     let changed = false;
4855     for (const [user, c] of _remoteCursors.entries()) {
4856         if (now - c.lastSeen > 5000) { _remoteCursors.delete(user); changed = true; }
4857     }
4858     if (changed) renderRemoteCursors();
4859 }, 2000);
4860
4861 function wsConnect() {
4862     wsClose();
4863     if (!state.current || !getToken()) return;
4864     const proto = location.protocol === 'https:' ? 'wss' : 'ws';
4865     const params = new URLSearchParams({

```

```

4866     source: refSource(), ref: refValue(), token: getToken(),
4867   });
4868   const url = `${proto}://${location.host}/ws/dicom?${params}`;
4869   let ws;
4870   try { ws = new WebSocket(url); } catch (e) { return; }
4871   _ws = ws;
4872   ws.onmessage = (e) => {
4873     let m;
4874     try { m = JSON.parse(e.data); } catch (err) { return; }
4875     if (m.type === 'presence') {
4876       state._presence = (m.users || []).filter(u => u !== state._user?.username);
4877       const pill = $('presencePill');
4878       if (state._presence.length) {
4879         pill.hidden = false;
4880         pill.textContent = state._presence.join(', ');
4881       } else {
4882         pill.hidden = true;
4883       }
4884     } else if (m.type === 'annotations_changed' && m.by !== state._user?.username) {
4885       setStatus(`${m.by} annotatsiyalarni yangiladi – qayta yuklanmoqda`);
4886       loadAnnotations();
4887     } else if (m.type === 'status_changed' && m.by !== state._user?.username) {
4888       setStatus(`${m.by}: ${m.annotation_id} → ${m.status}`);
4889       loadAnnotations();
4890     } else if (m.type === 'cursor' && m.user !== state._user?.username) {
4891       _remoteCursors.set(m.user, { x: m.x, y: m.y, lastSeen: Date.now() });
4892       renderRemoteCursors();
4893     }
4894   };
4895   ws.onclose = () => {
4896     if (_ws !== ws) return;
4897     _ws = null;
4898     if (state.current) {
4899       _wsRetryTimer = setTimeout(wsConnect, 3000);
4900     }
4901   };
4902   ws.onerror = () => {};
4903 }
4904
4905 (async function init() {
4906   try {
4907     await loadConfig();
4908     if (state.authRequired) {
4909       const ok = await attemptResumeSession();
4910       if (!ok) {
4911         showLoginModal();
4912         updateUserBar();
4913         return;
4914       }
4915     }
4916     updateUserBar();
4917     await loadAuthedAssets();
4918   } catch (e) {
4919     setStatus('ishga tushirish xatosi: ' + e.message);
4920   }
4921 })();

```

4. Model arxitekturalari (app/models_arch/)

app/models_arch/bca_yolo.py

Fayl yo'li: app/models_arch/bca_yolo.py | Qatorlar: 404 | Hajm: 14765 bayt

```
1  """
2  BCA-YOLO: Bilateral Cross-Attention YOLO for Mammographic Lesion Detection.
3
4  Reference implementation accompanying the methods paper.
5
6  Architecture overview
7  -----
8  Input: Four co-registered mammographic views per study,
9          $x_v \in \mathbb{R}^{1 \times H \times W}$ ,  $v \in \{L\_CC, R\_CC, L\_MLO, R\_MLO\}$ .
10
11  Components:
12      1. Shared YOLO backbone  $\Phi$  produces multi-scale features  $F_v$ .
13      2. Bilateral Cross-Attention (BCA) at scale  $s$  pairs  $(L_p, R_p)$  for
14         each projection  $p \in \{CC, MLO\}$  and produces asymmetry-aware features.
15      3. Inter-View Consistency (IVC) module fuses  $\{CC, MLO\}$  for each side.
16      4. Detection heads (per-view + fused-side auxiliary).
17      5. Ordinal BI-RADS regression head with cumulative link.
18
19  This module focuses on the BCA + IVC + ordinal head – the YOLO backbone
20  and detection heads are intentionally factored out so any Ultralytics
21  backbone can be swapped in.
22  """
23  from __future__ import annotations
24
25  from dataclasses import dataclass
26
27  import torch
28  import torch.nn as nn
29  import torch.nn.functional as F
30
31
32  # -----
33  # 1. Bilateral Cross-Attention (BCA)
34  # -----
35
36
37  class BilateralCrossAttention(nn.Module):
38      """Cross-attention between left/right contralateral feature maps.
39
40      Given  $F_L, F_R \in \mathbb{R}^{B \times C \times H \times W}$  for the same projection  $p$ , this module
41      horizontally mirrors  $F_R$  (so that anatomical regions are aligned) and
42      computes:
43
44           $Q_L = W_q F_L$ 
45           $K_R = W_k \text{mirror}(F_R)$ 
46           $V_R = W_v \text{mirror}(F_R)$ 
47           $A_{LR} = \text{softmax}(Q_L K_R^T / \sqrt{d_k})$ 
48           $F'_L = F_L + \gamma \cdot A_{LR} V_R$ 
49
50      Symmetrically,  $F'_R$  is updated using  $F_L$ . The learnable  $\gamma$  initialises
51      to a small value so the module starts as identity (which preserves
52      pre-trained backbone behaviour) and learns to attend to asymmetric
53      regions only when this reduces the loss.
54      """
55
```

```

56     def __init__(self, channels: int, num_heads: int = 4, dropout: float = 0.0):
57         super().__init__()
58         if channels % num_heads != 0:
59             raise ValueError("channels must be divisible by num_heads")
60         self.channels = channels
61         self.num_heads = num_heads
62         self.head_dim = channels // num_heads
63         self.scale = self.head_dim ** -0.5
64
65         self.qkv_l = nn.Conv2d(channels, channels * 3, kernel_size=1, bias=False)
66         self.qkv_r = nn.Conv2d(channels, channels * 3, kernel_size=1, bias=False)
67         self.proj_l = nn.Conv2d(channels, channels, kernel_size=1)
68         self.proj_r = nn.Conv2d(channels, channels, kernel_size=1)
69         self.gamma = nn.Parameter(torch.zeros(1))
70         self.drop = nn.Dropout(dropout)
71
72     @staticmethod
73     def mirror(x: torch.Tensor) -> torch.Tensor:
74         return torch.flip(x, dims=[-1])
75
76     def _attend(self, qkv_a: torch.Tensor, qkv_b_mirrored: torch.Tensor):
77         B, C3, H, W = qkv_a.shape
78         C = C3 // 3
79         qkv_a = qkv_a.view(B, 3, self.num_heads, self.head_dim, H * W)
80         qkv_b = qkv_b_mirrored.view(B, 3, self.num_heads, self.head_dim, H * W)
81         Q = qkv_a[:, 0]
82         K = qkv_b[:, 1]
83         V = qkv_b[:, 2]
84         attn = torch.einsum("bhdn,bhdm->bhnm", Q, K) * self.scale
85         attn = attn.softmax(dim=-1)
86         attn = self.drop(attn)
87         out = torch.einsum("bhnmbhdm->bhdn", attn, V)
88         out = out.reshape(B, C, H, W)
89         return out, attn
90
91     def forward(self, F_L: torch.Tensor, F_R: torch.Tensor):
92         F_R_m = self.mirror(F_R)
93         qkv_l = self.qkv_l(F_L)
94         qkv_r_m = self.qkv_r(F_R_m)
95
96         out_l, attn_lr = self._attend(qkv_l, qkv_r_m)
97         out_l = self.proj_l(out_l)
98
99         F_L_m = self.mirror(F_L)
100        qkv_l_m = self.qkv_l(F_L_m)
101        out_r, attn_rl = self._attend(qkv_r_m, qkv_l_m)
102        out_r = self.mirror(self.proj_r(out_r))
103
104        Fp_L = F_L + self.gamma * out_l
105        Fp_R = F_R + self.gamma * out_r
106        return Fp_L, Fp_R, {"attn_lr": attn_lr, "attn_rl": attn_rl}
107
108
109 # -----
110 # 2. Inter-View Consistency (IVC) fusion
111 # -----
112
113
114 class InterViewConsistency(nn.Module):
115     r"""Channel-wise gated fusion of CC and MLO views for the same side.
116
117     
$$F_{\text{fused}} = \sigma(W_g [F_{\text{CC}} ; F_{\text{MLO}}]) \odot F_{\text{CC}} + (1 - \sigma(\dots)) \odot F_{\text{MLO}}$$

118
119     The learnable gate  $\sigma(\cdot) \in (0, 1)$  is a per-channel sigmoid produced by a
120     1x1 convolution over the concatenated features. The fused feature is
121     used by an auxiliary detection head; the per-view heads also receive

```

```

122     a consistency penalty during training.
123     """
124
125     def __init__(self, channels: int):
126         super().__init__()
127         self.gate = nn.Conv2d(channels * 2, channels, kernel_size=1)
128
129     def forward(self, F_cc: torch.Tensor, F_mlo: torch.Tensor):
130         if F_cc.shape != F_mlo.shape:
131             F_mlo = F.interpolate(F_mlo, size=F_cc.shape[-2:], mode="bilinear",
132                                 align_corners=False)
133         cat = torch.cat([F_cc, F_mlo], dim=1)
134         g = torch.sigmoid(self.gate(cat))
135         return g * F_cc + (1.0 - g) * F_mlo
136
137
138 # -----
139 # 3. Ordinal BI-RADS regression head (cumulative link)
140 # -----
141
142
143 class OrdinalBIRADSHead(nn.Module):
144     r"""Cumulative-link ordinal regression for BI-RADS 0-6.
145
146     Standard cross-entropy treats BI-RADS 1+2 the same as 1+5; for an
147     ordinal label this loses information. Following McCullagh (1980) we
148     model:
149
150         
$$P(y \leq k \mid x) = \sigma(\theta_k - f(x)), \quad k = 0, 1, \dots, K-1$$

151         
$$P(y = k \mid x) = P(y \leq k \mid x) - P(y \leq k-1 \mid x)$$

152
153     The thresholds  $\theta_0 < \theta_1 < \dots < \theta_{K-1}$  are learnt as a strictly
154     increasing sequence by parameterising successive gaps as
155         
$$\theta_0 = \alpha_0, \quad \theta_k = \theta_{k-1} + \text{softplus}(\alpha_k).$$

156     """
157
158     def __init__(self, in_features: int, num_classes: int = 7):
159         super().__init__()
160         self.num_classes = num_classes
161         self.f = nn.Linear(in_features, 1)
162         self.alpha = nn.Parameter(torch.zeros(num_classes - 1))
163
164     def thresholds(self) -> torch.Tensor:
165         gaps = F.softplus(self.alpha[1:])
166         theta = torch.cat([self.alpha[1:], self.alpha[1:] + torch.cumsum(gaps, 0)])
167         return theta
168
169     def forward(self, h: torch.Tensor) -> torch.Tensor:
170         score = self.f(h).squeeze(-1)
171         theta = self.thresholds()
172         cdf = torch.sigmoid(theta.unsqueeze(0) - score.unsqueeze(-1))
173         zeros = torch.zeros_like(cdf[..., :1])
174         ones = torch.ones_like(cdf[..., :1])
175         cdf = torch.cat([zeros, cdf, ones], dim=-1)
176         pmf = cdf[..., 1:] - cdf[..., :-1]
177         pmf = pmf.clamp(min=1e-8)
178         return pmf
179
180
181 # -----
182 # 4. Loss functions
183 # -----
184
185
186 def bca_attention_loss(
187     attn_lr: torch.Tensor,

```

```

188     bbox_target_l: torch.Tensor,
189     eps: float = 1e-6,
190 ) -> torch.Tensor:
191     r"""Encourage attention from L to mirrored-R to peak at lesion locations.
192
193     L_bca = - mean over lesion pixels p of log( pooled_attn(p) + eps )
194
195     `bbox_target_l` is a binary map (B, 1, H, W) marking lesion pixels in
196     the left view; `attn_lr` is (B, num_heads, N, N) where N = H*W. We
197     average over heads and reduce to a (B, H, W) attention-on-self map.
198     """
199     B, Hh, N, _ = attn_lr.shape
200     attn_avg = attn_lr.mean(dim=1)
201     attn_diag = attn_avg.diagonal(dim1=-2, dim2=-1)
202     H = bbox_target_l.shape[-2]
203     W = bbox_target_l.shape[-1]
204     attn_map = attn_diag.view(B, H, W)
205     target = bbox_target_l.squeeze(1).clamp(0, 1)
206     masked = -(target * torch.log(attn_map + eps)).sum(dim=(1, 2))
207     norm = target.sum(dim=(1, 2)).clamp(min=1.0)
208     return (masked / norm).mean()
209
210
211 def ivc_consistency_loss(
212     obj_cc: torch.Tensor,
213     obj_mlo: torch.Tensor,
214 ) -> torch.Tensor:
215     r"""Smoothness penalty between objectness on CC and MLO views.
216
217     L_ivc = || pool(obj_cc) - pool(obj_mlo) ||_2^2
218
219     Both maps are global-average-pooled per detection scale to remove
220     spatial misalignment that a learnable loss should not need to handle.
221     """
222     p_cc = obj_cc.mean(dim=(-2, -1))
223     p_mlo = obj_mlo.mean(dim=(-2, -1))
224     return F.mse_loss(p_cc, p_mlo)
225
226
227 def ordinal_birads_loss(
228     pmf: torch.Tensor,
229     target: torch.Tensor,
230 ) -> torch.Tensor:
231     r"""Cumulative-link ordinal cross-entropy.
232
233     L_ord = - mean log p(y_target | x)
234             + λ_mono * Σ_k max(0, p_{k} - p_{k+1}) on tail
235     """
236     nll = -torch.log(pmf.gather(-1, target.long()).unsqueeze(-1)).squeeze(-1)
237             .clamp(min=1e-8))
238     return nll.mean()
239
240
241 # -----
242 # 5. Top-level orchestrator (skeleton)
243 # -----
244
245
246 @dataclass
247 class BCAYOLOConfig:
248     backbone_channels: tuple[int, int, int] = (256, 512, 1024)
249     num_classes: int = 4
250     num_birads: int = 7
251     bca_heads: int = 4
252     lambda_bca: float = 0.3
253     lambda_ivc: float = 0.1

```

```

254     lambda_ord: float = 0.5
255
256
257 class BCAYoloHead(nn.Module):
258     """Container module wiring BCA + IVC at three feature scales.
259
260     The actual YOLO detection / box-regression layers are intentionally
261     abstracted to `_make_det_head` so any Ultralytics-style head can be
262     plugged in (this preserves training compatibility with the broader
263     YOLOv8/v11 ecosystem).
264     """
265
266     def __init__(self, cfg: BCAYOLOConfig | None = None):
267         super().__init__()
268         cfg = cfg or BCAYOLOConfig()
269         self.cfg = cfg
270         self.bca = nn.ModuleList(
271             [BilateralCrossAttention(c, num_heads=cfg.bca_heads)
272              for c in cfg.backbone_channels]
273         )
274         self.ivc = nn.ModuleList(
275             [InterViewConsistency(c) for c in cfg.backbone_channels]
276         )
277         self.det_per_view = nn.ModuleList(
278             [self._make_det_head(c, cfg.num_classes) for c in cfg.backbone_channels]
279         )
280         self.det_fused = nn.ModuleList(
281             [self._make_det_head(c, cfg.num_classes) for c in cfg.backbone_channels]
282         )
283         self.gap = nn.AdaptiveAvgPool2d(1)
284         self.birads = OrdinalBIRADSHead(
285             in_features=sum(cfg.backbone_channels), num_classes=cfg.num_birads,
286         )
287
288     def _make_det_head(self, channels: int, num_classes: int) -> nn.Module:
289         return nn.Sequential(
290             nn.Conv2d(channels, channels, kernel_size=3, padding=1),
291             nn.SiLU(),
292             nn.Conv2d(channels, num_classes + 5, kernel_size=1),
293         )
294
295     def forward(
296         self,
297         feats: dict[str, list[torch.Tensor]],
298     ) -> dict:
299         """feats: {view_name: [F_s1, F_s2, F_s3]} where view_name is one of
300         L_CC, R_CC, L_MLO, R_MLO. Each tensor (B, C_s, H_s, W_s).
301         """
302         keys = ["L_CC", "R_CC", "L_MLO", "R_MLO"]
303         for k in keys:
304             assert k in feats, f"missing view {k}"
305
306         per_view_dets: dict[str, list[torch.Tensor]] = {k: [] for k in keys}
307         fused_dets: dict[str, list[torch.Tensor]] = {"L": [], "R": []}
308         attn_maps: list[dict] = []
309
310         for s, (bca_s, ivc_s, det_pv, det_f) in enumerate(
311             zip(self.bca, self.ivc, self.det_per_view, self.det_fused)
312         ):
313             FL_cc, FR_cc, attn_cc = bca_s(feats["L_CC"][s], feats["R_CC"][s])
314             FL_mlo, FR_mlo, attn_mlo = bca_s(feats["L_MLO"][s], feats["R_MLO"][s])
315             attn_maps.append({"CC": attn_cc, "MLO": attn_mlo})
316
317             for k, F_v in zip(["L_CC", "R_CC", "L_MLO", "R_MLO"],
318                             [FL_cc, FR_cc, FL_mlo, FR_mlo]):
319                 per_view_dets[k].append(det_pv(F_v))

```



```

320
321         FL_fused = ivc_s(FL_cc, FL_mlo)
322         FR_fused = ivc_s(FR_cc, FR_mlo)
323         fused_dets["L"].append(det_f(FL_fused))
324         fused_dets["R"].append(det_f(FR_fused))
325
326         h = []
327         for s in range(len(self.cfg.backbone_channels)):
328             for k in keys:
329                 h.append(self.gap(feats[k][s]).flatten(1))
330         h = torch.cat(h, dim=-1)
331         h = h.view(h.size(0), 4, -1).mean(dim=1)
332         birads_pmf = self.birads(h)
333
334         return {
335             "per_view": per_view_dets,
336             "fused": fused_dets,
337             "birads": birads_pmf,
338             "attn": attn_maps,
339         }
340
341
342 def total_loss(
343     out: dict,
344     targets: dict,
345     cfg: BCAYOLOConfig,
346     yolo_loss_fn,
347 ) -> dict[str, torch.Tensor]:
348     L_yolo = sum(
349         yolo_loss_fn(out["per_view"][k], targets["per_view"][k])
350         for k in out["per_view"]
351     )
352
353     L_bca = torch.zeros(), device=L_yolo.device)
354     for s_idx, attn_s in enumerate(out["attn"]):
355         for proj in ("CC", "MLO"):
356             attn = attn_s[proj]
357             target_l = targets["bbox_mask"][f"L_{proj}"][s_idx]
358             L_bca = L_bca + bca_attention_loss(attn["attn_lr"], target_l)
359             target_r = targets["bbox_mask"][f"R_{proj}"][s_idx]
360             L_bca = L_bca + bca_attention_loss(attn["attn_rl"], target_r)
361     L_bca = L_bca / (2 * 2 * len(out["attn"]))
362
363     L_ivc = torch.zeros(), device=L_yolo.device)
364     for s_idx in range(len(out["per_view"]["L_CC"])):
365         for side in ("L", "R"):
366             cc = out["per_view"][f"{side}_CC"][s_idx][:, 4:5]
367             mlo = out["per_view"][f"{side}_MLO"][s_idx][:, 4:5]
368             L_ivc = L_ivc + ivc_consistency_loss(cc, mlo)
369     L_ivc = L_ivc / (2 * len(out["per_view"]["L_CC"]))
370
371     L_ord = ordinal_birads_loss(out["birads"], targets["birads"])
372
373     L_total = L_yolo + cfg.lambda_bca * L_bca + cfg.lambda_ivc * L_ivc + cfg.lambda_ord * L_ord
374     return {
375         "total": L_total, "yolo": L_yolo,
376         "bca": L_bca, "ivc": L_ivc, "ord": L_ord,
377     }
378
379
380 def parameter_count(module: nn.Module) -> int:
381     return sum(p.numel() for p in module.parameters() if p.requires_grad)
382
383
384 if __name__ == "__main__":
385     import time

```

```

386     cfg = BCAYOLOConfig()
387     head = BCAYoloHead(cfg)
388     print(f"BCA-YOLO head parameters: {parameter_count(head):,}")
389
390     B, scales = 2, [(256, 80, 80), (512, 40, 40), (1024, 20, 20)]
391     feats = {
392         v: [torch.randn(B, c, h, w) for (c, h, w) in scales]
393         for v in ["L_CC", "R_CC", "L_MLO", "R_MLO"]
394     }
395     head.eval()
396     with torch.no_grad():
397         t0 = time.perf_counter()
398         for _ in range(5):
399             out = head(feats)
400             elapsed = (time.perf_counter() - t0) / 5
401     print(f"forward latency (CPU, B={B}): {elapsed*1000:.1f} ms")
402     print(f"per_view scales: {[d.shape for d in out['per_view']['L_CC']]}")
403     print(f"fused scales:    {[d.shape for d in out['fused']['L']]}")
404     print(f"birads pmf:      {out['birads'].shape}, sum={out['birads'].sum(-1)}")

```

app/models_arch/tillnet_det.py

Fayl yo'li: [app/models_arch/tillnet_det.py](#) | Qatorlar: 414 | Hajm: 17668 bayt

```

1  """TILLNet-Det – Text-Informed Lesion Localisation Network for Detection.
2
3  A multimodal mammography lesion detector that conditions the visual feature
4  pyramid on the radiology report. Novelty over standard one-stage detectors:
5
6      1. Cross-script text encoder. Character-level token embeddings shared
7         across Cyrillic and Latin scripts (Uzbek-Cy / Uzbek-Lat / Russian /
8         English) feed a 4-layer Transformer producing a single d_t-dim
9         semantic vector t.
10
11      2. FiLM-fused FPN. At every FPN level  $P_l \in \mathbb{R}^{C \times H_l \times W_l}$  we
12         compute  $(\gamma_l, \beta_l) = \text{MLP}_l(t)$  and modulate
13          $P_l' = (1 + \gamma_l) \odot P_l + \beta_l$ 
14         This is FiLM (Perez et al. 2018) – cheap, end-to-end differentiable,
15         and lets the report bias the detector toward the side / quadrant /
16         lesion type that the radiologist described.
17
18      3. Anchor-free FCOS-style head. Per pixel of each  $P_l$  we predict
19         (a) classification logits over K classes,
20         (b) regression: 4-vector (l, t, r, b) distances from pixel to bbox sides,
21         (c) centerness  $\in [0, 1]$  suppressing low-quality predictions.
22
23     The head weights are shared across pyramid levels (modulated only by
24     the FiLM parameters), giving 1.6M head params total – small enough to
25     train on a few-thousand-image mammography corpus without overfitting.
26
27     References (architecture inspirations only; implementation is original):
28     - Lin et al. 2017      "Feature Pyramid Networks"
29     - Tian et al. 2019     "FCOS: Fully Convolutional One-Stage Detection"
30     - Perez et al. 2018    "FiLM: Visual Reasoning with a Linear Layer"
31
32     Sanity entry point at the bottom runs a forward pass on synthetic data and
33     prints the output shapes, parameter count, and approximate GFLOPs.
34     """
35     from __future__ import annotations
36
37     import math
38     from dataclasses import dataclass
39
40     import torch

```

```

41 import torch.nn as nn
42 import torch.nn.functional as F
43 from torchvision.models import resnet50, ResNet50_Weights
44
45
46 # -----
47 # 1. Text encoder – multilingual char-level transformer
48 # -----
49
50
51 # 256 char buckets cover Latin + Cyrillic + digits + punctuation cleanly.
52 TEXT_VOCAB = 256
53 TEXT_PAD_ID = 0
54 TEXT_MAX_LEN = 512
55
56
57 def encode_text(text: str, max_len: int = TEXT_MAX_LEN) -> torch.LongTensor:
58     """Char-level encoding: each character → its Unicode codepoint mod 256.
59
60     Hash collisions across scripts are fine because the embedding table is
61     learned end-to-end; the bucket id only needs to be deterministic.
62     """
63     if not text:
64         return torch.zeros(max_len, dtype=torch.long)
65     ids = [(ord(c) % 254) + 1 for c in text[:max_len]] # reserve 0 for PAD
66     if len(ids) < max_len:
67         ids = ids + [TEXT_PAD_ID] * (max_len - len(ids))
68     return torch.tensor(ids, dtype=torch.long)
69
70
71 class TextEncoder(nn.Module):
72     """Character embedding → 4-layer Transformer → mean-pool → linear."""
73
74     def __init__(
75         self,
76         vocab: int = TEXT_VOCAB,
77         d_model: int = 256,
78         n_layers: int = 4,
79         n_heads: int = 4,
80         d_ff: int = 1024,
81         d_out: int = 256,
82         max_len: int = TEXT_MAX_LEN,
83         dropout: float = 0.1,
84     ):
85         super().__init__()
86         self.embed = nn.Embedding(vocab, d_model, padding_idx=TEXT_PAD_ID)
87         self.pos = nn.Parameter(torch.zeros(1, max_len, d_model))
88         nn.init.trunc_normal_(self.pos, std=0.02)
89         layer = nn.TransformerEncoderLayer(
90             d_model=d_model, nhead=n_heads, dim_feedforward=d_ff,
91             dropout=dropout, batch_first=True, norm_first=True, activation="gelu",
92         )
93         self.encoder = nn.TransformerEncoder(layer, num_layers=n_layers)
94         self.proj = nn.Linear(d_model, d_out)
95         self.d_out = d_out
96
97     def forward(self, token_ids: torch.LongTensor) -> torch.Tensor:
98         # token_ids: (B, L)
99         B, L = token_ids.shape
100         mask = token_ids == TEXT_PAD_ID # (B, L) True where pad
101         x = self.embed(token_ids) + self.pos[:, :L]
102         x = self.encoder(x, src_key_padding_mask=mask) # (B, L, d_model)
103         # masked mean-pool
104         keep = (~mask).float().unsqueeze(-1) # (B, L, 1)
105         denom = keep.sum(dim=1).clamp_min(1.0)
106         pooled = (x * keep).sum(dim=1) / denom # (B, d_model)

```

```

107         return self.proj(pooled)                                # (B, d_out)
108
109
110 # -----
111 # 2. Image backbone – ResNet-50 → C3, C4, C5
112 # -----
113
114
115 class ResNetBackbone(nn.Module):
116     """Returns C3 (s=8), C4 (s=16), C5 (s=32) feature maps."""
117
118     def __init__(self, in_chans: int = 1, pretrained: bool = True):
119         super().__init__()
120         weights = ResNet50_Weights.DEFAULT if pretrained else None
121         net = resnet50(weights=weights)
122         # Adapt 3-chan ImageNet stem to 1-chan grayscale by averaging.
123         if in_chans != 3:
124             with torch.no_grad():
125                 w = net.conv1.weight.data.mean(dim=1, keepdim=True)
126                 new_conv = nn.Conv2d(in_chans, 64, 7, 2, 3, bias=False)
127                 new_conv.weight.data.copy_(w.repeat(1, in_chans, 1, 1) / in_chans)
128                 net.conv1 = new_conv
129         self.stem = nn.Sequential(net.conv1, net.bn1, net.relu, net.maxpool)
130         self.layer1 = net.layer1 # → C2 (s=4, 256ch)
131         self.layer2 = net.layer2 # → C3 (s=8, 512ch)
132         self.layer3 = net.layer3 # → C4 (s=16, 1024ch)
133         self.layer4 = net.layer4 # → C5 (s=32, 2048ch)
134         self.out_channels = (512, 1024, 2048)
135
136     def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
137         x = self.stem(x)
138         c2 = self.layer1(x)
139         c3 = self.layer2(c2)
140         c4 = self.layer3(c3)
141         c5 = self.layer4(c4)
142         return c3, c4, c5
143
144
145 # -----
146 # 3. FPN with extra P6/P7 (FCOS canonical 5-level pyramid)
147 # -----
148
149
150 class FPN(nn.Module):
151     """C3, C4, C5 → P3, P4, P5, P6, P7 (all width=fpn_dim)."""
152
153     def __init__(self, in_channels: tuple[int, int, int], fpn_dim: int = 256):
154         super().__init__()
155         c3, c4, c5 = in_channels
156         self.lat3 = nn.Conv2d(c3, fpn_dim, 1)
157         self.lat4 = nn.Conv2d(c4, fpn_dim, 1)
158         self.lat5 = nn.Conv2d(c5, fpn_dim, 1)
159         self.smooth3 = nn.Conv2d(fpn_dim, fpn_dim, 3, 1, 1)
160         self.smooth4 = nn.Conv2d(fpn_dim, fpn_dim, 3, 1, 1)
161         self.smooth5 = nn.Conv2d(fpn_dim, fpn_dim, 3, 1, 1)
162         self.p6 = nn.Conv2d(fpn_dim, fpn_dim, 3, 2, 1)
163         self.p7 = nn.Conv2d(fpn_dim, fpn_dim, 3, 2, 1)
164         self.fpn_dim = fpn_dim
165
166     def forward(self, c3, c4, c5) -> list[torch.Tensor]:
167         p5 = self.lat5(c5)
168         p4 = self.lat4(c4) + F.interpolate(p5, size=c4.shape[-2:], mode="nearest")
169         p3 = self.lat3(c3) + F.interpolate(p4, size=c3.shape[-2:], mode="nearest")
170         p3 = self.smooth3(p3); p4 = self.smooth4(p4); p5 = self.smooth5(p5)
171         p6 = self.p6(F.relu(p5))
172         p7 = self.p7(F.relu(p6))

```

```

173         return [p3, p4, p5, p6, p7]
174
175
176 # -----
177 # 4. FiLM fusion – per pyramid level
178 # -----
179
180
181 class FiLM(nn.Module):
182     """Predict  $(\gamma, \beta) \in \mathbb{R}^{2C}$  from text vector and modulate a feature map.
183
184     Output =  $(1 + \gamma) \odot x + \beta$  ( $\gamma, \beta$  broadcast across H, W).
185     Initialised so the network starts as the identity transform ( $\gamma=\beta=0$ ).
186     """
187
188     def __init__(self, text_dim: int, channels: int, hidden: int = 256):
189         super().__init__()
190         self.mlp = nn.Sequential(
191             nn.Linear(text_dim, hidden),
192             nn.GELU(),
193             nn.Linear(hidden, channels * 2),
194         )
195         # zero-init the final layer → identity at start of training
196         nn.init.zeros_(self.mlp[-1].weight)
197         nn.init.zeros_(self.mlp[-1].bias)
198
199     def forward(self, x: torch.Tensor, t: torch.Tensor) -> torch.Tensor:
200         # x: (B, C, H, W)    t: (B, d_t)
201         gb = self.mlp(t)                # (B, 2C)
202         gamma, beta = gb.chunk(2, dim=-1)
203         gamma = gamma[:, :, None, None]
204         beta = beta[:, :, None, None]
205         return (1.0 + gamma) * x + beta
206
207
208 # -----
209 # 5. FCOS-style detection head (shared weights across levels)
210 # -----
211
212
213 class Scale(nn.Module):
214     """Per-level learnable scalar so shared-head outputs match each level's stride."""
215
216     def __init__(self, init: float = 1.0):
217         super().__init__()
218         self.scale = nn.Parameter(torch.tensor(float(init)))
219
220     def forward(self, x: torch.Tensor) -> torch.Tensor:
221         return x * self.scale
222
223
224 class DetectionHead(nn.Module):
225     """Shared classification + regression + centerness towers."""
226
227     def __init__(self, fpn_dim: int = 256, num_classes: int = 3, n_convs: int = 4):
228         super().__init__()
229
230         def tower():
231             layers = []
232             for _ in range(n_convs):
233                 layers.append(nn.Conv2d(fpn_dim, fpn_dim, 3, 1, 1))
234                 layers.append(nn.GroupNorm(32, fpn_dim))
235                 layers.append(nn.ReLU(inplace=True))
236             return nn.Sequential(*layers)
237
238         self.cls_tower = tower()

```

```

239     self.reg_tower = tower()
240     self.cls_head = nn.Conv2d(fpn_dim, num_classes, 3, 1, 1)
241     self.reg_head = nn.Conv2d(fpn_dim, 4, 3, 1, 1)
242     self.center_head = nn.Conv2d(fpn_dim, 1, 3, 1, 1)
243     self.scales = nn.ModuleList([Scale(1.0) for _ in range(5)])
244
245     # Bias init for cls head:  $p_a \approx 0.01$  (focal-loss convention).
246     prior = 0.01
247     nn.init.constant_(self.cls_head.bias, -math.log((1.0 - prior) / prior))
248     nn.init.normal_(self.cls_head.weight, std=0.01)
249     nn.init.normal_(self.reg_head.weight, std=0.01)
250     nn.init.zeros_(self.reg_head.bias)
251     nn.init.normal_(self.center_head.weight, std=0.01)
252     nn.init.zeros_(self.center_head.bias)
253
254     def forward(self, features: list[torch.Tensor]):
255         cls_outs, reg_outs, ctr_outs = [], [], []
256         for level, x in enumerate(features):
257             ct = self.cls_tower(x)
258             rt = self.reg_tower(x)
259             cls_outs.append(self.cls_head(ct))
260             reg_outs.append(F.relu(self.scales[level](self.reg_head(rt))))
261             ctr_outs.append(self.center_head(rt))
262         return cls_outs, reg_outs, ctr_outs
263
264
265     # -----
266     # 6. Top-level model
267     # -----
268
269
270     @dataclass
271     class TILLNetConfig:
272         num_classes: int = 3
273         text_dim: int = 256
274         fpn_dim: int = 256
275         pretrained_backbone: bool = True
276         in_chans: int = 1
277         use_text: bool = True          # set False for image-only ablation
278
279
280     class TILLNetDet(nn.Module):
281         def __init__(self, cfg: TILLNetConfig | None = None):
282             super().__init__()
283             cfg = cfg or TILLNetConfig()
284             self.cfg = cfg
285
286             self.text_enc = TextEncoder(d_out=cfg.text_dim) if cfg.use_text else None
287             self.backbone = ResNetBackbone(in_chans=cfg.in_chans, pretrained=cfg.pretrained_backbone)
288             self.fpn = FPN(self.backbone.out_channels, fpn_dim=cfg.fpn_dim)
289
290             if cfg.use_text:
291                 self.films = nn.ModuleList([
292                     FiLM(cfg.text_dim, cfg.fpn_dim) for _ in range(5)
293                 ])
294             else:
295                 self.films = None
296
297             self.head = DetectionHead(fpn_dim=cfg.fpn_dim, num_classes=cfg.num_classes)
298             self.fpn_strides = (8, 16, 32, 64, 128)
299
300         def forward(
301             self,
302             images: torch.Tensor,          # (B, in_chans, H, W)
303             text_ids: torch.LongTensor | None = None, # (B, L) or None for image-only
304         ):

```

```

305     c3, c4, c5 = self.backbone(images)
306     feats = self.fpn(c3, c4, c5)                                # 5 levels
307
308     if self.cfg.use_text and self.text_enc is not None:
309         if text_ids is None:
310             # Allow batched eval where text is optional - zero-pad encoding.
311             text_ids = torch.zeros(images.size(0), TEXT_MAX_LEN,
312                                   dtype=torch.long, device=images.device)
313             t = self.text_enc(text_ids)                        # (B, d_t)
314             feats = [film(f, t) for film, f in zip(self.films, feats)]
315
316     cls_outs, reg_outs, ctr_outs = self.head(feats)
317     return {
318         "cls_logits": cls_outs,      # list of (B, K, H_l, W_l)
319         "reg_dist": reg_outs,        # list of (B, 4, H_l, W_l) - distances l,t,r,b in stride units
320         "centerness": ctr_outs,     # list of (B, 1, H_l, W_l)
321         "fpn_strides": self.fpn_strides,
322     }
323
324 @torch.no_grad()
325 def predict(
326     self,
327     images: torch.Tensor,
328     text_ids: torch.LongTensor | None = None,
329     score_threshold: float = 0.05,
330     max_per_image: int = 100,
331 ) -> list[dict]:
332     """Decode dense predictions to per-image (boxes, scores, labels) lists."""
333     out = self.forward(images, text_ids)
334     cls_outs, reg_outs, ctr_outs = out["cls_logits"], out["reg_dist"], out["centerness"]
335     B = images.size(0)
336     H_img, W_img = images.shape[-2:]
337     results: list[dict] = []
338     for b in range(B):
339         all_boxes, all_scores, all_labels = [], [], []
340         for level, stride in enumerate(self.fpn_strides):
341             cls = cls_outs[level][b].sigmoid()                # (K, H, W)
342             reg = reg_outs[level][b]                          # (4, H, W)
343             ctr = ctr_outs[level][b].sigmoid()                # (1, H, W)
344             K, H, W = cls.shape
345             # Generate location grid (in original image coords).
346             ys, xs = torch.meshgrid(
347                 torch.arange(H, device=images.device),
348                 torch.arange(W, device=images.device),
349                 indexing="ij",
350             )
351             cx = (xs.float() + 0.5) * stride
352             cy = (ys.float() + 0.5) * stride
353             # FCOS regression: (l, t, r, b) distances → bbox xyxy
354             l = reg[0] * stride; t = reg[1] * stride
355             r = reg[2] * stride; b_ = reg[3] * stride
356             x0 = (cx - l).clamp(0, W_img); y0 = (cy - t).clamp(0, H_img)
357             x1 = (cx + r).clamp(0, W_img); y1 = (cy + b_).clamp(0, H_img)
358             boxes = torch.stack([x0, y0, x1, y1], dim=-1).reshape(-1, 4)
359             scores = (cls * ctr).reshape(K, -1)                # (K, HW)
360             top_score, top_label = scores.max(dim=0)           # per-location best class
361             keep = top_score >= score_threshold
362             if keep.any():
363                 all_boxes.append(boxes[keep])
364                 all_scores.append(top_score[keep])
365                 all_labels.append(top_label[keep])
366         if not all_boxes:
367             results.append({"boxes": torch.zeros(0, 4), "scores": torch.zeros(0),
368                             "labels": torch.zeros(0, dtype=torch.long)})
369         continue

```

```

370         boxes = torch.cat(all_boxes); scores = torch.cat(all_scores); labels =
torch.cat(all_labels)
371         order = scores.argsort(descending=True)[:max_per_image]
372         results.append({"boxes": boxes[order], "scores": scores[order], "labels": labels[order]})
373     return results
374
375
376 # -----
377 # 7. Sanity entry point – forward pass on synthetic data
378 # -----
379
380
381 def _param_count(m: nn.Module) -> int:
382     return sum(p.numel() for p in m.parameters() if p.requires_grad)
383
384
385 if __name__ == "__main__":
386     torch.manual_seed(0)
387     cfg = TILLNetConfig(num_classes=3, pretrained_backbone=False, use_text=True)
388     model = TILLNetDet(cfg).eval()
389
390     B, H, W = 2, 512, 512          # smaller for CPU smoke test
391     images = torch.randn(B, 1, H, W)
392     texts = torch.stack([
393         encode_text("Чап сут безида юқори-ташқи квадрантда хосила 18мм."),
394         encode_text("Right breast UOQ mass at 2 o'clock 15mm."),
395     ])
396     out = model(images, texts)
397
398     print(f"Backbone params      : {_param_count(model.backbone)/1e6:6.2f} M")
399     print(f"Text encoder params  : {_param_count(model.text_enc)/1e6:6.2f} M")
400     print(f"FPN params           : {_param_count(model.fpn)/1e6:6.2f} M")
401     print(f"FiLM params          : {_param_count(model.films)/1e6:6.2f} M")
402     print(f"Detection head params: {_param_count(model.head)/1e6:6.2f} M")
403     print(f"TOTAL                : {_param_count(model)/1e6:6.2f} M")
404     print()
405     for level, (cls, reg, ctr) in enumerate(
406         zip(out["cls_logits"], out["reg_dist"], out["centerness"])):
407     ):
408         print(f"  P{level+3}  stride={out['fpn_strides'][level]:3d}  "
409             f"cls={tuple(cls.shape)} reg={tuple(reg.shape)} ctr={tuple(ctr.shape)}")
410     preds = model.predict(images, texts, score_threshold=0.0, max_per_image=5)
411     print("\nDecoded predictions (top-5 per image):")
412     for i, p in enumerate(preds):
413         print(f"  img{i}: {len(p['boxes'])} boxes; first score={p['scores'][0].item():.3f}, "
414             f"label={p['labels'][0].item()}, box={p['boxes'][0].tolist()}")

```

[app/models_arch/xs_classifier.py](#)

Fayl yo'li: [app/models_arch/xs_classifier.py](#) | Qatorlar: 205 | Hajm: 6733 bayt

```

1  """
2  XS-Classifier – Cross-Script adaptive classifier for mammography reports.
3
4  The task: binary classification of mammography clinical text (Uzbek-Cyrillic,
5  Uzbek-Latin, Russian, frequently code-switched within a single document) into
6  {cancer, non-cancer}.
7
8  The novel idea: a *script-aware* feature stream. Each token is independently
9  classified into a script bucket via Unicode-block detection; we extract one
10 TF-IDF vector per bucket and concatenate them, allowing the linear classifier
11 to learn script-specific oncology vocabulary while implicitly sharing
12 character-level features across the two scripts of Uzbek.
13

```



```

14 Components:
15     - ScriptSegmenter          : per-token script detection
16     - CrossScriptVectorizer    : two-stream TF-IDF (Cyrillic + Latin) with
17                                   shared char-n-gram backbone
18     - XSClassifier             : sklearn-compatible Pipeline factory
19 """
20 from __future__ import annotations
21
22 import re
23 import unicodedata
24 from dataclasses import dataclass
25
26 import numpy as np
27 import scipy.sparse as sp
28 from sklearn.base import BaseEstimator, TransformerMixin
29 from sklearn.feature_extraction.text import TfidfVectorizer
30 from sklearn.linear_model import LogisticRegression
31 from sklearn.pipeline import FeatureUnion, Pipeline
32
33
34 # -----
35 # 1. Script detection
36 # -----
37
38
39 _TOKEN_RE = re.compile(r"\b\w+\b", flags=re.UNICODE)
40
41
42 def script_of_char(ch: str) -> str:
43     """Return 'cyrillic', 'latin', or 'other' for a single character."""
44     if not ch.isalpha():
45         return "other"
46     o = ord(ch)
47     if 0x0400 <= o <= 0x04FF or 0x0500 <= o <= 0x052F:
48         return "cyrillic"
49     if 0x0041 <= o <= 0x024F:
50         return "latin"
51     return "other"
52
53
54 def dominant_script(token: str) -> str:
55     counts = {"cyrillic": 0, "latin": 0, "other": 0}
56     for ch in token:
57         counts[script_of_char(ch)] += 1
58     if counts["cyrillic"] > counts["latin"]:
59         return "cyrillic"
60     if counts["latin"] > counts["cyrillic"]:
61         return "latin"
62     return "other"
63
64
65 def split_by_script(text: str) -> tuple[str, str]:
66     """Return (cyrillic_tokens_joined, latin_tokens_joined)."""
67     cyr, lat = [], []
68     for tok in _TOKEN_RE.findall(text or ""):
69         s = dominant_script(tok)
70         norm = unicodedata.normalize("NFKC", tok.lower())
71         if s == "cyrillic":
72             cyr.append(norm)
73         elif s == "latin":
74             lat.append(norm)
75         else:
76             cyr.append(norm)
77             lat.append(norm)
78     return " ".join(cyr), " ".join(lat)
79

```

```

80
81 # -----
82 # 2. Stream selectors (transformers picking one stream from a (text, label) pair)
83 # -----
84
85
86 class _StreamSelector(BaseEstimator, TransformerMixin):
87     def __init__(self, stream: str = "cyrillic"):
88         self.stream = stream
89
90     def fit(self, X, y=None):
91         return self
92
93     def transform(self, X):
94         out = []
95         for text in X:
96             cyr, lat = split_by_script(text)
97             out.append(cyr if self.stream == "cyrillic" else lat)
98         return out
99
100
101 # -----
102 # 3. Pipeline factory
103 # -----
104
105
106 @dataclass
107 class XSConfig:
108     word_min_df: int = 2
109     word_max_df: float = 0.95
110     word_ngram: tuple = (1, 2)
111     char_min_df: int = 2
112     char_max_df: float = 0.95
113     char_ngram: tuple = (3, 5)
114     sublinear_tf: bool = True
115     C: float = 1.0
116     class_weight: str | None = "balanced"
117     max_iter: int = 2000
118
119
120 def make_xs_pipeline(cfg: XSConfig | None = None) -> Pipeline:
121     """Two-stream Cross-Script classifier.
122
123     The architecture:
124     text -> ScriptSegmenter ->
125         |   |
126         |   | Cyrillic stream -> word TF-IDF + char TF-IDF
127         |   | Latin stream -> word TF-IDF + char TF-IDF
128         |   |
129         |   | -> concatenate sparse features -> Logistic Regression(L2)
130     """
131     cfg = cfg or XSConfig()
132
133     def stream_branch(stream: str) -> Pipeline:
134         return Pipeline([
135             ("select", _StreamSelector(stream=stream)),
136             ("union", FeatureUnion([
137                 ("word", TfidfVectorizer(
138                     analyzer="word", ngram_range=cfg.word_ngram,
139                     min_df=cfg.word_min_df, max_df=cfg.word_max_df,
140                     sublinear_tf=cfg.sublinear_tf, token_pattern=r"\b\w+\b",
141                 )),
142                 ("char", TfidfVectorizer(
143                     analyzer="char_wb", ngram_range=cfg.char_ngram,
144                     min_df=cfg.char_min_df, max_df=cfg.char_max_df,
145                     sublinear_tf=cfg.sublinear_tf,
146                 )),
147             ])),
148         ])

```

```

146     ])
147
148     return Pipeline([
149         ("xs", FeatureUnion([
150             ("cyrillic", stream_branch("cyrillic")),
151             ("latin", stream_branch("latin")),
152         ])),
153         ("clf", LogisticRegression(
154             C=cfg.C, max_iter=cfg.max_iter,
155             class_weight=cfg.class_weight, solver="liblinear",
156         )),
157     ])
158
159
160 def make_baseline_pipeline(
161     analyzer: str = "char_wb",
162     ngram: tuple = (3, 5),
163     C: float = 1.0,
164 ) -> Pipeline:
165     """Single-stream TF-IDF baseline (no script awareness)."""
166     return Pipeline([
167         ("tfidf", TfidfVectorizer(
168             analyzer=analyzer, ngram_range=ngram,
169             min_df=2, max_df=0.95, sublinear_tf=True,
170             token_pattern=r"\b\w+\b" if analyzer == "word" else None,
171         )),
172         ("clf", LogisticRegression(
173             C=C, max_iter=2000, class_weight="balanced", solver="liblinear",
174         )),
175     ])
176
177
178 def make_word_only_baseline() -> Pipeline:
179     return make_baseline_pipeline(analyzer="word", ngram=(1, 2))
180
181
182 def make_char_only_baseline() -> Pipeline:
183     return make_baseline_pipeline(analyzer="char_wb", ngram=(3, 5))
184
185
186 # -----
187 # 4. Sanity entry point
188 # -----
189
190
191 if __name__ == "__main__":
192     samples = [
193         "Шикояти: Унг сут безидаги хосилага. Анамнесис: бемор узи 2024 йилда",
194         "Shikoyati: O'ng sut bezidagi xosilaga. Anamnesis morbi: bemor o'zini",
195         "Жалобы: на образование в правой молочной железе.",
196     ]
197     for s in samples:
198         cyr, lat = split_by_script(s)
199         print(f"  cyr: {cyr[:60]!r}")
200         print(f"  lat: {lat[:60]!r}")
201         print()
202
203     pipe = make_xs_pipeline()
204     pipe.fit(samples + samples, [0, 1, 0, 1, 0, 1])
205     print(f"  XS pipeline fit OK, predict: {pipe.predict(samples)}")

```

5. O'qitish va tadqiqot skriptlari (app/research/)

app/research/download_models.py

Fayl yo'li: app/research/download_models.py | Qatorlar: 86 | Hajm: 3451 bayt

```
1  """Download base YOLO architectures as FINE-TUNING bases for mammography training.
2
3  IMPORTANT – read this first
4  =====
5  These are COCO-pretrained *general-purpose* detectors. Running them directly on
6  mammograms will NOT detect masses / calcifications – they detect everyday
7  objects. They are only useful as **starting weights** that you fine-tune on your
8  own annotated mammography dataset.
9
10 The honest path to "excellent results":
11  1. Annotate cases in MAMOGRAF (you already can).
12  2. Export a YOLO dataset: GET /api/export?format=yolo (the ↓ YOLO button).
13  3. Fine-tune one or more of these architectures on that dataset
14     (see app/research/train_detector.py).
15  4. Drop the trained *.pt files into app/models/ – they appear in the AI dropdown.
16  5. Turn on TTA and select "🦋 Ensemble (WBF)" to combine them.
17
18 Bigger architecture ≈ higher ceiling but slower + needs more data:
19   n (nano) < s (small) < m (medium) < l (large) < x (extra-large)
20
21 Usage
22 ----
23     python -m app.research.download_models          # default recommended set
24     python -m app.research.download_models yolo11x yolov8l
25     python -m app.research.download_models --dest app/research/pretrained
26 """
27 from __future__ import annotations
28
29 import argparse
30 import sys
31 from pathlib import Path
32
33 # Recommended training bases (downloaded by Ultralytics on first use).
34 RECOMMENDED = ["yolo11s", "yolo11m", "yolo11l", "yolo11x", "yolov8m"]
35
36 DEFAULT_DEST = Path(__file__).resolve().parent / "pretrained"
37
38
39 def download(names: list[str], dest: Path) -> list[Path]:
40     try:
41         from ultralytics import YOLO
42     except ImportError:
43         sys.exit("ultralytics is not installed. Run: pip install ultralytics")
44
45     dest.mkdir(parents=True, exist_ok=True)
46     saved: list[Path] = []
47     for name in names:
48         weight = name if name.endswith(".pt") else f"{name}.pt"
49         print(f"→ fetching {weight} ...")
50         # Constructing YOLO(weight) downloads the COCO-pretrained checkpoint.
51         model = YOLO(weight)
52         ckpt = Path(model.ckpt_path) if getattr(model, "ckpt_path", None) else Path(weight)
53         target = dest / ckpt.name
54         if ckpt.exists() and ckpt.resolve() != target.resolve():
55             target.write_bytes(ckpt.read_bytes())
```

```

56         print(f" saved base weights → {target}")
57         saved.append(target)
58     return saved
59
60
61 def main() -> None:
62     ap = argparse.ArgumentParser(description="Download YOLO training bases (NOT ready detectors).")
63     ap.add_argument("models", nargs="*", default=RECOMMENDED,
64                     help=f"architectures to fetch (default: {' '.join(RECOMMENDED)})")
65     ap.add_argument("--dest", default=str(DEFAULT_DEST),
66                     help="destination folder for training bases")
67     args = ap.parse_args()
68
69     names = args.models or RECOMMENDED
70     dest = Path(args.dest)
71     saved = download(names, dest)
72
73     print("\nDone. Downloaded training bases:")
74     for p in saved:
75         print(f" {p}")
76     print(
77         "\nNEXT: fine-tune on your exported YOLO dataset, e.g.\n"
78         " yolo detect train model={base}.pt data=path/to/data.yaml imgsz=1024 epochs=100\n"
79         "Then copy the resulting best.pt into app/models/ (rename meaningfully, e.g.\n"
80         "mammo_yolo11m_v1.pt). It will show up in the AI model dropdown automatically.\n"
81         "Do NOT put these raw COCO bases into app/models/ – they do not detect findings."
82     )
83
84
85 if __name__ == "__main__":
86     main()

```

app/research/eval.py

Fayl yo'li: app/research/eval.py | Qatorlar: 355 | Hajm: 15101 bayt

```

1  """Detection-benchmark evaluation for MAMOGRAF.
2
3  Reports per-configuration AP@0.5, AP@[0.5:0.95], and FROC (sensitivity at
4  0.25, 0.5, 1.0, 2.0, 4.0 false positives / image) with bootstrap 95% CIs.
5  Supports four configurations driven from a single dataset:
6
7  * each individual YOLO model (`--models a.pt b.pt ...`)
8  * the WBF ensemble of those models (`--ensemble`)
9  * the same with test-time augmentation (`--tta`)
10
11 Dataset layout follows the standard Ultralytics YOLO format::
12
13     data.yaml          # `path:`, `val:` (or `test:`), `names: {0: ..., 1: ...}`
14     images/<split>/*. [png|jpg]
15     labels/<split>/*.txt # one row per box: "<cls> cx cy w h" (normalised)
16
17 Usage:
18     python -m app.research.eval --data path/to/data.yaml \
19         --models app/models/yolo9_e.pt app/models/yolo11_x.pt \
20         --ensemble --tta --bootstrap 1000 \
21         --out paper/results.json --md-out paper/results_tables.md
22
23 Outputs a JSON file with raw metrics + bootstrap CIs and a Markdown file with
24 Tables 3 / FROC ready to paste into the manuscript.
25 """
26 from __future__ import annotations
27
28 import argparse

```

```

29 import json
30 import sys
31 import time
32 from collections import defaultdict
33 from pathlib import Path
34 from typing import Optional
35
36 import numpy as np
37 from PIL import Image
38
39 try:
40     import yaml
41 except ImportError:
42     sys.exit("PyYAML is required: pip install pyyaml")
43
44 # Reuse the WBF + TTA implementation from the platform.
45 sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
46 from app.inference import infer_png, weighted_boxes_fusion # noqa: E402
47
48
49 # ----- #
50 # Dataset helpers #
51 # ----- #
52 def load_dataset(yaml_path: Path, split: str | None) -> tuple[list[Path], dict[int, str]]:
53     cfg = yaml.safe_load(yaml_path.read_text(encoding="utf-8"))
54     root = Path(cfg.get("path", yaml_path.parent)).expanduser()
55     if not root.is_absolute():
56         root = (yaml_path.parent / root).resolve()
57     chosen = split or ("test" if "test" in cfg else "val")
58     images_dir = cfg.get(chosen)
59     if images_dir is None:
60         raise SystemExit(f"split '{chosen}' not present in {yaml_path}")
61     images_dir = root / images_dir
62     images = sorted(p for p in images_dir.rglob("*") if p.suffix.lower() in {".png", ".jpg", ".jpeg"})
63     if not images:
64         raise SystemExit(f"no images found under {images_dir}")
65     names_raw = cfg.get("names", {})
66     if isinstance(names_raw, list):
67         names = {i: n for i, n in enumerate(names_raw)}
68     else:
69         names = {int(k): v for k, v in names_raw.items()}
70     return images, names
71
72
73 def label_path_for(img: Path) -> Path:
74     return Path(str(img).replace("/images/", "/labels/").replace("\\images\\",
75 "\\labels\\")).with_suffix(".txt")
76
77
78 def read_yolo_labels(p: Path) -> list[tuple[int, float, float, float, float]]:
79     """Each line: <cls> <cx> <cy> <w> <h> (normalised); returns (cls, x1, y1, x2, y2)."""
80     out = []
81     if not p.exists():
82         return out
83     for line in p.read_text(encoding="utf-8").splitlines():
84         parts = line.split()
85         if len(parts) != 5:
86             continue
87         c, cx, cy, w, h = parts
88         cx, cy, w, h = float(cx), float(cy), float(w), float(h)
89         out.append((int(c), cx - w / 2, cy - h / 2, cx + w / 2, cy + h / 2))
90     return out
91
92 # ----- #
93 # Inference #

```

```

94 # ----- #
95 def detect_one(image_path: Path, models: list[str], ensemble: bool, tta: bool,
96               conf: float, iou: float, imgsz: int, names_inv: dict[str, int]
97               ) -> list[tuple[int, float, float, float, float, float]]:
98     """Return detections as (cls, x1, y1, x2, y2, score), normalised."""
99     png = image_path.read_bytes()
100     per_model: list[list[dict]] = []
101     for m in models:
102         r = infer_png(png, m, conf=conf, iou=iou, imgsz=imgsz, tta=tta)
103         per_model.append(r["detections"])
104     dets_dicts = (weighted_boxes_fusion(per_model, iou_thr=0.55, num_sources=len(models))
105                  if ensemble else per_model[0])
106     out = []
107     for d in dets_dicts:
108         cls = names_inv.get(str(d.get("label")), d.get("class_id", -1))
109         if cls is None or cls < 0:
110             continue
111         x, y, w, h = d["bbox"]
112         out.append((int(cls), x, y, x + w, y + h, float(d["confidence"])))
113     return out
114
115
116 # ----- #
117 # Metrics #
118 # ----- #
119 def iou_xyxy(a, b):
120     ix1, iy1 = max(a[0], b[0]), max(a[1], b[1])
121     ix2, iy2 = min(a[2], b[2]), min(a[3], b[3])
122     iw, ih = max(0.0, ix2 - ix1), max(0.0, iy2 - iy1)
123     inter = iw * ih
124     if inter <= 0:
125         return 0.0
126     area_a = max(0.0, a[2] - a[0]) * max(0.0, a[3] - a[1])
127     area_b = max(0.0, b[2] - b[0]) * max(0.0, b[3] - b[1])
128     return inter / (area_a + area_b - inter + 1e-12)
129
130
131 def match_image(preds, gts, iou_thr: float):
132     """Greedy IoU matching; returns list of (score, tp_flag) and #FP, plus per-image TP count."""
133     preds_sorted = sorted(preds, key=lambda p: -p[5])
134     used = [False] * len(gts)
135     out: list[tuple[float, int]] = []
136     tp = 0
137     fp = 0
138     for cls, x1, y1, x2, y2, s in preds_sorted:
139         best_i, best_iou = -1, iou_thr
140         for i, g in enumerate(gts):
141             if used[i] or g[0] != cls:
142                 continue
143             iou = iou_xyxy((x1, y1, x2, y2), g[1:5])
144             if iou > best_iou:
145                 best_iou, best_i = iou, i
146         if best_i >= 0:
147             used[best_i] = True
148             out.append((s, 1))
149             tp += 1
150         else:
151             out.append((s, 0))
152             fp += 1
153     fn = sum(1 for u in used if not u)
154     return out, tp, fp, fn
155
156
157 def ap_from_pr(scored: list[tuple[float, int]], n_gt: int) -> float:
158     """Average precision via the all-point interpolation (COCO style)."""
159     if n_gt == 0:

```

```

160         return 0.0 if not scored else float("nan")
161     scored = sorted(scored, key=lambda x: -x[0])
162     tp_cum, fp_cum = 0, 0
163     pr = []
164     for s, tp_flag in scored:
165         tp_cum += tp_flag
166         fp_cum += 1 - tp_flag
167         recall = tp_cum / n_gt
168         precision = tp_cum / (tp_cum + fp_cum)
169         pr.append((recall, precision))
170     if not pr:
171         return 0.0
172     rec = np.array([0.0] + [p[0] for p in pr] + [1.0])
173     prec = np.array([0.0] + [p[1] for p in pr] + [0.0])
174     for i in range(len(prec) - 2, -1, -1):
175         prec[i] = max(prec[i], prec[i + 1])
176     idx = np.where(rec[1:] != rec[:-1])[0]
177     return float(np.sum((rec[idx + 1] - rec[idx]) * prec[idx + 1]))
178
179
180 def map_at_iou(per_image_dets, per_image_gts, iou_thr: float) -> float:
181     """Mean AP across classes at a single IoU threshold."""
182     by_cls_scored: dict[int, list[tuple[float, int]]] = defaultdict(list)
183     by_cls_gt: dict[int, int] = defaultdict(int)
184     for preds, gts in zip(per_image_dets, per_image_gts):
185         for g in gts:
186             by_cls_gt[g[0]] += 1
187         # match per class
188         for cls in set([p[0] for p in preds] + [g[0] for g in gts]):
189             preds_c = [p for p in preds if p[0] == cls]
190             gts_c = [g for g in gts if g[0] == cls]
191             scored, _, _ = match_image(preds_c, gts_c, iou_thr)
192             by_cls_scored[cls].extend(scored)
193     if not by_cls_gt:
194         return float("nan")
195     aps = [ap_from_pr(by_cls_scored[c], by_cls_gt[c]) for c in by_cls_gt]
196     return float(np.mean(aps))
197
198
199 def coco_ap(per_image_dets, per_image_gts) -> float:
200     return float(np.mean([map_at_iou(per_image_dets, per_image_gts, t)
201                               for t in np.arange(0.5, 1.0, 0.05)]))
202
203
204 def froc(per_image_dets, per_image_gts, fppi_points=(0.25, 0.5, 1.0, 2.0, 4.0),
205          iou_thr: float = 0.5) -> dict[float, float]:
206     """Sensitivity at fixed FP-per-image operating points (single-class proxy)."""
207     pairs: list[tuple[float, int, int]] = [] # (score, tp, n_images_contribution=1)
208     n_gt = 0
209     n_img = max(1, len(per_image_dets))
210     for preds, gts in zip(per_image_dets, per_image_gts):
211         n_gt += len(gts)
212         scored, _tp, _fp, _fn = match_image(preds, gts, iou_thr)
213         for s, tp in scored:
214             pairs.append((s, tp, 1))
215     if n_gt == 0:
216         return {f: float("nan") for f in fppi_points}
217     pairs.sort(key=lambda x: -x[0])
218     tp_cum, fp_cum = 0, 0
219     curve = [] # (fppi, sensitivity)
220     for s, tp, _ in pairs:
221         tp_cum += tp
222         fp_cum += 1 - tp
223         curve.append((fp_cum / n_img, tp_cum / n_gt))
224     out: dict[float, float] = {}
225     for target in fppi_points:

```



```

226         best = 0.0
227         for fppi, sens in curve:
228             if fppi <= target and sens > best:
229                 best = sens
230         out[target] = float(best)
231     return out
232
233
234 def bootstrap_ci(per_image_dets, per_image_gts, metric_fn, n: int = 1000,
235                 seed: int = 0) -> tuple[float, float]:
236     rng = np.random.default_rng(seed)
237     m = len(per_image_dets)
238     vals = []
239     for _ in range(n):
240         idx = rng.integers(0, m, size=m)
241         v = metric_fn([per_image_dets[i] for i in idx], [per_image_gts[i] for i in idx])
242         if np.isfinite(v):
243             vals.append(v)
244     if not vals:
245         return float("nan"), float("nan")
246     return float(np.percentile(vals, 2.5)), float(np.percentile(vals, 97.5))
247
248
249 # ----- #
250 # Per-configuration evaluator #
251 # ----- #
252 def evaluate_config(name: str, images: list[Path], names: dict[int, str],
253                   models: list[str], ensemble: bool, tta: bool,
254                   conf: float, iou: float, imgsz: int,
255                   bootstrap: int) -> dict:
256     names_inv = {v: k for k, v in names.items()}
257     per_image_dets, per_image_gts = [], []
258     t0 = time.time()
259     for i, img in enumerate(images, 1):
260         dets = detect_one(img, models, ensemble, tta, conf, iou, imgsz, names_inv)
261         per_image_dets.append(dets)
262         per_image_gts.append(read_yolo_labels(label_path_for(img)))
263         if i % 25 == 0 or i == len(images):
264             print(f" {name}: {i}/{len(images)} elapsed {time.time() - t0:.1f}s", flush=True)
265
266     ap50 = map_at_iou(per_image_dets, per_image_gts, 0.5)
267     ap5095 = coco_ap(per_image_dets, per_image_gts)
268     froc_pts = froc(per_image_dets, per_image_gts)
269     res = {
270         "config": name, "n_images": len(images),
271         "AP@0.5": ap50, "AP@[0.5:0.95]": ap5095, "FROC": froc_pts,
272         "wall_time_s": time.time() - t0,
273     }
274     if bootstrap > 0:
275         ci_lo, ci_hi = bootstrap_ci(per_image_dets, per_image_gts,
276                                   lambda d, g: map_at_iou(d, g, 0.5),
277                                   n=bootstrap)
278         res["AP@0.5_ci95"] = [ci_lo, ci_hi]
279     return res
280
281
282 # ----- #
283 # Markdown table emitter #
284 # ----- #
285 def write_markdown(results: list[dict], path: Path) -> None:
286     fppi = (0.25, 0.5, 1.0, 2.0, 4.0)
287     lines = ["# Detection benchmark", ""]
288     lines.append("| Configuration | n | AP@0.5 (95% CI) | AP@[0.5:0.95] | "
289               + " | ".join(f"FROC@{f}" for f in fppi) + " |")
290     lines.append("|" + "|" .join(["---"] * (4 + len(fppi))) + "|")
291     for r in results:

```

```

292     ci = r.get("AP@0.5_ci95")
293     ci_s = f" ({ci[0]:.3f}-{ci[1]:.3f})" if ci else ""
294     froc_s = " | ".join(f"{r['FROC'][f]:.3f}" for f in fpfi)
295     lines.append(f"| {r['config']} | {r['n_images']} | "
296                 f"{r['AP@0.5']:.3f}{ci_s} | {r['AP@[0.5:0.95]']:.3f} | {froc_s} |")
297     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
298
299
300 # ----- #
301 # CLI #
302 # ----- #
303 def main() -> None:
304     ap = argparse.ArgumentParser(description="Detection benchmark (AP / FROC) for MAMOGRAF.")
305     ap.add_argument("--data", required=True, help="Ultralytics data.yaml")
306     ap.add_argument("--split", default=None, help="'test' (default if present) or 'val'")
307     ap.add_argument("--models", nargs="+", required=True,
308                     help="paths or filenames inside app/models/")
309     ap.add_argument("--ensemble", action="store_true",
310                     help="also evaluate the WBF ensemble of all --models")
311     ap.add_argument("--tta", action="store_true",
312                     help="also evaluate the TTA variant of each configuration")
313     ap.add_argument("--conf", type=float, default=0.05)
314     ap.add_argument("--iou", type=float, default=0.5)
315     ap.add_argument("--imgsz", type=int, default=1024)
316     ap.add_argument("--bootstrap", type=int, default=1000,
317                     help="bootstrap iterations for AP@0.5 CI (0 to disable)")
318     ap.add_argument("--out", default="paper/results.json")
319     ap.add_argument("--md-out", default="paper/results_tables.md")
320     args = ap.parse_args()
321
322     images, names = load_dataset(Path(args.data), args.split)
323     print(f"dataset: {len(images)} images, {len(names)} classes: {list(names.values())}")
324
325     configs: list[tuple[str, list[str], bool, bool]] = []
326     for m in args.models:
327         stem = Path(m).stem
328         configs.append((f"{stem} (single)", [m], False, False))
329         if args.tta:
330             configs.append((f"{stem} + TTA", [m], False, True))
331     if args.ensemble and len(args.models) >= 2:
332         configs.append((f"WBF ensemble x{len(args.models)}", args.models, True, False))
333         if args.tta:
334             configs.append((f"WBF ensemble x{len(args.models)} + TTA",
335                             args.models, True, True))
336
337     results = []
338     for cfg_name, mlist, ens, tta in configs:
339         print(f"\n>>> {cfg_name}")
340         r = evaluate_config(cfg_name, images, names, mlist, ens, tta,
341                             args.conf, args.iou, args.imgsz, args.bootstrap)
342         results.append(r)
343         print(f"    AP@0.5={r['AP@0.5']:.3f}    AP@[0.5:0.95]={r['AP@[0.5:0.95]']:.3f}    "
344               f"FROC@1FP={r['FROC'][1.0]:.3f}")
345
346     Path(args.out).parent.mkdir(parents=True, exist_ok=True)
347     Path(args.out).write_text(json.dumps({"results": results, "models": args.models,
348                                           "classes": names}, indent=2), encoding="utf-8")
349     Path(args.md_out).parent.mkdir(parents=True, exist_ok=True)
350     write_markdown(results, Path(args.md_out))
351     print(f"\nWrote {args.out} and {args.md_out}")
352
353
354 if __name__ == "__main__":
355     main()

```

app/research/explore_labels.py

Fayl yo'li: app/research/explore_labels.py | Qatorlar: 105 | Hajm: 3651 bayt

```
1  """Explore real labels in the SQLite DB to design the classification task."""
2  from __future__ import annotations
3
4  import re
5  import sys
6  from pathlib import Path
7
8  sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
9
10 from app.db import get_conn
11
12 CANCER_PATTERNS = [
13     r"\bC\s*50\b", r"C\s*50",
14     r"\bsarat[oa]n", r"\bсарат[oa]н",
15     r"\bрак\b", r"\braka?\b",
16     r"саратон", r"saraton",
17     r"karsinom", r"карцином",
18     r"malign", r"малигн",
19     r"\bM\s*8500/3\b",
20 ]
21 CANCER_RE = re.compile("|".join(CANCER_PATTERNS), re.IGNORECASE | re.UNICODE)
22
23
24 def detect_cancer(text: str) -> bool:
25     if not text:
26         return False
27     return bool(CANCER_RE.search(text))
28
29
30 def script_ratio(text: str) -> dict:
31     """Cyrillic vs Latin character ratio."""
32     if not text:
33         return {"cyrillic": 0, "latin": 0, "other": 0}
34     cyr = sum(1 for c in text if "Ё" <= c <= "я")
35     lat = sum(1 for c in text if c.isalpha() and ord(c) < 128)
36     other = sum(1 for c in text if c.isalpha()) - cyr - lat
37     total = max(1, cyr + lat + other)
38     return {"cyrillic": cyr / total, "latin": lat / total, "other": other / total}
39
40
41 def main():
42     with get_conn() as c:
43         rows = c.execute("""
44             SELECT id, diagnosis, complaints, history, clinical_findings,
45                    recommendations, report
46             FROM records
47             """).fetchall()
48     rows = [dict(r) for r in rows]
49     print(f"Total records: {len(rows)}")
50
51     has_dx = [r for r in rows if r.get("diagnosis")]
52     print(f"Records with non-null diagnosis: {len(has_dx)}")
53     cancer_rows = [r for r in rows if detect_cancer(r.get("diagnosis") or "")]
54     print(f"Records with cancer in diagnosis: {len(cancer_rows)}")
55     print(f"{len(cancer_rows)/len(rows)*100:.1f}%")
56
57     input_rows = [r for r in rows
58                   if (r.get("complaints") or r.get("history") or r.get("clinical_findings"))]
59     print(f"Records with at least one input field: {len(input_rows)}")
60
61     usable = []
62     for r in rows:
```

```

62     inp = " ".join(filter(None, [
63         r.get("complaints") or "", r.get("history") or "",
64         r.get("clinical_findings") or "",
65     ]))
66     if not inp.strip():
67         continue
68     dx = r.get("diagnosis") or ""
69     rep = r.get("report") or ""
70     is_cancer = detect_cancer(dx) or detect_cancer(rep)
71     usable.append({"id": r["id"], "text": inp, "dx": dx, "is_cancer": is_cancer})
72
73     print(f"\nUsable for ML (has input text): {len(usable)}")
74     n_pos = sum(1 for r in usable if r["is_cancer"])
75     print(f"   positive (cancer): {n_pos} ({n_pos/len(usable)*100:.1f}%)")
76     print(f"   negative: {len(usable)-n_pos} ({(len(usable)-n_pos)/len(usable)*100:.1f}%)")
77
78     # Script distribution
79     print("\nScript distribution (first 200 usable):")
80     cyr_pure = lat_pure = mixed = 0
81     for r in usable[:200]:
82         sr = script_ratio(r["text"])
83         if sr["cyrillic"] > 0.7:
84             cyr_pure += 1
85         elif sr["latin"] > 0.7:
86             lat_pure += 1
87         else:
88             mixed += 1
89     print(f"   Cyrillic-dominant: {cyr_pure}")
90     print(f"   Latin-dominant:    {lat_pure}")
91     print(f"   Mixed/other:         {mixed}")
92
93     # Sample positives and negatives
94     print("\nSample POSITIVE (cancer):")
95     for r in [r for r in usable if r["is_cancer"]][:2]:
96         print(f"   id={r['id']}   text[:200]={r['text'][:200]!r}")
97         print(f"   dx={r['dx'][:100]!r}")
98     print("\nSample NEGATIVE:")
99     for r in [r for r in usable if not r["is_cancer"]][:2]:
100         print(f"   id={r['id']}   text[:200]={r['text'][:200]!r}")
101         print(f"   dx={r['dx'][:80]!r}")
102
103
104 if __name__ == "__main__":
105     main()

```

app/research/gold_to_yolo.py

Fayl yo'li: app/research/gold_to_yolo.py | Qatorlar: 277 | Hajm: 10119 bayt

```

1  """Convert radiologist-verified gold labels (review_decisions table) into a
2  Ultralytics-compatible YOLO dataset, ready for the TILLNet-Det Stage-3
3  fine-tune.
4
5  Source of truth is the project SQLite DB: rows with `status IN ('accepted',
6  'edited')` are exported. For each row:
7
8      1. The preprocessed PNG (as referenced by `preprocessed_png_path`) is
9      symlinked / copied into <out>/images/<split>/<base>.png.
10     2. `final_bboxes_json` (pixel coords on the preprocessed canvas, same
11     coordinate system as `pseudo_labels.py` produced) is converted to
12     YOLO format and written to <out>/labels/<split>/<base>.txt.
13     3. A patient-level train/val split is computed from the SOP UID prefix
14     (study UID prefix) so two views of the same study cannot leak across
15     splits.

```

```

16
17 The resulting `dataset.yaml` is plug-compatible with both
18 `app.research.train_detector` (YOLOv8) and `app.research.train_tillnet`.
19
20 Usage:
21     python -m app.research.gold_to_yolo \
22         --out /path/to/yolo_gold \
23         --target-size 1024 \
24         --val-frac 0.15 \
25         --include accepted,edited
26
27 Or pass a pre-exported JSONL bundle (from the /api/research/review/export
28 endpoint) instead of reading the DB directly:
29
30     python -m app.research.gold_to_yolo \
31         --jsonl gold_labels_2026-05-07.jsonl \
32         --out /path/to/yolo_gold
33 """
34 from __future__ import annotations
35
36 import argparse
37 import hashlib
38 import json
39 import random
40 import shutil
41 import sys
42 import warnings
43 from pathlib import Path
44
45 warnings.filterwarnings("ignore")
46 if hasattr(sys.stdout, "reconfigure"):
47     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
48
49 sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
50
51 from app.db import get_conn, init_db
52
53
54 # Default 3-class layout matches pseudo_labels.DEFAULT_CLASS_MAP.
55 CLASS_NAMES = ["mass", "calcification", "asymmetry"]
56
57
58 # -----
59 # Sources: DB rows or pre-exported JSONL
60 # -----
61
62
63 def _rows_from_db(include: list[str]):
64     init_db()
65     placeholders = ",".join("?" * len(include))
66     with get_conn() as c:
67         return [dict(r) for r in c.execute(
68             f"SELECT sop_uid, dicom_path, preprocessed_png_path, view, laterality, "
69             f"      final_bboxes_json, status, reviewer "
70             f"FROM review_decisions WHERE status IN ({placeholders})",
71             include,
72         ).fetchall()]
73
74
75 def _rows_from_jsonl(path: Path):
76     rows = []
77     for line in path.read_text(encoding="utf-8").splitlines():
78         if not line.strip():
79             continue
80         try:
81             d = json.loads(line)

```

```

82         except json.JSONDecodeError:
83             continue
84         rows.append({
85             "sop_uid": d.get("sop_uid", ""),
86             "dicom_path": d.get("dicom_path", ""),
87             "preprocessed_png_path": d.get("preprocessed_png", ""),
88             "view": d.get("view", ""),
89             "laterality": d.get("laterality", ""),
90             "final_bboxes_json": json.dumps(d.get("bboxes", [])),
91             "status": d.get("status", ""),
92             "reviewer": d.get("reviewer", ""),
93         })
94     return rows
95
96
97 # -----
98 # YOLO label writer (final_bboxes are pixel coords on a target_size canvas)
99 # -----
100
101
102 def _bbox_to_yolo(bbox: dict, target_size: int) -> str | None:
103     try:
104         x0 = float(bbox["x0"]); y0 = float(bbox["y0"])
105         x1 = float(bbox["x1"]); y1 = float(bbox["y1"])
106         cls = int(bbox.get("cls", 0))
107     except (KeyError, TypeError, ValueError):
108         return None
109     if x1 <= x0 or y1 <= y0:
110         return None
111     cx = (x0 + x1) / 2.0 / target_size
112     cy = (y0 + y1) / 2.0 / target_size
113     w = (x1 - x0) / target_size
114     h = (y1 - y0) / target_size
115     cx = max(0.0, min(1.0, cx)); cy = max(0.0, min(1.0, cy))
116     w = max(0.0, min(1.0, w)); h = max(0.0, min(1.0, h))
117     if w <= 0 or h <= 0:
118         return None
119     return f"{cls} {cx:.6f} {cy:.6f} {w:.6f} {h:.6f}"
120
121
122 def _write_label(path: Path, bboxes: list[dict], target_size: int) -> int:
123     path.parent.mkdir(parents=True, exist_ok=True)
124     n = 0
125     with path.open("w", encoding="utf-8") as f:
126         for b in bboxes:
127             line = _bbox_to_yolo(b, target_size)
128             if line:
129                 f.write(line + "\n"); n += 1
130     return n
131
132
133 # -----
134 # Patient-level split – keys on the SOP UID's study root so two views of the
135 # same study stay together.
136 # -----
137
138
139 def _study_key(sop_uid: str) -> str:
140     """Use the SOP UID without its final dotted segment (instance suffix) as
141     a study-level grouping key. If nothing parses, fall back to the full UID.
142     """
143     if not sop_uid:
144         return ""
145     parts = sop_uid.rsplit(".", 1)
146     return parts[0] if len(parts) == 2 else sop_uid
147

```

```

148
149 def _assign_split(study_keys: list[str], val_frac: float, seed: int) -> dict[str, str]:
150     rng = random.Random(seed)
151     keys = sorted(set(study_keys))
152     rng.shuffle(keys)
153     n_val = max(1, int(round(len(keys) * val_frac))) if keys else 0
154     val_set = set(keys[:n_val])
155     return {k: ("val" if k in val_set else "train") for k in keys}
156
157
158 # -----
159 # Main
160 # -----
161
162
163 def _safe_basename(row: dict) -> str:
164     """Stable, filesystem-safe filename derived from SOP UID (full UIDs can
165     exceed Windows path limits, so we hash long ones)."""
166     sop = row.get("sop_uid") or ""
167     if not sop:
168         return hashlib.sha1((row.get("preprocessed_png_path") or "").encode()).hexdigest()[:16]
169     safe = sop.replace(".", "_")
170     if len(safe) <= 80:
171         return safe
172     return safe[:40] + "_" + hashlib.sha1(sop.encode()).hexdigest()[:16]
173
174
175 def main():
176     ap = argparse.ArgumentParser(description="Gold-label → YOLO dataset converter")
177     ap.add_argument("--out", required=True, help="Destination YOLO dataset folder")
178     ap.add_argument("--include", default="accepted,edited",
179                     help="Comma-separated review statuses to include")
180     ap.add_argument("--target-size", type=int, default=1024,
181                     help="Image size assumed for bbox normalisation (must match preprocess)")
182     ap.add_argument("--val-frac", type=float, default=0.15)
183     ap.add_argument("--seed", type=int, default=42)
184     ap.add_argument("--copy", action="store_true",
185                     help="Copy PNGs (default: try symlink, fall back to copy on Windows)")
186     ap.add_argument("--single-class", action="store_true",
187                     help="Collapse all classes into one 'lesion' class")
188     ap.add_argument("--jsonl", default=None,
189                     help="Pre-exported JSONL (from /api/research/review/export); skips DB read")
190     args = ap.parse_args()
191
192     out_root = Path(args.out).resolve()
193     out_root.mkdir(parents=True, exist_ok=True)
194
195     include = [s.strip() for s in args.include.split(",") if s.strip()]
196     if args.jsonl:
197         rows = _rows_from_jsonl(Path(args.jsonl))
198         print(f"[load] {len(rows)} rows from {args.jsonl}")
199     else:
200         rows = _rows_from_db(include)
201         print(f"[load] {len(rows)} rows from DB (status in {include})")
202
203     if not rows:
204         raise SystemExit("no gold-label rows found – did the radiologist review anything?")
205
206     # Plan train/val split first.
207     study_keys = [_study_key(r["sop_uid"]) for r in rows]
208     split_map = _assign_split(study_keys, args.val_frac, args.seed)
209
210     n_ok = n_skip_no_png = n_skip_no_bbox = n_train = n_val = 0
211     for r in rows:
212         png_src = (r.get("preprocessed_png_path") or "").strip()
213         if not png_src or not Path(png_src).exists():

```

```

214         n_skip_no_png += 1
215         continue
216     try:
217         bboxes = json.loads(r.get("final_bboxes_json") or "[]")
218     except json.JSONDecodeError:
219         bboxes = []
220     if not bboxes:
221         n_skip_no_bbox += 1
222         continue
223
224     if args.single_class:
225         for b in bboxes:
226             b["cls"] = 0
227
228     split = split_map.get(_study_key(r["sop_uid"]), "train")
229     base = _safe_basename(r)
230     img_dst = out_root / "images" / split / f"{base}.png"
231     lbl_dst = out_root / "labels" / split / f"{base}.txt"
232     img_dst.parent.mkdir(parents=True, exist_ok=True)
233
234     # Try symlink first (fast, no duplication); fall back to copy.
235     if not img_dst.exists():
236         try:
237             if args.copy:
238                 raise OSError("copy mode forced")
239             img_dst.symlink_to(Path(png_src).resolve())
240         except (OSError, NotImplementedError):
241             shutil.copy2(png_src, img_dst)
242
243     n_written = _write_label(lbl_dst, bboxes, args.target_size)
244     if n_written == 0:
245         n_skip_no_bbox += 1
246         img_dst.unlink(missing_ok=True)
247         continue
248     n_ok += 1
249     if split == "train":
250         n_train += 1
251     else:
252         n_val += 1
253
254     names = ["lesion"] if args.single_class else CLASS_NAMES
255     yaml_path = out_root / "dataset.yaml"
256     yaml_path.write_text(
257         f"# Auto-generated by app.research.gold_to_yolo\n"
258         f"# Source: review_decisions table (statuses={include})\n"
259         f"path: {out_root.as_posix()}\n"
260         f"train: images/train\n"
261         f"val: images/val\n"
262         f"test: images/val\n"
263         f"nc: {len(names)}\n"
264         f"names: {names}\n",
265         encoding="utf-8",
266     )
267
268     print(f"\n[done] wrote {n_ok} samples (train={n_train}, val={n_val})")
269     if n_skip_no_png:
270         print(f"  skipped {n_skip_no_png} - preprocessed PNG missing")
271     if n_skip_no_bbox:
272         print(f"  skipped {n_skip_no_bbox} - no usable bboxes after conversion")
273     print(f"  dataset.yaml → {yaml_path}")
274
275
276 if __name__ == "__main__":
277     main()

```


app/research/load_review_queue.py

Fayl yo'li: app/research/load_review_queue.py | Qatorlar: 120 | Hajm: 4397 bayt

```
1  """Load a `review_queue.jsonl` (produced by app.research.pseudo_labels) into
2  the project SQLite DB so radiologists can verify the pseudo-bboxes through
3  the MAMOGRAF UI.
4
5  Each JSONL line becomes one row in `review_decisions` with status='pending'.
6  Re-running with the same source-queue is idempotent – existing rows
7  (matched by `(sop_uid, source_queue)`) are skipped, not duplicated.
8
9  Usage:
10     python -m app.research.load_review_queue \\\
11         --queue      runs/full/local_pseudo/review_queue.jsonl \\\
12         --preproc-out runs/full/local_preprocessed
13 """
14 from __future__ import annotations
15
16 import argparse
17 import json
18 import sys
19 import warnings
20 from datetime import datetime, timezone
21 from pathlib import Path
22
23 warnings.filterwarnings("ignore")
24 if hasattr(sys.stdout, "reconfigure"):
25     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
26
27 import sqlite3
28
29 # Touch the project's init_db so the table exists if this is the first call.
30 sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
31 from app.db import get_conn, init_db
32
33
34 def _read_jsonl(path: Path):
35     for line in path.read_text(encoding="utf-8").splitlines():
36         if not line.strip():
37             continue
38         try:
39             yield json.loads(line)
40         except json.JSONDecodeError:
41             continue
42
43
44 def _resolve_dicom_path(sop_uid: str, manifest_path: Path | None) -> str | None:
45     """Look up the source DICOM path from the preprocess manifest by sop_uid."""
46     if manifest_path is None or not manifest_path.exists():
47         return None
48     for d in _read_jsonl(manifest_path):
49         if d.get("sop_uid") == sop_uid:
50             return d.get("src_path")
51     return None
52
53
54 def main():
55     ap = argparse.ArgumentParser(description="Ingest review_queue.jsonl into review_decisions")
56     ap.add_argument("--queue", required=True, help="Path to review_queue.jsonl")
57     ap.add_argument("--preproc-out", default=None,
58                     help="Path to preprocessing output dir (used to resolve PNG + DICOM paths)")
59     ap.add_argument("--source-tag", default=None,
60                     help="Identifier for this queue (default: queue file basename)")
61     args = ap.parse_args()
62
```

```

63     queue_path = Path(args.queue).resolve()
64     if not queue_path.exists():
65         raise SystemExit(f"queue file not found: {queue_path}")
66
67     preproc_out = Path(args.preproc_out).resolve() if args.preproc_out else None
68     manifest_path = (preproc_out / "manifest.jsonl") if preproc_out else None
69     source_tag = args.source_tag or queue_path.name
70
71     init_db()
72     now = datetime.now(timezone.utc).isoformat()
73     n_inserted = n_skipped = n_failed = 0
74
75     with get_conn() as c:
76         for entry in _read_jsonl(queue_path):
77             sop_uid = entry.get("sop_uid") or ""
78             if not sop_uid:
79                 n_failed += 1
80                 continue
81
82             png_rel = entry.get("image") or ""
83             png_abs = ""
84             if preproc_out and png_rel:
85                 cand = (preproc_out / png_rel).resolve()
86                 if cand.exists():
87                     png_abs = str(cand)
88
89             dicom_path = _resolve_dicom_path(sop_uid, manifest_path) or ""
90
91             try:
92                 c.execute(
93                     "INSERT INTO review_decisions ("
94                     " sop_uid, dicom_path, preprocessed_png_path, view, laterality,"
95                     " findings_json, pseudo_bboxes_json, status, imported_at, source_queue"
96                     ") VALUES (?, ?, ?, ?, ?, ?, ?, 'pending', ?, ?)",
97                     (
98                         sop_uid,
99                         dicom_path,
100                        png_abs,
101                        entry.get("view", "") or "",
102                        entry.get("laterality", "") or "",
103                        json.dumps(entry.get("findings") or {}, ensure_ascii=False),
104                        json.dumps(entry.get("bboxes") or [], ensure_ascii=False),
105                        now,
106                        source_tag,
107                    ),
108                 )
109                 n_inserted += 1
110             except sqlite3.IntegrityError:
111                 n_skipped += 1
112         c.commit()
113
114     print(f"[done] inserted={n_inserted} skipped(existing)={n_skipped} failed={n_failed}")
115     print(f"[source_queue] = {source_tag!r}")
116     print(f"[next] open MAMOGRAF UI →  Review modal to verify each item")
117
118
119 if __name__ == "__main__":
120     main()

```

app/research/preprocess.py

Fayl yo'li: app/research/preprocess.py | Qatorlar: 484 | Hajm: 16718 bayt

```
1 """Mammography DICOM preprocessing pipeline.
```

```

2
3 Implements the 9-step pipeline used in the breast lesion detection project:
4
5     1. Modality LUT      (RescaleSlope / RescaleIntercept)
6     2. VOI LUT          (WindowCenter / WindowWidth → 8-bit)
7     3. Photometric inv. (MONOCHROME1 → MONOCHROME2)
8     4. Breast segment.  (Otsu + largest connected component)
9     5. Crop              (tight bbox around breast)
10    6. Pectoral removal  (MLO views: Hough line + triangle mask)
11    7. CLAHE              (clipLimit=2.0, tileGrid=8x8)
12    8. Letterbox resize  (1024x1024, square pad)
13    9. Save PNG + manifest entry (view, laterality, breast_bbox, spacing)
14
15 Usage (CLI):
16     python -m app.research.preprocess --input <dicom_root> --output <png_root>
17
18 Programmatic:
19     from app.research.preprocess import preprocess_mammogram
20     img8, meta = preprocess_mammogram(Path("study/file.dcm"), target_size=1024)
21 """
22 from __future__ import annotations
23
24 import argparse
25 import json
26 import sys
27 import warnings
28 from dataclasses import dataclass, asdict
29 from pathlib import Path
30 from typing import Optional
31
32 warnings.filterwarnings("ignore")
33 if hasattr(sys.stdout, "reconfigure"):
34     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
35
36 import numpy as np
37 import pydicom
38 from PIL import Image
39 from pydicom.uid import ImplicitVRLittleEndian
40
41 try:
42     from pydicom.pixels.processing import apply_modality_lut, apply_voi_lut
43 except ImportError:
44     from pydicom.pixel_data_handlers.util import apply_modality_lut, apply_voi_lut
45
46 import cv2
47 from scipy import ndimage as ndi
48
49
50 # -----
51 # Data containers
52 # -----
53
54
55 @dataclass
56 class PreprocessMeta:
57     """Metadata describing a preprocessed mammogram."""
58     src_path: str
59     out_path: str
60     patient_id: str
61     study_uid: str
62     series_uid: str
63     sop_uid: str
64     laterality: str      # "L" / "R" / ""
65     view: str            # "CC" / "MLO" / ""
66     photometric: str     # "MONOCHROME1" / "MONOCHROME2"
67     rows_orig: int

```

```

68     cols_orig: int
69     pixel_spacing: tuple[float, float] | None # (row, col) mm
70     breast_bbox: tuple[int, int, int, int] # (x0, y0, x1, y1) in original coords
71     pectoral_removed: bool
72     target_size: int
73     flipped_to_left: bool # True if R-laterality was horizontally flipped
74
75
76 # -----
77 # DICOM helpers (mirror app.dicom_utils style)
78 # -----
79
80
81 def _ensure_transfer_syntax(ds: pydicom.Dataset) -> None:
82     fm = getattr(ds, "file_meta", None)
83     if fm is None or not getattr(fm, "TransferSyntaxUID", None):
84         meta = pydicom.dataset.FileMetaDataset()
85         meta.TransferSyntaxUID = ImplicitVRLittleEndian
86         ds.file_meta = meta
87
88
89 def _read_dicom(path: Path) -> pydicom.Dataset:
90     ds = pydicom.dcmread(str(path), force=True)
91     _ensure_transfer_syntax(ds)
92     return ds
93
94
95 def _get_pixel_spacing(ds: pydicom.Dataset) -> Optional[tuple[float, float]]:
96     for attr in ("PixelSpacing", "ImagerPixelSpacing"):
97         v = getattr(ds, attr, None)
98         if v is None:
99             continue
100         try:
101             row, col = float(v[0]), float(v[1])
102             return (row, col)
103         except Exception:
104             continue
105     return None
106
107
108 def _str_attr(ds: pydicom.Dataset, key: str) -> str:
109     v = getattr(ds, key, "")
110     return str(v) if v is not None else ""
111
112
113 # -----
114 # Step 1+2: Modality + VOI LUT → float32 in [0, 1]
115 # -----
116
117
118 def _to_normalized_float(ds: pydicom.Dataset) -> tuple[np.ndarray, str]:
119     """Apply Modality LUT, then VOI LUT, return float32 in [0,1] and photometric."""
120     if "PixelData" not in ds:
121         raise ValueError("DICOM has no pixel data")
122
123     raw = ds.pixel_array
124     if raw.ndim == 3 and raw.shape[0] > 1 and raw.shape[-1] not in (3, 4):
125         raw = raw[0]
126     if raw.ndim == 3 and raw.shape[-1] in (3, 4):
127         raw = raw[..., 0]
128
129     try:
130         raw = apply_modality_lut(raw, ds)
131     except Exception:
132         pass
133

```

```

134     try:
135         windowed = apply_voi_lut(raw.astype(np.float32), ds, prefer_lut=True)
136     except Exception:
137         wc = getattr(ds, "WindowCenter", None)
138         ww = getattr(ds, "WindowWidth", None)
139         if isinstance(wc, pydicom.multival.MultiValue):
140             wc = wc[0]
141         if isinstance(ww, pydicom.multival.MultiValue):
142             ww = ww[0]
143         try:
144             wc = float(wc); ww = float(ww)
145         except Exception:
146             wc, ww = None, None
147         if wc is None or ww is None or ww <= 0:
148             lo, hi = float(np.min(raw)), float(np.max(raw))
149             wc = (lo + hi) / 2.0
150             ww = max(hi - lo, 1.0)
151         lo = wc - ww / 2.0
152         hi = wc + ww / 2.0
153         windowed = np.clip(raw.astype(np.float32), lo, hi)
154
155     arr = windowed.astype(np.float32)
156     lo, hi = float(np.min(arr)), float(np.max(arr))
157     if hi <= lo:
158         hi = lo + 1.0
159     norm = (arr - lo) / (hi - lo)
160     return norm, str(getattr(ds, "PhotometricInterpretation", "MONOCHROME2"))
161
162
163 # -----
164 # Step 4: Breast segmentation (Otsu + largest connected component)
165 # -----
166
167
168 def _segment_breast(img01: np.ndarray) -> tuple[np.ndarray, tuple[int, int, int, int]]:
169     """Return (binary_mask, bbox=(x0,y0,x1,y1)) for the breast region."""
170     u8 = (img01 * 255).astype(np.uint8)
171     blur = cv2.GaussianBlur(u8, (5, 5), 0)
172     _, mask = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
173     kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (15, 15))
174     mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
175     mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
176
177     labels, n = ndi.label(mask > 0)
178     if n == 0:
179         h, w = img01.shape
180         return np.ones_like(img01, dtype=np.uint8), (0, 0, w, h)
181     sizes = ndi.sum(mask > 0, labels, index=range(1, n + 1))
182     biggest = int(np.argmax(sizes)) + 1
183     breast = (labels == biggest).astype(np.uint8)
184
185     breast = ndi.binary_fill_holes(breast).astype(np.uint8)
186     ys, xs = np.where(breast > 0)
187     if ys.size == 0:
188         h, w = img01.shape
189         return breast, (0, 0, w, h)
190     x0, x1 = int(xs.min()), int(xs.max()) + 1
191     y0, y1 = int(ys.min()), int(ys.max()) + 1
192     return breast, (x0, y0, x1, y1)
193
194
195 # -----
196 # Step 6: Pectoral muscle removal (MLO views only)
197 # -----
198
199

```

```

200 def _remove_pectoral(img01: np.ndarray, side: str) -> tuple[np.ndarray, bool]:
201     """Erase the pectoral muscle triangle on MLO views.
202
203     Strategy: the pectoral muscle is the brightest triangular region in the
204     upper-left (left-laterality) or upper-right (right-laterality) corner.
205     We Hough-detect the dominant straight edge and zero out the triangle
206     above it, but only if its slope is plausible (avoids false positives on
207     CC views or heterogeneously dense breasts).
208     """
209     h, w = img01.shape
210     side = (side or "L").upper()
211
212     crop_w = max(int(w * 0.45), 32)
213     crop_h = max(int(h * 0.55), 32)
214     if side == "L":
215         roi = (img01[:crop_h, :crop_w] * 255).astype(np.uint8)
216         x_off, y_off = 0, 0
217     else:
218         roi = (img01[:crop_h, w - crop_w:] * 255).astype(np.uint8)
219         x_off, y_off = w - crop_w, 0
220
221     edges = cv2.Canny(roi, 30, 100)
222     lines = cv2.HoughLines(edges, 1, np.pi / 180, threshold=int(min(crop_h, crop_w) * 0.4))
223     if lines is None:
224         return img01, False
225
226     best = None
227     for rho, theta in lines[:, 0]:
228         deg = np.degrees(theta)
229         if side == "L":
230             ok = 100.0 < deg < 170.0
231         else:
232             ok = 10.0 < deg < 80.0
233         if ok:
234             best = (rho, theta)
235             break
236     if best is None:
237         return img01, False
238
239     rho, theta = best
240     a, b = np.cos(theta), np.sin(theta)
241     if abs(b) < 1e-6:
242         return img01, False
243
244     out = img01.copy()
245     yy, xx = np.indices(roi.shape)
246     line_y = (rho - a * xx) / b
247     if side == "L":
248         keep = yy >= line_y
249     else:
250         keep = yy >= line_y
251     erase = ~keep
252     sub = out[y_off:y_off + crop_h, x_off:x_off + crop_w]
253     sub[erase] = 0.0
254     out[y_off:y_off + crop_h, x_off:x_off + crop_w] = sub
255     return out, True
256
257
258 # -----
259 # Step 7: CLAHE
260 # -----
261
262
263 def _apply_clahe(img01: np.ndarray, clip: float = 2.0, tile: int = 8) -> np.ndarray:
264     u8 = (np.clip(img01, 0, 1) * 255).astype(np.uint8)
265     clahe = cv2.createCLAHE(clipLimit=clip, tileGridSize=(tile, tile))

```

```

266     out = clahe.apply(u8)
267     return out.astype(np.float32) / 255.0
268
269
270 # -----
271 # Step 8: Letterbox resize to a square
272 # -----
273
274
275 def _letterbox(img01: np.ndarray, size: int = 1024, pad: float = 0.0) -> np.ndarray:
276     h, w = img01.shape
277     if h == 0 or w == 0:
278         return np.full((size, size), pad, dtype=np.float32)
279     scale = min(size / w, size / h)
280     new_w = max(1, int(round(w * scale)))
281     new_h = max(1, int(round(h * scale)))
282     resized = cv2.resize(img01, (new_w, new_h), interpolation=cv2.INTER_AREA)
283     canvas = np.full((size, size), pad, dtype=np.float32)
284     y0 = (size - new_h) // 2
285     x0 = (size - new_w) // 2
286     canvas[y0:y0 + new_h, x0:x0 + new_w] = resized
287     return canvas
288
289
290 # -----
291 # Top-level pipeline
292 # -----
293
294
295 def preprocess_mammogram(
296     dcm_path: Path,
297     target_size: int = 1024,
298     standardize_to_left: bool = True,
299     do_pectoral: bool = True,
300     do_clahe: bool = True,
301 ) -> tuple[np.ndarray, PreprocessMeta]:
302     """Run the full 9-step preprocessing pipeline on one DICOM file.
303
304     Returns:
305         img8 : uint8 HxW image of size target_sizextarget_size
306         meta : PreprocessMeta describing provenance
307     """
308     dcm_path = Path(dcm_path)
309     ds = _read_dicom(dcm_path)
310
311     laterality = _str_attr(ds, "ImageLaterality").upper()
312     view = _str_attr(ds, "ViewPosition").upper()
313     rows_orig = int(getattr(ds, "Rows", 0) or 0)
314     cols_orig = int(getattr(ds, "Columns", 0) or 0)
315     spacing = _get_pixel_spacing(ds)
316
317     # 1+2. Modality LUT + VOI LUT
318     img, photometric = _to_normalized_float(ds)
319
320     # 3. Photometric inversion
321     if photometric == "MONOCHROME1":
322         img = 1.0 - img
323
324     # 4+5. Breast segmentation + crop
325     mask, bbox = _segment_breast(img)
326     x0, y0, x1, y1 = bbox
327     img_crop = img[y0:y1, x0:x1] * mask[y0:y1, x0:x1]
328
329     # Standardize laterality so all breasts face the same direction (left).
330     flipped = False
331     if standardize_to_left and laterality == "R":

```

```

332     img_crop = np.fliplr(img_crop)
333     flipped = True
334     side_for_pectoral = "L" if (laterality != "R" or flipped) else "R"
335
336     # 6. Pectoral muscle removal (MLO only)
337     pectoral_removed = False
338     if do_pectoral and "MLO" in view:
339         img_crop, pectoral_removed = _remove_pectoral(img_crop, side_for_pectoral)
340
341     # 7. CLAHE
342     if do_clahe:
343         img_crop = _apply_clahe(img_crop, clip=2.0, tile=8)
344
345     # 8. Letterbox to square
346     canvas = _letterbox(img_crop, size=target_size, pad=0.0)
347     img8 = (np.clip(canvas, 0, 1) * 255).astype(np.uint8)
348
349     meta = PreprocessMeta(
350         src_path=str(dcm_path),
351         out_path="",
352         patient_id=_str_attr(ds, "PatientID"),
353         study_uid=_str_attr(ds, "StudyInstanceUID"),
354         series_uid=_str_attr(ds, "SeriesInstanceUID"),
355         sop_uid=_str_attr(ds, "SOPInstanceUID"),
356         laterality=laterality,
357         view=view,
358         photometric=photometric,
359         rows_orig=rows_orig,
360         cols_orig=cols_orig,
361         pixel_spacing=spacing,
362         breast_bbox=(int(x0), int(y0), int(x1), int(y1)),
363         pectoral_removed=pectoral_removed,
364         target_size=target_size,
365         flipped_to_left=flipped,
366     )
367     return img8, meta
368
369
370 # -----
371 # BBox transform (map original DICOM coords → preprocessed image coords)
372 # -----
373
374
375 def transform_bbox(
376     bbox_orig: tuple[int, int, int, int],
377     meta: PreprocessMeta,
378 ) -> tuple[float, float, float, float] | None:
379     """Map a bbox from original DICOM coordinates to preprocessed image coords.
380
381     Mirrors the geometric transforms in preprocess_mammogram: crop → flip → letterbox.
382     Returns (x0, y0, x1, y1) in pixels of the preprocessed canvas, or None if the
383     bbox falls outside the breast crop.
384     """
385     bx0, by0, bx1, by1 = meta.breast_bbox
386     crop_w = max(bx1 - bx0, 1)
387     crop_h = max(by1 - by0, 1)
388
389     x0, y0, x1, y1 = bbox_orig
390     # 1. Translate into breast-crop frame
391     x0 -= bx0; x1 -= bx0
392     y0 -= by0; y1 -= by0
393     # 2. Clip to crop
394     x0 = max(0, min(x0, crop_w)); x1 = max(0, min(x1, crop_w))
395     y0 = max(0, min(y0, crop_h)); y1 = max(0, min(y1, crop_h))
396     if x1 <= x0 or y1 <= y0:
397         return None

```



```

398 # 3. Horizontal flip (R → L)
399 if meta.flipped_to_left:
400     x0, x1 = crop_w - x1, crop_w - x0
401 # 4. Letterbox scale + pad
402 s = meta.target_size
403 scale = min(s / crop_w, s / crop_h)
404 new_w = crop_w * scale
405 new_h = crop_h * scale
406 pad_x = (s - new_w) / 2.0
407 pad_y = (s - new_h) / 2.0
408 return (
409     x0 * scale + pad_x,
410     y0 * scale + pad_y,
411     x1 * scale + pad_x,
412     y1 * scale + pad_y,
413 )
414
415
416 # -----
417 # Batch CLI
418 # -----
419
420
421 def _iter_dicoms(root: Path):
422     for p in root.rglob("*"):
423         if p.is_file() and p.suffix.lower() in (".dcm", ".dicom", ""):
424             yield p
425
426
427 def _save_png(img8: np.ndarray, out_path: Path) -> None:
428     out_path.parent.mkdir(parents=True, exist_ok=True)
429     Image.fromarray(img8, mode="L").save(out_path, format="PNG", optimize=False, compress_level=3)
430
431
432 def main():
433     ap = argparse.ArgumentParser(description="Mammography DICOM preprocessing")
434     ap.add_argument("--input", required=True, help="Folder of DICOM files (recursive)")
435     ap.add_argument("--output", required=True, help="Folder to write PNGs")
436     ap.add_argument("--size", type=int, default=1024, help="Target square size (default 1024)")
437     ap.add_argument("--no-pectoral", action="store_true")
438     ap.add_argument("--no-clahe", action="store_true")
439     ap.add_argument("--no-flip", action="store_true",
440                     help="Disable R→L laterality standardization")
441     ap.add_argument("--manifest", default="manifest.jsonl",
442                     help="Manifest filename inside output folder (default manifest.jsonl)")
443     args = ap.parse_args()
444
445     in_root = Path(args.input).resolve()
446     out_root = Path(args.output).resolve()
447     out_root.mkdir(parents=True, exist_ok=True)
448
449     if not in_root.exists():
450         raise SystemExit(f"input folder not found: {in_root}")
451
452     manifest_path = out_root / args.manifest
453     n_ok = n_fail = 0
454
455     with manifest_path.open("w", encoding="utf-8") as mf:
456         for dcm in _iter_dicoms(in_root):
457             try:
458                 img8, meta = preprocess_mammogram(
459                     dcm,
460                     target_size=args.size,
461                     standardize_to_left=not args.no_flip,
462                     do_pectoral=not args.no_pectoral,
463                     do_clahe=not args.no_clahe,

```

```

464         )
465     except Exception as e:
466         n_fail += 1
467         print(f"[skip] {dcm.name}: {e}")
468         continue
469
470     rel = dcm.relative_to(in_root).with_suffix(".png")
471     out_path = out_root / "images" / rel
472     _save_png(img8, out_path)
473     meta.out_path = str(out_path.relative_to(out_root)).replace("\\", "/")
474     mf.write(json.dumps(asdict(meta), ensure_ascii=False) + "\n")
475     n_ok += 1
476     if n_ok % 25 == 0:
477         print(f" ... {n_ok} ok, {n_fail} failed")
478
479     print(f"\n[done] {n_ok} preprocessed, {n_fail} failed")
480     print(f"[manifest] {manifest_path}")
481
482
483 if __name__ == "__main__":
484     main()

```

app/research/pseudo_labels.py

Fayl yo'li: [app/research/pseudo_labels.py](#) | Qatorlar: 586 | Hajm: 21878 bayt

```

1  """Text-guided pseudo-label generator for mammography lesion detection.
2
3  When local DICOMs lack ground-truth bounding boxes but DO have radiology
4  reports, this module mines the reports for laterality / quadrant / clock /
5  size cues and synthesises *weak* bounding boxes on the preprocessed image.
6  The bboxes are saved as YOLO labels with a `confidence` field in a sidecar
7  manifest so a radiologist can later verify them in the MAMOGRAF UI.
8
9  Languages handled:
10     - Uzbek (Cyrillic)
11     - Uzbek (Latin)
12     - Russian
13     - English (literature dumps)
14
15  Pipeline:
16     1. `extract_findings(text)` runs all multilingual regexes and returns a
17        structured `Findings` object.
18     2. `findings_to_bboxes(findings, meta)` projects the findings onto the
19        preprocessed image (using `PreprocessMeta` from `preprocess.py`).
20     3. CLI: reads the project SQLite DB, joins DICOM links → records, runs
21        the preprocessing manifest, and emits a YOLO-format `pseudo_labels/`
22        dataset plus a verification queue JSONL.
23
24  Spatial conventions (after R→L standardisation in preprocess_mammogram):
25     - LEFT side of image = chest wall (posterior)
26     - RIGHT side of image = nipple (anterior)
27     - TOP of image = upper / superior (in MLO views)
28     - BOTTOM of image = lower / inferior (in MLO views)
29     - In CC views the upper/lower information is collapsed; we still place
30       the bbox using the same y-axis heuristic but flag it as low-confidence.
31
32  These pseudo-labels are deliberately *coarse*: typical bbox is ~25% of the
33  breast crop. They exist to bootstrap a fine-tuning stage and to seed the
34  radiologist verification UI – not to be trained on directly without review.
35  """
36  from __future__ import annotations
37
38  import argparse

```

```

39 import json
40 import re
41 import sys
42 import warnings
43 from dataclasses import dataclass, field, asdict
44 from pathlib import Path
45 from typing import Iterable
46
47 warnings.filterwarnings("ignore")
48 if hasattr(sys.stdout, "reconfigure"):
49     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
50
51 import numpy as np
52
53 from app.research.preprocess import PreprocessMeta
54
55
56 # -----
57 # 1. Regex patterns (multilingual)
58 # -----
59
60
61 # Laterality
62 # Note: trailing word boundaries are intentionally relaxed because Uzbek
63 # (locative -da, ablative -dan, ...) and Russian (genitive/dative endings) attach
64 # productive suffixes onto the lemma. We anchor the *start* of the keyword and
65 # let inflection trail.
66 _LAT_RIGHT = re.compile(
67     r"\b("
68     r"right\s+breast|"
69     r"o['']?ng\s+(?:sut\s+bezi|ko['']?krak)\w*|" # Uzbek-Latin
70     r"o['']?ng\s+tomon\w*|"
71     r"ў?нг\s+(?:сут\s+бези|кўкрак)\w*|у?нг\s+сут\s+бези\w*|" # Uzbek-Cyr
72     r"ў?нг\s+тараф\w*|у?нг\s+тараф\w*|"
73     r"правая?\s+(?:молочная|грудь)\w*|" # Russian
74     r"справа|правой\s+молочн\w*"
75     r")",
76     re.IGNORECASE | re.UNICODE,
77 )
78 _LAT_LEFT = re.compile(
79     r"\b("
80     r"left\s+breast|"
81     r"chap\s+(?:sut\s+bezi|ko['']?krak)\w*|"
82     r"chap\s+tomon\w*|"
83     r"чап\s+(?:сут\s+бези|кўкрак)\w*|чап\s+сут\s+бези\w*|"
84     r"чап\s+тараф\w*|"
85     r"лева[йй]\s+(?:молочная|грудь)\w*|"
86     r"слева|левой\s+молочн\w*"
87     r")",
88     re.IGNORECASE | re.UNICODE,
89 )
90
91 # Quadrants. Abbreviations are common (UOQ = upper outer quadrant) plus
92 # spelled-out forms in three languages.
93 _QUAD_PATTERNS = {
94     "UOQ": re.compile(
95         r"\b("
96         r"UOQ|upper[-\s]?outer\s+quadrant|"
97         r"yuqori[-\s]?tashqi(?:\s+kvadrant)?|"
98         r"юкори[-\s]?ташқи(?:\s+квадрант)?|юкори[-\s]?ташки|"
99         r"верхне[-\s]?наружн(?:ый|ом|ого)(?:\s+квадрант)?"
100         r")\b", re.IGNORECASE | re.UNICODE),
101     "UIQ": re.compile(
102         r"\b("
103         r"UIQ|upper[-\s]?inner\s+quadrant|"
104         r"yuqori[-\s]?ichki(?:\s+kvadrant)?|"

```

```

105     r"юкори[-\s]?ички(?:\s+квадрант)?|юкори[-\s]?ички|"
106     r"верхне[-\s]?внутренн(?:ий|ем|его)(?:\s+квадрант)?"
107     r")\b", re.IGNORECASE | re.UNICODE),
108     "LOQ": re.compile(
109         r"\b("
110         r"LOQ|lower[-\s]?outer\s+quadrant|"
111         r"pastki[-\s]?tashqi(?:\s+kvadrant)?|"
112         r"пастки[-\s]?ташки(?:\s+квадрант)?|пастки[-\s]?ташки|"
113         r"нижне[-\s]?наружн(?:ый|ом|ого)(?:\s+квадрант)?"
114         r")\b", re.IGNORECASE | re.UNICODE),
115     "LIQ": re.compile(
116         r"\b("
117         r"LIQ|lower[-\s]?inner\s+quadrant|"
118         r"pastki[-\s]?ichki(?:\s+kvadrant)?|"
119         r"пастки[-\s]?ички(?:\s+квадрант)?|"
120         r"нижне[-\s]?внутренн(?:ий|ем|его)(?:\s+квадрант)?"
121         r")\b", re.IGNORECASE | re.UNICODE),
122     "central": re.compile(
123         r"\b("
124         r"central|sub[-\s]?areolar|retro[-\s]?areolar|"
125         r"markaziy|марказий|"
126         r"центральный(?:ый|ом|ого)|"
127         r"za\s+aerolasi|за\s+ареолой"
128         r")\b", re.IGNORECASE | re.UNICODE),
129     "axillary_tail": re.compile(
130         r"\b("
131         r"axillary\s+tail|aksillyar(?:\s+dum)?|"
132         r"аксилляр(?:ная|ной)|подмышечн(?:ый|ой)\s+(?:отрост|хвост)|"
133         r"qo[']?']?ltiq\s+osti|кўлтиқ\s+ости"
134         r")\b", re.IGNORECASE | re.UNICODE),
135 }
136
137 # Clock position: "X o'clock", "соат X да", "soat X da", "на X часах"
138 _CLOCK_RE = re.compile(
139     r"(?:"
140     r"(?:at\s+|на\s+|do\s+|ra|ra\s+d{1,2}\s+мин)?"
141     r"(?:soat|соат)\s+(?P<c1>d{1,2})|"
142     r"\b(?:P<c2>d{1,2})\s*(?:o[']?']?\s*clock|часа?х?|час[аов]?)\b|"
143     r"(?:position\s+|positsiya\s+|позици[я]\s+)(?P<c3>d{1,2})"
144     r")",
145     re.IGNORECASE | re.UNICODE,
146 )
147
148 # Size: "20x15 mm", "12 mm", "1.5 sm", "2 cm"
149 _SIZE_RE = re.compile(
150     r"(?P<a>d+(?:[.,]\d+)?)\s*(?:[xxx]\s*(?P<b>d+(?:[.,]\d+)?))?"
151     r"\s*(?P<unit>mm|мм|sm|cm|cm)\b",
152     re.IGNORECASE | re.UNICODE,
153 )
154
155 # Distance from nipple
156 _DIST_NIPPLE_RE = re.compile(
157     r"(\d+(?:[.,]\d+)?)\s*(mm|мм|sm|cm|cm)\s+(?:от|nipple|emchak|эмчак|"
158     r"so[']?']?rg[']?']?ich|сўғич)",
159     re.IGNORECASE | re.UNICODE,
160 )
161
162 # Lesion type → class
163 _LESION_PATTERNS = {
164     "calcification": re.compile(
165         r"\b("
166         r"calcification\w*|microcalcif\w*|"
167         r"mikrokalts\w*|mikrokalsi\w*|kal[']?']?siy\s+to[']?']?plama\w*|"
168         r"микрoкальцин\w*|кальцификат\w*|кальциноз\w*|кальцин\w*"
169         r")",
170         re.IGNORECASE | re.UNICODE),

```

```

171 "asymmetry": re.compile(
172     r"\b("
173     r"asymmetr\w*|architectural\s+distortion|"
174     r"asimetri\w*|tuzilma\s+buzilish\w*|"
175     r"асимметрии\w*|нарушение\s+архитектоник\w*"
176     r")",
177     re.IGNORECASE | re.UNICODE),
178 "mass": re.compile(
179     r"\b("
180     r"mass\b|lesion\w*|tumou?r\w*|nodul\w*|"
181     r"hosil[ая]? \w*|massa\b|o['']?simta\w*|"
182     r"хосил[ая]? \w*|масс[аы]\b|ўсимта\w*|"
183     r"образован\w*|опухол\w*|узел\w*|узлов\w*"
184     r")",
185     re.IGNORECASE | re.UNICODE),
186 }
187
188
189 # -----
190 # 2. Findings dataclass
191 # -----
192
193
194 @dataclass
195 class Findings:
196     laterality: str = "" # "L" / "R" / ""
197     quadrant: str = "" # one of {"UOQ", "UIQ", "LOQ", "LIQ", "central", "axillary_tail", ""}
198     clock: int | None = None # 1..12 or None
199     lesion_types: list[str] = field(default_factory=list)
200     size_mm: tuple[float, float] | None = None # (long, short) in mm
201     distance_from_nipple_mm: float | None = None
202     raw_hits: dict = field(default_factory=dict)
203
204
205 def _to_mm(value: str, unit: str) -> float:
206     v = float(value.replace(",", "."))
207     u = unit.lower()
208     if u in ("sm", "cm", "cm"):
209         return v * 10.0
210     return v
211
212
213 def extract_findings(text: str) -> Findings:
214     """Run all multilingual regexes on `text` and return structured Findings."""
215     f = Findings()
216     if not text:
217         return f
218     t = text
219
220     # Laterality (a single report can mention both – prefer the first hit; if
221     # both are present we still mark which appears earliest, others added to raw).
222     r_match = _LAT_RIGHT.search(t)
223     l_match = _LAT_LEFT.search(t)
224     if r_match and (not l_match or r_match.start() < l_match.start()):
225         f.laterality = "R"
226     elif l_match:
227         f.laterality = "L"
228     f.raw_hits["lat_right"] = bool(r_match)
229     f.raw_hits["lat_left"] = bool(l_match)
230
231     # Quadrant
232     for name, pat in _QUAD_PATTERNS.items():
233         if pat.search(t):
234             f.quadrant = name
235             break
236

```

```

237     # Clock
238     m = _CLOCK_RE.search(t)
239     if m:
240         for k in ("c1", "c2", "c3"):
241             v = m.group(k)
242             if v:
243                 try:
244                     n = int(v)
245                 except ValueError:
246                     continue
247                 if 1 <= n <= 12:
248                     f.clock = n
249                     break
250
251     # Lesion types (multi-label)
252     for cls, pat in _LESION_PATTERNS.items():
253         if pat.search(t):
254             f.lesion_types.append(cls)
255
256     # Size
257     m = _SIZE_RE.search(t)
258     if m:
259         a = _to_mm(m.group("a"), m.group("unit"))
260         b_raw = m.group("b")
261         b = _to_mm(b_raw, m.group("unit")) if b_raw else a
262         f.size_mm = (max(a, b), min(a, b))
263
264     # Distance from nipple
265     m = _DIST_NIPPLE_RE.search(t)
266     if m:
267         f.distance_from_nipple_mm = _to_mm(m.group(1), m.group(2))
268
269     return f
270
271
272 # -----
273 # 3. Spatial mapper: Findings × PreprocessMeta → bboxes
274 # -----
275
276
277 # Lesion class → YOLO class id (must match dataset.yaml)
278 DEFAULT_CLASS_MAP = {
279     "mass": 0,
280     "calcification": 1,
281     "asymmetry": 2,
282 }
283
284
285 def _quadrant_center(quadrant: str, view: str) -> tuple[float, float]:
286     """Return (x_frac, y_frac) of the quadrant centre inside the breast crop.
287
288     After R→L standardisation: image x∈[0,1] maps chest-wall→nipple,
289     y∈[0,1] maps top→bottom. Outer = far from chest wall = high x.
290     """
291     if "MLO" in view:
292         # Y-axis = upper/lower (well-defined); X-axis = posterior/anterior
293         mapping = {
294             "UOQ": (0.70, 0.30),
295             "UIQ": (0.30, 0.30),
296             "LOQ": (0.70, 0.70),
297             "LIQ": (0.30, 0.70),
298             "central": (0.55, 0.50),
299             "axillary_tail": (0.20, 0.20), # toward chest wall + upper
300         }
301     else:
302         # CC view: Y-axis is medial/lateral (collapsed). Heuristic: place

```

```

303     # outer at the lateral side (top), inner at medial (bottom). Without a
304     # second view this is ambiguous, so we keep the bbox large.
305     mapping = {
306         "UOQ": (0.65, 0.30),
307         "UIQ": (0.65, 0.70),
308         "LOQ": (0.40, 0.30),
309         "LIQ": (0.40, 0.70),
310         "central": (0.55, 0.50),
311         "axillary_tail": (0.20, 0.40),
312     }
313     return mapping.get(quadrant, (0.55, 0.50))
314
315
316 def _clock_to_xy(clock: int, view: str) -> tuple[float, float]:
317     """Clock face -> (x_frac, y_frac) inside the breast crop.
318
319     Convention (post-R->L): nipple at right edge, chest wall at left.
320     Clock face is centred on the nipple. 12 -> straight up (toward image top).
321     For LEFT-side conventions (which is what we see post-flip), 3 o'clock ->
322     medial (toward chest wall), 9 o'clock -> lateral (away from body).
323     """
324     angle = (clock % 12) * (np.pi / 6.0) - np.pi / 2.0 # 12 o'clock -> top
325     cx, cy = 0.85, 0.50 # rotate around the nipple area
326     radius = 0.30
327     x = cx + radius * np.cos(angle)
328     if "MLO" in view:
329         y = cy + radius * np.sin(angle)
330     else:
331         # CC view collapses superior/inferior; halve the y excursion.
332         y = cy + radius * np.sin(angle) * 0.5
333     return float(np.clip(x, 0.05, 0.95)), float(np.clip(y, 0.05, 0.95))
334
335
336 def _size_to_pixels(
337     size_mm: tuple[float, float] | None,
338     meta: PreprocessMeta,
339     fallback_frac: float = 0.20,
340 ) -> tuple[float, float]:
341     """Convert a real-world lesion size in mm to pixels on the letterboxed image."""
342     s = meta.target_size
343     if size_mm is None or meta.pixel_spacing is None:
344         side = s * fallback_frac
345         return side, side
346
347     # Scale factor: we cropped to breast bbox then letterboxed to sxs.
348     bx0, by0, bx1, by1 = meta.breast_bbox
349     crop_w = max(bx1 - bx0, 1)
350     crop_h = max(by1 - by0, 1)
351     scale = min(s / crop_w, s / crop_h)
352     px_per_mm_x = scale / max(meta.pixel_spacing[1], 1e-3)
353     px_per_mm_y = scale / max(meta.pixel_spacing[0], 1e-3)
354
355     long_mm, short_mm = size_mm
356     w_px = long_mm * px_per_mm_x * 1.4 # 40% margin (the bbox encloses, not equals)
357     h_px = short_mm * px_per_mm_y * 1.4
358     # Clamp to a sensible range
359     w_px = float(np.clip(w_px, s * 0.05, s * 0.50))
360     h_px = float(np.clip(h_px, s * 0.05, s * 0.50))
361     return w_px, h_px
362
363
364 @dataclass
365 class WeakBBox:
366     cls: int
367     x0: float
368     y0: float

```

```

369     x1: float
370     y1: float
371     confidence: float          # 0..1, our self-rating of how trustworthy this label is
372     rationale: str            # human-readable: "MLO + UOQ + size 20mm"
373
374
375 def _confidence_score(f: Findings, view: str) -> float:
376     """Heuristic 0-1 score for how reliable the synthesised bbox is."""
377     s = 0.30                    # base
378     if f.quadrant: s += 0.20
379     if f.clock is not None: s += 0.15
380     if f.size_mm is not None: s += 0.15
381     if f.distance_from_nipple_mm is not None: s += 0.10
382     if "MLO" in view: s += 0.10          # MLO localisation is less ambiguous
383     return float(min(s, 0.95))
384
385
386 def findings_to_bboxes(
387     findings: Findings,
388     meta: PreprocessMeta,
389     class_map: dict[str, int] | None = None,
390 ) -> list[WeakBBox]:
391     """Synthesise weak bboxes from parsed Findings + PreprocessMeta."""
392     class_map = class_map or DEFAULT_CLASS_MAP
393     if not findings.lesion_types and findings.quadrant:
394         findings.lesion_types = ["mass"]    # default class when only location is mentioned
395     if not findings.lesion_types:
396         return []
397
398     # Verify laterality matches the image; if not, this report is about the
399     # other breast and we should not place a bbox here.
400     if findings.laterality and findings.laterality != meta.laterality:
401         return []
402
403     s = meta.target_size
404     if findings.clock is not None:
405         cx_frac, cy_frac = _clock_to_xy(findings.clock, meta.view)
406         rationale_loc = f'clock={findings.clock}"
407     elif findings.quadrant:
408         cx_frac, cy_frac = _quadrant_center(findings.quadrant, meta.view)
409         rationale_loc = f"quadrant={findings.quadrant}"
410     else:
411         cx_frac, cy_frac = 0.55, 0.50
412         rationale_loc = "default-center"
413
414     w_px, h_px = _size_to_pixels(findings.size_mm, meta)
415     cx, cy = cx_frac * s, cy_frac * s
416     x0 = float(np.clip(cx - w_px / 2, 0, s))
417     y0 = float(np.clip(cy - h_px / 2, 0, s))
418     x1 = float(np.clip(cx + w_px / 2, 0, s))
419     y1 = float(np.clip(cy + h_px / 2, 0, s))
420     if x1 <= x0 + 4 or y1 <= y0 + 4:
421         return []
422
423     conf = _confidence_score(findings, meta.view)
424     rationale = f"{meta.view or '?'} | {rationale_loc} | size_mm={findings.size_mm} | conf={conf:.2f}"
425
426     out: list[WeakBBox] = []
427     for cls_name in findings.lesion_types:
428         cls_id = class_map.get(cls_name)
429         if cls_id is None:
430             continue
431         out.append(WeakBBox(
432             cls=cls_id, x0=x0, y0=y0, x1=x1, y1=y1,
433             confidence=conf, rationale=f"{cls_name}: {rationale}",
434         ))

```



```

435     return out
436
437
438 # -----
439 # 4. CLI: project DB + preprocess manifest → YOLO pseudo-labels
440 # -----
441
442
443 def _load_text_from_db(db_path: Path, sop_uid: str) -> str:
444     """Look up a record's clinical text by SOP UID via dicom_patient_links → records.
445
446     Falls back to PatientID matching if no SOP-level link exists.
447     Returns concatenated free-text fields (complaints + history + clinical_findings + report).
448     """
449     import sqlite3
450     if not db_path.exists():
451         return ""
452     con = sqlite3.connect(str(db_path))
453     con.row_factory = sqlite3.Row
454     try:
455         # Try SOP-UID join via dicom_patient_links if such a column exists
456         try:
457             row = con.execute(
458                 "SELECT r.complaints, r.history, r.clinical_findings, r.report, r.diagnosis "
459                 "FROM dicom_patient_links l "
460                 "JOIN records r ON r.patient_id = l.patient_id "
461                 "WHERE l.sop_uid = ?",
462                 (sop_uid,),
463             ).fetchone()
464         except sqlite3.OperationalError:
465             row = None
466         if row is None:
467             return ""
468         parts = [row[k] for k in ("complaints", "history", "clinical_findings",
469                                 "report", "diagnosis") if row[k]]
470         return "\n".join(parts)
471     finally:
472         con.close()
473
474
475 def _read_preprocess_manifest(manifest_path: Path) -> Iterable[PreprocessMeta]:
476     for line in manifest_path.read_text(encoding="utf-8").splitlines():
477         if not line.strip():
478             continue
479         d = json.loads(line)
480         if d.get("pixel_spacing") and isinstance(d["pixel_spacing"], list):
481             d["pixel_spacing"] = tuple(d["pixel_spacing"])
482         if d.get("breast_bbox") and isinstance(d["breast_bbox"], list):
483             d["breast_bbox"] = tuple(d["breast_bbox"])
484         yield PreprocessMeta(**d)
485
486
487 def _write_yolo_label(path: Path, bboxes: list[WeakBBox], img_size: int) -> None:
488     path.parent.mkdir(parents=True, exist_ok=True)
489     with path.open("w", encoding="utf-8") as f:
490         for b in bboxes:
491             cx = (b.x0 + b.x1) / 2.0 / img_size
492             cy = (b.y0 + b.y1) / 2.0 / img_size
493             w = (b.x1 - b.x0) / img_size
494             h = (b.y1 - b.y0) / img_size
495             if w <= 0 or h <= 0:
496                 continue
497             f.write(f"{b.cls} {cx:.6f} {cy:.6f} {w:.6f} {h:.6f}\n")
498
499
500 def main():

```

```

501 ap = argparse.ArgumentParser(description="Text-guided pseudo-label generator")
502 ap.add_argument("--manifest", required=True,
503                 help="JSONL produced by app.research.preprocess (rows of PreprocessMeta)")
504 ap.add_argument("--images-root", required=True,
505                 help="Folder where preprocess.py wrote images/")
506 ap.add_argument("--db", default="app/db.sqlite3",
507                 help="SQLite DB to source clinical text from")
508 ap.add_argument("--out", required=True,
509                 help="Destination folder for the pseudo-label YOLO dataset")
510 ap.add_argument("--text-csv", default=None,
511                 help="Optional CSV with columns [sop_uid, text]; bypasses DB lookup")
512 ap.add_argument("--min-confidence", type=float, default=0.50,
513                 help="Skip labels below this self-rated confidence")
514 ap.add_argument("--single-class", action="store_true",
515                 help="Collapse mass/calcification/asymmetry into one class")
516 args = ap.parse_args()
517
518 manifest_path = Path(args.manifest).resolve()
519 images_root = Path(args.images_root).resolve()
520 out_root = Path(args.out).resolve()
521 db_path = Path(args.db).resolve()
522 out_root.mkdir(parents=True, exist_ok=True)
523
524 text_lookup: dict[str, str] = {}
525 if args.text_csv:
526     import csv
527     with open(args.text_csv, "r", encoding="utf-8", newline="") as f:
528         for row in csv.DictReader(f):
529             if row.get("sop_uid") and row.get("text"):
530                 text_lookup[row["sop_uid"]] = row["text"]
531
532 class_map = {"mass": 0} if args.single_class else dict(DEFAULT_CLASS_MAP)
533 review_path = out_root / "review_queue.jsonl"
534 n_labelled = n_empty = n_filtered = 0
535
536 with review_path.open("w", encoding="utf-8") as rf:
537     for meta in _read_preprocess_manifest(manifest_path):
538         text = text_lookup.get(meta.sop_uid) or _load_text_from_db(db_path, meta.sop_uid)
539         findings = extract_findings(text)
540         if not findings.lesion_types and not findings.quadrant and findings.clock is None:
541             n_empty += 1
542             continue
543
544         bboxes_all = findings_to_bboxes(findings, meta, class_map=class_map)
545         bboxes = [b for b in bboxes_all if b.confidence >= args.min_confidence]
546         if not bboxes:
547             n_filtered += 1
548             continue
549
550         img_rel = Path(meta.out_path).with_suffix(".png").name
551         label_path = out_root / "labels" / img_rel.replace(".png", ".txt")
552         _write_yolo_label(label_path, bboxes, meta.target_size)
553
554         rf.write(json.dumps({
555             "sop_uid": meta.sop_uid,
556             "image": meta.out_path,
557             "view": meta.view,
558             "laterality": meta.laterality,
559             "findings": asdict(findings),
560             "bboxes": [asdict(b) for b in bboxes],
561             "needs_radiologist_review": True,
562         }, ensure_ascii=False) + "\n")
563         n_labelled += 1
564         if n_labelled % 50 == 0:
565             print(f" ... {n_labelled} labelled, {n_empty} empty, {n_filtered} below conf")
566

```

```

567     names = ["lesion"] if args.single_class else ["mass", "calcification", "asymmetry"]
568     yaml_path = out_root / "dataset.yaml"
569     yaml_path.write_text(
570         f"# Auto-generated by app.research.pseudo_labels\n"
571         f"# Pseudo-labels – REQUIRES radiologist verification before training!\n"
572         f"path: {out_root.as_posix()}\n"
573         f"train: {images_root.as_posix()}\n"
574         f"nc: {len(names)}\n"
575         f"names: {names}\n",
576         encoding="utf-8",
577     )
578     print(f"\n[done] {n_labelled} labelled, {n_empty} no findings, {n_filtered} below confidence")
579     print(f"  review queue : {review_path}")
580     print(f"  dataset.yaml : {yaml_path}")
581     print(f"\nNext step: open the MAMOGRAF UI, run a verification pass on")
582     print(f"the review queue, then re-export with high-confidence-only labels.")
583
584
585 if __name__ == "__main__":
586     main()

```

app/research/run_pipeline.py

Fayl yo'li: `app/research/run_pipeline.py` | Qatorlar: 599 | Hajm: 24665 bayt

```

1  """End-to-end mammography lesion-detection pipeline runner.
2
3  Orchestrates the six research stages we built as one resumable, idempotent
4  command. Each stage runs as a subprocess (isolated failure, full logs
5  captured), and each stage skips itself if its sentinel output already
6  exists (use --force to override per-stage).
7
8  Stages:
9      0 preprocess      DICOM (local)      → preprocessed PNG + manifest
10     1 cbis_convert     CBIS-DDSM root     → YOLO-format dataset
11     2a train_yolo      cbis_yolo/dataset.yaml → YOLOv8 baseline + FROC
12     2b train_tillnet0  cbis_yolo + (img-only) → stage-1 TILLNet on CBIS
13     3 pseudo_labels   local manifest + DB → weak YOLO labels + review queue
14     3b load_review     review_queue.jsonl → review_decisions DB rows (pending)
15     4 train_tillnet1   stage1 ckpt + pseudo → stage-2 multimodal TILLNet
16     --- HUMAN STEP: radiologist reviews via MAMOGRAF UI; statuses flip to ---
17     --- accepted/edited/rejected. Re-run from `gold_to_yolo` afterwards. ---
18     4b gold_to_yolo    review_decisions DB → gold-label YOLO dataset
19     4c train_tillnet2  stage2 ckpt + gold → stage-3 fine-tune
20     5 summarise        all FROC summaries → run.summary.{json,md}
21
22  Every stage's output goes under <out_root>/, with one log file per stage
23  under <out_root>/logs/. Re-running with the same flags resumes where the
24  last invocation stopped.
25
26  Example:
27      python -m app.research.run_pipeline \\\
28          --local-dicoms /data/MAMOGRAF/uploads \\\
29          --cbis-root /data/CBIS-DDSM \\\
30          --texts-csv /data/MAMOGRAF/reports.csv \\\
31          --out-root runs/full \\\
32          --gpu 0 \\\
33          --imgsz 1024 \\\
34          --epochs-stage1 100 \\\
35          --epochs-stage2 30
36  """
37  from __future__ import annotations
38
39  import argparse

```

```

40 import json
41 import os
42 import subprocess
43 import sys
44 import time
45 import warnings
46 from dataclasses import dataclass, field, asdict
47 from pathlib import Path
48 from typing import Callable
49
50 warnings.filterwarnings("ignore")
51 if hasattr(sys.stdout, "reconfigure"):
52     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
53
54
55 # -----
56 # Config
57 # -----
58
59
60 @dataclass
61 class PipelineCfg:
62     out_root: Path
63     local_dicoms: Path | None
64     cbis_root: Path | None
65     texts_csv: Path | None
66     db_path: Path
67     imgsz: int = 1024
68     batch: int = 8
69     epochs_stage1: int = 100
70     epochs_stage2: int = 30
71     gpu: str = "0"
72     yolo_model: str = "yolov8m.pt"
73     workers: int = 4
74     skip: set[str] = field(default_factory=set)
75     only: set[str] = field(default_factory=set)
76     force: set[str] = field(default_factory=set)
77     dry_run: bool = False
78     min_pseudo_conf: float = 0.5
79
80     @property
81     def logs_dir(self) -> Path:
82         return self.out_root / "logs"
83
84     @property
85     def preprocessed_dir(self) -> Path:
86         return self.out_root / "local_preprocessed"
87
88     @property
89     def cbis_yolo_dir(self) -> Path:
90         return self.out_root / "cbis_yolo"
91
92     @property
93     def pseudo_dir(self) -> Path:
94         return self.out_root / "local_pseudo"
95
96     @property
97     def runs_dir(self) -> Path:
98         return self.out_root / "runs"
99
100     @property
101     def gold_yolo_dir(self) -> Path:
102         return self.out_root / "gold_yolo"
103
104
105 # -----

```

```

106 # Stage runner – subprocess with tee'd log
107 # -----
108
109
110 def _run(cmd: list[str], log_path: Path, dry_run: bool) -> int:
111     """Run a subprocess, streaming stdout to console + log file."""
112     log_path.parent.mkdir(parents=True, exist_ok=True)
113     print(f"\n[$] {' '.join(str(c) for c in cmd)}")
114     if dry_run:
115         return 0
116     t0 = time.time()
117     with log_path.open("w", encoding="utf-8") as logf:
118         logf.write(f"# {' '.join(str(c) for c in cmd)}\n")
119         logf.flush()
120     try:
121         proc = subprocess.Popen(
122             cmd, stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
123             text=True, encoding="utf-8", errors="replace", bufsize=1,
124         )
125     except FileNotFoundError as e:
126         print(f"[err] {e}")
127         return 127
128     for line in proc.stdout:
129         sys.stdout.write(line)
130         logf.write(line)
131         logf.flush()
132     rc = proc.wait()
133     dt = time.time() - t0
134     print(f"[done] rc={rc} ({dt:.1f}s) log → {log_path}")
135     return rc
136
137
138 def _exists_nonempty(path: Path) -> bool:
139     if not path.exists():
140         return False
141     if path.is_file():
142         return path.stat().st_size > 0
143     # directory: at least one file inside
144     for _ in path.rglob("*"):
145         return True
146     return False
147
148
149 def _python() -> str:
150     """Return the current Python executable so children inherit our venv."""
151     return sys.executable or "python"
152
153
154 # -----
155 # Stages
156 # -----
157
158
159 class Pipeline:
160     def __init__(self, cfg: PipelineCfg):
161         self.cfg = cfg
162         cfg.out_root.mkdir(parents=True, exist_ok=True)
163         cfg.logs_dir.mkdir(parents=True, exist_ok=True)
164
165     def _device_arg(self) -> list[str]:
166         return ["--device", self.cfg.gpu] if self.cfg.gpu else []
167
168     # 0 -----
169     def stage_preprocess(self) -> None:
170         cfg = self.cfg
171         manifest = cfg.preprocessed_dir / "manifest.jsonl"

```

```

172     if not cfg.local_dicoms:
173         print("[skip] preprocess: --local-dicoms not provided")
174         return
175     if _exists_nonempty(manifest) and "preprocess" not in cfg.force:
176         print(f"[skip] preprocess: {manifest} already populated")
177         return
178     rc = _run(
179         [_python(), "-m", "app.research.preprocess",
180          "--input", str(cfg.local_dicoms),
181          "--output", str(cfg.preprocessed_dir),
182          "--size", str(cfg.imgsz)],
183         cfg.logs_dir / "0_preprocess.log",
184         cfg.dry_run,
185     )
186     if rc != 0:
187         raise RuntimeError(f"preprocess stage failed (rc={rc})")
188
189     # 1 -----
190     def stage_cbis_convert(self) -> None:
191         cfg = self.cfg
192         yaml_path = cfg.cbis_yolo_dir / "dataset.yaml"
193         if not cfg.cbis_root:
194             print("[skip] cbis_convert: --cbis-root not provided")
195             return
196         if _exists_nonempty(yaml_path) and "cbis_convert" not in cfg.force:
197             print(f"[skip] cbis_convert: {yaml_path} exists")
198             return
199         rc = _run(
200             [_python(), "-m", "app.research.datasets.cbis_ddsm",
201              "--root", str(cfg.cbis_root),
202              "--out", str(cfg.cbis_yolo_dir),
203              "--size", str(cfg.imgsz)],
204             cfg.logs_dir / "1_cbis_convert.log",
205             cfg.dry_run,
206         )
207         if rc != 0:
208             raise RuntimeError(f"cbis_convert stage failed (rc={rc})")
209
210     # 2a -----
211     def stage_train_yolo(self) -> None:
212         cfg = self.cfg
213         run_dir = cfg.runs_dir / "cbis_yolo_baseline"
214         best = run_dir / "weights" / "best.pt"
215         if not (cfg.cbis_yolo_dir / "dataset.yaml").exists():
216             print("[skip] train_yolo: cbis dataset.yaml missing")
217             return
218         if _exists_nonempty(best) and "train_yolo" not in cfg.force:
219             print(f"[skip] train_yolo: {best} exists")
220             return
221         rc = _run(
222             [_python(), "-m", "app.research.train_detector",
223              "--data", str(cfg.cbis_yolo_dir / "dataset.yaml"),
224              "--model", cfg.yolo_model,
225              "--epochs", str(cfg.epochs_stage1),
226              "--imgsz", str(cfg.imgsz),
227              "--batch", str(cfg.batch),
228              "--workers", str(cfg.workers),
229              "--project", str(cfg.runs_dir),
230              "--name", "cbis_yolo_baseline",
231              *self._device_arg()],
232             cfg.logs_dir / "2a_train_yolo.log",
233             cfg.dry_run,
234         )
235         if rc != 0:
236             raise RuntimeError(f"train_yolo stage failed (rc={rc})")
237

```

```

238 # 2b -----
239 def stage_train_tillnet0(self) -> None:
240     cfg = self.cfg
241     run_dir = cfg.runs_dir / "cbis_tillnet_imgonly"
242     best = run_dir / "best.pt"
243     if not (cfg.cbis_yolo_dir / "dataset.yaml").exists():
244         print("[skip] train_tillnet0: cbis dataset.yaml missing")
245         return
246     if _exists_nonempty(best) and "train_tillnet0" not in cfg.force:
247         print(f"[skip] train_tillnet0: {best} exists")
248         return
249     # Stage-1: train image-only on CBIS (no text branch).
250     rc = _run(
251         [_python(), "-m", "app.research.train_tillnet",
252          "--data", str(cfg.cbis_yolo_dir / "dataset.yaml"),
253          "--epochs", str(cfg.epochs_stage1),
254          "--imgsz", str(cfg.imgsz),
255          "--batch", str(cfg.batch),
256          "--workers", str(cfg.workers),
257          "--device", cfg.gpu or "cpu",
258          "--out", str(run_dir),
259          "--no-text"],
260         cfg.logs_dir / "2b_train_tillnet0.log",
261         cfg.dry_run,
262     )
263     if rc != 0:
264         raise RuntimeError(f"train_tillnet0 stage failed (rc={rc})")
265
266 # 3 -----
267 def stage_pseudo_labels(self) -> None:
268     cfg = self.cfg
269     manifest = cfg.preprocessed_dir / "manifest.jsonl"
270     review = cfg.pseudo_dir / "review_queue.jsonl"
271     if not _exists_nonempty(manifest):
272         print("[skip] pseudo_labels: preprocessed manifest missing")
273         return
274     if _exists_nonempty(review) and "pseudo_labels" not in cfg.force:
275         print(f"[skip] pseudo_labels: {review} exists")
276         return
277     cmd = [_python(), "-m", "app.research.pseudo_labels",
278            "--manifest", str(manifest),
279            "--images-root", str(cfg.preprocessed_dir / "images"),
280            "--db", str(cfg.db_path),
281            "--out", str(cfg.pseudo_dir),
282            "--min-confidence", str(cfg.min_pseudo_conf)]
283     if cfg.texts_csv:
284         cmd += ["--text-csv", str(cfg.texts_csv)]
285     rc = _run(cmd, cfg.logs_dir / "3_pseudo_labels.log", cfg.dry_run)
286     if rc != 0:
287         raise RuntimeError(f"pseudo_labels stage failed (rc={rc})")
288
289 # 4 -----
290 def stage_train_tillnet1(self) -> None:
291     cfg = self.cfg
292     prev_best = cfg.runs_dir / "cbis_tillnet_imgonly" / "best.pt"
293     run_dir = cfg.runs_dir / "local_tillnet_stage2"
294     best = run_dir / "best.pt"
295     if not _exists_nonempty(prev_best):
296         print("[skip] train_tillnet1: stage-1 best.pt missing")
297         return
298     if not (cfg.pseudo_dir / "dataset.yaml").exists():
299         print("[skip] train_tillnet1: pseudo dataset.yaml missing")
300         return
301     if _exists_nonempty(best) and "train_tillnet1" not in cfg.force:
302         print(f"[skip] train_tillnet1: {best} exists")
303         return

```

```

304 cmd = [_python(), "-m", "app.research.train_tillnet",
305         "--data", str(cfg.pseudo_dir / "dataset.yaml"),
306         "--epochs", str(cfg.epochs_stage2),
307         "--imgsz", str(cfg.imgsz),
308         "--batch", str(cfg.batch),
309         "--workers", str(cfg.workers),
310         "--device", cfg.gpu or "cpu",
311         "--out", str(run_dir),
312         "--resume", str(prev_best),
313         "--lr", "5e-5"]
314 if cfg.texts_csv:
315     cmd += ["--texts", str(cfg.texts_csv)]
316 rc = _run(cmd, cfg.logs_dir / "4_train_tillnet1.log", cfg.dry_run)
317 if rc != 0:
318     raise RuntimeError(f"train_tillnet1 stage failed (rc={rc})")
319
320 # 3b -----
321 def stage_load_review(self) -> None:
322     """Ingest the pseudo_labels/review_queue.jsonl into review_decisions DB."""
323     cfg = self.cfg
324     queue = cfg.pseudo_dir / "review_queue.jsonl"
325     if not _exists_nonempty(queue):
326         print("[skip] load_review: pseudo_dir/review_queue.jsonl missing")
327         return
328     # Idempotency: this stage is itself idempotent at the loader level
329     # (UNIQUE constraint on (sop_uid, source_queue)). We always re-invoke
330     # so newly-generated queue rows get loaded – `--force` is not needed.
331     rc = _run(
332         [_python(), "-m", "app.research.load_review_queue",
333          "--queue", str(queue),
334          "--preproc-out", str(cfg.preprocessed_dir),
335          "--source-tag", f"pipeline_{cfg.out_root.name}"],
336         cfg.logs_dir / "3b_load_review.log",
337         cfg.dry_run,
338     )
339     if rc != 0:
340         raise RuntimeError(f"load_review stage failed (rc={rc})")
341
342 # 4b -----
343 def stage_gold_to_yolo(self) -> None:
344     """Convert radiologist-verified rows into a YOLO dataset."""
345     cfg = self.cfg
346     yaml_path = cfg.gold_yolo_dir / "dataset.yaml"
347     if _exists_nonempty(yaml_path) and "gold_to_yolo" not in cfg.force:
348         print(f"[skip] gold_to_yolo: {yaml_path} exists")
349         return
350     rc = _run(
351         [_python(), "-m", "app.research.gold_to_yolo",
352          "--out", str(cfg.gold_yolo_dir),
353          "--target-size", str(cfg.imgsz),
354          "--include", "accepted,edited",
355          "--copy"],
356         cfg.logs_dir / "4b_gold_to_yolo.log",
357         cfg.dry_run,
358     )
359     if rc != 0:
360         print("[note] gold_to_yolo returned non-zero – likely no gold rows yet")
361         print("        (radiologist must finish reviewing through the MAMOGRAF UI)")
362
363 # 4c -----
364 def stage_train_tillnet2(self) -> None:
365     """Stage-3 fine-tune of TILLNet on radiologist-verified gold labels."""
366     cfg = self.cfg
367     prev_best = cfg.runs_dir / "local_tillnet_stage2" / "best.pt"
368     run_dir = cfg.runs_dir / "local_tillnet_stage3"
369     best = run_dir / "best.pt"

```



```

370     if not _exists_nonempty(prev_best):
371         print("[skip] train_tillnet2: stage-2 best.pt missing")
372         return
373     if not (cfg.gold_yolo_dir / "dataset.yaml").exists():
374         print("[skip] train_tillnet2: gold dataset.yaml missing")
375         return
376     if _exists_nonempty(best) and "train_tillnet2" not in cfg.force:
377         print(f"[skip] train_tillnet2: {best} exists")
378         return
379     cmd = [_python(), "-m", "app.research.train_tillnet",
380           "--data", str(cfg.gold_yolo_dir / "dataset.yaml"),
381           "--epochs", str(max(10, cfg.epochs_stage2 // 2)),      # shorter, lower LR
382           "--imgsz", str(cfg.imgsz),
383           "--batch", str(cfg.batch),
384           "--workers", str(cfg.workers),
385           "--device", cfg.gpu or "cpu",
386           "--out", str(run_dir),
387           "--resume", str(prev_best),
388           "--lr", "2e-5"]
389     if cfg.texts_csv:
390         cmd += ["--texts", str(cfg.texts_csv)]
391     rc = _run(cmd, cfg.logs_dir / "4c_train_tillnet2.log", cfg.dry_run)
392     if rc != 0:
393         raise RuntimeError(f"train_tillnet2 stage failed (rc={rc})")
394
395     # 5 -----
396     def stage_summarise(self) -> None:
397         cfg = self.cfg
398         out: dict = {"stages": {}, "config": {k: str(v) for k, v in asdict(cfg).items()}}
399
400         def _read_json(p: Path):
401             try:
402                 return json.loads(p.read_text(encoding="utf-8"))
403             except Exception:
404                 return None
405
406         def _last_history(p: Path):
407             try:
408                 lines = [l for l in p.read_text(encoding="utf-8").splitlines() if l.strip()]
409                 return json.loads(lines[-1]) if lines else None
410             except Exception:
411                 return None
412
413         out["stages"]["preprocess"] = {
414             "manifest_lines": sum(1 for _ in (cfg.preprocessed_dir / "manifest.jsonl").open(
415                 "r", encoding="utf-8")) if (cfg.preprocessed_dir / "manifest.jsonl").exists() else 0,
416         }
417         out["stages"]["cbis_convert"] = {
418             "exists": (cfg.cbis_yolo_dir / "dataset.yaml").exists(),
419         }
420
421         yolo_summary = _read_json(cfg.runs_dir / "cbis_yolo_baseline" / "froc_summary.json")
422         if yolo_summary:
423             out["stages"]["train_yolo"] = {
424                 "n_images": yolo_summary.get("n_images"),
425                 "sens_at_fp": yolo_summary.get("sens_at_fp"),
426                 "iou_threshold": yolo_summary.get("iou_threshold"),
427             }
428
429         tn0 = _last_history(cfg.runs_dir / "cbis_tillnet_imgonly" / "history.jsonl")
430         if tn0:
431             out["stages"]["train_tillnet0"] = {
432                 "epoch": tn0.get("epoch"),
433                 "val_sens_at_fp": (tn0.get("val") or {}).get("sens_at_fp"),
434                 "train_loss": (tn0.get("train") or {}).get("loss"),
435             }

```

```

436 tn1 = _last_history(cfg.runs_dir / "local_tillnet_stage2" / "history.jsonl")
437 if tn1:
438     out["stages"]["train_tillnet1"] = {
439         "epoch": tn1.get("epoch"),
440         "val_sens_at_fp": (tn1.get("val") or {}).get("sens_at_fp"),
441         "train_loss": (tn1.get("train") or {}).get("loss"),
442     }
443
444 tn2 = _last_history(cfg.runs_dir / "local_tillnet_stage3" / "history.jsonl")
445 if tn2:
446     out["stages"]["train_tillnet2"] = {
447         "epoch": tn2.get("epoch"),
448         "val_sens_at_fp": (tn2.get("val") or {}).get("sens_at_fp"),
449         "train_loss": (tn2.get("train") or {}).get("loss"),
450     }
451
452 out["stages"]["pseudo_labels"] = {
453     "review_queue_lines": sum(1 for _ in (cfg.pseudo_dir / "review_queue.jsonl").open(
454         "r", encoding="utf-8")) if (cfg.pseudo_dir / "review_queue.jsonl").exists() else 0,
455 }
456
457 # Review-decisions DB stats for radiologist progress tracking
458 try:
459     from app.db import get_conn
460     with get_conn() as c:
461         rows = c.execute(
462             "SELECT status, COUNT(*) AS n FROM review_decisions GROUP BY status"
463         ).fetchall()
464         out["stages"]["review_decisions"] = {r["status"]: r["n"] for r in rows}
465 except Exception:
466     pass
467
468 if (cfg.gold_yolo_dir / "dataset.yaml").exists():
469     n_train = len(list((cfg.gold_yolo_dir / "labels" / "train").glob("*.txt"))) \
470         if (cfg.gold_yolo_dir / "labels" / "train").exists() else 0
471     n_val = len(list((cfg.gold_yolo_dir / "labels" / "val").glob("*.txt"))) \
472         if (cfg.gold_yolo_dir / "labels" / "val").exists() else 0
473     out["stages"]["gold_to_yolo"] = {"train": n_train, "val": n_val}
474
475 json_path = cfg.out_root / "run.summary.json"
476 md_path = cfg.out_root / "run.summary.md"
477 json_path.write_text(json.dumps(out, indent=2, ensure_ascii=False), encoding="utf-8")
478
479 # Human-readable markdown summary
480 lines = ["# Pipeline run summary", "", "## Configuration", "```json",
481         json.dumps(out["config"], indent=2), "```", "", "## Stages", ""]
482 for name, st in out["stages"].items():
483     lines.append(f"### {name}")
484     lines.append("```json")
485     lines.append(json.dumps(st, indent=2, ensure_ascii=False))
486     lines.append("```")
487     lines.append("")
488 md_path.write_text("\n".join(lines), encoding="utf-8")
489 print(f"\n[summary] {json_path}")
490 print(f"[summary] {md_path}")
491
492
493
494 # -----
495 # CLI
496 # -----
497
498
499 _STAGES: dict[str, str] = {
500     "preprocess": "stage_preprocess",
501     "cbis_convert": "stage_cbis_convert",

```

```

502     "train_yolo":      "stage_train_yolo",
503     "train_tillnet0":  "stage_train_tillnet0",
504     "pseudo_labels":  "stage_pseudo_labels",
505     "load_review":     "stage_load_review",
506     "train_tillnet1":  "stage_train_tillnet1",
507     "gold_to_yolo":    "stage_gold_to_yolo",
508     "train_tillnet2":  "stage_train_tillnet2",
509     "summarise":       "stage_summarise",
510 }
511
512
513 def _comma(s: str) -> set[str]:
514     return {x.strip() for x in s.split(",") if x.strip()}
515
516
517 def main():
518     ap = argparse.ArgumentParser(description="End-to-end mammography research pipeline runner")
519     ap.add_argument("--out-root", required=True, help="Where all artifacts will be written")
520     ap.add_argument("--local-dicoms", default=None, help="Local DICOM root (recursive)")
521     ap.add_argument("--cbis-root", default=None, help="Extracted CBIS-DDSM root folder")
522     ap.add_argument("--texts-csv", default=None, help="CSV/JSONL of (filename,text) for TILLNet")
523     ap.add_argument("--db", default="app/db.sqlite3", help="SQLite DB for clinical text lookup")
524     ap.add_argument("--imgsz", type=int, default=1024)
525     ap.add_argument("--batch", type=int, default=8)
526     ap.add_argument("--epochs-stage1", type=int, default=100)
527     ap.add_argument("--epochs-stage2", type=int, default=30)
528     ap.add_argument("--gpu", default="0", help="GPU id (or 'cpu', or '' to omit)")
529     ap.add_argument("--yolo-model", default="yolov8m.pt")
530     ap.add_argument("--workers", type=int, default=4)
531     ap.add_argument("--min-pseudo-conf", type=float, default=0.5)
532     ap.add_argument("--skip", default="",
533                     help=f"Comma-separated stages to skip. Stages: {' '.join(_STAGES)}")
534     ap.add_argument("--only", default="",
535                     help="Comma-separated stages to run (overrides --skip)")
536     ap.add_argument("--force", default="",
537                     help="Comma-separated stages to re-run even if outputs exist")
538     ap.add_argument("--dry-run", action="store_true", help="Print commands without executing")
539     args = ap.parse_args()
540
541     cfg = PipelineCfg(
542         out_root=Path(args.out_root).resolve(),
543         local_dicoms=Path(args.local_dicoms).resolve() if args.local_dicoms else None,
544         cbis_root=Path(args.cbis_root).resolve() if args.cbis_root else None,
545         texts_csv=Path(args.texts_csv).resolve() if args.texts_csv else None,
546         db_path=Path(args.db).resolve(),
547         imgsz=args.imgsz, batch=args.batch,
548         epochs_stage1=args.epochs_stage1, epochs_stage2=args.epochs_stage2,
549         gpu=args.gpu, yolo_model=args.yolo_model, workers=args.workers,
550         skip=_comma(args.skip), only=_comma(args.only), force=_comma(args.force),
551         dry_run=args.dry_run, min_pseudo_conf=args.min_pseudo_conf,
552     )
553
554     bad = (cfg.skip | cfg.only | cfg.force) - set(_STAGES)
555     if bad:
556         raise SystemExit(f"unknown stage names: {sorted(bad)}; valid: {sorted(_STAGES)}")
557
558     pipeline = Pipeline(cfg)
559
560     print(f"[start] out_root = {cfg.out_root}")
561     print(f"          stages   = {list(_STAGES)}")
562     if cfg.only:
563         print(f"          only      = {sorted(cfg.only)}")
564     if cfg.skip:
565         print(f"          skip      = {sorted(cfg.skip)}")
566     if cfg.force:
567         print(f"          force     = {sorted(cfg.force)}")

```

```

568
569     n_run = n_skipped = n_failed = 0
570     failures: list[tuple[str, str]] = []
571     t_global = time.time()
572     for name, method_name in _STAGES.items():
573         if cfg.only and name not in cfg.only:
574             continue
575         if name in cfg.skip:
576             print(f"\n[skip] {name} (--skip)")
577             n_skipped += 1
578             continue
579         print(f"\n===== STAGE: {name} =====")
580         try:
581             getattr(pipeline, method_name)()
582             n_run += 1
583         except Exception as e:
584             print(f"[fail] {name}: {e}")
585             failures.append((name, str(e)))
586             n_failed += 1
587             # Continue downstream stages – they'll skip themselves if their inputs are missing.
588
589     dt = time.time() - t_global
590     print(f"\n[summary] ran={n_run} skipped={n_skipped} failed={n_failed} ({dt/60:.1f} min total)")
591     if failures:
592         print("[failed stages]")
593         for name, err in failures:
594             print(f"  - {name}: {err}")
595         raise SystemExit(1)
596
597
598 if __name__ == "__main__":
599     main()

```

app/research/stat_test.py

Fayl yo'li: app/research/stat_test.py | Qatorlar: 79 | Hajm: 2842 bayt

```

1  """Paired statistical significance test (XS-Classfier vs baselines)."""
2  from __future__ import annotations
3
4  import sys
5  import warnings
6  from pathlib import Path
7
8  warnings.filterwarnings("ignore")
9  if hasattr(sys.stdout, "reconfigure"):
10     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
11
12  import numpy as np
13  from sklearn.metrics import accuracy_score, f1_score
14  from sklearn.model_selection import StratifiedKFold
15  from scipy.stats import ttest_rel, wilcoxon
16
17  sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
18
19  from app.research.train_classifier import load_dataset
20  from app.models_arch.xs_classifier import (
21     make_xs_pipeline, make_word_only_baseline,
22     make_char_only_baseline, make_baseline_pipeline,
23 )
24
25
26 def fold_scores(pipeline, X, y, n_splits=5, seed=42):
27     skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)

```

```

28     accs, f1s = [], []
29     for tr, te in skf.split(X, y):
30         X_tr = [X[i] for i in tr]; X_te = [X[i] for i in te]
31         pipeline.fit(X_tr, y[tr])
32         y_p = pipeline.predict(X_te)
33         accs.append(accuracy_score(y[te], y_p))
34         f1s.append(f1_score(y[te], y_p))
35     return np.asarray(accs), np.asarray(f1s)
36
37
38 def main():
39     X, y, _ = load_dataset()
40     print(f"n={len(X)}, positives={int(y.sum())}\n")
41
42     print("Running 5-fold CV per model (deterministic seed)...")
43     xs_acc, xs_f1 = fold_scores(make_xs_pipeline(), X, y)
44     print(f"  XS-Classifer:          acc={xs_acc.mean():.4f}, f1={xs_f1.mean():.4f}")
45
46     word_acc, word_f1 = fold_scores(make_word_only_baseline(), X, y)
47     print(f"  Word-only:          acc={word_acc.mean():.4f}, f1={word_f1.mean():.4f}")
48
49     char_acc, char_f1 = fold_scores(make_char_only_baseline(), X, y)
50     print(f"  Char n-gram:        acc={char_acc.mean():.4f}, f1={char_f1.mean():.4f}")
51
52     print("\nPaired tests (XS-Classifer vs baselines, 5 folds):")
53     for label, base_acc, base_f1 in [
54         ("Word-only", word_acc, word_f1),
55         ("Char n-gram", char_acc, char_f1),
56     ]:
57         d_acc = xs_acc - base_acc
58         d_f1 = xs_f1 - base_f1
59         try:
60             t_acc = ttest_rel(xs_acc, base_acc)
61             t_f1 = ttest_rel(xs_f1, base_f1)
62             print(f"    vs {label}:")
63             print(f"      Δacc = {d_acc.mean()*100:+.3f}pp, "
64                   f"paired-t p={t_acc.pvalue:.4f}")
65             print(f"      Δf1  = {d_f1.mean()*100:+.3f}pp, "
66                   f"paired-t p={t_f1.pvalue:.4f}")
67         try:
68             w_acc = wilcoxon(xs_acc, base_acc)
69             w_f1 = wilcoxon(xs_f1, base_f1)
70             print(f"      Wilcoxon acc p={w_acc.pvalue:.4f}, "
71                   f"f1 p={w_f1.pvalue:.4f}")
72         except Exception:
73             pass
74     except Exception as e:
75         print(f"  test failed: {e}")
76
77
78 if __name__ == "__main__":
79     main()

```

app/research/train_classifier.py

Fayl yo'li: app/research/train_classifier.py | Qatorlar: 304 | Hajm: 10662 bayt

```

1 """
2 Train and evaluate XS-Classifer on the local mammography corpus.
3
4 Outputs (under paper/figures/):
5     metrics.json          - full per-model results
6     confusion_*.png       - confusion matrices
7     pr_curves.png         - precision-recall curves

```

```

8     roc_curves.png           - ROC curves
9     feature_importances.txt - top features per class
10
11 Usage:
12     python -m app.research.train_classifier
13 """
14 from __future__ import annotations
15
16 import json
17 import re
18 import sys
19 import warnings
20 from pathlib import Path
21
22 import numpy as np
23
24 warnings.filterwarnings("ignore", category=UserWarning)
25
26 if hasattr(sys.stdout, "reconfigure"):
27     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
28
29 import matplotlib
30
31 matplotlib.use("Agg")
32 import matplotlib.pyplot as plt
33 from sklearn.metrics import (
34     accuracy_score, average_precision_score, confusion_matrix,
35     f1_score, precision_recall_curve, precision_score, recall_score,
36     roc_auc_score, roc_curve,
37 )
38 from sklearn.model_selection import StratifiedKFold
39
40 ROOT = Path(__file__).resolve().parents[2]
41 sys.path.insert(0, str(ROOT))
42
43 from app.db import get_conn # noqa: E402
44 from app.models_arch.xs_classifier import ( # noqa: E402
45     make_xs_pipeline, make_word_only_baseline, make_char_only_baseline,
46     make_baseline_pipeline, XSConfig, split_by_script,
47 )
48
49
50 CANCER_PATTERNS = [
51     r"\bC\s*[\-\.\.]\s*50", r"C\s*[\-\.\.]\s*50",
52     r"\bs\s*[\-\.\.]\s*50",
53     r"\bsarat[oa]n", r"\бсарат[oa]н",
54     r"\брак\b", r"раков", r"раком", r"\braka?\b",
55     r"саратон", r"saraton",
56     r"karsinom", r"карцином",
57     r"\bM\s*8500/3\b", r"орух", r"опух",
58     r"malign", r"малигн",
59     r"neoplasm", r"новообраз",
60 ]
61 CANCER_RE = re.compile("|".join(CANCER_PATTERNS), re.IGNORECASE | re.UNICODE)
62
63
64 def detect_cancer(text: str) -> bool:
65     return bool(CANCER_RE.search(text)) if text else False
66
67
68 OUT = ROOT / "paper" / "figures"
69 OUT.mkdir(parents=True, exist_ok=True)
70
71
72 def load_dataset():
73     with get_conn() as c:

```

```

74     rows = [
75         dict(r) for r in c.execute(
76             "SELECT id, complaints, history, clinical_findings, "
77             "diagnosis, report FROM records"
78         ).fetchall()
79     ]
80     X, y, ids = [], [], []
81     for r in rows:
82         text = " ".join(filter(None, [
83             r.get("complaints") or "",
84             r.get("history") or "",
85             r.get("clinical_findings") or "",
86         ]))
87         text = text.strip()
88         if not text:
89             continue
90         target_text = " ".join(filter(None, [
91             r.get("diagnosis") or "",
92             r.get("report") or "",
93         ]))
94         is_cancer = detect_cancer(target_text)
95         X.append(text)
96         y.append(int(is_cancer))
97         ids.append(r["id"])
98     return X, np.asarray(y), ids
99
100
101 def script_summary(X):
102     cyr_dom = lat_dom = mixed = 0
103     for t in X:
104         cyr, lat = split_by_script(t)
105         cl, ll = len(cyr), len(lat)
106         if cl > 2 * ll:
107             cyr_dom += 1
108         elif ll > 2 * cl:
109             lat_dom += 1
110         else:
111             mixed += 1
112     return {"cyrillic_dominant": cyr_dom, "latin_dominant": lat_dom, "mixed": mixed}
113
114
115 def cv_evaluate(name, pipeline, X, y, n_splits=5, seed=42):
116     skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
117     accs, f1s, precs, recs, aurops, auprs = [], [], [], [], [], []
118     all_y_true, all_y_pred, all_y_score = [], [], []
119
120     for fold, (tr, te) in enumerate(skf.split(X, y)):
121         X_tr = [X[i] for i in tr]
122         X_te = [X[i] for i in te]
123         y_tr, y_te = y[tr], y[te]
124         pipeline.fit(X_tr, y_tr)
125         y_pred = pipeline.predict(X_te)
126         try:
127             y_score = pipeline.predict_proba(X_te)[:, 1]
128         except Exception:
129             y_score = pipeline.decision_function(X_te)
130         accs.append(accuracy_score(y_te, y_pred))
131         f1s.append(f1_score(y_te, y_pred))
132         precs.append(precision_score(y_te, y_pred))
133         recs.append(recall_score(y_te, y_pred))
134         try:
135             aurops.append(roc_auc_score(y_te, y_score))
136             auprs.append(average_precision_score(y_te, y_score))
137         except Exception:
138             pass
139     all_y_true.extend(y_te.tolist())

```

```

140     all_y_pred.extend(y_pred.tolist())
141     all_y_score.extend(np.asarray(y_score).tolist())
142
143     return {
144         "name": name,
145         "accuracy": (np.mean(accs), np.std(accs)),
146         "f1": (np.mean(f1s), np.std(f1s)),
147         "precision": (np.mean(precs), np.std(precs)),
148         "recall": (np.mean(recs), np.std(recs)),
149         "auroc": (np.mean(aurocs), np.std(aurocs)) if aurocs else (None, None),
150         "auprc": (np.mean(auprs), np.std(auprs)) if auprs else (None, None),
151         "y_true": np.asarray(all_y_true),
152         "y_pred": np.asarray(all_y_pred),
153         "y_score": np.asarray(all_y_score),
154     }
155
156
157 def plot_confusion(y_true, y_pred, title, fname):
158     cm = confusion_matrix(y_true, y_pred)
159     fig, ax = plt.subplots(figsize=(4, 3.6))
160     im = ax.imshow(cm, cmap="Blues")
161     ax.set_xticks([0, 1]); ax.set_yticks([0, 1])
162     ax.set_xticklabels(["non-cancer", "cancer"])
163     ax.set_yticklabels(["non-cancer", "cancer"])
164     ax.set_xlabel("Predicted"); ax.set_ylabel("True")
165     ax.set_title(title)
166     for i in range(2):
167         for j in range(2):
168             color = "white" if cm[i, j] > cm.max() / 2 else "black"
169             ax.text(j, i, str(cm[i, j]), ha="center", va="center",
170                 color=color, fontsize=12, fontweight="bold")
171     fig.colorbar(im, fraction=0.046, pad=0.04)
172     fig.tight_layout()
173     fig.savefig(OUT / fname, dpi=150)
174     plt.close(fig)
175
176
177 def plot_curves(results, fname_pr, fname_roc):
178     fig, ax = plt.subplots(figsize=(5, 4))
179     for r in results:
180         if "y_score" not in r or r["auprc"][0] is None:
181             continue
182         prec, rec, _ = precision_recall_curve(r["y_true"], r["y_score"])
183         ax.plot(rec, prec, label=f"{r['name']} (AUPRC={r['auprc'][0]:.3f})")
184     ax.set_xlabel("Recall"); ax.set_ylabel("Precision")
185     ax.set_title("Precision-Recall (5-fold CV pooled)")
186     ax.legend(loc="lower left", fontsize=8); ax.grid(alpha=0.3)
187     fig.tight_layout(); fig.savefig(OUT / fname_pr, dpi=150); plt.close(fig)
188
189     fig, ax = plt.subplots(figsize=(5, 4))
190     for r in results:
191         if r["auroc"][0] is None:
192             continue
193         fpr, tpr, _ = roc_curve(r["y_true"], r["y_score"])
194         ax.plot(fpr, tpr, label=f"{r['name']} (AUROC={r['auroc'][0]:.3f})")
195     ax.plot([0, 1], [0, 1], "--", color="gray", lw=0.8)
196     ax.set_xlabel("False Positive Rate"); ax.set_ylabel("True Positive Rate")
197     ax.set_title("ROC (5-fold CV pooled)")
198     ax.legend(loc="lower right", fontsize=8); ax.grid(alpha=0.3)
199     fig.tight_layout(); fig.savefig(OUT / fname_roc, dpi=150); plt.close(fig)
200
201
202 def plot_class_balance(y, fname):
203     fig, ax = plt.subplots(figsize=(4, 3))
204     counts = np.bincount(y)
205     ax.bar(["non-cancer", "cancer"], counts, color=["#3b82f6", "#ef4444"])

```



```

206     for i, c in enumerate(counts):
207         ax.text(i, c + 5, str(c), ha="center", fontweight="bold")
208     ax.set_ylabel("count")
209     ax.set_title(f"Class distribution (n={int(counts.sum())})")
210     fig.tight_layout(); fig.savefig(OUT / fname, dpi=150); plt.close(fig)
211
212
213 def plot_script_distribution(stats, fname):
214     fig, ax = plt.subplots(figsize=(4, 3))
215     keys = ["cyrillic_dominant", "latin_dominant", "mixed"]
216     vals = [stats[k] for k in keys]
217     colors = ["#7c3aed", "#0ea5e9", "#10b981"]
218     ax.bar([k.replace("_", "\n") for k in keys], vals, color=colors)
219     for i, v in enumerate(vals):
220         ax.text(i, v + 8, str(v), ha="center", fontweight="bold")
221     ax.set_ylabel("documents")
222     ax.set_title("Script distribution")
223     fig.tight_layout(); fig.savefig(OUT / fname, dpi=150); plt.close(fig)
224
225
226 def main():
227     print("=" * 60)
228     print("Loading dataset...")
229     X, y, ids = load_dataset()
230     n = len(X)
231     n_pos = int(y.sum())
232     print(f"  total: {n}, positive: {n_pos} ({n_pos/n*100:.1f}%), "
233           f"negative: {n-n_pos}")
234
235     plot_class_balance(y, "class_balance.png")
236
237     print("Script analysis...")
238     stats = script_summary(X)
239     print(f"  {stats}")
240     plot_script_distribution(stats, "script_distribution.png")
241
242     models = {
243         "Word-only TF-IDF (Latin-blind)": make_word_only_baseline(),
244         "Char n-gram TF-IDF (script-blind)": make_char_only_baseline(),
245         "Word + char hybrid (single stream)": make_baseline_pipeline(
246             analyzer="char_wb", ngram=(3, 5)),
247         "XS-Classifer (proposed)": make_xs_pipeline(),
248     }
249
250     print("\nRunning 5-fold stratified CV...")
251     print("-" * 60)
252     results = []
253     for name, pipe in models.items():
254         print(f"  · {name}")
255         r = cv_evaluate(name, pipe, X, y, n_splits=5)
256         results.append(r)
257         print(f"    acc={r['accuracy'][0]:.4f}±{r['accuracy'][1]:.3f}, "
258               f"F1={r['f1'][0]:.4f}±{r['f1'][1]:.3f}, "
259               f"AUROC={r['auroc'][0]:.4f}, AUPRC={r['auprc'][0]:.4f}")
260
261     print("\nGenerating plots...")
262     plot_curves(results, "pr_curves.png", "roc_curves.png")
263     for r in results:
264         slug = re.sub(r"^[^a-z0-9]+", "_", r["name"].lower()).strip("_")
265         plot_confusion(r["y_true"], r["y_pred"], r["name"], f"cm_{slug}.png")
266
267     out_metrics = {
268         "n_samples": n, "n_positive": n_pos, "n_negative": n - n_pos,
269         "script_distribution": stats,
270         "models": [{
271             "name": r["name"],

```

```

272         "accuracy_mean": float(r["accuracy"][0]),
273         "accuracy_std": float(r["accuracy"][1]),
274         "f1_mean": float(r["f1"][0]),
275         "f1_std": float(r["f1"][1]),
276         "precision_mean": float(r["precision"][0]),
277         "precision_std": float(r["precision"][1]),
278         "recall_mean": float(r["recall"][0]),
279         "recall_std": float(r["recall"][1]),
280         "auroc_mean": float(r["auroc"][0]) if r["auroc"][0] is not None else None,
281         "auprc_mean": float(r["auprc"][0]) if r["auprc"][0] is not None else None,
282     } for r in results],
283 }
284 (OUT / "metrics.json").write_text(
285     json.dumps(out_metrics, indent=2, ensure_ascii=False), encoding="utf-8")
286
287 print("\n" + "=" * 60)
288 print("RESULTS SUMMARY (5-fold CV)")
289 print("=" * 60)
290 print(f"{'Model':<42} {'Acc':>10} {'F1':>10} {'AUROC':>10} {'AUPRC':>10}")
291 print("-" * 90)
292 for r in results:
293     print(f"{r['name']:<42} "
294           f"{r['accuracy'][0]:.4f} "
295           f"{r['f1'][0]:.4f} "
296           f"{r['auroc'][0]:.4f} "
297           f"{r['auprc'][0]:.4f}")
298
299 print(f"\nFigures saved to: {OUT}")
300 print(f"Metrics JSON: {OUT / 'metrics.json'}")
301
302
303 if __name__ == "__main__":
304     main()

```

app/research/train_detector.py

Fayl yo'li: app/research/train_detector.py | Qatorlar: 398 | Hajm: 13773 bayt

```

1  """Stage-1 YOLOv8/v11 detector training on CBIS-DDSM (or any YOLO-format
2  mammography dataset).
3
4  Tuned for mammography:
5
6      - fliplr=0.0    images are pre-standardised to LEFT laterality
7      - flipud=0.0    up/down flip changes anatomy
8      - scale=0.1     conservative; lesion size is diagnostically meaningful
9      - degrees=5     small rotations only
10     - mosaic=0.3     low mosaic – multi-breast composites are unrealistic
11     - mixup=0.0      blending two mammograms invents non-existent lesions
12     - hsv_*=0.0      mammograms are grayscale; HSV jitter is meaningless
13     - copy_paste=0.0 same reason as mosaic
14     - close_mosaic   disables mosaic for the last N epochs
15
16  Beyond stock Ultralytics metrics (mAP@0.5, mAP@0.5:0.95) the script computes
17  **FROC** (Free-Response ROC) and **sensitivity at fixed FP/image** rates,
18  which are the standard mammography lesion-detection benchmarks.
19
20  Usage:
21      python -m app.research.train_detector \
22          --data /path/to/yolo_cbis/dataset.yaml \
23          --model yolov8m.pt \
24          --epochs 100 \
25          --imgsz 1024 \
26          --batch 8 \

```

```

27     --project runs/cbis_stage1
28     """
29     from __future__ import annotations
30
31     import argparse
32     import json
33     import sys
34     import warnings
35     from dataclasses import dataclass, asdict
36     from pathlib import Path
37
38     warnings.filterwarnings("ignore")
39     if hasattr(sys.stdout, "reconfigure"):
40         sys.stdout.reconfigure(encoding="utf-8", errors="replace")
41
42     import numpy as np
43
44
45     # -----
46     # Mammography-specific train hyper-parameters
47     # -----
48
49
50     MAMMO_HYPS = dict(
51         # Augmentation
52         hsv_h=0.0, hsv_s=0.0, hsv_v=0.05,
53         degrees=5.0,
54         translate=0.05,
55         scale=0.10,
56         shear=0.0,
57         perspective=0.0,
58         flipud=0.0,
59        fliplr=0.0,
60         mosaic=0.3,
61         mixup=0.0,
62         copy_paste=0.0,
63         erasing=0.0,
64         close_mosaic=10,
65         # Optimization (sane defaults – Ultralytics auto-tunes lr0 anyway)
66         optimizer="AdamW",
67         lr0=1e-3,
68         lrf=0.01,
69         momentum=0.937,
70         weight_decay=5e-4,
71         warmup_epochs=3.0,
72         cos_lr=True,
73         # Detection-specific
74         box=7.5, cls=0.5, dfl=1.5,
75         label_smoothing=0.0,
76     )
77
78
79     # -----
80     # YOLO label I/O
81     # -----
82
83
84     def _read_yolo_label(path: Path, img_size: int) -> list[tuple[int, float, float, float, float]]:
85         """Return list of (cls, x0, y0, x1, y1) in pixel coords."""
86         if not path.exists():
87             return []
88         out = []
89         for line in path.read_text(encoding="utf-8").splitlines():
90             parts = line.strip().split()
91             if len(parts) != 5:
92                 continue

```

```

93     cls = int(parts[0])
94     cx, cy, w, h = (float(x) for x in parts[1:])
95     x0 = (cx - w / 2) * img_size
96     y0 = (cy - h / 2) * img_size
97     x1 = (cx + w / 2) * img_size
98     y1 = (cy + h / 2) * img_size
99     out.append((cls, x0, y0, x1, y1))
100 return out
101
102
103 # -----
104 # IoU + bbox matching for FROC
105 # -----
106
107
108 def _iou(a: tuple, b: tuple) -> float:
109     ax0, ay0, ax1, ay1 = a
110     bx0, by0, bx1, by1 = b
111     ix0 = max(ax0, bx0); iy0 = max(ay0, by0)
112     ix1 = min(ax1, bx1); iy1 = min(ay1, by1)
113     iw = max(0.0, ix1 - ix0); ih = max(0.0, iy1 - iy0)
114     inter = iw * ih
115     area_a = max(0.0, ax1 - ax0) * max(0.0, ay1 - ay0)
116     area_b = max(0.0, bx1 - bx0) * max(0.0, by1 - by0)
117     union = area_a + area_b - inter
118     if union <= 0:
119         return 0.0
120     return inter / union
121
122
123 @dataclass
124 class _PredHit:
125     score: float
126     is_tp: bool          # matched a GT
127
128
129 def _match_image(
130     preds: list[tuple[float, float, float, float, float]], # (score, x0, y0, x1, y1)
131     gts: list[tuple[int, float, float, float, float]],
132     iou_thr: float,
133 ) -> tuple[list[_PredHit], int]:
134     """Greedy match preds to GTs by descending score. Returns (hits, n_gt)."""
135     n_gt = len(gts)
136     used = [False] * n_gt
137     hits: list[_PredHit] = []
138     preds_sorted = sorted(preds, key=lambda p: -p[0])
139     for score, *box in preds_sorted:
140         best_iou = 0.0
141         best_idx = -1
142         for j, g in enumerate(gts):
143             if used[j]:
144                 continue
145             i = _iou(tuple(box), tuple(g[1:]))
146             if i > best_iou:
147                 best_iou = i
148                 best_idx = j
149         if best_idx >= 0 and best_iou >= iou_thr:
150             used[best_idx] = True
151             hits.append(_PredHit(score=score, is_tp=True))
152         else:
153             hits.append(_PredHit(score=score, is_tp=False))
154     return hits, n_gt
155
156
157 # -----
158 # FROC curve

```

```

159 # -----
160
161
162 @dataclass
163 class FrocPoint:
164     score: float
165     sensitivity: float
166     fp_per_image: float
167
168
169 def compute_froc(
170     per_image: list[tuple[list[_PredHit], int]],
171     n_images: int,
172 ) -> list[FrocPoint]:
173     """Sweep score threshold; return one (sens, fp/img) per unique score."""
174     all_hits: list[_PredHit] = []
175     total_gt = 0
176     for hits, n_gt in per_image:
177         all_hits.extend(hits)
178         total_gt += n_gt
179     if total_gt == 0 or n_images == 0:
180         return []
181
182     all_hits.sort(key=lambda h: -h.score)
183     tp = fp = 0
184     pts: list[FrocPoint] = []
185     last_score = None
186     for h in all_hits:
187         if h.is_tp:
188             tp += 1
189         else:
190             fp += 1
191         if last_score is None or h.score < last_score - 1e-9:
192             pts.append(FrocPoint(
193                 score=float(h.score),
194                 sensitivity=tp / total_gt,
195                 fp_per_image=fp / n_images,
196             ))
197             last_score = h.score
198     return pts
199
200
201 def sensitivity_at_fp(curve: list[FrocPoint], fp_target: float) -> float:
202     """Highest sensitivity achievable at fp_per_image ≤ fp_target."""
203     best = 0.0
204     for p in curve:
205         if p.fp_per_image <= fp_target and p.sensitivity > best:
206             best = p.sensitivity
207     return best
208
209
210 # -----
211 # Run YOLO inference on the test split and collect preds + GTs
212 # -----
213
214
215 def evaluate_froc(
216     weights: Path,
217     data_yaml: Path,
218     split: str,
219     imgsz: int,
220     conf_min: float = 0.001,
221     iou_thr: float = 0.3,
222     device: str | None = None,
223 ) -> dict:
224     """Run weights over `split` images and compute FROC."""

```

```

225 from ultralytics import YOLO # local import – only needed for eval
226 import yaml as _yaml
227
228 cfg = _yaml.safe_load(data_yaml.read_text(encoding="utf-8"))
229 root = Path(cfg.get("path", data_yaml.parent)).resolve()
230 img_dir = root / cfg.get(split, f"images/{split}")
231 if not img_dir.exists():
232     raise FileNotFoundError(f"split dir not found: {img_dir}")
233 label_dir = Path(str(img_dir).replace("/images/", "/labels/").replace("\\images\\", "\\labels\\"))
234
235 images = sorted(p for p in img_dir.iterdir() if p.suffix.lower() in (".png", ".jpg", ".jpeg"))
236 if not images:
237     raise RuntimeError(f"no images under {img_dir}")
238
239 model = YOLO(str(weights))
240
241 per_image: list[tuple[list[_PredHit], int]] = []
242 for batch_start in range(0, len(images), 32):
243     batch = images[batch_start:batch_start + 32]
244     results = model.predict(
245         source=[str(p) for p in batch],
246         imgsz=imgsz,
247         conf=conf_min,
248         iou=0.5,
249         device=device,
250         verbose=False,
251         save=False,
252     )
253     for img_path, res in zip(batch, results):
254         preds: list[tuple[float, float, float, float, float]] = []
255         if res.bboxes is not None and len(res.bboxes) > 0:
256             xyxy = res.bboxes.xyxy.cpu().numpy()
257             conf = res.bboxes.conf.cpu().numpy()
258             for (x0, y0, x1, y1), s in zip(xyxy, conf):
259                 preds.append((float(s), float(x0), float(y0), float(x1), float(y1)))
260             gt_path = label_dir / (img_path.stem + ".txt")
261             gts = _read_yolo_label(gt_path, imgsz)
262             per_image.append(_match_image(preds, gts, iou_thr))
263
264 curve = compute_froc(per_image, n_images=len(images))
265 fp_targets = [0.5, 1.0, 2.0, 4.0]
266 summary = {
267     "n_images": len(images),
268     "iou_threshold": iou_thr,
269     "n_total_gt": sum(n for _, n in per_image),
270     "sens_at_fp": {f"{t:g}": sensitivity_at_fp(curve, t) for t in fp_targets},
271     "froc_curve": [asdict(p) for p in curve],
272 }
273 return summary
274
275
276 def plot_froc(summary: dict, out_path: Path) -> None:
277     """Save a FROC plot PNG (skips silently if matplotlib is missing)."""
278     try:
279         import matplotlib
280         matplotlib.use("Agg")
281         import matplotlib.pyplot as plt
282     except ImportError:
283         return
284     curve = summary.get("froc_curve") or []
285     if not curve:
286         return
287     fp = [p["fp_per_image"] for p in curve]
288     se = [p["sensitivity"] for p in curve]
289     fig, ax = plt.subplots(figsize=(5.5, 4.5))
290     ax.plot(fp, se, "-", linewidth=2.0, color="#1f77b4")

```

```

291     for t, val in summary["sens_at_fp"].items():
292         ax.axvline(float(t), color="#888", linestyle=":", linewidth=0.8)
293         ax.text(float(t), 0.02, f" {val:.2f}@{t}", fontsize=8, color="#444")
294     ax.set_xlabel("False positives per image")
295     ax.set_ylabel("Sensitivity (recall)")
296     ax.set_title(f"FROC - IoU≥{summary.get('iou_threshold',0.3):.1f}, "
297                 f"n={summary.get('n_images',0)}")
298     ax.set_xscale("symlog", lincthresh=0.5)
299     ax.set_xlim(0, max(8.0, max(fp) if fp else 8.0))
300     ax.set_ylim(0, 1.02)
301     ax.grid(alpha=0.3)
302     fig.tight_layout()
303     fig.savefig(out_path, dpi=150)
304     plt.close(fig)
305
306
307 # -----
308 # CLI
309 # -----
310
311
312 def main():
313     ap = argparse.ArgumentParser(description="Stage-1 YOLO mammography detector")
314     ap.add_argument("--data", required=True, help="Path to dataset.yaml")
315     ap.add_argument("--model", default="yolov8m.pt",
316                     help="Pretrained weights or .yaml architecture spec")
317     ap.add_argument("--epochs", type=int, default=100)
318     ap.add_argument("--imgsz", type=int, default=1024)
319     ap.add_argument("--batch", type=int, default=8)
320     ap.add_argument("--device", default=None,
321                     help="GPU id ('0'), 'cpu', or None for auto")
322     ap.add_argument("--project", default="runs/detect", help="Output dir for runs")
323     ap.add_argument("--name", default="cbis_stage1")
324     ap.add_argument("--patience", type=int, default=20)
325     ap.add_argument("--workers", type=int, default=4)
326     ap.add_argument("--resume", action="store_true",
327                     help="Resume the last run with --name")
328     ap.add_argument("--no-train", action="store_true",
329                     help="Skip training; only run FROC eval on existing weights")
330     ap.add_argument("--weights", default=None,
331                     help="When --no-train, path to weights .pt; else uses best.pt of the run")
332     ap.add_argument("--eval-split", default="test",
333                     help="Which split to FROC-evaluate (default: test)")
334     ap.add_argument("--iou-thr", type=float, default=0.3,
335                     help="IoU threshold for hit detection in FROC (mammo standard 0.2-0.5)")
336     args = ap.parse_args()
337
338     data_yaml = Path(args.data).resolve()
339     if not data_yaml.exists():
340         raise SystemExit(f"dataset.yaml not found: {data_yaml}")
341
342     project_dir = Path(args.project).resolve()
343     project_dir.mkdir(parents=True, exist_ok=True)
344
345     if not args.no_train:
346         from ultralytics import YOLO
347         print(f"[train] model={args.model} imgsz={args.imgsz} batch={args.batch} "
348               f"epochs={args.epochs} device={args.device or 'auto'}")
349         model = YOLO(args.model)
350         results = model.train(
351             data=str(data_yaml),
352             epochs=args.epochs,
353             imgsz=args.imgsz,
354             batch=args.batch,
355             device=args.device,
356             workers=args.workers,

```

```

357         project=str(project_dir),
358         name=args.name,
359         patience=args.patience,
360         resume=args.resume,
361         exist_ok=True,
362         **MAMMO_HYPS,
363     )
364     run_dir = Path(results.save_dir) if hasattr(results, "save_dir") else (project_dir / args.name)
365     weights = run_dir / "weights" / "best.pt"
366 else:
367     run_dir = project_dir / args.name
368     weights = Path(args.weights) if args.weights else (run_dir / "weights" / "best.pt")
369
370 if not weights.exists():
371     raise SystemExit(f"best.pt not found at {weights}")
372
373 print(f"\n[eval] FROC on split='{args.eval_split}', IoU≥{args.iou_thr}")
374 summary = evaluate_froc(
375     weights=weights,
376     data_yaml=data_yaml,
377     split=args.eval_split,
378     imgsz=args.imgsz,
379     iou_thr=args.iou_thr,
380     device=args.device,
381 )
382
383 print(f"  n_images = {summary['n_images']}, n_gt = {summary['n_total_gt']}")
384 for fp_t, sens in summary["sens_at_fp"].items():
385     print(f"    sens @ {fp_t:>4} FP/img : {sens:.4f}")
386
387 out_dir = Path(run_dir)
388 out_dir.mkdir(parents=True, exist_ok=True)
389 (out_dir / "froc_summary.json").write_text(
390     json.dumps(summary, indent=2, ensure_ascii=False), encoding="utf-8",
391 )
392 plot_froc(summary, out_dir / "froc.png")
393 print(f"\n[saved] {out_dir / 'froc_summary.json'}")
394 print(f"[saved] {out_dir / 'froc.png'}")
395
396
397 if __name__ == "__main__":
398     main()

```

app/research/train_tillnet.py

Fayl yo'li: `app/research/train_tillnet.py` | Qatorlar: 674 | Hajm: 27096 bayt

```

1  """TILLNet-Det training script.
2
3  End-to-end trainer for the multimodal mammography detector defined in
4  `app.models_arch.tillnet_det`. Implements:
5
6      - FCOS-style dense target assignment (per-level regress ranges + center sampling)
7      - Composite loss: focal_cls + GIoU_reg + BCE_centerness
8      - Paired (image, text) dataloader: PNG + YOLO .txt + clinical text
9      - AdamW + cosine LR schedule + linear warmup
10     - Periodic FROC validation (best checkpoint by sensitivity@1FP/image)
11
12  The script reuses the FROC math from `app.research.train_detector` so the
13  metric numbers are directly comparable to the YOLOv8 stage-1 baseline.
14
15  Usage:
16      python -m app.research.train_tillnet \
17          --data /path/to/yolo_cbis/dataset.yaml \

```



```

18     --texts    /path/to/texts.csv          # columns: filename,text
19     --epochs  60 \\
20     --imgsz   1024 \\
21     --batch   2 \\
22     --device  0 \\
23     --out     runs/tillnet
24     """
25     from __future__ import annotations
26
27     import argparse
28     import json
29     import math
30     import sys
31     import time
32     import warnings
33     from dataclasses import dataclass
34     from pathlib import Path
35
36     warnings.filterwarnings("ignore")
37     if hasattr(sys.stdout, "reconfigure"):
38         sys.stdout.reconfigure(encoding="utf-8", errors="replace")
39
40     import numpy as np
41     import torch
42     import torch.nn as nn
43     import torch.nn.functional as F
44     from torch.utils.data import Dataset, DataLoader
45     from PIL import Image
46
47     from app.models_arch.tillnet_det import (
48         TILLNetDet, TILLNetConfig, encode_text, TEXT_MAX_LEN,
49     )
50     from app.research.train_detector import (
51         _PredHit, _match_image, compute_froc, sensitivity_at_fp, plot_froc,
52     )
53
54
55     # Standard FCOS regress ranges (in original-image pixels) per pyramid level.
56     # Locations whose maximum (l,t,r,b) falls in the level's range are "responsible"
57     # for that GT, which gives the network a natural multi-scale specialisation.
58     REGRESS_RANGES = (
59         (-1, 64),      # P3 stride 8      small lesions ≤ 64 px
60         (64, 128),     # P4 stride 16
61         (128, 256),    # P5 stride 32
62         (256, 512),    # P6 stride 64
63         (512, 10_000) # P7 stride 128   large masses
64     )
65
66
67     # -----
68     # 1. Dataset – paired (image, YOLO labels, text)
69     # -----
70
71
72     def _read_yolo_label(path: Path, img_size: int) -> tuple[np.ndarray, np.ndarray]:
73         """Returns (boxes Nx4 xyxy in pixels, labels N int64)."""
74         boxes, labels = [], []
75         if path.exists():
76             for line in path.read_text(encoding="utf-8").splitlines():
77                 parts = line.strip().split()
78                 if len(parts) != 5:
79                     continue
80                 cls = int(parts[0])
81                 cx, cy, w, h = (float(x) for x in parts[1:])
82                 x0 = (cx - w / 2) * img_size
83                 y0 = (cy - h / 2) * img_size

```

```

84         x1 = (cx + w / 2) * img_size
85         y1 = (cy + h / 2) * img_size
86         if x1 > x0 and y1 > y0:
87             boxes.append([x0, y0, x1, y1])
88             labels.append(cls)
89     return (np.asarray(boxes, dtype=np.float32).reshape(-1, 4),
90           np.asarray(labels, dtype=np.int64))
91
92
93 class MammoDetTextDataset(Dataset):
94     """One sample = (image tensor, GT boxes, GT labels, text token ids)."""
95
96     def __init__(
97         self,
98         images_dir: Path,
99         labels_dir: Path,
100         texts_lookup: dict[str, str],
101         target_size: int,
102         augment: bool = False,
103     ):
104         self.images_dir = Path(images_dir)
105         self.labels_dir = Path(labels_dir)
106         self.target_size = target_size
107         self.augment = augment
108         self.texts_lookup = texts_lookup
109         self.files = sorted(p for p in self.images_dir.iterdir()
110                             if p.suffix.lower() in (".png", ".jpg", ".jpeg"))
111         if not self.files:
112             raise RuntimeError(f"no images under {images_dir}")
113
114     def __len__(self) -> int:
115         return len(self.files)
116
117     def __getitem__(self, idx: int):
118         img_path = self.files[idx]
119         # PIL grayscale, scale to target_size if mismatched (preprocess.py already
120         # produces target-size PNGs, but this guards against mixed runs).
121         img = Image.open(img_path).convert("L")
122         if img.size != (self.target_size, self.target_size):
123             img = img.resize((self.target_size, self.target_size), Image.BILINEAR)
124         arr = np.asarray(img, dtype=np.float32) / 255.0
125
126         if self.augment:
127             # Mammography-conservative: slight intensity jitter only.
128             arr = arr * float(np.random.uniform(0.9, 1.1))
129             arr = np.clip(arr + float(np.random.uniform(-0.05, 0.05)), 0.0, 1.0)
130
131         img_t = torch.from_numpy(arr)[None] # (1, H, W)
132
133         lbl_path = self.labels_dir / (img_path.stem + ".txt")
134         boxes, labels = _read_yolo_label(lbl_path, self.target_size)
135         boxes_t = torch.from_numpy(boxes)
136         labels_t = torch.from_numpy(labels)
137
138         text = self.texts_lookup.get(img_path.name, "")
139         text_ids = encode_text(text)
140
141         return {
142             "image": img_t,
143             "boxes": boxes_t, # (N, 4)
144             "labels": labels_t, # (N,)
145             "text_ids": text_ids, # (L,)
146             "image_id": img_path.stem,
147         }
148
149

```

```

150 def collate_fn(batch: list[dict]) -> dict:
151     return {
152         "images": torch.stack([b["image"] for b in batch], dim=0),
153         "text_ids": torch.stack([b["text_ids"] for b in batch], dim=0),
154         "boxes": [b["boxes"] for b in batch],
155         "labels": [b["labels"] for b in batch],
156         "image_ids": [b["image_id"] for b in batch],
157     }
158
159
160 # -----
161 # 2. FCOS target assignment
162 # -----
163
164
165 def _grid_locations(H: int, W: int, stride: int, device) -> torch.Tensor:
166     """Return (H*W, 2) tensor of (x, y) pixel locations at feature-pixel centers."""
167     ys = torch.arange(H, device=device, dtype=torch.float32)
168     xs = torch.arange(W, device=device, dtype=torch.float32)
169     yy, xx = torch.meshgrid(ys, xs, indexing="ij")
170     cx = (xx.reshape(-1) + 0.5) * stride
171     cy = (yy.reshape(-1) + 0.5) * stride
172     return torch.stack([cx, cy], dim=-1) # (HW, 2)
173
174
175 def _assign_targets_one_image(
176     gt_boxes: torch.Tensor, # (G, 4) xyxy
177     gt_labels: torch.Tensor, # (G,)
178     locations_per_level: list[torch.Tensor], # each (HW_l, 2)
179     strides: tuple[int, ...],
180     regress_ranges: tuple[tuple[float, float], ...],
181     num_classes: int,
182     center_sample_radius: float = 1.5,
183 ):
184     """For one image, return cls_target / reg_target / ctr_target per level."""
185     device = locations_per_level[0].device
186     cls_t_levels: list[torch.Tensor] = []
187     reg_t_levels: list[torch.Tensor] = []
188     ctr_t_levels: list[torch.Tensor] = []
189
190     G = int(gt_boxes.shape[0])
191     if G == 0:
192         # All-background masks
193         for loc in locations_per_level:
194             N = loc.shape[0]
195             cls_t_levels.append(torch.full((N,), -1, dtype=torch.long, device=device))
196             reg_t_levels.append(torch.zeros((N, 4), device=device))
197             ctr_t_levels.append(torch.zeros((N,), device=device))
198         return cls_t_levels, reg_t_levels, ctr_t_levels
199
200     gt_areas = (gt_boxes[:, 2] - gt_boxes[:, 0]) * (gt_boxes[:, 3] - gt_boxes[:, 1]) # (G,)
201     gt_cx = (gt_boxes[:, 0] + gt_boxes[:, 2]) * 0.5
202     gt_cy = (gt_boxes[:, 1] + gt_boxes[:, 3]) * 0.5
203
204     for level, loc in enumerate(locations_per_level):
205         N = loc.shape[0]
206         stride = strides[level]
207         lo, hi = regress_ranges[level]
208         x = loc[:, 0:1] # (N, 1)
209         y = loc[:, 1:2]
210         l = x - gt_boxes[:, 0] # (N, G)
211         t = y - gt_boxes[:, 1]
212         r = gt_boxes[:, 2] - x
213         b = gt_boxes[:, 3] - y
214         ltrb = torch.stack([l, t, r, b], dim=-1) # (N, G, 4)
215

```

```

216     inside_box = ltrb.min(dim=-1).values > 0 # (N, G)
217
218     # Center sampling: location must also be within radius * stride of GT center
219     radius = center_sample_radius * stride
220     cs_x0 = torch.clamp_min(gt_cx - radius, gt_boxes[:, 0])
221     cs_y0 = torch.clamp_min(gt_cy - radius, gt_boxes[:, 1])
222     cs_x1 = torch.clamp_max(gt_cx + radius, gt_boxes[:, 2])
223     cs_y1 = torch.clamp_max(gt_cy + radius, gt_boxes[:, 3])
224     cs_inside = (x >= cs_x0) & (x <= cs_x1) & (y >= cs_y0) & (y <= cs_y1)
225
226     max_ltrb = ltrb.max(dim=-1).values # (N, G)
227     in_range = (max_ltrb >= lo) & (max_ltrb < hi) # (N, G)
228
229     valid = inside_box & cs_inside & in_range # (N, G)
230
231     # Among valid GTs, pick the one with smallest area (FCOS heuristic).
232     BIG = 1e10
233     areas_exp = gt_areas[None, :].expand(N, G).clone()
234     areas_exp[~valid] = BIG
235     min_area, gt_idx = areas_exp.min(dim=1) # (N,) each
236     is_pos = min_area < BIG
237
238     cls_t = torch.full((N,), -1, dtype=torch.long, device=device) # -1 = ignore/background
239     cls_t[is_pos] = gt_labels[gt_idx[is_pos]]
240
241     chosen_ltrb = ltrb[torch.arange(N, device=device), gt_idx] # (N, 4)
242     reg_t = chosen_ltrb / stride # normalised by stride
243     reg_t[~is_pos] = 0.0
244
245     # Centerness target - only meaningful for positive locations
246     if is_pos.any():
247         l_, t_, r_, b_ = chosen_ltrb[is_pos].unbind(dim=-1)
248         ctr_full = torch.zeros((N,), device=device)
249         ctr = ((torch.minimum(l_, r_) / torch.maximum(l_, r_)) *
250               (torch.minimum(t_, b_) / torch.maximum(t_, b_))).clamp_min(0).sqrt()
251         ctr_full[is_pos] = ctr
252         ctr_t = ctr_full
253     else:
254         ctr_t = torch.zeros((N,), device=device)
255
256     cls_t_levels.append(cls_t)
257     reg_t_levels.append(reg_t)
258     ctr_t_levels.append(ctr_t)
259
260     return cls_t_levels, reg_t_levels, ctr_t_levels
261
262
263 # -----
264 # 3. Loss functions
265 # -----
266
267
268 def _sigmoid_focal_loss(
269     logits: torch.Tensor, targets: torch.Tensor,
270     alpha: float = 0.25, gamma: float = 2.0,
271 ) -> torch.Tensor:
272     """Multi-class focal loss on per-class binary outputs (FCOS / RetinaNet style).
273
274     logits : (M, K)
275     targets : (M, K) one-hot (positives only)
276     """
277     p = logits.sigmoid()
278     ce = F.binary_cross_entropy_with_logits(logits, targets, reduction="none")
279     pt = p * targets + (1 - p) * (1 - targets)
280     alpha_t = alpha * targets + (1 - alpha) * (1 - targets)
281     loss = alpha_t * (1 - pt).pow(gamma) * ce

```

```

282     return loss.sum()
283
284
285 def _giou_loss(pred_ltrb: torch.Tensor, target_ltrb: torch.Tensor) -> torch.Tensor:
286     """Generalized IoU loss between two sets of (l, t, r, b) regressions.
287
288     All values are in stride units; computed without converting to xyxy because
289     GIoU on l/t/r/b distances is mathematically equivalent.
290     """
291     pl, pt, pr, pb = pred_ltrb.unbind(dim=-1)
292     tl, tt, tr, tb = target_ltrb.unbind(dim=-1)
293
294     pred_area = (pl + pr) * (pt + pb)
295     target_area = (tl + tr) * (tt + tb)
296
297     inter_w = torch.minimum(pl, tl) + torch.minimum(pr, tr)
298     inter_h = torch.minimum(pt, tt) + torch.minimum(pb, tb)
299     inter = inter_w.clamp_min(0) * inter_h.clamp_min(0)
300
301     union = pred_area + target_area - inter + 1e-6
302     iou = inter / union
303
304     enclose_w = torch.maximum(pl, tl) + torch.maximum(pr, tr)
305     enclose_h = torch.maximum(pt, tt) + torch.maximum(pb, tb)
306     enclose = enclose_w.clamp_min(0) * enclose_h.clamp_min(0) + 1e-6
307     giou = iou - (enclose - union) / enclose
308
309     return (1 - giou).sum()
310
311
312 def fcos_loss(
313     out: dict,
314     gt_boxes: list[torch.Tensor],
315     gt_labels: list[torch.Tensor],
316     image_size: int,
317     num_classes: int,
318     cls_weight: float = 1.0,
319     reg_weight: float = 1.0,
320     ctr_weight: float = 1.0,
321 ) -> dict:
322     """Compute FCOS-style composite loss over a batch."""
323     cls_outs = out["cls_logits"] # list of (B, K, H, W)
324     reg_outs = out["reg_dist"] # list of (B, 4, H, W)
325     ctr_outs = out["centerness"] # list of (B, 1, H, W)
326     strides = out["fpn_strides"]
327     device = cls_outs[0].device
328     B = cls_outs[0].shape[0]
329
330     # Precompute per-level location grids (shared across the batch).
331     locations_per_level = [
332         _grid_locations(c.shape[-2], c.shape[-1], strides[i], device)
333         for i, c in enumerate(cls_outs)
334     ]
335
336     # Assign targets per image, then concatenate across images per level.
337     cls_t_all = [[] for _ in cls_outs]
338     reg_t_all = [[] for _ in cls_outs]
339     ctr_t_all = [[] for _ in cls_outs]
340     for b in range(B):
341         c_lvls, r_lvls, k_lvls = _assign_targets_one_image(
342             gt_boxes[b].to(device), gt_labels[b].to(device),
343             locations_per_level, strides, REGRESS_RANGES, num_classes,
344         )
345         for i in range(len(cls_outs)):
346             cls_t_all[i].append(c_lvls[i])
347             reg_t_all[i].append(r_lvls[i])

```

```

348         ctr_t_all[i].append(k_lvls[i])
349
350     total_cls = total_reg = total_ctr = torch.zeros((), device=device)
351     n_pos_total = 0
352     for i in range(len(cls_outs)):
353         cls_logits = cls_outs[i].permute(0, 2, 3, 1).reshape(-1, num_classes)
354         reg_pred = reg_outs[i].permute(0, 2, 3, 1).reshape(-1, 4)
355         ctr_pred = ctr_outs[i].permute(0, 2, 3, 1).reshape(-1)
356
357         cls_t = torch.cat(cls_t_all[i])
358         reg_t = torch.cat(reg_t_all[i])
359         ctr_t = torch.cat(ctr_t_all[i])
360
361         # Build one-hot targets for focal loss (positives only contribute).
362         valid = cls_t >= 0
363         cls_one_hot = torch.zeros_like(cls_logits)
364         if valid.any():
365             cls_one_hot[valid, cls_t[valid]] = 1.0
366         total_cls = total_cls + _sigmoid_focal_loss(cls_logits, cls_one_hot)
367
368         if valid.any():
369             total_reg = total_reg + _giou_loss(reg_pred[valid], reg_t[valid])
370             total_ctr = total_ctr + F.binary_cross_entropy_with_logits(
371                 ctr_pred[valid], ctr_t[valid], reduction="sum",
372             )
373             n_pos_total += int(valid.sum())
374
375     n_pos = max(n_pos_total, 1)
376     loss_cls = total_cls / n_pos
377     loss_reg = total_reg / n_pos
378     loss_ctr = total_ctr / n_pos
379     loss = cls_weight * loss_cls + reg_weight * loss_reg + ctr_weight * loss_ctr
380     return {
381         "loss": loss,
382         "loss_cls": loss_cls.detach(),
383         "loss_reg": loss_reg.detach(),
384         "loss_ctr": loss_ctr.detach(),
385         "n_pos": n_pos,
386     }
387
388
389 # -----
390 # 4. NMS + FROC eval (using TILLNet's predict() then matching to GT)
391 # -----
392
393
394 def _nms(bboxes: torch.Tensor, scores: torch.Tensor, iou_thr: float = 0.5) -> torch.Tensor:
395     """Class-agnostic NMS. Returns indices to keep, sorted by score."""
396     if bboxes.numel() == 0:
397         return torch.zeros((0,), dtype=torch.long, device=bboxes.device)
398     order = scores.argsort(descending=True)
399     keep = []
400     while order.numel() > 0:
401         i = order[0].item()
402         keep.append(i)
403         if order.numel() == 1:
404             break
405         rest = order[1:]
406         xx0 = torch.maximum(bboxes[i, 0], bboxes[rest, 0])
407         yy0 = torch.maximum(bboxes[i, 1], bboxes[rest, 1])
408         xx1 = torch.minimum(bboxes[i, 2], bboxes[rest, 2])
409         yy1 = torch.minimum(bboxes[i, 3], bboxes[rest, 3])
410         iw = (xx1 - xx0).clamp_min(0)
411         ih = (yy1 - yy0).clamp_min(0)
412         inter = iw * ih
413         a_i = (bboxes[i, 2] - bboxes[i, 0]) * (bboxes[i, 3] - bboxes[i, 1])

```

```

414     a_r = (boxes[rest, 2] - boxes[rest, 0]) * (boxes[rest, 3] - boxes[rest, 1])
415     iou = inter / (a_i + a_r - inter + 1e-6)
416     order = rest[iou <= iou_thr]
417     return torch.tensor(keep, dtype=torch.long, device=boxes.device)
418
419
420 @torch.no_grad()
421 def evaluate_froc_tillnet(
422     model: TILLNetDet,
423     loader: DataLoader,
424     device,
425     iou_thr_match: float = 0.3,
426     nms_iou: float = 0.5,
427     score_min: float = 0.01,
428 ) -> dict:
429     """Run model over loader and compute FROC + sensitivity@FP-rates."""
430     model.eval()
431     per_image: list[tuple[list[_PredHit], int]] = []
432     n_images = 0
433     for batch in loader:
434         images = batch["images"].to(device)
435         text_ids = batch["text_ids"].to(device)
436         preds = model.predict(images, text_ids,
437                               score_threshold=score_min, max_per_image=300)
438         for i, p in enumerate(preds):
439             n_images += 1
440             keep = _nms(p["boxes"], p["scores"], iou_thr=nms_iou)
441             boxes = p["boxes"][keep].cpu().numpy()
442             scores = p["scores"][keep].cpu().numpy()
443             preds_list = [
444                 (float(s), float(b[0]), float(b[1]), float(b[2]), float(b[3]))
445                 for s, b in zip(scores, boxes)
446             ]
447             gt_boxes = batch["boxes"][i].cpu().numpy()
448             gt_labels = batch["labels"][i].cpu().numpy()
449             gts = [(int(c), float(x0), float(y0), float(x1), float(y1))
450                    for (x0, y0, x1, y1), c in zip(gt_boxes, gt_labels)]
451             per_image.append(_match_image(preds_list, gts, iou_thr_match))
452
453     curve = compute_froc(per_image, n_images=n_images)
454     fp_targets = [0.5, 1.0, 2.0, 4.0]
455     return {
456         "n_images": n_images,
457         "iou_threshold": iou_thr_match,
458         "n_total_gt": sum(n for _, n in per_image),
459         "sens_at_fp": {f"{t:g}": sensitivity_at_fp(curve, t) for t in fp_targets},
460         "froc_curve": [{"score": p.score, "sensitivity": p.sensitivity,
461                        "fp_per_image": p.fp_per_image} for p in curve],
462     }
463
464
465 # -----
466 # 5. LR schedule (warmup + cosine)
467 # -----
468
469
470 def _make_scheduler(optimizer, total_steps: int, warmup_steps: int):
471     def lr_lambda(step: int) -> float:
472         if step < warmup_steps:
473             return step / max(1, warmup_steps)
474         prog = (step - warmup_steps) / max(1, total_steps - warmup_steps)
475         return 0.5 * (1 + math.cos(math.pi * prog))
476     return torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)
477
478
479 # -----

```

```

480 # 6. CLI
481 # -----
482
483
484 def _load_texts(path: Path | None) -> dict[str, str]:
485     if path is None:
486         return {}
487     p = Path(path)
488     if not p.exists():
489         return {}
490     out: dict[str, str] = {}
491     if p.suffix.lower() == ".jsonl":
492         for line in p.read_text(encoding="utf-8").splitlines():
493             if not line.strip():
494                 continue
495             r = json.loads(line)
496             fn = r.get("filename") or r.get("image") or r.get("file")
497             if fn:
498                 out[Path(fn).name] = r.get("text", "") or ""
499     else:
500         import csv as _csv
501         with p.open("r", encoding="utf-8", newline="") as f:
502             for r in _csv.DictReader(f):
503                 fn = r.get("filename") or r.get("image") or r.get("file")
504                 if fn:
505                     out[Path(fn).name] = r.get("text", "") or ""
506     return out
507
508
509 def _resolve_split_dirs(data_yaml: Path):
510     import yaml as _yaml
511     cfg = _yaml.safe_load(data_yaml.read_text(encoding="utf-8"))
512     root = Path(cfg.get("path", data_yaml.parent)).resolve()
513
514     def split_dirs(name: str):
515         rel = cfg.get(name, f"images/{name}")
516         img = (root / rel).resolve()
517         lbl = Path(str(img).replace("/images/", "/labels/").replace("\\images\\", "\\labels\\"))
518         return img, lbl
519
520     return cfg, root, split_dirs
521
522
523 def main():
524     ap = argparse.ArgumentParser(description="TILLNet-Det multimodal detector trainer")
525     ap.add_argument("--data", required=True)
526     ap.add_argument("--texts",
527                     help="CSV or JSONL with 'filename,text' columns (filename = image basename)")
528     ap.add_argument("--epochs", type=int, default=60)
529     ap.add_argument("--imgsz", type=int, default=1024)
530     ap.add_argument("--batch", type=int, default=2)
531     ap.add_argument("--lr", type=float, default=1e-4)
532     ap.add_argument("--weight-decay", type=float, default=5e-4)
533     ap.add_argument("--warmup-epochs", type=float, default=2.0)
534     ap.add_argument("--workers", type=int, default=2)
535     ap.add_argument("--device", default="cuda" if torch.cuda.is_available() else "cpu")
536     ap.add_argument("--out", default="runs/tillnet")
537     ap.add_argument("--no-text", action="store_true",
538                     help="Image-only ablation (drops text branch entirely)")
539     ap.add_argument("--no-pretrained", action="store_true",
540                     help="Train backbone from scratch")
541     ap.add_argument("--clip-grad", type=float, default=10.0)
542     ap.add_argument("--eval-every", type=int, default=2,
543                     help="Run FROC validation every N epochs")
544     ap.add_argument("--num-classes", type=int, default=2,
545                     help="Must match dataset.yaml's nc")

```



```

546     ap.add_argument("--resume", default=None,
547                     help="Path to .pt checkpoint to resume training")
548     args = ap.parse_args()
549
550     data_yaml = Path(args.data).resolve()
551     out_dir = Path(args.out).resolve()
552     out_dir.mkdir(parents=True, exist_ok=True)
553     device = torch.device(args.device)
554
555     cfg, root, split_dirs = _resolve_split_dirs(data_yaml)
556     nc = int(cfg.get("nc", args.num_classes))
557     if nc != args.num_classes:
558         print(f"[warn] dataset.yaml nc={nc} differs from --num-classes={args.num_classes}; using {nc}")
559     args.num_classes = nc
560
561     train_img, train_lbl = split_dirs("train")
562     val_img, val_lbl = split_dirs("val")
563     texts = _load_texts(Path(args.texts) if args.texts else None)
564     print(f"[data] train={train_img} val={val_img} texts={len(texts)} entries")
565
566     train_ds = MammoDetTextDataset(train_img, train_lbl, texts, args.imgsz, augment=True)
567     val_ds = MammoDetTextDataset(val_img, val_lbl, texts, args.imgsz, augment=False)
568     train_loader = DataLoader(train_ds, batch_size=args.batch, shuffle=True,
569                              num_workers=args.workers, collate_fn=collate_fn, drop_last=True)
570     val_loader = DataLoader(val_ds, batch_size=max(1, args.batch),
571                            shuffle=False, num_workers=args.workers, collate_fn=collate_fn)
572
573     cfg_m = TILLNetConfig(
574         num_classes=args.num_classes,
575         pretrained_backbone=not args.no_pretrained,
576         use_text=not args.no_text,
577     )
578     model = TILLNetDet(cfg_m).to(device)
579
580     start_epoch = 0
581     best_sens_at_1 = 0.0
582     if args.resume:
583         ckpt = torch.load(args.resume, map_location=device)
584         model.load_state_dict(ckpt["model"])
585         start_epoch = ckpt.get("epoch", 0) + 1
586         best_sens_at_1 = ckpt.get("best_sens_at_1", 0.0)
587         print(f"[resume] from epoch {start_epoch}, best sens@1FP = {best_sens_at_1:.4f}")
588
589     optim = torch.optim.AdamW(
590         model.parameters(), lr=args.lr, weight_decay=args.weight_decay,
591         betas=(0.9, 0.999),
592     )
593     steps_per_epoch = max(1, len(train_loader))
594     total_steps = steps_per_epoch * args.epochs
595     warmup_steps = int(steps_per_epoch * args.warmup_epochs)
596     scheduler = _make_scheduler(optim, total_steps, warmup_steps)
597
598     print(f"[model] params = {sum(p.numel() for p in model.parameters() if p.requires_grad)/1e6:.2f}M")
599
600     print(f"[train] epochs={args.epochs} batch={args.batch} imgsz={args.imgsz} "
601           f"lr={args.lr} steps/epoch={steps_per_epoch} device={device}")
602
603     history = []
604     for epoch in range(start_epoch, args.epochs):
605         model.train()
606         ep_t0 = time.time()
607         running = {"loss": 0.0, "cls": 0.0, "reg": 0.0, "ctr": 0.0}
608         n_batches = 0
609         for batch in train_loader:
610             images = batch["images"].to(device, non_blocking=True)

```

```

611     text_ids = batch["text_ids"].to(device, non_blocking=True)
612     out = model(images, text_ids if cfg_m.use_text else None)
613     losses = fcos_loss(
614         out, batch["boxes"], batch["labels"],
615         image_size=args.imgsz, num_classes=args.num_classes,
616     )
617     loss = losses["loss"]
618     optim.zero_grad(set_to_none=True)
619     loss.backward()
620     if args.clip_grad > 0:
621         nn.utils.clip_grad_norm_(model.parameters(), args.clip_grad)
622     optim.step()
623     scheduler.step()
624     running["loss"] += float(loss.detach())
625     running["cls"] += float(losses["loss_cls"])
626     running["reg"] += float(losses["loss_reg"])
627     running["ctr"] += float(losses["loss_ctr"])
628     n_batches += 1
629
630     avg = {k: v / max(1, n_batches) for k, v in running.items()}
631     ep_dt = time.time() - ep_t0
632     print(f"[ep {epoch+1:3d}]/[args.epochs]] "
633           f"loss={avg['loss']:.4f} cls={avg['cls']:.4f} "
634           f"reg={avg['reg']:.4f} ctr={avg['ctr']:.4f} "
635           f"({epoch_dt:.1f}s, lr={scheduler.get_last_lr()[0]:.2e})")
636
637     do_eval = ((epoch + 1) % args.eval_every == 0) or (epoch + 1 == args.epochs)
638     if do_eval and len(val_ds) > 0:
639         summary = evaluate_froc_tillnet(model, val_loader, device)
640         sens1 = summary["sens_at_fp"].get("1", 0.0)
641         print(f"        val FROC: sens@1FP={sens1:.4f} "
642               f"f"sens@2FP={summary['sens_at_fp'].get('2',0.0):.4f} "
643               f"({summary['n_images']} imgs, {summary['n_total_gt']} GTs)")
644         history.append({"epoch": epoch + 1, "train": avg, "val": summary})
645         (out_dir / "history.jsonl").write_text(
646             "\n".join(json.dumps(h, ensure_ascii=False) for h in history),
647             encoding="utf-8",
648         )
649         if sens1 > best_sens_at_1:
650             best_sens_at_1 = sens1
651             torch.save({
652                 "model": model.state_dict(),
653                 "epoch": epoch,
654                 "best_sens_at_1": best_sens_at_1,
655                 "config": cfg_m.__dict__,
656             }, out_dir / "best.pt")
657             plot_froc(summary, out_dir / "best_froc.png")
658             print(f"        [save] new best best.pt sens@1FP={best_sens_at_1:.4f}")
659
660     # Always keep the most recent weights for resume.
661     torch.save({
662         "model": model.state_dict(),
663         "epoch": epoch,
664         "best_sens_at_1": best_sens_at_1,
665         "config": cfg_m.__dict__,
666     }, out_dir / "last.pt")
667
668     print(f"\n[done] best sens@1FP = {best_sens_at_1:.4f}")
669     print(f"[saved] {out_dir / 'best.pt'}")
670     print(f"[saved] {out_dir / 'history.jsonl'}")
671
672
673 if __name__ == "__main__":
674     main()

```

6. Qo'shimcha utilita skriptlari

build_pdf.py

Fayl yo'li: build_pdf.py | Qatorlar: 481 | Hajm: 15701 bayt

```
1  """
2  Build PDF documentation with screenshots.
3
4  Usage:
5      python build_pdf.py
6
7  Yaratadi:
8      docs/screenshots/*.png
9      FOYDALANUVCHI_QOLLANMA.pdf
10 """
11 from __future__ import annotations
12
13 import asyncio
14 import re
15 import shutil
16 import subprocess
17 import sys
18 import time
19 from pathlib import Path
20
21 if hasattr(sys.stdout, "reconfigure"):
22     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
23     sys.stderr.reconfigure(encoding="utf-8", errors="replace")
24
25 import markdown
26 from playwright.async_api import async_playwright
27
28 ROOT = Path(__file__).resolve().parent
29 DOCS = ROOT / "docs"
30 SHOTS = DOCS / "screenshots"
31 MD_FILE = ROOT / "FOYDALANUVCHI_QOLLANMA.md"
32 PDF_FILE = ROOT / "FOYDALANUVCHI_QOLLANMA.pdf"
33 HTML_FILE = DOCS / "_manual.html"
34
35 BASE_URL = "http://127.0.0.1:8765"
36 ADMIN_USER = "admin"
37 ADMIN_PASS = "admin123"
38
39
40 async def wait_visible(page, sel, timeout=8000):
41     await page.wait_for_selector(sel + ":not([hidden])", timeout=timeout)
42
43
44 async def login(page):
45     await page.goto(BASE_URL, wait_until="networkidle")
46     await wait_visible(page, "#loginModal")
47     await page.fill("#loginUsername", ADMIN_USER)
48     await page.fill("#loginPassword", ADMIN_PASS)
49     await page.click("#loginSubmit")
50     await page.wait_for_function(
51         "() => { const m = document.getElementById('loginModal'); return !m || m.hidden; }",
52         timeout=10000,
53     )
54     await page.wait_for_timeout(800)
55
```

```

56
57 async def shoot(page, name: str, full_page: bool = False):
58     SHOTS.mkdir(parents=True, exist_ok=True)
59     out = SHOTS / f"{name}.png"
60     await page.screenshot(path=str(out), full_page=full_page)
61     print(f"    ✓ {out.name}")
62     return out.name
63
64
65 async def open_dicom_local(page, ref: str):
66     await page.click('button.tab[data-tab="local"]')
67     await page.wait_for_timeout(400)
68     await page.click(f'#localList li[data-path="{ref}"]')
69     await page.wait_for_function(
70         "() => !document.getElementById('imgStack').hidden",
71         timeout=15000,
72     )
73     await page.wait_for_timeout(1500)
74
75
76 async def add_demo_annotation(page):
77     label_sel = await page.query_selector("#labelSelect")
78     if label_sel:
79         opts = await label_sel.query_selector_all("option")
80         if opts:
81             await page.select_option("#labelSelect", index=0)
82     await page.click("#toolBbox")
83     box = await page.bounding_box("#canvasWrap")
84     if box:
85         cx, cy = box["x"] + box["width"] * 0.45, box["y"] + box["height"] * 0.40
86         await page.mouse.move(cx, cy)
87         await page.mouse.down()
88         await page.mouse.move(cx + 100, cy + 100, steps=10)
89         await page.mouse.up()
90         await page.wait_for_timeout(1500)
91     await page.click("#toolSelect")
92     await page.wait_for_timeout(400)
93
94
95 async def take_all(page):
96     print(" 1. Login modal...")
97     await page.goto(BASE_URL, wait_until="networkidle")
98     await wait_visible(page, "#loginModal")
99     await shoot(page, "01_login")
100
101     print(" 2. Login...")
102     await page.fill("#loginUsername", ADMIN_USER)
103     await page.fill("#loginPassword", ADMIN_PASS)
104     await page.click("#loginSubmit")
105     await page.wait_for_function(
106         "() => { const m = document.getElementById('loginModal'); return !m || m.hidden; }",
107         timeout=10000,
108     )
109     await page.wait_for_timeout(1000)
110
111     print(" 3. Empty main UI...")
112     await shoot(page, "02_empty_ui")
113
114     print(" 4. Open DICOM...")
115     try:
116         await open_dicom_local(page, "DCMDT/D00000000.dcm")
117     except Exception as e:
118         print(f"    ! DICOM ochishda xato: {e}")
119
120     print(" 5. DICOM viewer with image...")
121     await shoot(page, "03_dicom_viewer")

```

```

122
123     print(" 6. Add demo bbox...")
124     try:
125         await add_demo_annotation(page)
126         await shoot(page, "04_with_annotation")
127     except Exception as e:
128         print(f"      ! annotation xato: {e}")
129
130     print(" 7. Right pane: Annotations tab...")
131     try:
132         await page.click('.meta-pane .tab[data-rtab="anno"]')
133         await page.wait_for_timeout(400)
134         await shoot(page, "05_annotations_tab")
135     except Exception:
136         pass
137
138     print(" 8. Right pane: Hisobot tab...")
139     try:
140         await page.click('.meta-pane .tab[data-rtab="report"]')
141         await page.wait_for_timeout(800)
142         await shoot(page, "06_report_tab")
143     except Exception:
144         pass
145
146     print(" 9. Stats modal...")
147     try:
148         await page.click("#statsBtn")
149         await wait_visible(page, "#statsModal")
150         await page.wait_for_timeout(800)
151         await shoot(page, "07_stats_modal")
152         await page.click("#statsCloseBtn")
153         await page.wait_for_timeout(300)
154     except Exception as e:
155         print(f"      ! stats: {e}")
156
157     print("10. Audit timeline...")
158     try:
159         await page.click("#auditBtn")
160         await wait_visible(page, "#auditModal")
161         await page.wait_for_timeout(600)
162         await shoot(page, "08_audit_modal")
163         await page.click("#auditCloseBtn")
164         await page.wait_for_timeout(300)
165     except Exception as e:
166         print(f"      ! audit: {e}")
167
168     print("11. Overview modal...")
169     try:
170         await page.click("#overviewBtn")
171         await wait_visible(page, "#overviewModal")
172         await page.wait_for_timeout(600)
173         await shoot(page, "09_overview_modal")
174         await page.click("#overviewCloseBtn")
175         await page.wait_for_timeout(300)
176     except Exception as e:
177         print(f"      ! overview: {e}")
178
179     print("12. PACS modal...")
180     try:
181         await page.click("#pacsBtn")
182         await wait_visible(page, "#pacsModal")
183         await page.wait_for_timeout(500)
184         await shoot(page, "10_pacs_modal")
185         await page.click("#pacsCloseBtn")
186         await page.wait_for_timeout(300)
187     except Exception as e:

```

```

188         print(f"    ! pacs: {e}")
189
190     print("13. Admin modal...")
191     try:
192         await page.click("#adminBtn")
193         await wait_visible(page, "#adminModal")
194         await page.wait_for_timeout(800)
195         await shoot(page, "11_admin_modal")
196         await page.click("#adminCloseBtn")
197         await page.wait_for_timeout(300)
198     except Exception as e:
199         print(f"    ! admin: {e}")
200
201     print("14. TOTP modal...")
202     try:
203         await page.click("#totpBtn")
204         await wait_visible(page, "#totpModal")
205         await page.wait_for_timeout(1500)
206         await shoot(page, "12_totp_modal")
207         await page.click("#totpCloseBtn")
208         await page.wait_for_timeout(300)
209     except Exception as e:
210         print(f"    ! totp: {e}")
211
212     print("15. Worklist tab...")
213     try:
214         await page.click('button.tab[data-tab="worklist"]')
215         await page.wait_for_timeout(800)
216         await shoot(page, "13_worklist")
217     except Exception as e:
218         print(f"    ! worklist: {e}")
219
220     print("16. Mening ishim tab...")
221     try:
222         await page.click('button.tab[data-tab="dashboard"]')
223         await page.wait_for_timeout(800)
224         await shoot(page, "14_dashboard")
225     except Exception as e:
226         print(f"    ! dashboard: {e}")
227
228     print("17. Cleanup demo annotations...")
229     try:
230         token = await page.evaluate("() => localStorage.getItem('mamograf_jwt')")
231         import urllib.request
232         req = urllib.request.Request(
233             f"{BASE_URL}/api/annotations",
234             data=b'{"source": "local", "ref": "DCMDT/D0000000.dcm", "annotations": []}',
235             headers={
236                 "Content-Type": "application/json",
237                 "Authorization": f"Bearer {token}",
238             },
239             method="PUT",
240         )
241         urllib.request.urlopen(req, timeout=5).read()
242     except Exception:
243         pass
244
245
246     SHOT_PLACEMENTS = {
247         r"## 1\. Boshlash": "01_login",
248         r"## 2\. Asosiy interfeys": "02_empty_ui",
249         r"## 4\. DICOM ochish va ko'rish": "03_dicom_viewer",
250         r"## 5\. Annotatsiya jarayoni": "04_with_annotation",
251         r"### 5\.5 O'ng panelda annotatsiyalar": "05_annotations_tab",
252         r"## 6\. Hisobot tab": "06_report_tab",
253         r"## 9\. PACS integratsiyasi": "10_pacs_modal",

```

```

254     r"## 12\. Admin paneli": "11_admin_modal",
255     r"## 13\. Statistika": "07_stats_modal",
256     r"## 14\. Audit timeline": "08_audit_modal",
257     r"## 15\. Annotated DICOMs overview": "09_overview_modal",
258     r"## 16\. 2FA \((Two-Factor Authentication\)": "12_totp_modal",
259     r"## 17\. Worklist": "13_worklist",
260     r"### Annotator \((radiolog stajori\)": "14_dashboard",
261 }
262
263
264 CSS = ""
265 @page {
266     size: A4;
267     margin: 18mm 16mm 18mm 18mm;
268     @bottom-right {
269         content: counter(page) " / " counter(pages);
270         color: #888;
271         font-size: 9pt;
272     }
273 }
274 body {
275     font-family: "Segoe UI", "Arial", sans-serif;
276     font-size: 10pt;
277     line-height: 1.5;
278     color: #1a1a1a;
279     margin: 0;
280 }
281 h1 { color: #1f6feb; border-bottom: 2px solid #1f6feb; padding-bottom: 4px; }
282 h2 { color: #1f6feb; margin-top: 22px; border-bottom: 1px solid #cdd5e0; padding-bottom: 3px; page-
break-after: avoid; }
283 h3 { color: #2b3344; margin-top: 14px; page-break-after: avoid; }
284 h4 { color: #2b3344; margin-top: 10px; }
285 table { border-collapse: collapse; width: 100%; margin: 8px 0 14px; font-size: 9pt; page-break-inside:
avoid; }
286 table th, table td { border: 1px solid #d0d7de; padding: 4px 7px; text-align: left; vertical-align:
top; }
287 table th { background: #f0f4fa; font-weight: 600; }
288 code, pre { background: #f4f5f7; border-radius: 3px; font-family: "Cascadia Mono", "Consolas",
monospace; }
289 code { padding: 1px 4px; font-size: 9pt; }
290 pre { padding: 8px 10px; font-size: 8.5pt; overflow-x: auto; page-break-inside: avoid; }
291 ul, ol { margin: 6px 0 12px 22px; padding: 0; }
292 li { margin: 2px 0; }
293 img.shot { max-width: 100%; border: 1px solid #cdd5e0; border-radius: 4px; margin: 12px 0; box-shadow:
0 2px 6px rgba(0,0,0,0.08); page-break-inside: avoid; }
294 .cover { text-align: center; margin-top: 80mm; page-break-after: always; }
295 .cover h1 { font-size: 26pt; border: none; }
296 .cover .sub { color: #555; font-size: 13pt; margin-top: 6px; }
297 .cover .meta { color: #888; font-size: 10pt; margin-top: 24px; }
298 .toc { page-break-after: always; }
299 .toc ul { list-style: none; margin-left: 0; }
300 .toc li { margin: 3px 0; }
301 .toc a { text-decoration: none; color: #1f6feb; }
302 hr { border: none; border-top: 1px solid #cdd5e0; margin: 18px 0; }
303 ""
304
305
306 def _slugify(text: str) -> str:
307     s = re.sub(r"[^\w\s-]", "", text).strip().lower()
308     return re.sub(r"\s+", "-", s)
309
310
311 def build_html(md_text: str, screenshots: dict[str, str]) -> str:
312     lines = md_text.splitlines()
313     out_lines = []
314     h1_done = False

```

```

315     for i, ln in enumerate(lines):
316         for pattern, name in SHOT_PLACEMENTS.items():
317             if re.match(pattern, ln) and name in screenshots:
318                 pass
319             if ln.startswith("## "):
320                 for pattern, name in SHOT_PLACEMENTS.items():
321                     if re.match(pattern, ln) and name in screenshots:
322                         out_lines.append(ln)
323                         out_lines.append("")
324                         out_lines.append(f"![{name}](screenshots/{name}.png)")
325                         out_lines.append("")
326                         break
327                 else:
328                     out_lines.append(ln)
329             elif ln.startswith("### "):
330                 for pattern, name in SHOT_PLACEMENTS.items():
331                     if re.match(pattern, ln) and name in screenshots:
332                         out_lines.append(ln)
333                         out_lines.append("")
334                         out_lines.append(f"![{name}](screenshots/{name}.png)")
335                         out_lines.append("")
336                         break
337                 else:
338                     out_lines.append(ln)
339             else:
340                 out_lines.append(ln)
341
342     annotated = "\n".join(out_lines)
343     body_html = markdown.markdown(
344         annotated,
345         extensions=["tables", "fenced_code", "toc"],
346     )
347     body_html = re.sub(r'<img([>]*)>', r'<img class="shot"\1>', body_html)
348
349     headings = []
350     for ln in lines:
351         m = re.match(r"^(#{1,3}) (.+)$", ln)
352         if m and len(m.group(1)) <= 2:
353             level = len(m.group(1))
354             text = m.group(2).strip()
355             if level == 1 and h1_done:
356                 continue
357             if level == 1:
358                 h1_done = True
359                 continue
360             headings.append((level, text))
361
362     toc_items = []
363     for level, text in headings[:30]:
364         toc_items.append(f'<li style="margin-left:{(level-2)*14}px">{text}</li>')
365     toc_html = "<ul>" + "".join(toc_items) + "</ul>"
366
367     cover = f"""
368     <div class="cover">
369         <h1>MAMOGRAF DICOM Viewer</h1>
370         <div class="sub">Foydalanuvchi qo'llanmasi</div>
371         <div class="meta">{time.strftime('%Y-%m-%d')}</div>
372     </div>
373     <div class="toc">
374         <h2 style="border:none">Mundarija</h2>
375         {toc_html}
376     </div>
377     """
378
379     return f'<!doctype html>
380     <html lang="uz">

```



```

381 <head>
382 <meta charset="utf-8" />
383 <title>MAMOGRAF – Foydalanuvchi qo'llanmasi</title>
384 <style>{CSS}</style>
385 </head>
386 <body>
387 {cover}
388 {body_html}
389 </body>
390 </html>
391 ""
392
393
394 async def render_pdf(html_path: Path, pdf_path: Path):
395     print(f"\n[render] {pdf_path.name} chiqarilmoqda...")
396     async with async_playwright() as p:
397         browser = await p.chromium.launch()
398         page = await browser.new_page()
399         await page.goto(html_path.resolve().as_uri(), wait_until="networkidle")
400         await page.wait_for_timeout(800)
401         await page.pdf(
402             path=str(pdf_path),
403             format="A4",
404             margin={"top": "18mm", "bottom": "18mm", "left": "18mm", "right": "16mm"},
405             print_background=True,
406             display_header_footer=False,
407         )
408         await browser.close()
409     sz = pdf_path.stat().st_size
410     print(f"[ok] {pdf_path} – {sz/1024/1024:.2f} MB")
411
412
413 async def capture_phase():
414     print("[capture] brauzer ochilmoqda...")
415     async with async_playwright() as p:
416         browser = await p.chromium.launch()
417         ctx = await browser.new_context(
418             viewport={"width": 1600, "height": 950},
419             device_scale_factor=1,
420         )
421         page = await ctx.new_page()
422         try:
423             await take_all(page)
424         finally:
425             await browser.close()
426
427
428 def start_server() -> subprocess.Popen:
429     print("[server] ishga tushirilmoqda...")
430     env = {
431         **__import__("os").environ,
432         "LOCAL_DICOM_ROOT": str(ROOT.parent / "dicomfiles"),
433     }
434     cmd = [
435         str(ROOT / ".venv" / "Scripts" / "python.exe"),
436         "-m", "uvicorn", "app.main:app",
437         "--host", "127.0.0.1", "--port", "8765",
438         "--log-level", "warning",
439     ]
440     proc = subprocess.Popen(cmd, cwd=str(ROOT), env=env,
441                             stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
442     import urllib.request, urllib.error
443     for _ in range(40):
444         try:
445             urllib.request.urlopen(f"{BASE_URL}/api/config", timeout=1).read()
446             print("[server] tayyor")

```

```

447         return proc
448     except (urllib.error.URLError, ConnectionError):
449         time.sleep(0.5)
450     raise RuntimeError("server javob bermadi")
451
452
453 async def main():
454     if not MD_FILE.exists():
455         print(f"[xato] {MD_FILE} topilmadi", file=sys.stderr)
456         sys.exit(1)
457     DOCS.mkdir(parents=True, exist_ok=True)
458     SHOTS.mkdir(parents=True, exist_ok=True)
459
460     server = start_server()
461     try:
462         await capture_phase()
463     finally:
464         print("[server] to'xtatilmoqda...")
465         server.terminate()
466     try:
467         server.wait(timeout=5)
468     except subprocess.TimeoutExpired:
469         server.kill()
470
471     md = MD_FILE.read_text(encoding="utf-8")
472     available = {p.stem for p in SHOTS.glob("*.png")}
473     print(f"\n[shots] {len(available)} ta rasm: {sorted(available)}")
474
475     html = build_html(md, available)
476     HTML_FILE.write_text(html, encoding="utf-8")
477     await render_pdf(HTML_FILE, PDF_FILE)
478
479
480 if __name__ == "__main__":
481     asyncio.run(main())

```

build_researcher_pdf.py

Fayl yo'li: build_researcher_pdf.py | Qatorlar: 470 | Hajm: 16301 bayt

```

1  """
2  Researcher PDF – real DICOM workflow bilan amaliy skreeshotlar.
3  """
4  from __future__ import annotations
5
6  import asyncio
7  import re
8  import subprocess
9  import sys
10 import time
11 import urllib.error
12 import urllib.request
13 from pathlib import Path
14
15 if hasattr(sys.stdout, "reconfigure"):
16     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
17     sys.stderr.reconfigure(encoding="utf-8", errors="replace")
18
19 import markdown
20 from playwright.async_api import async_playwright
21
22 ROOT = Path(__file__).resolve().parent
23 DOCS = ROOT / "docs"
24 SHOTS = DOCS / "researcher_screenshots"

```

```

25 MD_FILE = ROOT / "TADQIQOTCHI_QOLLANMA.md"
26 PDF_FILE = ROOT / "TADQIQOTCHI_QOLLANMA.pdf"
27 HTML_FILE = DOCS / "_researcher.html"
28
29 BASE_URL = "http://127.0.0.1:8765"
30 ADMIN_USER = "admin"
31 ADMIN_PASS = "admin123"
32
33
34 async def shoot(page, name: str, full_page: bool = False):
35     SHOTS.mkdir(parents=True, exist_ok=True)
36     out = SHOTS / f"{name}.png"
37     await page.screenshot(path=str(out), full_page=full_page)
38     print(f"    ✓ {out.name}")
39     return out.name
40
41
42 async def login(page):
43     await page.goto(BASE_URL, wait_until="networkidle")
44     await page.wait_for_selector("#loginModal:not([hidden])", timeout=8000)
45     await page.fill("#loginUsername", ADMIN_USER)
46     await page.fill("#loginPassword", ADMIN_PASS)
47     await page.click("#loginSubmit")
48     await page.wait_for_function(
49         "() => { const m = document.getElementById('loginModal'); return !m || m.hidden; }",
50         timeout=10000,
51     )
52     await page.wait_for_timeout(800)
53
54
55 async def open_dicom_local(page, ref: str):
56     await page.click('button.tab[data-tab="local"]')
57     await page.wait_for_timeout(400)
58     parts = ref.split("/")
59     for i in range(len(parts) - 1):
60         sub = "/".join(parts[: i + 1])
61         try:
62             await page.click(f'#localList li[data-path="{sub}"]', timeout=5000)
63             await page.wait_for_timeout(500)
64         except Exception:
65             pass
66     await page.click(f'#localList li[data-path="{ref}"]', timeout=10000)
67     await page.wait_for_function(
68         "() => !document.getElementById('imgStack').hidden",
69         timeout=20000,
70     )
71     await page.wait_for_timeout(2500)
72
73
74 async def cleanup_annotations(page, ref="DCMDT/D0000000.dcm"):
75     token = await page.evaluate("() => localStorage.getItem('mamograf_jwt')")
76     try:
77         req = urllib.request.Request(
78             f"{BASE_URL}/api/annotations",
79             data=f'{{"source": "local", "ref": "{ref}", "annotations": []}}'.encode(),
80             headers={
81                 "Content-Type": "application/json",
82                 "Authorization": f"Bearer {token}",
83             },
84             method="PUT",
85         )
86         urllib.request.urlopen(req, timeout=5).read()
87     except Exception:
88         pass
89
90

```

```

91 async def take_all(page):
92     print("[1] Login + DICOM ochish...")
93     await login(page)
94     try:
95         await open_dicom_local(page, "DCMDT/D0000000.dcm")
96     except Exception as e:
97         print(f" ! DICOM ochish: {e}")
98     await shoot(page, "01_dicom_open")
99
100     print("[2] Metadata tab...")
101     try:
102         await page.click('.meta-pane .tab[data-rtab="meta"]', timeout=3000)
103         await page.wait_for_timeout(500)
104         await shoot(page, "02_metadata")
105     except Exception as e:
106         print(f" ! meta: {e}")
107
108     print("[3] Hisobot tab (xlsx integratsiyasi)...")
109     try:
110         await page.click('.meta-pane .tab[data-rtab="report"]', timeout=3000)
111         await page.wait_for_timeout(1500)
112         await shoot(page, "03_hisobot_match")
113     except Exception as e:
114         print(f" ! hisobot: {e}")
115
116     print("[4] AI model tanlash...")
117     try:
118         await page.select_option("#aiModelSelect", value="digitaleye_yolo11_l.pt")
119         await page.wait_for_timeout(400)
120     except Exception as e:
121         print(f" ! ai-select: {e}")
122
123     print("[5] AI tahlil yugurish (real digitaleye, ~10 sec)...")
124     try:
125         await page.click("#aiRunBtn", timeout=3000)
126         for _ in range(40):
127             await page.wait_for_timeout(500)
128             status_text = await page.text_content("#status")
129             if status_text and ("AI:" in status_text or "taklif" in status_text):
130                 break
131         await page.wait_for_timeout(1500)
132         await shoot(page, "04_ai_suggestions")
133     except Exception as e:
134         print(f" ! ai-run: {e}")
135
136     print("[6] Threshold slayder pastga...")
137     try:
138         await page.fill("#aiConfSlider", "10")
139         await page.dispatch_event("#aiConfSlider", "input")
140         await page.wait_for_timeout(800)
141         await shoot(page, "05_threshold_low")
142     except Exception as e:
143         print(f" ! threshold: {e}")
144
145     print("[7] Hammasini qabul qilish...")
146     try:
147         await page.evaluate("""
148             () => {
149                 if (typeof state !== 'undefined' && state.allSuggestions) {
150                     state.allSuggestions.forEach(s => {
151                         const ann = {
152                             id: 'demo_' + Math.random().toString(36).slice(2,8),
153                             type: 'bbox', label: 'mass', bi_rads: '4B',
154                             note: 'AI: ' + s.label + ' ' + (s.confidence*100).toFixed(1) + '%',
155                             frame: 0, bbox: s.bbox.slice(),
156                             created_at: new Date().toISOString(),

```

```

157         updated_at: new Date().toISOString(),
158     };
159     state.annotations.push(ann);
160 });
161 state.allSuggestions = [];
162 state.suggestions = [];
163 if (typeof renderSvg === 'function') renderSvg();
164 if (typeof renderAnnoList === 'function') renderAnnoList();
165 if (typeof scheduleAutosave === 'function') scheduleAutosave();
166     }
167 }
168 """)
169 await page.wait_for_timeout(2000)
170 await shoot(page, "06_after_accept")
171 except Exception as e:
172     print(f" ! accept: {e}")
173
174 print("[8] Annotatsiyalar tab...")
175 try:
176     await page.click('.meta-pane .tab[data-rtab="anno"]', timeout=3000)
177     await page.wait_for_timeout(500)
178     await shoot(page, "07_annotations_tab")
179 except Exception as e:
180     print(f" ! anno-tab: {e}")
181
182 print("[9] Polygon chizish...")
183 try:
184     await page.click("#toolPoly", timeout=3000)
185     await page.wait_for_timeout(300)
186     wrap = await page.evaluate(
187         "() => { const r = document.getElementById('canvasWrap').getBoundingClientRect();"
188         " return {x: r.x, y: r.y, w: r.width, h: r.height}; }"
189     )
190     cx, cy = wrap["x"] + wrap["w"] * 0.3, wrap["y"] + wrap["h"] * 0.6
191     for dx, dy in [(0, 0), (40, 5), (60, 30), (50, 60), (10, 50), (-10, 20)]:
192         await page.mouse.click(cx + dx, cy + dy)
193         await page.wait_for_timeout(150)
194         await page.keyboard.press("Enter")
195         await page.wait_for_timeout(1500)
196         await shoot(page, "08_polygon")
197         await page.click("#toolSelect", timeout=2000)
198 except Exception as e:
199     print(f" ! poly: {e}")
200
201 print("[10] Eksport tugmasi (COCO)...")
202 try:
203     await page.evaluate "() => { const b = document.getElementById('exportBtn'); if (b)
204     b.scrollIntoView(); }")
205     await page.wait_for_timeout(300)
206     await shoot(page, "09_export_buttons")
207 except Exception as e:
208     print(f" ! export-btn: {e}")
209
210 print("[11] Stats modal (dataset-darajadagi)...")
211 try:
212     await page.click("#statsBtn", timeout=3000)
213     await page.wait_for_selector("#statsModal:not([hidden])", timeout=5000)
214     await page.wait_for_timeout(1000)
215     await shoot(page, "10_stats")
216     await page.click("#statsCloseBtn")
217     await page.wait_for_timeout(300)
218 except Exception as e:
219     print(f" ! stats: {e}")
220
221 print("[12] De-ID modal – real PHI tag'lar...")
222 try:

```

```

222     await page.click("#deidBtn", timeout=3000)
223     await page.wait_for_selector("#deidModal:not([hidden])", timeout=5000)
224     await page.wait_for_timeout(1500)
225     await shoot(page, "11_deid_phi")
226     await page.click("#deidCloseBtn")
227     await page.wait_for_timeout(300)
228 except Exception as e:
229     print(f" ! deid: {e}")
230
231 print("[13] Mening ishim tab – approved annotatsiyalar...")
232 try:
233     await page.click('button.tab[data-tab="dashboard"]', timeout=3000)
234     await page.wait_for_timeout(800)
235     await shoot(page, "12_dashboard")
236 except Exception as e:
237     print(f" ! dash: {e}")
238
239 print("[14] Cleanup...")
240 await cleanup_annotations(page)
241
242
243 async def capture_phase():
244     print("[capture] brauzer ochilmoqda...")
245     async with async_playwright() as p:
246         browser = await p.chromium.launch()
247         ctx = await browser.new_context(
248             viewport={"width": 1600, "height": 950},
249             device_scale_factor=1,
250         )
251         page = await ctx.new_page()
252         try:
253             await take_all(page)
254         finally:
255             await browser.close()
256
257
258 SHOT_PLACEMENTS = {
259     r"### 1\. Maqsad va workflow": "01_dicom_open",
260     r"### 2\.1 DICOM yig'ish": "02_metadata",
261     r"### 2\.2 xlsx bilan bog'lash": "03_hisobot_match",
262     r"### 2\.3 AI bootstrap": "04_ai_suggestions",
263     r"#### Threshold sozlash": "05_threshold_low",
264     r"### 2\.4 Batch AI inference": "06_after_accept",
265     r"### 2\.5 Manual tuzatish": "07_annotations_tab",
266     r"### 3\. Eksport formatlari": "09_export_buttons",
267     r"### 3\.4 De-identification": "11_deid_phi",
268     r"### 6\. Natijalarni baholash": "10_stats",
269     r"### 7\.4 Quality monitoring": "12_dashboard",
270     r"### 10\. Tipik tadqiqot stseneriya'lari": "08_polygon",
271 }
272
273
274 CSS = """
275 @page {
276     size: A4;
277     margin: 18mm 16mm 18mm 18mm;
278     @bottom-right {
279         content: counter(page) " / " counter(pages);
280         color: #888;
281         font-size: 9pt;
282     }
283 }
284 body {
285     font-family: "Segoe UI", "Arial", sans-serif;
286     font-size: 10pt;
287     line-height: 1.5;

```

```

288     color: #1a1a1a;
289     margin: 0;
290 }
291 h1 { color: #2563eb; border-bottom: 2px solid #2563eb; padding-bottom: 4px; }
292 h2 { color: #2563eb; margin-top: 22px; border-bottom: 1px solid #cdd5e0; padding-bottom: 3px; page-
break-after: avoid; }
293 h3 { color: #1e3a8a; margin-top: 14px; page-break-after: avoid; }
294 h4 { color: #1e3a8a; margin-top: 10px; }
295 table { border-collapse: collapse; width: 100%; margin: 8px 0 14px; font-size: 9pt; page-break-inside:
avoid; }
296 table th, table td { border: 1px solid #d0d7de; padding: 4px 7px; text-align: left; vertical-align:
top; }
297 table th { background: #eff6ff; font-weight: 600; color: #1e3a8a; }
298 code, pre { background: #f4f5f7; border-radius: 3px; font-family: "Cascadia Mono", "Consolas",
monospace; }
299 code { padding: 1px 4px; font-size: 9pt; color: #be185d; }
300 pre { padding: 8px 10px; font-size: 8.5pt; overflow-x: auto; page-break-inside: avoid; border: 1px
solid #e2e8f0; }
301 pre code { color: #1a1a1a; padding: 0; background: none; }
302 ul, ol { margin: 6px 0 12px 22px; padding: 0; }
303 li { margin: 2px 0; }
304 img.shot {
305     max-width: 100%; max-height: 140mm;
306     border: 1px solid #cdd5e0; border-radius: 4px;
307     margin: 12px auto; display: block;
308     box-shadow: 0 2px 6px rgba(0,0,0,0.08);
309     page-break-inside: avoid;
310 }
311 .cover { text-align: center; margin-top: 60mm; page-break-after: always; }
312 .cover h1 { font-size: 24pt; border: none; }
313 .cover .sub { color: #555; font-size: 13pt; margin-top: 6px; }
314 .cover .badge {
315     display: inline-block; margin-top: 18px; padding: 6px 14px;
316     background: #eff6ff; color: #1e3a8a; border-radius: 18px;
317     font-size: 11pt; font-weight: 600;
318 }
319 .cover .meta { color: #888; font-size: 10pt; margin-top: 24px; }
320 .toc { page-break-after: always; }
321 .toc ul { list-style: none; margin-left: 0; }
322 .toc li { margin: 3px 0; }
323 .callout {
324     background: #eff6ff; border-left: 3px solid #2563eb;
325     padding: 8px 12px; margin: 12px 0; font-size: 9.5pt;
326 }
327 hr { border: none; border-top: 1px solid #cdd5e0; margin: 18px 0; }
328 blockquote {
329     border-left: 3px solid #94a3b8; padding-left: 10px; color: #475569;
330     margin: 8px 0; font-style: italic;
331 }
332 """"
333
334
335 def build_html(md_text: str, screenshots: set) -> str:
336     lines = md_text.splitlines()
337     out_lines = []
338     for ln in lines:
339         out_lines.append(ln)
340         for pattern, name in SHOT_PLACEMENTS.items():
341             if re.match(pattern, ln) and name in screenshots:
342                 out_lines.append("")
343                 out_lines.append(f"! [{name}] (researcher_screenshots/{name}.png)")
344                 out_lines.append("")
345                 break
346
347     annotated = "\n".join(out_lines)
348     body_html = markdown.markdown(

```

```

349         annotated,
350         extensions=["tables", "fenced_code", "toc"],
351     )
352     body_html = re.sub(r'<img([>]*)>', r'<img class="shot"\1>', body_html)
353
354     headings = []
355     for ln in lines:
356         m = re.match(r"^(#{1,3}) (.+)$", ln)
357         if m and len(m.group(1)) <= 2:
358             level = len(m.group(1))
359             text = m.group(2).strip()
360             if level == 1:
361                 continue
362             headings.append((level, text))
363
364     toc_items = []
365     for level, text in headings[:35]:
366         indent = (level - 2) * 14
367         toc_items.append(f'<li style="margin-left:{indent}px">{text}</li>')
368     toc_html = "<ul>" + "".join(toc_items) + "</ul>"
369
370     cover = f"""
371     <div class="cover">
372         <div class="badge"> 📄 Tibbiy AI Tadqiqotchi</div>
373         <h1>MAMOGRAF DICOM Viewer</h1>
374         <div class="sub">Trening dataset yaratish va custom YOLO modellari o'rgatish</div>
375         <div class="meta">{time.strftime('%Y-%m-%d')} · {len(screenshots)} screenshot · amaliy
qo'llanma</div>
376     </div>
377     <div class="toc">
378         <h2 style="border:none">Mundarija</h2>
379         {toc_html}
380     </div>
381     """
382
383     return f"""<!doctype html>
384 <html lang="uz">
385 <head>
386 <meta charset="utf-8" />
387 <title>MAMOGRAF – Tadqiqotchi qo'llanmasi</title>
388 <style>{CSS}</style>
389 </head>
390 <body>
391 {cover}
392 {body_html}
393 </body>
394 </html>
395 """
396
397
398 async def render_pdf(html_path: Path, pdf_path: Path):
399     print(f"\n[render] {pdf_path.name} chiqarilmoqda...")
400     async with async_playwright() as p:
401         browser = await p.chromium.launch()
402         page = await browser.new_page()
403         await page.goto(html_path.resolve().as_uri(), wait_until="networkidle")
404         await page.wait_for_timeout(800)
405         await page.pdf(
406             path=str(pdf_path),
407             format="A4",
408             margin={"top": "18mm", "bottom": "18mm", "left": "18mm", "right": "16mm"},
409             print_background=True,
410             display_header_footer=False,
411         )
412         await browser.close()
413     sz = pdf_path.stat().st_size

```



```

414     print(f"[ok] {pdf_path} - {sz/1024/1024:.2f} MB")
415
416
417 def start_server() -> subprocess.Popen:
418     print("[server] ishga tushirilmoqda...")
419     import os
420     env = {
421         **os.environ,
422         "LOCAL_DICOM_ROOT": str(ROOT.parent / "dicomfiles"),
423     }
424     cmd = [
425         str(ROOT / ".venv" / "Scripts" / "python.exe"),
426         "-m", "uvicorn", "app.main:app",
427         "--host", "127.0.0.1", "--port", "8765",
428         "--log-level", "warning",
429     ]
430     proc = subprocess.Popen(cmd, cwd=str(ROOT), env=env,
431                             stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
432     for _ in range(40):
433         try:
434             urllib.request.urlopen(f"{BASE_URL}/api/config", timeout=1).read()
435             print("[server] tayyor")
436             return proc
437         except (urllib.error.URLError, ConnectionError):
438             time.sleep(0.5)
439     raise RuntimeError("server javob bermadi")
440
441
442 async def main():
443     if not MD_FILE.exists():
444         print(f"[xato] {MD_FILE} topilmadi", file=sys.stderr)
445         sys.exit(1)
446     DOCS.mkdir(parents=True, exist_ok=True)
447     SHOTS.mkdir(parents=True, exist_ok=True)
448
449     server = start_server()
450     try:
451         await capture_phase()
452     finally:
453         print("[server] to'xtatilmoqda...")
454         server.terminate()
455         try:
456             server.wait(timeout=5)
457         except subprocess.TimeoutExpired:
458             server.kill()
459
460     md = MD_FILE.read_text(encoding="utf-8")
461     available = {p.stem for p in SHOTS.glob("*.png")}
462     print(f"\n[shots] {len(available)} ta rasm")
463
464     html = build_html(md, available)
465     HTML_FILE.write_text(html, encoding="utf-8")
466     await render_pdf(HTML_FILE, PDF_FILE)
467
468
469 if __name__ == "__main__":
470     asyncio.run(main())

```

_render_only.py

Fayl yo'li: _render_only.py | Qatorlar: 27 | Hajm: 796 bayt

```

1 """Faqat HTML/PDF render – capture qaytariladi."""
2 import asyncio

```

```

3 import sys
4 from pathlib import Path
5
6 if hasattr(sys.stdout, "reconfigure"):
7     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
8
9 sys.path.insert(0, str(Path(__file__).resolve().parent))
10 from build_pdf import build_html, render_pdf, MD_FILE, SHOTS, HTML_FILE, PDF_FILE, DOCS
11
12
13 async def main():
14     DOCS.mkdir(parents=True, exist_ok=True)
15     md = MD_FILE.read_text(encoding="utf-8")
16     available = {p.stem for p in SHOTS.glob("*.png")}
17     print(f"[shots] {len(available)} ta rasm")
18
19     html = build_html(md, available)
20     HTML_FILE.write_text(html, encoding="utf-8")
21     print(f"[html] {HTML_FILE} ({len(html)} bayt)")
22
23     await render_pdf(HTML_FILE, PDF_FILE)
24
25
26 if __name__ == "__main__":
27     asyncio.run(main())

```

